Report

0.2

Customer

Epineon



Fourth Review Round

NAPQES v4 — AEAD Scheme Audit

October 20, 2026

ABDK Consulting

Contents

1. Document Revisions **3**

2. Introduction **5**

3. System Overview **6**

4. Methodology **11**

5. Findings **12**

5.1. Major 13

5.2. Moderate 32

5.3. Minor 112

6. Round 5: Review of Fixes at Commit f6ced5d **177**

6.1. Moderate 178

6.2. Minor 210

7. Conclusion **269**

8. Out-of-Scope Observations (Code Not Reviewed) **272**

9. Evidence Provenance: The Section 9 Statistical Report **273**

About Us **275**

Contacts **275**

1. Document Revisions

#	Date	Author	Description
0.1	September 21, 2026	Dmitry Khovratovich	Initial report for the fourth round, covering the Rust reference implementation delivered with the version 4 paper. The specification itself was audited separately and its findings are reported elsewhere; the paper appears here only as the conformance oracle. The round comprised two independent reviews of the same artifacts, reconciled into a single set of thirty-seven findings (three major, seventeen moderate, seventeen minor). A four-region specification-to-implementation conformance pass was run, whose central result is that an encryptor written from the paper alone reproduces none of the twelve positive known-answer vectors, and all twelve under the undocumented format-subkey substitution the code performs.
0.2	October 20, 2026	Dmitry Khovratovich	Added the round-5 chapter, a review of the client's fixes at commit f6ced5d. All thirty-seven round-4 findings were re-checked: twenty-five are fixed and twelve partially fixed, and their statuses in this report are updated accordingly. The chapter reports thirty-one findings, CVF-V5-1 to CVF-V5-31 (eight moderate, twenty-three

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

Client Comment

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

NAPQES (Noise-Augmented Post-Quantum Encryption Scheme) is an Authenticated Encryption with Associated Data (AEAD) scheme whose security is claimed to reduce entirely to the pseudorandomness of `HMAC-SHA256`, deliberately avoiding the finite-field and permutation structures used by `AES-GCM` and `ChaCha20-Poly1305`. **This report audits the Rust reference implementation of NAPQES**, delivered for the first time in this engagement. The version 4 paper is not the subject of this document. It appears here in one role only, as the conformance oracle: the normative text against which the delivered code was measured. The specification was audited in its own right, separately, and those findings are reported elsewhere.

The specification, as the oracle. Because most of what follows is a comparison, the specified scheme has to be on the page. Version 4 keys two independently sampled secrets. The arithmetic-layer key $\mathbf{k} = (k_1, \ldots, k_K)$ is an ordered tuple of K distinct primes drawn uniformly without replacement from the set $\mathcal{P}$ of primes in $[10^6,\ 9.9\times10^6]$ (default $K = 10$), serialised as five-byte big-endian encodings; the derivation secret $\mathsf{sk} \in \{0,1\}^{256}$ is drawn uniformly and independently of it. The paper states that sk is the only secret that ever keys an HMAC call, and that **k** is used only, and un-keyed, by the arithmetic token map, so that no security theorem depends on the min-entropy of **k**; the first half of that statement is not true of any delivered port, and we return to it below. The size of $\mathcal{P}$ is stated exactly, as $|\mathcal{P}| = 579{,}947$: we reproduced the count by sieve and confirm it, together with $H_\infty(\mathbf{k}) = 191.46$ bits and the post-Grover figure of $2^{95.73}$ at the default K.

All internal values are produced by a domain-separated derivation

$$\mathsf{Derive}_d(\mathsf{sk}, N, \mathsf{ctx}) = \mathsf{HMAC}\big(\mathsf{sk},\ d \,\|\, N \,\|\, \mathsf{ctx}\big)$$

over nine domains covering the noise schedule, the padding codepoints, the per-position addends, the keystream and the authentication tag. The tag input is factored into a single named quantity, $\mathsf{Tagln}(A, \mathcal{B}) = \mathsf{be8}(|A|) \,\|\, A \,\|\, \mathcal{B}$, so the encryptor and the decryptor cannot disagree on the width of the length prefix. The nonce is synthetic, in the manner of RFC 5297 SIV:

$$N = \mathsf{HMAC}\big(\mathsf{sk},\ \texttt{0x0A} \,\|\, \mathsf{be8}(|A|) \,\|\, A \,\|\, M\big)[0{:}16],$$

so encryption is a pure function of the associated data and the message, and accidental nonce reuse across distinct inputs is a collision event rather than a sampling failure.

A plaintext of n codepoints is padded to a block size B given by a padding profile, a replaceable deployment parameter rather than a hard-wired rule. The normative default is the power-of-two bucket

$$B = \max(16,\ 2^{\lceil \log_2(n+1) \rceil}),$$

with a stride-thinned ladder and a fixed-frame profile as alternatives. All three take values in the same thirteen-element set, so the set of legal ciphertext lengths is profile-independent and no decryptor needs to know which profile a sender used. Padding yields $R = B + 2$ real tokens after a two-codepoint length prefix. Each padded codepoint c becomes a token $c \cdot k + a$ for a prime $k \in$ **k** and a derived addend $a \in [1, k-1]$, with pseudorandom noise tokens interleaved among the real ones. Runs of consecutive noise tokens are capped at MAX_NOISE_RUN $= 19$ and the stream is then padded with filler tokens to exactly $20R$ tokens, so the token count is a deterministic function of the padding bucket alone. Tokens are serialised in fixed-width eight-byte fields, masked with a keystream derived from domain `0x07`, and authenticated by a 32-byte tag over TagIn. The ciphertext is $C = N \,\|\, \mathcal{B} \,\|\, T$, of length $48 + 160(B + 2)$ bytes, a minimum of $2{,}928$ bytes for any message of at most fifteen codepoints. The message space is Unicode scalar values, excluding the surrogate range. Decryption carries two structural checks, both placed after tag verification: the recovered real-token count must equal $B+2$ for one of the thirteen reachable block sizes, and the recovered length prefix must not exceed the padded buffer.

The implementation as delivered. The Rust crate implements two wire formats side by side. The v8 format is the one the paper specifies; the v7 format is its predecessor, retained for compatibility, and differs in the key schedule, in the width of the associated-data length prefix and in the absence of the noise-run cap. Both live in `src/lib.rs`, which at 827 non-test nSLOC is 84% of the reviewable logic in scope and contains the entire AEAD: prime generation and key validation, the nine derivation domains, the padding and token map, the emission loop, the masking keystream, the tag, and the inverse of all of it. The v8 entry points are `encrypt_bytes_v8` and `decrypt_bytes_v8`; six further public functions expose the legacy v7 path, and `encrypt_raw` and `decrypt_raw` expose it to the crate's own transport modules.

Around `lib.rs` sit four smaller in-scope files. `src/self_test.rs` is the FIPS 140-3 power-on self-test module, executed at crate startup rather than under `cargo test`; 380 of its lines are nSLOC but 299 of those are a single embedded hexadecimal ciphertext constant, leaving 81 lines of logic. `src/kat_cross_check.rs` is the known-answer parity harness that consumes the JSON corpora and is the evidence behind the paper's byte-parity claim; it is gated behind `#[cfg(test)]`, so it compiles only under `cargo test`. `fuzz/fuzz_targets/decode_bytes.rs` and `examples/dudect_harness.rs` back the paper's fuzzing and constant-time claims respectively. `Cargo.toml` and `README.md` complete the in-scope set.

What the code computes, and what the specification requires. The single most consequential result of this round is that these are not the same function. Both v8 entry points begin by deriving a format subkey under a domain byte `0x0B` that appears nowhere in the

paper,

$$\mathrm{sk}_{\mathrm{fmt}} = \mathrm{HMAC}\big(\mathrm{sk},\ \texttt{0x0B||0x01}\big),$$

and then thread $\mathrm{sk}_{\mathrm{fmt}}$, not sk, through every one of the nine tabulated domains — the synthetic nonce, the noise threshold and positions, the padding codepoints, both addend domains, the noise characters, the keystream and the tag. On the v8 path sk itself keys exactly one HMAC call per operation, for a domain the paper never mentions.

We established this against the shipped corpus rather than by reading alone. An independent re-implementation of Enc, written from the paper's text only, was run against all twelve positive vectors of the delivered v8 known-answer corpus: keyed by sk exactly as the specification and its Table 1 caption require, **0 of 12** reproduce, and the disagreement is visible in the first sixteen bytes. With the single substitution $\mathrm{sk} \to \mathrm{sk}_{\mathrm{fmt}}$ and no other change, **12 of 12** reproduce byte for byte, ciphertext and tag included, across every padding bucket, every key size and every associated-data length in the corpus. The implemented scheme is therefore exactly the specified scheme under one uniform key substitution: every byte string, field order and width, every truncation and modular reduction, the fixed-point noise-threshold arithmetic, the run cap and the tag-input layout are correct. The key is the only divergence, and it is total.

The extra layer is deliberate and useful — it binds every derivation to one wire format, so a tag produced under one format cannot verify under another sharing the same key material — and all three ports agree with one another in applying it. It is the paper that disagrees with all three. Two corollaries follow and are developed in the findings below: the byte-identical conformance requirement of Section 10 cannot be satisfied by any implementation faithful to Table 1, so the specification's own conformance test rejects the specification; and because $\mathrm{sk}_{\mathrm{fmt}}$ is a pseudorandom output rather than a uniform draw, each of the four security theorems needs one additional $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_0)$ term for the extra hop. The composition is sound and the added term is negligible, so this is not a security regression; but the object proved about is not the object computed.

/ (root directory)
Cargo.toml README.md

/docs/

/examples/
dudect_harness.rs

/fuzz/

/fuzz/fuzz_targets/
decode_bytes.rs

/src/
kat_cross_check.rs lib.rs self_test.rs

Scope of this round. The reviewed surface is the Rust reference implementation, restricted to the seven files named by the client in the round's scope note: `src/lib.rs`, `src/self_test.rs`, `src/kat_cross_check.rs`, `fuzz/fuzz_targets/decode_bytes.rs`, `examples/dudect_harness.rs`, `Cargo.toml` and `README.md`. That is **1,283 reviewable non-test**

Rust nSLOC, or **984** once the 299-line embedded hexadecimal known-answer constant in `self_test.rs` is set aside as a single reviewable fact rather than 299 lines of logic. `src/lib.rs` alone is 827 nSLOC, 84% of that reviewable logic. A further 508 lines of in-scope Rust are test code; they are not nSLOC, but they are audited surface under the scope note and were read in full. We record one discrepancy for the client's attention: the scope note sizes `lib.rs` at 1,420 lines and the delivered file is 1,530, so the file grew by 7.7% between the scope being agreed and the snapshot we received.

The specification is not the subject of this report. This must be said plainly, because a reader could otherwise mistake this document's silence for approval. **The version 4 paper, `napseq-eprint-v4.tex` (2,281 lines), was audited separately, and its findings are reported elsewhere.** They are not restated here, and nothing in this report should be read as a clean bill of health for the specification: the absence of paper findings below reflects a division of deliverables, not an absence of paper findings. The paper enters this document in one capacity only, as the conformance oracle against which the code was measured, and the findings that cite it are all specification-to-implementation divergences, which need the specification side to be findings at all. Where such a finding quotes the paper, it quotes it to establish what the code was supposed to do.

What was not reviewed. The delivered tree is substantially larger than the reviewed surface, and the remainder was not audited. Outside scope, and not reviewed, are `src/kem.rs`, `src/kem_exchange.rs`, `src/ot_frame.rs`, the protocol adapters under `src/protocols/`, and the three binaries under `src/bin/` — together 1,983 non-test nSLOC, 1.55 times the in-scope figure, of which the 1,420 nSLOC of binaries carry no tests at all. Also outside scope are the Python reference suite under `tests/` (1,844 code lines, the oracle from which the known-answer corpora were generated) and the known-answer corpora themselves. Two consequences should be stated plainly. First, `src/lib.rs` — an in-scope file — declares the unreviewed modules as public modules of the audited crate, so a consumer of the crate sees reviewed and unreviewed cryptographic code as one public surface. Second, the byte-parity evidence the paper relies on establishes that the Rust code agrees with the Python reference implementation, not that either agrees with the specification; the reference implementation that would settle that has not been reviewed.

The two undelivered ports. The paper names three reference implementations and asserts properties of all three. Only the Rust one has ever been delivered: `napqes.py` and `C/napqes.c` have not been supplied in any round of this engagement. Every claim in this report about cross-port agreement or divergence therefore rests on the delivered Rust source, on the stored known-answer corpora, and on what the paper and the crate's own comments say about the other two ports — not on their code. Where a finding turns on that distinction, it says so.

How this round was conducted. The round comprised two independent reviews of the same artifacts, carried out separately and then reconciled into the single finding set pre-

sented below. Reconciliation resolved every claim from both reviews against the delivered source before it was kept, merged or set aside: duplicates were collapsed onto one finding, overlapping-but-distinct claims were kept separate, and claims that did not survive verification were recorded and dropped. The findings in this report are the reconciled set and are presented as the work of the engagement as a whole; no finding is attributed to an individual reviewer.

Within the implementation review, a dedicated specification-to-implementation conformance pass compared the paper against the code in four regions taken in turn — parameters and key material; the derivation schedule and the synthetic nonce; padding, the noise schedule and the token map; and the wire format and the algorithm triple. Each region was worked as a row-by-row matrix in which every normative statement in the paper was matched against the code that implements it and marked as conforming or divergent. The great majority of rows conform, and the conformance findings below are the divergences. The most consequential of them, and the central result of the round, is the format-subkey divergence described in the system overview: an independent re-implementation of Enc written from the paper alone reproduces **0 of 12** positive known-answer vectors when keyed as the specification requires, and **12 of 12** under the substitution the code actually performs. That experiment was run twice, independently, with the same outcome.

Verification method. No client code was compiled, executed, fuzzed or benchmarked at any point in this round. Every statement in this report about the implementation was established by reading it. Where a numeric claim is made — prime counts and densities, entropy figures, iteration counts, token magnitudes, collision cases — it was recomputed independently, in code written for this review and run on data generated for it, never on client artifacts. The known-answer experiment described above reproduces the client's stored vectors with an independent encryptor; it does not run the client's encryptor.

Client Comment

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the NAPQES v4 — AEAD Scheme Audit project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment
: The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis
: Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis
: For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis
: The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical
Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major
Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate
Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor
Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the NAPQES v4 — AEAD Scheme Audit codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Major	3	2
Moderate	17	8

	Found	Fixed
Minor	**17**	**15**

Fixed 25 out of 37 issues reported.

5.1. Major

CVF-11: fixed Category: **behavior**

Every derivation in the implementation is keyed by an undocumented domain-`0x0B` format subkey instead of by sk, so the reference port computes a different function from the one the paper specifies and no implementation written from the paper alone can interoperate with it — demonstrated by reproducing the shipped corpus: 0 of 12 positive vectors under sk, 12 of 12 under $\mathsf{sk}_{\mathsf{fmt}}$

Table 1: Domain-separated HMAC derivation schedule. Every domain is keyed by sk only. The nonce N is applied uniformly by Derive_d for $d \neq$ `0x0A` and is therefore omitted from the Context column above.

Reference PDF p. 6 — Table 1's caption states the property the implementation does not have: "Every domain is keyed by sk *only."*

function of its inputs (Definition 3.10), no nonce field appears and none is needed. Conforming implementations must produce byte-identical ciphertext for every positive vector and reject every negative vector. The generator `tests/gen_kats_v8.py` regenerates the file and verifies parity on

Reference PDF p. 25 — Section 10 requires that "Conforming implementations must produce byte-identical ciphertext for every positive vector" — a requirement no Table-1-faithful implementation can meet.

Section 3.3 defines the entire derivation schedule as

$$\mathrm{Derive}_d(\mathsf{sk}, N, \mathsf{ctx}) = \mathrm{HMAC}(\mathsf{sk},\ d \| N \| \mathsf{ctx})$$

, and the caption of Table 1 states the point without qualification: "Every domain is keyed by sk only."

The Rust port never keys any derivation by sk. Both v8 entry points begin by computing a format subkey under a domain byte `0x0B`:

```
fn derive_format_subkey(sk: &[u8], format_id: u8) -> [u8; 32] {
    hmac_digest(sk, &[0x0B, format_id])
}
```

and then thread `sk_fmt` — not `sk` — through every single derivation: the synthetic nonce (`0x0A`), the noise threshold (`0x02`), the padding codepoints (`0x06`), the noise positions (`0x00`), both addend domains (`0x01`, `0x05`), the noise characters (`0x04`), the keystream (`0x07`) and the authentication tag (`0x03`). The effective HMAC key for the whole scheme is therefore $\mathrm{HMAC}(\mathsf{sk},\ \texttt{0x0B} \| \texttt{0x01})$.

Domain `0x0B` does not exist in the specification. The 2,281-line source contains zero occurrences of `0x0B`, of "subkey", and of "format subkey"; the schedule in Table 1 enumerates `0x00` through `0x07` plus `0x0A` and nothing else. The one pointer the code offers is to a document that was not submitted and is not this paper — the module comment cites `docs/napseq-eprint-preprint.tex` and `docs/CAVEATS.md` for the "V8 Key Schedule and Synthetic Nonce" specification.

Nothing escapes the extra layer. We enumerated the schedule call site by call site. All nine tabulated domains are keyed by `sk_fmt`: `0x00` at `lib.rs:1160` on the encrypt side and `:580` on the decrypt side; `0x01` at `:1169` and `:592`; `0x02` at `:1150` and `:571`; `0x03` at `:1193` and `:1217`; `0x04` at `:1162` and `:1182`; `0x05` at `:1163` and `:1183`; `0x06` at `:1152`; `0x07` at `:1188` and `:1223`; and the synthetic nonce `0x0A` at `:1144`. On the v8 path sk itself keys exactly **one** HMAC call per operation — the two-byte input `0x0B||0x01` at `lib.rs:1078` — for a domain the paper never mentions. There is no residual site at which the specified keying survives.

The divergence was confirmed against the shipped corpus, not merely read off the source. We re-implemented `Enc` from the paper alone — `Definition 3.4`, Table 1, Section 3.4, Section 3.5, `Definition 3.6`, Listing 1, `Definition 3.8` and Section 3.9 — in an independent script, and ran it against all twelve positive vectors of `tests/kat/v8_vectors.json`. Keyed by sk exactly as Table 1's caption specifies, **0 of 12** reproduce; the divergence is visible in the first sixteen bytes, since for `W001` the paper-faithful nonce is `be219a83d40477dc37de106c68d81dc3` where the vector's is `e5bb595068e24fd5630555256b5d3446`. With the single substitution $\mathsf{sk} \to \mathsf{sk}_{\mathsf{fmt}} = \mathrm{HMAC}(\mathsf{sk},\ \texttt{0x0B} \| \texttt{0x01})$ and no other change, **12 of 12** reproduce byte for byte, ciphertext and tag included, across every bucket ($B \in \{16, 32, 64\}$), every key size ($K \in \{1, 4, 10\}$) and every AAD length in the corpus. So the implemented scheme is exactly the specified scheme under one uniform key substitution: every byte string, field order and width, every truncation and modular reduction, the $\theta(N)$ fixed-point arithmetic, the run cap, the ceiling and the `TagIn` layout are all correct. The key is the only divergence, and it is total.

Four consequences follow.

- **Three separate prose assertions are falsified, not one.** Table 1's caption, "Every domain is keyed by sk only" (lines 387–388); the abstract, "a uniform 256-bit se-

cret sk that keys every HMAC domain derivation used for noise injection, plaintext padding, keystream masking, and message authentication" (lines 66–67); and the Section 12 caveat *Scope of* sk *and* **k** *compromise*, "every HMAC domain derivation — the noise schedule, the padding, the per-position addends, the keystream, and the tag — is keyed by sk alone (Section 3.3)" (lines 2066–2070). All three are false of all three ports. `Lemma 3.5` is a fourth casualty of a different kind: it claims injectivity "on its *entire* argument domain", and its proof enumerates the domains as $\{$`0x00`, `0x01`, `0x02`, `0x04`, `0x05`, `0x06`, `0x07`$\}$ plus `0x03` plus `0x0A` (lines 415–417). Domain `0x0B` is absent from an enumeration the lemma explicitly declares exhaustive. The lemma's *conclusion* survives — `0x0B||0x01` is a two-byte input, shorter than the minimum of every other domain (17 bytes for the nonce-carrying domains, 9 for `0x0A`), so it cannot collide — but the enumeration needs `0x0B` added when Section 3.3 is fixed.

- **Section 10's conformance requirement is unsatisfiable as written.** Lines 1992–1993 state that "Conforming implementations must produce byte-identical ciphertext for every positive vector". No Table-1-faithful implementation can satisfy that, and we established it experimentally rather than by inference: 0 of 12 under the specified keying. The paper's own conformance test rejects the paper's own specification, and because the failure is a whole-ciphertext disagreement the corpus cannot distinguish it from an ordinary implementation bug.
- **Every byte of every v8 ciphertext differs from the specified construction.** The divergence is in the key of the outermost PRF, so it propagates to the nonce, the noise schedule, the addends, the padding, the keystream and the tag simultaneously. There is no partial agreement and no graceful failure: the two constructions share only their byte layout.
- **The four security bounds are each missing a term.** $\mathrm{sk}_{\mathrm{fmt}}$ is a PRF output, not a uniform draw, so `Assumption 4.2` — which is stated against the uniform-key distribution — does not apply to it directly. It applies after one extra PRF hop over the single-query oracle $\mathrm{HMAC}(\mathrm{sk}, \cdot)$. `Theorem 4.8`, `Theorem 4.12`, `Theorem 4.18` and `Theorem 4.20` should therefore each carry an additional $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_0)$ term for that hop. Composing an extra PRF layer is sound and the omission is quantitatively negligible, so this is not a security regression — but the object proved about is not the object computed.

We emphasise that the defect is in the specification rather than in the port. The extra layer is deliberate and beneficial: it binds every derivation to one wire format so that a tag produced under one format cannot verify under another sharing the same $(\mathbf{k}, \mathrm{sk})$. It is also consistent across the implementation family — `derive_format_subkey` is documented as matching `napqes.py::_derive_format_subkey` and `C/napqes.c::derive_format_subkey` — and `kat_cross_check.rs` records that Rust once lacked it and that the omission "went undetected" because the v7 corpus exercises no v8 code. So all three ports agree with each other and the paper disagrees with all three.

The format-identifier space, stated accurately. The code declares two identifiers, `FORMAT_BLOCK_V8: u8 = 0x01` and `FORMAT_STREAM_AE_V8: u8 = 0x02` (`lib.rs:1066` and `:1068`); there is no `FORMAT_AEAD_V8`. `FORMAT_BLOCK_V8` is used at exactly two sites, `lib.rs:1143` and `:1213`. **`FORMAT_STREAM_AE_V8` has zero uses anywhere in the delivered tree** — a search across `src/`, `tests/`, `examples/` and `fuzz/` returns only its

declaration. The format identifier never reaches the wire in any case, since the ciphertext $C = N \| \mathcal{B} \| T$ carries no format byte; it enters the key derivation only, so a ciphertext produced under a different format subkey simply fails the tag comparison and is indistinguishable from a wrong key.

The three `api != "stream_ae"` filters in the parity harness (`kat_cross_check.rs:65`, `:126`, `:191`) are therefore *not* evidence of a second v8 wire format, and an earlier version of this finding said they were. The accurate account is narrower. `tests/kat/v8_vectors.json` declares `spec_version: "v8"`, carries no `api` field at all on any of its 20 vectors, and contains no streaming vector. All six `stream_ae` vectors live in `tests/kat/v6_vectors.json`, the v6/v7-era corpus the paper itself places out of scope, and `tests/gen_kats.py:5` records that those vectors "are unaffected and still use LEB128 tokens" — that is, the streaming format is the *pre-CVF1* variable-width encoding, the very encoding retired because the blob length leaked plaintext content. The streaming API is Python-only (`napqes.encrypt_stream_ae` / `decrypt_stream_ae`); there is no Rust streaming entry point, which is precisely why the harness filters those vectors out — Rust cannot run them, not because a second v8 format is being exercised. So `0x02` is an allocated identifier with no v8 format behind it: the cross-format separation the subkey layer provides is real but currently untested, because there is only one v8 format.

Recommendation

Specify domain `0x0B` and the format-subkey layer, then restate the schedule in terms of the key that is actually used.

Concretely, in Section 3.3:

- Add `0x0B` to Table 1 as the format-subkey domain, with context `format_id` and a 32-byte output, and note that it takes neither a nonce nor any other input.
- Define $\mathsf{sk}_{\mathsf{fmt}} = \mathrm{HMAC}(\mathsf{sk},\ \texttt{0x0B} \| \mathsf{format_id})$ as a named quantity, in the same style as `Definition 3.4` already does for TagIn, and fix $\mathsf{format_id} = \texttt{0x01}$ for the block format specified here.
- Redefine the schedule as $\mathrm{Derive}_d(\mathsf{sk}_{\mathsf{fmt}}, N, \mathsf{ctx})$ for every $d \neq \texttt{0x0B}$, and replace Table 1's caption sentence "Every domain is keyed by sk only" with a statement that every domain other than `0x0B` is keyed by $\mathsf{sk}_{\mathsf{fmt}}$, and that `0x0B` alone is keyed by sk.
- Make the same correction in the two other places the paper asserts sk-keying: the abstract (lines 66–67) and the Section 12 caveat *Scope of* sk *and* **k** *compromise* (lines 2066–2070). The caveat's substantive argument is unaffected — an adversary holding sk derives $\mathsf{sk}_{\mathsf{fmt}}$ in one HMAC call and the $\mathcal{O}(|\mathcal{P}|)$ trial-division search proceeds exactly as described — but the sentence as printed is false.
- Add `0x0B` to `Lemma 3.5`'s domain enumeration (lines 415–417) and to its proof. The lemma's conclusion holds unchanged: a two-byte input cannot collide with any tabulated domain's input, all of which are at least nine bytes.
- State the reason the layer exists — cross-format tag separation — since that is the property it buys and it is currently invisible to a reader.
- Reserve the format-identifier space explicitly, recording `0x01` for block mode and `0x02` for the streaming-AE format, and say that the streaming format is out of scope for this paper. Since `0x02` is currently allocated in the Rust crate with no

Rust implementation and no v8 vectors, either describe it as reserved-for-future-use or remove the constant; leaving it declared and unused invites a reader to conclude a second v8 format exists.

Then propagate the change through Section 4. Each theorem's reduction currently hops from HMAC keyed by a uniform sk; with the subkey layer it hops twice, and the bounds of `Theorem 4.8`, `Theorem 4.12`, `Theorem 4.18` and `Theorem 4.20` each gain one $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_0)$ term for the single-query `0x0B` call. `Assumption 4.2` already supplies what is needed, since sk is uniform on $\{0,1\}^{256}$ and $\mathrm{sk}_{\mathrm{fmt}}$ is then pseudorandom; state the extra hop rather than leaving the reader to infer it.

Finally, since the paper names `rust/src/lib.rs` as a reference implementation in Section 8.2 and Section 10 requires byte-identical output, add a known-answer vector that a reader can reproduce from the paper's text alone — at minimum the format subkey for `format_id = 0x01` under a stated sk, and ideally the synthetic nonce for vector `W001` as well, so that an independent implementer localises a `0x0B` mismatch in the first sixteen bytes instead of discovering it as a whole-ciphertext disagreement.

Client Comment

/docs/napseq-eprint-v4.tex

```
66 prime-indexed token cipher, and a uniform 256-bit secret
   $\mathsf{sk}$ that
67 keys every HMAC domain derivation used for noise injection, plaintext
```

same file

```
345 All internal values are computed as
346 \[
347   \mathrm{Derive}_d(\mathsf{sk}, N, \mathsf{ctx}) =
   \mathrm{HMAC}(\mathsf{sk},\ d \| N \| \mathsf{ctx}),
348 \]
```

same file

```
387 \caption{Domain-separated HMAC derivation schedule. Every domain is
    keyed
388 by $\mathsf{sk}$ only. The nonce $N$ is applied uniformly by
```

same file

```
415 context encoding. For domains
416 $\{\texttt{0x00},\texttt{0x01},\texttt{0x02},\texttt{0x04},\texttt{0x05},\texttt
417 the context is a fixed-width positional counter (or empty), trivially
```

same file

```
1088 \[
```

```
\mathrm{Adv}^{\mathrm{IND\text{-}CPA\text{-}det}}_{\Pi}(\mathcal{A})
  \;\leq\;
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_1)
  + \frac{q^2}{2^{128}},
```

same file

```
\[
  \mathrm{Adv}^{\mathrm{LH\text{-}IND\text{-}CPA\text{-}det}}_{\Pi}(\mathcal{A})
  \;\leq\;
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_1)
  + \frac{q^2}{2^{128}},
\]
```

same file

```
\[
  \mathrm{Adv}^{\mathrm{INT\text{-}CTXT}}_{\Pi}(\mathcal{A})
  \leq
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_2)
  + \frac{q_v}{2^{256}}.
\]
```

same file

```
\[
\mathrm{Adv}^{\mathrm{IND\text{-}CCA\text{-}det}}_{\Pi}(\mathcal{A})
  \leq
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_1)
  +
  \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-}SHA256}}(\mathcal{B}_2)
  + \frac{q^2}{2^{128}}
  + \frac{q_d}{2^{256}}.
\]
```

same file

```
needed. Conforming implementations must produce byte-identical ciphertext
for every positive vector and reject every negative vector. The generator
```

same file

```
  \item[Scope of $\mathsf{sk}$ and $\mathbf{k}$ compromise.] $\mathsf{sk}$
        is the sole cryptographic secret in this construction: every HMAC
        domain derivation — the noise schedule, the padding, the
        per-position addends, the keystream, and the tag — is keyed by
        $\mathsf{sk}$ alone (Section~\ref{sec:domains}). Consequently, an
```

/src/kat_cross_check.rs

```
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "negative" && v["api"] != "stream_ae")
```

same file

```
//
// These vectors exist because the v7 corpus above does not exercise v8 at
// all, which is how the Rust port's missing domain-0x0B format subkey went
// undetected (docs/CAVEATS.md, V3-CVF1 Residual 4).
```

/src/lib.rs

```
    let noise_theta = derive_noise_threshold_v8(kb, nonce);
```

same file

```
            && is_noise_pos_v8(kb, nonce, ct_pos, noise_theta)
```

same file

```
        let addend = derive_addend(kb, nonce, real_idx, k);
```

same file

```
pub const FORMAT_BLOCK_V8: u8 = 0x01;
/// Domain `0x0B` format-subkey identifier for the v8 streaming-AE format.
pub const FORMAT_STREAM_AE_V8: u8 = 0x02;
```

same file

```
fn derive_format_subkey(sk: &[u8], format_id: u8) -> [u8; 32] {
    hmac_digest(sk, &[0x0B, format_id])
}
```

same file

```
    let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```
    let nonce = synthetic_nonce(&sk_fmt, aad, message.as_bytes());
```

same file

```
    let noise_theta = derive_noise_threshold_v8(&sk_fmt, &nonce);
```

same file

```
    let padded = pad_to_block(&codepoints, &sk_fmt, &nonce,
block_size);
```

same file

```
            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt,
&nonce, ct_pos, noise_theta) {
```

same file

```
                let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
```

same file

```
                let na = derive_noise_token_addend(&sk_fmt, &nonce,
ct_pos, k);
```

same file

```
                let addend = derive_addend(&sk_fmt, &nonce, real_idx,
k);
```

same file

```
        let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
```

same file

```
        let na = derive_noise_token_addend(&sk_fmt, &nonce, ct_pos,
k);
```

same file

```
    let ks = varint_keystream(&sk_fmt, &nonce, blob.len());
```

same file

```
    let tag = compute_auth_tag(&sk_fmt, aad, &payload,
AAD_LEN_WIDTH_V8);
```

same file

```
    let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```
    let calc_tag = compute_auth_tag(&sk_fmt, aad, payload,
AAD_LEN_WIDTH_V8);
```

same file

```
1223     let ks = varint_keystream(&sk_fmt, nonce, masked.len());
```

same file

```
1226     let codepoints = decrypt_core_v8(nonce, &tokens, primes,
&sk_fmt)?;
```

CVF-13: fixed Category: **flaw**

Six public v7 entry points short-circuit the empty input to an empty output, so a zero-length ciphertext decrypts successfully to the empty string under any key and any associated data — a universal forgery that the specification's own Enc excludes by construction, and one the crate's own PDU layer exposes to the network

Every legacy entry point begins by returning early on empty input. On the encrypt side:

```
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
  Result<Vec<u8>, String> {
    if message.is_empty() {
        return Ok(Vec::new());
    }
```

with the same three lines in `encrypt_str`, `encrypt_raw` and `encrypt_bytes_with_nonce`. On the decrypt side:

```
pub fn decrypt_bytes(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
  Result<String, String> {
    if ciphertext.is_empty() {
        return Ok(String::new());
    }
```

with the same in `decrypt_str` and `decrypt_raw`.

The decrypt-side short-circuit is the problem. It returns `Ok` before the key is touched, before the tag is computed and before the associated data is examined. So for any key and any AAD whatsoever, the zero-length ciphertext is accepted as a valid encryption of the empty message. An attacker who truncates a ciphertext to nothing, or drops it and substitutes an empty frame, obtains a successful decryption rather than an authentication failure. Nothing distinguishes this from a legitimate empty message, because a legitimate empty message is encoded identically — as nothing at all. `decrypt_raw`'s own doc comment states the contract the branch below it breaks: "Verifies the HMAC tag (constant-time) before decrypting. Returns `Err` on authentication failure."

This is a universal forgery for one message, available without the key. It is narrow, in that the only plaintext an attacker can produce this way is the empty string, but it is exactly the event the integrity property is supposed to make infeasible: `Theorem 4.18`

bounds the INT-CTXT advantage by $\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC}}(\mathcal{B}_2) + q_v/2^{256}$, and here it is 1 after a single verification query.

The behaviour also diverges from the specification, which defines Enc on the empty message rather than excluding it. Under `bucket`, $B = \max(16, 2^{\lceil \log_2 1 \rceil}) = 16$, so $R = 18$ and $|C| = 48 + 160 \cdot 18 = 2{,}928$ bytes — the paper's own stated minimum well-formed ciphertext. The specified scheme therefore emits a full 2,928-byte authenticated ciphertext for the empty message, and the v7 API emits zero bytes.

The v8 path is correct and should be read as the model: `encrypt_bytes_v8_with_profile` has no empty short-circuit, `bucket_block_size(0)` returns 16, and the empty message produces the full 2,928-byte ciphertext the specification requires. Vector `W001` — "Empty message produces a full authenticated ciphertext" — is exactly 2,928 bytes and confirms it. Only the legacy surface is affected.

Reach: the bypass is exposed to the network by the crate's own protocol layer. This evidence comes from `src/ot_frame.rs`, which is **outside the reviewed artifact surface** for this engagement and is cited here as impact only; the root cause is wholly inside `src/lib.rs`, at the six short-circuits listed above. `ot_frame::unwrap_pdu` parses the 23-byte OT envelope header — which validates only the two-byte magic, the protocol identifier and the declared blob length, and is itself unauthenticated — and then calls `decrypt_raw` on the declared blob. An envelope carrying `blob_len = 0` therefore reaches `decrypt_raw(&[])`, hits the short-circuit, and is returned to the caller as a **successfully authenticated, replay-checked PDU under any key**. A 23-byte injected frame is accepted. The envelope header is bound in only as AAD to a tag that on this path is never computed, so nothing else authenticates it either. That converts an API-level hazard into a live protocol-level authentication bypass in the delivered tree.

Two tests entrench the behaviour rather than catching it, and one of them fixes the remediation cost. `lib.rs` asserts the round trip:

```
#[test]
fn empty_message_roundtrip() {
    let k = test_key();
    assert_eq!(decrypt_str(&encrypt_str("", &k, b"").unwrap(), &k,
  b"").unwrap(), "");
}
```

which passes without executing a single cryptographic operation — both calls return through their short-circuits, so the assertion compares "" to "". The in-crate parity harness goes further and **asserts the empty↔empty mapping as a conformance requirement**:

```
if message.is_empty() {
    let ct = encrypt_bytes_with_nonce(message, &[], [0u8;
  NONCE_SIZE], b"");
    assert!(ct.is_empty(), "[{}] empty message should produce empty
  ciphertext", id);
```

under an **empty key** and an **all-zero nonce**. So the behaviour this finding reports as a flaw is pinned by the test suite as correct, and any fix will fail that assertion. This is a material addition to the remediation estimate and should be planned for rather than discovered.

Severity: raised from medium to high in round 4, with the reasoning recorded. Two amplifications were put forward. We accept one and decline the other.

- *Accepted.* The bypass is a categorical break of the property `Theorem 4.18` asserts, not a degradation of it: an adversary with no key material obtains a `Dec` output of `Ok` on a chosen input, at advantage 1 with a single query. It is reachable from public, non-deprecated API, and — the point that decides the severity — the crate's own PDU layer routes network input into it, so no third-party misuse is required to reach it. Under this engagement's scale, an unauthenticated party obtaining an accepted frame on an authenticated channel is unauthorized access, which is the high band. Against that, the forged plaintext is a single fixed value (the empty string), so the attacker injects a null frame rather than chosen content, and the defect is confined to the v7 surface, which the paper explicitly disclaims as "not the scheme analysed here". Those two facts bound the exploit, but they do not soften the break, and in an OT command-carrying context an accepted empty frame that the receiver cannot distinguish from a genuine suppressed message is a meaningful integrity failure in its own right. On balance, high.

- *Declined.* It was also proposed that the severity be raised because `ot_frame::unwrap_pdu` commits the replay counter (`ot_frame.rs:347`, writing `*last = seq` at `:175`) *before* calling `decrypt_raw` at `:355`, so one injected envelope advances the sequence counter to any attacker-chosen `seq` and permanently locks out the genuine peer. That behaviour is real and we confirmed it, but it does not belong to this finding: it holds for **any** frame whose tag fails, not only for the empty one, so it is not caused by the empty-input short-circuit and would survive a complete fix to `src/lib.rs`. Its root cause is the ordering inside `ot_frame.rs`, which is outside the reviewed surface. It is recorded separately for a scope decision rather than counted toward this severity.

Recommendation

Remove the decrypt-side short-circuits first, since those are the ones that accept a forgery. In `decrypt_bytes`, `decrypt_str` and `decrypt_raw`, delete the `if ciphertext.is_empty() { return Ok(..) }` block and let the existing length floor reject the input:

```
if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
    return Err(format!(
        "Ciphertext too short: {} bytes; header+tag require at
  least {}.",
        ciphertext.len(), NONCE_SIZE + TAG_SIZE
    ));
}
```

A zero-length ciphertext then fails this check like any other truncated input, and the empty message stops being representable as an empty ciphertext. This is the change that closes the protocol-layer exposure, and it can be made on its own, ahead of the encrypt-side work, if a staged fix is preferred.

Then remove the encrypt-side short-circuits so the empty message is encrypted rather than elided, matching both the v8 path and the specification. `pad_message` and `bucket_block_size` already handle $n = 0$ correctly — `bit_length(0)` is 0,

floored to `PAD_MIN_EXP`, giving $B = 16$ — so no other change is needed and the result is the specified 2,928-byte ciphertext.

If backward compatibility with stored v7 ciphertexts forbids changing the encrypt side, then at minimum keep the decrypt-side fix and have the empty message return an explicit error from the encrypt entry points, so the two sides cannot disagree about whether the empty message has a representation.

Budget for the test work, which is not incidental. `empty_message_roundtrip` should assert that the ciphertext is a full 2,928 bytes and that it round-trips through a real tag verification; as written it would pass even if both functions were replaced by `Ok(String::new())`. The parity harness's empty-message branch **asserts the behaviour being removed** and will fail the moment the fix lands, so it must be deleted along with the vector it services, or rewritten to assert the new behaviour — and it should stop passing an empty key, which is unrelated to what the branch claims to check and is itself rejected by `validate_key` on the v8 path. Any Python or C corpus entry that pins the same empty↔empty mapping needs the same treatment, or the three ports will disagree after the fix.

Separately, and independently of this finding, review the ordering in `ot_frame::unwrap_pdu`: the replay counter is committed before the payload is authenticated, so any frame with a forged tag advances it. That is out of the reviewed scope here and is recorded on its own, but it should be fixed in the same pass, since both defects are reached by the same injected envelope.

Client Comment

/src/kat_cross_check.rs

```
133
134         // Empty-message vector: nonce_hex is "" and ciphertext_hex
    is ""
135         if message.is_empty() {
136             let ct = encrypt_bytes_with_nonce(message, &[], [0u8;
    NONCE_SIZE], b"");
137             assert!(ct.is_empty(), "[{}] empty message should produce
    empty ciphertext", id);
138             eprintln!("[PASS] {} — empty message", id);
139             tested += 1;
140             continue;
```

/src/lib.rs

```
731     if message.is_empty() {
732         return Ok(Vec::new());
733     }
```

same file

```
    if message.is_empty() {
        return Vec::new();
    }
```

same file

```
    if ciphertext.is_empty() {
        return Ok(String::new());
    }
```

same file

```
    if message.is_empty() {
        return Ok(String::new());
    }
```

same file

```
    if cypher.is_empty() {
        return Ok(String::new());
    }
```

same file

```
    if data.is_empty() {
        return Ok(Vec::new());
    }
```

same file

```
/// Verifies the HMAC tag (constant-time) before decrypting.  Returns
/// `Err` on authentication failure — the caller must never use the
/// ciphertext for any purpose if this returns an error.
```

same file

```
    if ciphertext.is_empty() {
        return Ok(Vec::new());
    }
```

same file

```
    fn empty_message_roundtrip() {
        let k = test_key();
        assert_eq!(decrypt_str(&encrypt_str("", &k, b"").unwrap(),
&k, b"").unwrap(), "");
    }
```

CVF-18: open Category: **procedural**

All three known-answer tests in the FIPS power-on self-test exercise the legacy v7 format and none touches the specified scheme, so the mandatory pre-operation check would pass with the entire v8 derivation schedule broken; the fourth test is a version-string comparison that passes today by tautology and fails on the first version bump.

The module presents itself as a FIPS 140-3 cryptographic module with a hard pre-operation contract:

> Call `run_power_on_self_tests()` before using the module in production. It returns `Ok(())` only if all tests pass. On any failure it returns `Err(SelfTestError)` — the caller MUST NOT perform any cryptographic operations after a self-test failure.

citing NIST SP 800-140B §4.9.1. Four tests run: KAT-1 encrypt, KAT-2 decrypt, KAT-3 tamper rejection, INT-1 software integrity.

All three KATs are v7. KAT-1 calls `crate::encrypt_bytes_with_nonce`, and KAT-2 and KAT-3 both call `crate::decrypt_bytes`. None of the four calls `encrypt_bytes_v8` or `decrypt_bytes_v8`, so the following code has no coverage in the power-on self-test at all: `derive_format_subkey` and the domain-`0x0B` subkey, `synthetic_nonce` and domain `0x0A`, `derive_noise_threshold_v8` and the integer $\theta(N)$ computation, `is_noise_pos_v8`, the capped emission loop and the token ceiling, `decrypt_core_v8` with its divisibility, bucket and length-prefix checks, and the `AAD_LEN_WIDTH_V8` tag prefix. The specified construction could be broken in any of those places and `run_power_on_self_tests()` would still return `Ok(())`, whereupon the module's own contract certifies it fit for production use.

The vector confirms which format it belongs to without any need to run it. `KAT_CIPHERTEXT_HEX` decodes to 4,784 bytes, so $N_{\text{tok}} = (4784 - 48)/8 = 592$ for the one-codepoint message `"A"`, whose padded block is $R = 18$. That is 32.9 tokens per padded codepoint where the specification caps the ratio at 20, and 592 is not a multiple of 20, so no ceiling was applied. The specified v8 ciphertext for the same message is exactly 360 tokens and 2,928 bytes — a figure the delivered v8 corpus independently confirms, since `W002` has that message and that length. See CVF-17, for which this vector is also the evidence.

INT-1 is not an integrity check. It is:

```
const BUILD_HASH: &str = env!("CARGO_PKG_VERSION");

fn integrity_check() -> Result<(), SelfTestError> {
    if BUILD_HASH == "0.1.0" {
        Ok(())
    } else {
        Err(SelfTestError::IntegrityCheckFailed)
    }
}
```

The constant is named `BUILD_HASH` but holds the crate's declared version string, and the test compares it to a literal. It detects no modification of the binary, of the source, or of anything else — any tampering that leaves `Cargo.toml`'s version field alone passes, and the version field is the one thing an attacker modifying the module has no reason to touch.

The comment above it is candid about this ("This stub verifies a compile-time build metadata string rather than a full binary HMAC", deferred to "Phase 4 workstream 4.1"), but the candour lives in a source comment while `SelfTestError::IntegrityCheckFailed` and the module header present INT-1 to the operator as a software integrity check.

The manifest is now delivered and the outcome is settled rather than unknown. `Cargo.toml` declares `version = "0.1.0"` at line 3, so `CARGO_PKG_VERSION` expands to exactly the literal `integrity_check` compares against: INT-1 returns `Ok(())` on every build of this version, unconditionally. There is therefore no version of the crate at which the test tests anything — it is a tautology now and a contradiction from the first bump. That second half is the latent failure: the first release past `0.1.0` makes `integrity_check` return `Err`, which makes `run_power_on_self_tests` return `Err`, which by the module's stated contract forbids the caller from performing any cryptographic operation. A routine version bump disables the module, and it does so reporting an integrity-check failure — the most alarming diagnostic in the set — rather than a stale constant.

On the staleness question the scope file raises for this file, the answer is that the vector has not gone stale, and the corpus now settles it directly rather than by inference. Reading the v7 derivation path against the vector's provenance comment, we found no change since the vector was regenerated: every v7 derivation still keys on `kb = key_bytes(key)`, retains the `d || N || ctx` layout the comment names as the last format-affecting change, and uses `AAD_LEN_WIDTH_V7 = 4`; the v8 work added new functions rather than altering the v7 ones. Decoding the constant confirms it. `KAT_CIPHERTEXT_HEX` is a `concat!` of 299 sixteen-byte hex groups spanning three hundred lines of source — which is why a search for a single long hex literal does not find it — totalling 9,568 hex characters, and the 4,784 bytes it decodes to are **byte-identical** to the `ciphertext_hex` field of vector `V002` in `tests/kat/v6_vectors.json`: same key `[1000003, 1000033, 1000037, 1000039]`, same nonce `9c6c0b921a83849cdbf2fe7efb743fe9`, same message `"A"`, same empty AAD, same 4,784 bytes including the tag, with no differing byte anywhere. The self-test vector and the corpus agree exactly.

Two residues remain from that check. The provenance comment mischaracterises what the constant is: it says the vector is "derived from the current (v7, post-CVF1-fix) Python reference implementation, using the same key/nonce/message as the retired v6 vector `tests/kat/v6_vectors.json` V002", when the constant simply *is* V002, byte for byte — and V002 is not retired, since three of the seven tests in `kat_cross_check.rs` still consume the corpus it lives in (CVF-25). And the two copies are unsynchronised: one 4,784-byte value now exists in a three-hundred-line `concat!` literal and in a JSON file, with nothing in the crate comparing them. They agree today; nothing makes them agree tomorrow, and a regeneration of the corpus that leaves the literal untouched reproduces exactly the drift the scope file anticipated — silently, in the one test a FIPS operator is instructed to trust before doing anything else.

Reconciliation note (round 4). The independent audit filed the INT-1 half of this finding separately, as id 1171, at high severity; that issue is excluded as a duplicate of this one, which is retained because it is strictly broader — it covers the v7-only KAT coverage gap as well as INT-1, and it confirms the INT-1 outcome against `Cargo.toml:3` rather than inferring it. On the user's direction that a duplicate takes the independent auditor's severity, this finding is raised from medium to **high**. We record that we consider high

defensible on the merits as well, though for the combination rather than for either half: INT-1 alone is a candidly documented stub, but a FIPS module whose mandatory pre-operation gate does not execute a single line of the scheme the paper specifies is a materially stronger claim, and it is the conjunction that this finding establishes. See also CVF-47, which reports that nothing enforces the self-test outcome at run time — the gate is neither complete in what it tests nor binding on what it permits.

Recommendation

Add v8 known-answer tests to the power-on suite, and keep the v7 ones only for as long as the v7 format remains supported.

Since v8 encryption is deterministic in $(\mathbf{k}, \mathsf{sk}, A, M)$ — the nonce is a synthetic IV — a v8 KAT needs no nonce-injection entry point and can call the public API directly, which is simpler than the v7 case:

```
const KAT_V8_PRIMES: &[u64] = &[/* ... */];
const KAT_V8_SK: [u8; SK_SIZE] = [/* ... */];
const KAT_V8_MESSAGE: &str = "A";
const KAT_V8_CIPHERTEXT_HEX: &str = /* 2,928 bytes for the
  empty-AAD, one-codepoint case */;
```

with the three tests mirroring the existing ones: encrypt to the reference bytes, decrypt to the reference plaintext, and reject a tag-flipped copy. The delivered corpus supplies all four values directly — `W002` is that exact case — so this needs no new vector generation. Add a fourth test that exercises the post-authentication structural checks, since those are new in v4 and currently untested anywhere in the self-test: a ciphertext whose token count is a multiple of 20 but not $20(B+2)$ must be rejected by `decrypt_core_v8`'s bucket check, and `W-N07` in the corpus is already that vector.

Prefer generating the vectors from the KAT corpus at build time over embedding them as hex literals. Embedding is what created the staleness trap the scope file warned about, and the corpus now exists; `include_str!` on the vector file, or a `build.rs` that emits the constants, keeps the self-test and the corpus from drifting apart. If a literal must be kept for a no-dependency power-on path, add a test that cross-checks the literal against the corpus so drift fails loudly in CI rather than silently in the field. That applies to the existing v7 literal too, now that it is confirmed to duplicate `V002` byte for byte — a three-line test comparing `decode_hex(KAT_CIPHERTEXT_HEX)` against that vector's `ciphertext_hex` would make the duplication self-policing at no cost.

Correct the provenance comment while doing so. It should say that the constant is vector `V002`, not that it shares that vector's inputs, and it should not describe as retired a vector that three live tests consume.

Replace or rename INT-1. If the binary HMAC is genuinely deferred, then the honest interim is to remove `IntegrityCheckFailed` from the operator-visible result set and report the stub as what it is — a build-provenance check — rather than as an integrity check; a self-test that names a property it does not test is worse than an absent one, because an operator reads `Ok(())` as covering it. If it is kept in the interim, stop comparing to a hard-coded version literal, and in particular do not "fix" it by editing the literal at each release, which preserves a check that tests nothing while removing the one symptom that reveals it. Compare against a value generated at build time — `option_env!` on a build-injected digest, defaulting to a skip — so that a version bump

cannot brick the module. The `build.rs` design sketched in the comment (HMAC over `.text` and `.rodata`, written to `OUT_DIR`, pulled in with `include_bytes!`) is the right target and would make INT-1 mean what its name says.

Finally, note that the module as compiled contains an algorithm the power-on suite does not cover at all: `pub mod kem` exposes a FrodoKEM key exchange with no self-test. Under a FIPS framing every approved algorithm inside the module boundary needs one, and CVF-23 covers the boundary problem that makes this hard to see.

Sequence this ahead of CVF-47. That finding asks for the self-test outcome to be latched and enforced on every cryptographic entry point, which is the right end state; enabling it while the three KATs are v7-only and INT-1 is a version literal would inhibit the module on the strength of tests that say nothing about the specified scheme, and would brick it outright at the first version bump.

Client Comment

/Cargo.toml

```
version = "0.1.0"
```

/src/self_test.rs

```
//! FIPS 140-3 power-on self-tests for the NAPQES Cryptographic Module.
//!
//! Call `run_power_on_self_tests()` before using the module in production.
//! It returns `Ok(())` only if all tests pass.  On any failure it returns
//! `Err(SelfTestError)` — the caller MUST NOT perform any cryptographic
//! operations after a self-test failure.
//!
```

same file

```
    /// KAT-3: tampered ciphertext was not rejected.
```

same file

```
    /// INT-1: software integrity check failed.
```

same file

```
pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
    kat_encrypt()?;
    kat_decrypt()?;
    kat_tamper_rejection()?;
```

```
    integrity_check()?;
    Ok(())
}
```

same file

```
fn kat_encrypt() -> Result<(), SelfTestError> {
    let expected = decode_hex(KAT_CIPHERTEXT_HEX)?;
    let got = crate::encrypt_bytes_with_nonce(KAT_MESSAGE, KAT_KEY,
KAT_NONCE, b"");
    if got == expected {
        Ok(())
    } else {
        Err(SelfTestError::KatEncryptMismatch)
    }
}
```

same file

```
fn kat_decrypt() -> Result<(), SelfTestError> {
    let ciphertext = decode_hex(KAT_CIPHERTEXT_HEX)?;
    match crate::decrypt_bytes(&ciphertext, KAT_KEY, b"") {
        Ok(pt) if pt == KAT_MESSAGE => Ok(()),
        Ok(_) => Err(SelfTestError::KatDecryptMismatch),
        Err(_) => Err(SelfTestError::KatDecryptMismatch),
    }
}
```

same file

```
fn kat_tamper_rejection() -> Result<(), SelfTestError> {
    let mut ciphertext = decode_hex(KAT_CIPHERTEXT_HEX)?;
    // Flip the last byte of the authentication tag.
    let last = ciphertext.len() - 1;
    ciphertext[last] ^= 0xFF;
    match crate::decrypt_bytes(&ciphertext, KAT_KEY, b"") {
        Err(_) => Ok(()),
        Ok(_) => Err(SelfTestError::KatTamperNotRejected),
    }
}
```

same file

```
// ── INT-1: software integrity ──────────────────────────────────
//
// The integrity check verifies that the module binary has not been
modified
// since it was compiled.  The reference hash is embedded at compile
time by
// `build.rs` (see below).
```

```
//
// IMPLEMENTATION STATUS:
//   This stub verifies a compile-time build metadata string rather
than a
//   full binary HMAC.  Replacing it with a binary HMAC requires a
build.rs
//   that:
//     1. Computes HMAC-SHA256 of the compiled `.text` + `.rodata`
sections.
//     2. Writes the digest to `OUT_DIR/module_integrity.bin`.
//     3. include_bytes! pulls it in here.
//   This is a Phase 4 workstream 4.1 item.  The current
implementation
//   satisfies the Level 1 pre-attestation requirement to demonstrate
the
//   integrity-check mechanism is in place, pending the full binary
HMAC.
```

same file

```
const BUILD_HASH: &str = env!("CARGO_PKG_VERSION");

fn integrity_check() -> Result<(), SelfTestError> {
    // In the full implementation this will compare an HMAC over the
loaded
    // module binary to a reference digest embedded by build.rs.
    // For now, verify that the build version string matches a
compile-time
    // constant to ensure the binary was built from this source tree.
    if BUILD_HASH == "0.1.0" {
        Ok(())
    } else {
        Err(SelfTestError::IntegrityCheckFailed)
    }
}
```

/tests/kat/v6_vectors.json

```
      "id": "V002",
```

/tests/kat/v8_vectors.json

```
      "id": "W002",
```

same file

```
      "id": "W-N07",
```

5.2. Moderate

CVF-12: fixed Category: **behavior**

The paper credits the Rust port's constant-time tag comparison to the `subtle` crate; the manifest does declare `subtle` and no call site in the crate ever invokes it, so a reviewer who checks the manifest finds the claim apparently substantiated while every comparison in fact runs through a hand-rolled volatile loop, and the one caveat the paper reserves for the C port is precisely the caveat that applies to Rust.

Section 8.4 states, of tag comparison, that it "uses `hmac.compare_digest` (Python), `constant_time_eq` from the `subtle` crate (Rust), and an OR-accumulator byte-difference loop (C)", and then attaches a caveat to the C implementation only: "The C accumulator is not declared `volatile`, so nothing in the C standard prevents a sufficiently aggressive compiler from reintroducing an early exit; this is the usual caveat for hand-rolled constant-time C and is not addressed here."

`Cargo.toml` is now delivered, and it does declare the dependency: `subtle = "2"` at line 11. No call site anywhere in the crate invokes it. `subtle`, `ConstantTimeEq`, `constant_time_eq` and `Choice` appear in no `use` declaration and in no expression under `src/`, `examples/` or `fuzz/`; outside the manifest the string `subtle` occurs once, in a doc comment. Tag comparison at all three verification sites, and the CRNG nonce check, go instead to a hand-rolled function:

```
#[inline(never)]
fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    debug_assert_eq!(a.len(), b.len());
    let mut diff = 0u8;
    for (x, y) in a.iter().zip(b.iter()) {
        unsafe {
            diff |= std::ptr::read_volatile(x) ^
  std::ptr::read_volatile(y);
            std::ptr::write_volatile(&mut diff, diff);
        }
    }
    diff == 0
}
```

Its own doc comment argues at length that `#[inline(never)]`, `read_volatile` and a per-iteration `write_volatile` together "prevent LLVM from generating an early-exit

loop" and that the result "is a fixed-count loop". That reasoning is plausible and considerably more careful than the C accumulator's, but it is exactly the same *kind* of argument — a hand-rolled construction resting on what an optimiser is expected not to do — and Rust offers no more of a language-level guarantee here than C does. The paper's framing inverts the situation: it presents Rust as the case where a vetted library discharges the obligation and C as the lone hand-rolled exception, when in fact two of the three ports are hand-rolled and only Python delegates to a standard-library primitive.

A declared dependency that nothing calls is a worse state than an absent one, and this is the substance of the finding rather than an aside. Cargo emits no warning for an unused dependency, so nothing in an ordinary build surfaces it, and `subtle` is compiled and linked into every consumer to no purpose. More to the point, the manifest is the artifact a reviewer consults to check Section 8.4's attribution — the scope for this review names `Cargo.toml` for exactly that, "confirms `subtle` (constant-time tag compare claim, paper §Constant-Time)". It does not confirm it. It corroborates a claim the code does not implement, and a reader who finds the dependency declared has no reason to go on to the call sites, which are the only place the discrepancy is visible. Had the dependency simply been absent, the check would have failed at the first step.

This matters beyond attribution. A reader auditing the deployment for FIPS or common-criteria purposes will look for the `subtle` dependency, find it declared, and conclude that the obligation is discharged by a reviewed library — the one conclusion the source does not support. And the caveat that the paper does state, that a sufficiently aggressive compiler may reintroduce an early exit, is left attached to the wrong implementation: it applies to the Rust loop as well, and the Rust loop is what the deployment runs.

Assessed on its own merits, now that it is established as the real comparator, `ct_eq_bytes` is sound in the respect that matters and unsound in one that does not currently bite.

What it gets right. There is no early return and no `break`; the loop runs to completion on every call. The trip count is `min(a.len(), b.len())`, a function of the operand lengths and not of their contents, so nothing about the position of the first differing byte reaches the control flow. `read_volatile` on both operands makes every load an observable side effect that LLVM may not eliminate; the `write_volatile` to `diff` is a store to an address-taken local that it may likewise not elide, so iterations cannot be deleted or coalesced; and `#[inline(never)]` keeps the caller's `if !ct_eq_bytes(...)` branch from being sunk into the body. The accumulator is a byte-wise `|=` over an `xor`, both single-cycle and data-independent on every architecture of interest. The returned `bool` does reintroduce a branch at each call site, but on the single accumulated result rather than on any tag byte, which is unavoidable and correct. As hand-rolled constant-time code goes the reasoning is careful, and we found no defect in it.

What it gets wrong is the length. The precondition is enforced only by `debug_assert_eq!`, which is compiled out in release builds — and `Cargo.toml` declares no `[profile]` section, so the release profile takes Cargo's default `debug-assertions = false` and the check is absent from any shipped binary (CVF-23). The loop then uses `a.iter().zip(b.iter())`, and `zip` stops at the shorter operand, so on a length mismatch the function compares only the common prefix and returns `true` whenever that prefix agrees. All four call sites pass slices that are equal-length by construction — `recv_tag` is cut at `ciphertext.len() - TAG_SIZE` and `calc_tag` is a 32-byte

HMAC output at each of the three verification sites, and the CRNG check compares two `[u8; 16]` — so this is latent rather than live, and we found no path that reaches it. It is nonetheless a comparison primitive guarding tag verification whose failure mode on misuse is to report equality, which is the wrong direction: a caller passing a truncated tag, or any future reuse against a variable-length operand, gets a forgery accepted rather than rejected.

The declared-but-uncalled dependency is precisely what would fix that. `subtle`'s `ConstantTimeEq` implementation for slices returns `Choice::from(0)` on a length mismatch rather than comparing a prefix, so it fails closed. The crate has been carrying, unused, the dependency that supplies the one property its own comparator lacks.

One further note on scope. The function is named and documented as a constant-time equality test, but early-exit resistance is the only constant-time property it has or claims; it does not hide the length of its operands, which is fine for its four present uses and a hazard for a fifth. And the harness that would detect a regression in it times the whole of `decrypt_bytes` rather than the comparison, which per CVF-26 leaves the discriminating signal well below the measurement noise.

Recommendation

Either make the paper match the code or make the code match the paper. Both are defensible; the second is cheaper and closes the secondary defects too.

To make the code match the paper, no manifest change is needed — `subtle` is already declared. Replace the body of `ct_eq_bytes` with `subtle::ConstantTimeEq`:

```
use subtle::ConstantTimeEq;

fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    a.ct_eq(b).into()
}
```

`ConstantTimeEq` for slices returns `Choice::from(0)` on a length mismatch rather than comparing a prefix, so this fixes the `zip` truncation at the same time, and it removes two `unsafe` blocks and the optimiser argument that justifies them. It also makes the dependency earn its place in the manifest.

To make the paper match the code instead, rewrite the Section 8.4 sentence to say that Python delegates to `hmac.compare_digest` while both Rust and C use hand-rolled accumulator loops, and move the compiler caveat so it covers both. If that route is taken, note in the same place that the Rust loop does declare its accesses volatile and is therefore on firmer ground than the C one, and state explicitly that neither rests on a language-level guarantee. Remove `subtle` from `Cargo.toml` in the same change, so the manifest stops corroborating a claim the paper will no longer be making.

Independently of which route is chosen:

- Replace the `debug_assert_eq!` length check with a real one that survives release builds, returning `false` on a mismatch, so the primitive cannot report equality for operands of different lengths. A profile setting should not be load-bearing for a tag comparator; see CVF-23 on the absent `[profile]` section.
- Add a test for the mismatch case. There is currently no test anywhere that calls

`ct_eq_bytes` with unequal lengths, which is why a comparator that fails open has gone unremarked.

- Add an unused-dependency check to CI — `cargo udeps` or `cargo machete` — so that a manifest entry with no call site fails a build rather than serving as evidence. This is the mechanism that let the discrepancy survive: nothing in the toolchain objects to a declared dependency that is never used.
- Since the timing harness is the only thing that would catch a regression in this function, narrow it to measure the comparison itself, per CVF-26.

Client Comment

/Cargo.toml

```
11 subtle = "2"
```

same file

```
40 [lib]
41 name = "napqes"
42 path = "src/lib.rs"
```

/src/lib.rs

```
56         if ct_eq_bytes(&nonce, &prev_nonce) {
```

same file

```
637 #[inline(never)]
638 fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
639     debug_assert_eq!(a.len(), b.len());
640     let mut diff = 0u8;
641     for (x, y) in a.iter().zip(b.iter()) {
642         unsafe {
643             diff |= std::ptr::read_volatile(x) ^
std::ptr::read_volatile(y);
644             // The volatile write forces this accumulated value to be
645             // materialised every iteration; LLVM cannot skip
iterations.
646             std::ptr::write_volatile(&mut diff, diff);
647         }
648     }
649     diff == 0
650 }
```

same file

```
847     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
949     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
1218    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

CVF-14: open Category: **behavior**

The Rust port realises both competing message-space encodings itself, not just the codepoint one the paper credits it with: `encrypt_raw`/`decrypt_raw` implement the C port's byte-to-codepoint convention under the same key and the same tag construction, and `decrypt_raw` additionally masks each recovered codepoint to its low byte, so the cross-port plaintext confusion of CVF-3 is reproducible inside a single Rust process.

This amplifies the existing round-4 finding CVF-3 rather than duplicating it. The root cause is the one CVF-3 already identifies — the encoding from byte strings to codepoint sequences is not normative and no domain separator distinguishes the conventions — and the recommendation CVF-3 already makes would close both. What is new is that the paper's evidence for the Rust port being on the safe side does not hold.

Section 3.10 states: "The Python and Rust reference implementations realise $\mathcal{M}$ as specified: Rust's `&str` excludes surrogates by construction". Read against the source, that claim is correct as far as it goes. `encrypt_bytes`, `encrypt_str`, `encrypt_bytes_with_nonce` and `encrypt_bytes_v8_with_profile` all take `&str` and go through `message.chars().map(|c| c as u32)`, which yields Unicode scalar values and cannot yield a surrogate, so those four entry points do realise `Definition 3.10`'s message space exactly. On the decrypt side the v8 path is careful in the matching way: `decrypt_core_v8` rejects any recovered value above `0x10FFFF` or failing `char::from_u32`, and `decrypt_bytes_v8` then maps rather than filters, returning an error on a non-scalar value instead of dropping it.

But `&str` is not the crate's only interface to the token layer. Two further public functions take and return raw bytes:

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
  Result<Vec<u8>, String> {
    ...
    let codepoints: Vec<u32> = data.iter().map(|&b| b as
  u32).collect();
```

That is byte-to-codepoint — the exact convention CVF-3 identifies as the C port's, and which the paper describes as realising $\mathcal{M}$ "faithfully only for ASCII input". Its doc comment states the convention plainly: "Each byte is treated as a codepoint in [0, 255]."

So the two encodings the paper places in different implementations are both present in this one, keyed identically, sharing `compute_auth_tag` with the same `AAD_LEN_WIDTH_`

`V7` prefix and the same domain byte `0x03`. Nothing in the wire format distinguishes them, exactly as CVF-3 observes for the cross-port case. The consequence is that CVF-3's worked example no longer needs two implementations. Encrypt the string `é` with `encrypt_bytes` and one codepoint `U+00E9` enters the token layer; hand the resulting ciphertext to `decrypt_raw` and it authenticates, because the tag is computed over the masked blob and the AAD and knows nothing about how the caller obtained its codepoints, and the caller receives the single byte `0xE9`. Encrypt the two bytes `0xC3 0xA9` with `encrypt_raw` and hand the ciphertext to `decrypt_bytes`, and the caller receives the two-character string `Ã©`. Both directions succeed, silently, with no error and no indication that the plaintext is not the one that was encrypted.

`decrypt_raw` makes this worse than a symmetric encoding disagreement, because its inverse is lossy:

```
Ok(codepoints.into_iter().map(|c| (c & 0xFF) as u8).collect())
```

Any recovered codepoint above 255 is silently truncated to its low byte rather than rejected. So `decrypt_raw` is not merely a different-but-self-consistent reading of the token stream; it discards information without signalling, and it will return a plausible byte string for a ciphertext that encodes codepoints it cannot represent. Compare `decrypt_core_v8`, which rejects out-of-range values explicitly, and note that the v7 `decrypt_core` performs no range check at all.

The v7 `&str` decrypt has a third variant of the same defect. `decrypt_bytes` finishes with

```
let s: String =
  codepoints.into_iter().filter_map(char::from_u32).collect();
```

`filter_map` drops every recovered value that is not a Unicode scalar value, so a v7 ciphertext whose token stream decodes to surrogates or out-of-range values yields a *shorter* string than was encrypted, successfully and with no error. The v8 path's comment explicitly contrasts itself with this: "so this maps every recovered codepoint rather than silently dropping some."

The practical reach of this is not hypothetical within the delivered snapshot: the out-of-scope `ot_frame.rs` PDU framing layer, which is the module that would carry industrial traffic, consumes `encrypt_raw` and `decrypt_raw` rather than the v8 entry points.

Recommendation

Adopt CVF-3's primary recommendation — make the encoding normative and add a domain-separated encoding tag to the tag input so that a ciphertext produced under one convention is rejected rather than silently mis-decrypted under the other — and note that doing so now has to cover an intra-crate case, not only a cross-port one. CVF-3 argues that defining the message space as byte strings is the cheaper route; if that is taken, `encrypt_raw` becomes the specification and the `&str` entry points become thin wrappers that pass their UTF-8 bytes through, which also removes the surrogate question and leaves one encoding in the crate instead of two.

Whichever encoding becomes normative, three source-level changes are needed regardless:

- Remove the `& 0xFF` mask in `decrypt_raw` and return an error for any recovered codepoint above 255, in the style `decrypt_core_v8` already uses. Silently truncating a value that cannot be represented is worse than refusing it, because the caller cannot tell the two apart.
- Replace `filter_map(char::from_u32)` in `decrypt_bytes` with the fallible map that `decrypt_bytes_v8` already uses, so a non-scalar value is an error rather than a silent deletion.
- Add a range check to the v7 `decrypt_core` mirroring the one in `decrypt_core_v8`, or route both through a shared checked inverse. As it stands `decrypt_core` pushes `((token - addend) / k) as u32` with no validation, and the `as u32` truncation there is a fourth instance of the same silent-narrowing pattern.

  If the byte-oriented and codepoint-oriented interfaces are both to be retained deliberately, then bind the choice into the tag: give each a distinct `format_id` in the domain-`0x0B` subkey space that CVF-11 recommends specifying, so the two are cryptographically separated and a ciphertext from one cannot verify under the other. That is the mechanism the code already has for exactly this purpose, currently used only to separate block mode from streaming mode.

  Finally, Section 3.10's sentence should be narrowed to what is true. Rust's `&str` does exclude surrogates by construction, but that is a property of four of the crate's entry points, not of the crate; the byte-oriented pair realises the same convention the paper attributes to the C port alone. As written, the sentence tells a reader that choosing Rust avoids the hazard, and it does not.

Client Comment

/src/lib.rs

```
513             padded.push(((token - addend) / k) as u32);
```

same file

```
606         if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
607             return Err(format!(
608                 "Malformed v8 ciphertext: recovered value {} at
position {} is not a \
609                  Unicode scalar value.",
610                 cp, ct_pos
611             ));
612         }
```

same file

```
857     let s: String =
codepoints.into_iter().filter_map(char::from_u32).collect();
```

same file

```
886 pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
887     if data.is_empty() {
888         return Ok(Vec::new());
889     }
890     let nonce = generate_nonce_with_crng_check()?;
891     let codepoints: Vec<u32> = data.iter().map(|&b| b as
u32).collect();
```

same file

```
959     Ok(codepoints.into_iter().map(|c| (c & 0xFF) as u8).collect())
```

same file

```
1145     let codepoints: Vec<u32> = message.chars().map(|c| c as
u32).collect();
```

same file

```
1229     let s: String = codepoints
1230         .into_iter()
1231         .map(|c| {
1232             char::from_u32(c)
1233                 .ok_or_else(|| format!("Malformed v8 plaintext: {} is
not a Unicode scalar value.", c))
1234         })
1235         .collect::<Result<String, String>>()?;
```

CVF-15: fixed Category: **flaw**

The paper's no-panic attestation covers one entry point, and five other public decryptors panic: the legacy decrypt core aborts on a malformed padded buffer by design, subtracts without checking, and no legacy entry point validates its key, so an empty or degenerate key divides by zero.

Section 11 reports that the Rust fuzz harness "targets the Rust `decrypt_bytes_v8` entry point with multiple key sizes and AAD values" and that "no panics were observed across 10^6 corpus entries". The harness has since been delivered and it does not do that: it targets the legacy `decrypt_bytes`, and by construction reaches nothing past the tag check. That is CVF-22, and it inverts the scoping this finding originally assumed. What survives is the substantive half of the reading, which is about the source rather than about the harness: `decrypt_bytes_v8` genuinely is the careful path. It is written throughout in the fallible style, `decrypt_core_v8` returns `Result` at every branch, `fixed_decode_tokens` checks its length precondition, and `unpad_message` validates the length prefix

instead of slicing blindly. What is not true is that the fuzz result attests to any of that, since it never enters the v8 path at all.

The crate exports five other decryptors, and they are not written that way.

The shared legacy core ends with an explicit abort, and its comment states that this is deliberate:

```
    // v7 low-level API: signature is `-> Vec<u32>`, so a malformed
  padded
    // buffer panics here exactly as it did before V3-CVF8. The v8
  path
    // (`decrypt_core_v8`) returns the error instead.
    unpad_message(&padded).expect("v7 decrypt: malformed padded
  buffer")
```

`decrypt_core` is reached by `decrypt`, `decrypt_bytes`, `decrypt_str` and `decrypt_raw`. For the three that verify a tag first, the panic is reachable only by a party holding the key, so it is a robustness defect rather than an attack surface. `decrypt` itself is `pub` and takes a raw token slice with no tag verification at all, so for that one entry point an arbitrary caller-supplied token vector reaches the panic directly.

The same function subtracts without checking:

```
padded.push(((token - addend) / k) as u32);
```

For a token stream that is not of the form $ck + a$, `token - addend` underflows — a panic in debug builds and a wrap in release. `decrypt_core_v8` guards precisely this case, and its comment explains why: "the bare subtraction panics here in debug builds and wraps in release builds and in C." The guard was added to the v8 core and not to the v7 core that four public functions still use. The delivered manifest confirms which branch a shipped build takes: with no `[profile]` section the release default is `overflow-checks = false`, so the wrap is the deployed behaviour and the panic only the debug one (CVF-16, CVF-23).

Separately, `validate_key` is called from exactly two places — `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8` — so no legacy entry point validates its key material at all. Two degenerate keys panic:

- An empty key. `let kk = key.len() as u64` is zero and the next line indexes `key[(real_idx % kk) as usize]`, so the remainder operation divides by zero. This reaches `encrypt`, `encrypt_bytes`, `encrypt_raw`, `encrypt_bytes_with_nonce` and, through `decrypt_core`, all four legacy decryptors. `encrypt_bytes("x", &[], b"")` panics.
- A key element equal to 1, or 0. `derive_addend` and `derive_noise_token_addend` both compute `% (key_element - 1)`, which is a division by zero when the element is 1, and an underflow of the subtraction itself when the element is 0.

None of these is attacker-reachable, since the key is the caller's own; the defect is that a public cryptographic API accepts obviously invalid key material and aborts the process rather than returning the `Result` its signature already provides. The v8 entry points return a descriptive error for all three cases, so the fix is a call, not new logic.

The KAT harness now supplies an illustration of how easily the empty-key case is passed. `kat_cross_check.rs` calls `encrypt_bytes_with_nonce(message, &[], [0u8; NONCE_SIZE], b"")` with a literal empty key; that call does not panic only because the empty-message short-circuit returns before the key is read, and it would divide by zero for any non-empty message. See CVF-25, where the same line is a test defect for a different reason.

One incidental observation while reading `validate_key`. Its doc comment justifies the primality requirement on correctness grounds: "The correctness argument recovers `c` from `c * k + a` by exact division, which needs `gcd(a, k) = 1` for every addend `a` in `[1, k - 1]` – true only when `k` is prime." That is the same claim the existing round-4 finding CVF-8 identifies as incorrect in the paper, now carried into the code. It is not correct here either: for $t = ck + a$ with $0 < a < k$, the recomputed addend is identical on both sides and $t - a = ck$ exactly, so the division is exact for composite k as well. What invertibility needs is $a < k$, not coprimality. Primality remains a specification requirement because $\mathcal{P}$ is a set of primes, so the check itself should stay — only its stated reason is wrong, and correcting it here would keep the code from being cited as independent support for the paper's version of the claim.

Recommendation

Bring the legacy path up to the v8 path's error discipline rather than leaving two standards in one crate.

For the malformed-buffer abort, change `decrypt_core` to return `Result<Vec<u32>, String>` and propagate. The four callers already return `Result`, so each needs only a `?`; `decrypt`, whose signature is `-> Vec<u32>`, is the one that needs a real decision. Since it is a `pub` low-level entry point reachable with an arbitrary token slice and no tag check, prefer changing its signature to `Result` over keeping the panic — and if that is too disruptive, deprecate it.

For the unchecked subtraction, lift the guard that `decrypt_core_v8` already has into the shared inverse:

```
if token < addend || (token - addend) % k != 0 {
    return Err(format!(
        "Malformed ciphertext: token at position {} is not of the
  form \
         codepoint * k + addend for key element {}.", ct_pos, k
    ));
}
```

Doing it once in a shared helper used by both cores is preferable to a second copy, since the divergence between the two cores is how this arose.

For key validation, call `validate_key(key)?` at the head of every legacy entry point that takes a key: `encrypt_bytes`, `encrypt_str`, `encrypt_raw`, `decrypt_bytes`, `decrypt_str`, `decrypt_raw`, and `encrypt_bytes_with_nonce`. `validate_key` already rejects the empty key and any element below `MIN_KEY_PRIME`, which covers 0 and 1, so no new checks are required — and see CVF-16 for the upper bound it is still missing. `encrypt` and `decrypt` take `&[u64]` with `Vec<u32>` returns and cannot report an error; if they are kept, document that the caller must have validated the key, and prefer deprecating them. CVF-24 recommends the same deprecation for a

different reason, and one `#[deprecated]` attribute per function serves both.

On the attestation itself: the harness is now delivered and targets the legacy `decrypt_bytes`, not `decrypt_bytes_v8`, so Section 11 needs correcting on that point — and correcting it does not rescue the no-panic result for the legacy surface, because every panic listed above sits either past the tag check or behind key material the harness's fixtures never supply. CVF-22 sets out what the harness would have to do to reach them: derive the plaintext side from the fuzzer input, construct a valid ciphertext, mutate it under a recomputed tag, add a target for the unauthenticated `decrypt`, and put degenerate keys in the fixture set. Until that is done, Section 11 should say that the no-panic result covers the parse floor and the tag comparison, because that is what it covers.

Finally, correct the `validate_key` doc comment in step with the CVF-8 fix, so that the primality check is justified by $\mathcal{P}$'s definition and by the $a < k$ requirement rather than by a coprimality argument that does not hold.

Client Comment

/fuzz/fuzz_targets/decode_bytes.rs

```
use napqes::decrypt_bytes;
```

same file

```
/// Key fixtures — three representative sizes selected by fuzzer byte[0].
fn key_for(selector: u8) -> Vec<u64> {
    match selector % 3 {
        0 => vec![1_000_003, 1_000_033],
        1 => vec![7_999_993],
        _ => vec![
            1_000_003, 1_000_033, 1_000_037, 1_000_039,
            1_000_081, 1_000_099, 1_000_117, 1_000_121,
            1_000_133, 1_000_151,
        ],
    }
}
```

/src/kat_cross_check.rs

```
            let ct = encrypt_bytes_with_nonce(message, &[], [0u8; NONCE_SIZE], b"");
```

/src/lib.rs

```
    (u32_from_be4(&d[..4]) as u64 % (key_element - 1)) + 1
```

same file

```
205     (u32_from_be4(&d[..4]) as u64 % (key_element - 1)) + 1
```

same file

```
222 /// Reject a prime tuple that is empty, composite, undersized or
    repeating.
223 ///
224 /// The correctness argument recovers `c` from `c * k + a` by exact
    division,
225 /// which needs `gcd(a, k) = 1` for every addend `a` in `[1, k - 1]`
    -- true
226 /// only when `k` is prime. Called from both v8 entry points so that
    a caller
227 /// supplying a malformed key gets an error here rather than a
    silently
228 /// undecryptable ciphertext, matching `_validate_key` in the Python
    port.
```

same file

```
229 fn validate_key(key: &[u64]) -> Result<(), String> {
```

same file

```
513             padded.push(((token - addend) / k) as u32);
```

same file

```
517     // v7 low-level API: signature is `-> Vec<u32>`, so a malformed
    padded
518     // buffer panics here exactly as it did before V3-CVF8. The v8
    path
519     // (`decrypt_core_v8`) returns the error instead.
520     unpad_message(&padded).expect("v7 decrypt: malformed padded
    buffer")
```

same file

```
739     let kk = key.len() as u64;
```

same file

```
746                 let k = key[(real_idx % kk) as usize];
```

same file

```
1149    validate_key(primes)?;
```

same file

```
1212    validate_key(primes)?;
```

CVF-16: fixed Category: **over-underflow**

Key validation deliberately omits the normative upper bound on prime key elements, so the token-width bound that `Remark 3.11` calls unconditional is not enforced anywhere, and the token multiplication overflows `u64` for a key element the validator accepts.

Section 3 makes the prime interval normative — $\mathcal{P}$ is the primes in $[10^6,\ 9.9 \times 10^6]$, and v4 added the requirement that implementations must use this interval. `Remark 3.11` then discharges the notation section's outstanding obligation that tokens fit their field, by bounding the extreme token at $c =$ `0x10FFFF` and $k = 9{,}899{,}999$:

$$1{,}114{,}112 \times 9{,}899{,}999 - 1 = 11{,}029{,}707{,}685{,}887 \approx 2^{43.33}$$

leaving a margin of over 1.6×10^6 against the 8-byte serialised field. The paper describes this bound as "unconditional: it holds for every" admissible input. It is unconditional in c, but it is entirely conditional on $k \le 9{,}899{,}999$.

The implementation defines the bound and then declines to check it. `MAX_KEY_PRIME` is `9_900_000`, and its doc comment states the omission as a decision:

> This bound constrains key *generation* only. Validation and decryption accept any prime `>= MIN_KEY_PRIME`, so keys generated before this bound was tightened remain usable.

`validate_key` accordingly tests four things — non-empty, prime, at least `MIN_KEY_PRIME`, and pairwise distinct — and no upper bound. Token construction is then

```
cypher.push(c as u64 * k + addend);
```

with `c` at most `0x10FFFF`, so the product overflows `u64` once k exceeds roughly $2^{64}/2^{20.09} \approx 1.66 \times 10^{13}$. Every key element in $[1.66 \times 10^{13},\ 2^{64})$ that happens to be prime passes `validate_key` and overflows this multiplication. The same expression appears at all five token-emission sites, on both the v7 and v8 paths.

`Cargo.toml` has now been delivered and settles which way the overflow goes in a deployed build. The manifest declares no `[profile]` section, so the release profile takes Cargo's defaults, `overflow-checks = false` and `debug-assertions = false`. The product therefore wraps silently in release and panics only in debug: the safe failure mode is confined to the configuration that is not shipped, and the corrupting one to the configuration that is. See CVF-23, which records the absent profile section and its second consequence for the tag comparator.

The downstream behaviour differs by path but is bad in both. On v8, `decrypt_core_v8` checks that each token is genuinely of the form $ck + a$, so a wrapped token fails `(token - addend) % k != 0` and the ciphertext is rejected — the sender therefore produces authenticated ciphertexts that the receiver cannot decrypt, with an error message that points at ciphertext malformation rather than at the key. On v7, `decrypt_core` performs no such check and computes `((token - addend) / k) as u32` on the wrapped value, yielding an arbitrary codepoint and a silently corrupt plaintext.

Nothing in the API steers a caller toward the normative interval either. `generate_prime_numbers` and `generate_v8_key` both take `min_val` and `max_val` as parameters, so the

bounds are the caller's choice at generation time as well as unchecked at validation time; the constants exist but are only conventions.

A second consequence of the missing bound is a performance cliff on the decrypt path. `is_prime` is trial division:

```
let mut i: u64 = 3;
while i.saturating_mul(i) <= n {
    if n % i == 0 { return false; }
    i += 2;
}
```

which is correct — `saturating_mul` avoids the overflow that the naive form would have — but costs $\Theta(\sqrt{n})$. Inside the normative interval that is at most about 1,600 iterations per element and negligible. For a key element near 2^{63} it is on the order of 2^{31} iterations per element, and `validate_key` runs on *every* `encrypt_bytes_v8` and `decrypt_bytes_v8` call rather than once at key load, so a key the validator accepts can make every decryption take minutes. Even within the intended interval, re-testing primality of all K elements on every message is avoidable work on the hot path.

Recommendation

Add the upper bound to `validate_key`, alongside the existing lower-bound check:

```
if k > MAX_KEY_PRIME {
    return Err(format!(
        "Key element at index {} ({}) exceeds the maximum of {}.",
        i, k, MAX_KEY_PRIME
    ));
}
```

This is what makes `Remark 3.11`'s bound hold in the implementation, and it is the check that turns the remark's "unconditional" into a fact about the code rather than about the intended input.

The doc comment's stated reason for omitting it — that keys generated before the bound was tightened must remain usable — should be resolved explicitly rather than by leaving validation open. If such keys exist, they lie in $[9{,}900{,}000,\ 10^7)$ or thereabouts, nowhere near the overflow threshold; so accept a documented legacy ceiling that still preserves the token-width bound (any $k < 2^{43}$ is safe for the full codepoint range) instead of accepting every `u64` prime. Write the safe ceiling in terms of the bound it protects:

```
// c <= 0x10FFFF, so c * k + a < 2^64 requires k < 2^64 / (0x10FFFF
  + 1).
const MAX_SAFE_KEY_PRIME: u64 = u64::MAX / (0x10FFFF + 1);
```

and reject above it unconditionally, even if a wider legacy window is allowed below it.

Independently, make the arithmetic state its own precondition. Use `checked_mul`/`checked_add` at the five token-emission sites and return an error on overflow rather than relying on the validator having been called, since two of those sites are on legacy paths that never call it (CVF-15). A crate that computes on secret key material should not depend on the release profile's overflow setting for correctness

— and with the manifest now in hand, that setting is confirmed to be off. Adding `[profile.release] overflow-checks = true` per CVF-23 is worth doing as a backstop, but it is not a substitute: a profile setting is not a precondition, and a downstream crate can override it.

Move key validation off the per-message path. Validating K primes by trial division on every encrypt and every decrypt is wasted work at best and a denial-of-service vector at worst. Prefer a validated key type constructed once — a `Key` newtype whose constructor runs `validate_key` and whose existence is the proof — and have `encrypt_bytes_v8`/`decrypt_bytes_v8` take that instead of `&[u64]`. If the current signatures must be kept, at minimum bound the trial division by `MAX_KEY_PRIME` so its cost cannot exceed the interval's.

Finally, since $\mathcal{P}$ is now normative, consider having `generate_prime_numbers` and `generate_v8_key` default to the interval rather than taking it as a parameter, or reject bounds outside it. As written, the normative interval is enforced by neither generation nor validation.

Client Comment

/Cargo.toml

```
40 [lib]
41 name = "napqes"
42 path = "src/lib.rs"
```

/src/lib.rs

```
70 /// Upper bound of the normative prime interval, per
71 /// `docs/napseq-eprint-v3.tex` §Notation. `P` contains exactly
   579_947
72 /// primes (verified by sieve), giving `P(579_947, 10) = 2^191.46`
   ordered
73 /// 10-tuples (`2^95.73` post-Grover).
74 ///
75 /// This bound constrains key *generation* only. Validation and
   decryption
76 /// accept any prime `>= MIN_KEY_PRIME`, so keys generated before
   this bound
77 /// was tightened remain usable. Matches `napqes.py::MAX_KEY_PRIME`
   and
78 /// `C/napqes.h::NAPQES_MAX_KEY_PRIME`.
79 pub const MAX_KEY_PRIME: u64 = 9_900_000;
```

same file

```
81 pub fn is_prime(n: u64) -> bool {
```

```
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not
prime.", i, k));
        }
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of
{}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        if key[..i].contains(&k) {
            return Err(format!(
                "Key element {} at index {} is a duplicate; all
elements must be distinct.",
                k, i
            ));
        }
    }
    Ok(())
}
```

same file

```
490                 cypher.push(c as u64 * k + addend);
```

same file

```
754                 cypher.push(c as u64 * k + addend);
```

same file

```
813                 cypher.push(c as u64 * k + addend);
```

same file

```
910                 cypher.push(c as u64 * k + addend);
```

same file

```
1170                cypher.push(c as u64 * k + addend);
```

CVF-17: open Category: **flaw**

The legacy v7 token-emission loop applies neither the noise-run cap nor the token ceiling, so its ciphertext length is a random function of the nonce rather than of the padding bucket — this is a v7-only defect, the v8 loop having since been verified statement-for-statement against Listing 1, but the length leak the client documented and fixed for v8 remains open on the four public v7 encryptors, and the delivered self-test vector demonstrates it.

Scope of this finding. It concerns the four legacy v7 encryptors only. The v8 emission loop implements both bounds correctly and is *not* a defect site; the v8 line ranges cited below are given as the conforming reference against which the v7 loops are compared, not as instances of the problem.

The specified emission loop is bounded twice. Per real codepoint it emits noise while $noise_run < 19$ and the position is a noise position, and afterwards it appends filler tokens until exactly

$$N_{\text{tok}} = R \cdot (\text{MAX_NOISE_RUN} + 1) = 20R$$

tokens have been emitted. `Lemma 3.9` depends on both: $|C| = 48 + 160(B(|M|) + 2)$ is a function of the padding bucket alone.

The v8 implementation applies both bounds correctly. `encrypt_bytes_v8_with_profile` carries a `noise_run` counter tested against `MAX_NOISE_RUN`, and then tops up to the ceiling:

```
let ceiling = (padded.len() as u64) * (MAX_NOISE_RUN + 1);
while (cypher.len() as u64) < ceiling { /* emit filler */ }
```

Since the inner loop emits at most 19 noise plus one real token per codepoint, the count can never overshoot, and the top-up makes it exact. We re-checked the whole of

`encrypt_bytes_v8_with_profile`'s emission region statement for statement against `Listing 1` and it conforms on every point: `noise_run` is reset once per padded codepoint; the guard is `noise_run < MAX_NOISE_RUN && is_noise_pos_v8(...)`, matching the listing's `while noise_run < MAX_NOISE_RUN and is_noise(ct_pos)` including the short-circuit that skips the noise oracle once the run reaches 19; `MAX_NOISE_RUN` is `19`; the key element is `key[real_idx % K]` for noise and real tokens alike, so noise tokens emitted inside a run use the key element of the real token they precede; `ct_pos` advances on every emission and `real_idx` only on a real one; the ceiling is `padded.len() * (MAX_NOISE_RUN + 1)` with `padded.len() == R`; and in the filler loop `real_idx` is **frozen** at R while `ct_pos` continues to advance, exactly as the listing does, so the filler tokens are structurally identical to genuine noise tokens. The v8 path therefore satisfies `Lemma 3.9` as written, and nothing in this finding asks for a change to it.

The four legacy encryptors — `encrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce` and `encrypt_raw` — carry neither bound. Each contains the same unguarded loop:

```
for &c in &padded {
    loop {
        if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
            /* emit noise, ct_pos += 1 */
        } else {
            /* emit real token, break */
        }
    }
}
```

with no `noise_run` counter, no cap in the condition, and no ceiling top-up afterwards. The omission is deliberate and documented: the `MAX_NOISE_RUN` doc comment states the cap is "never applied to v7, matching `napqes.py` / `C/napqes.c`", and the same comment sets out precisely what the missing ceiling costs, in the client's own words —

> Without this, the *natural* token count varies with the message-derived synthetic nonce even for a fixed padding bucket, letting an observer who collects several ciphertexts of one message under varying AAD average out the noise and reliably recover the padding bucket

That is the V2-CVF2 finding. It was fixed for v8 and left standing on v7, which is still the crate's most prominent public API and, in the delivered snapshot, the API the transport layers actually use.

On v7 the leak is worse than the comment describes, because the v7 nonce is a fresh CSPRNG draw rather than a synthetic IV. The realised noise pattern is a deterministic function of (kb, N), so with N random the token count is a fresh random variable on every call. An observer therefore does not need to collect ciphertexts of one message under varying AAD to average out the noise — the ciphertext length of a single message varies across encryptions of *that same message*, and the distribution's location is set by the padding bucket. Length determinism does not hold at all on this path, so `Lemma 3.9`, `Corollary 4.14`'s $\log_2 \beta$ leakage bound, and the whole length-hiding argument of Section 4.3 apply only to v8.

The magnitude of the spread is computable from the paper's own parameters. Without the cap, the expected number of tokens per padded codepoint is $1+\sum_{j\geq 1} p^j = 1/(1-p)$ with $p = \theta(N)/2^{64}$; under the cap it is $1 + \sum_{j=1}^{19} p^j$, which runs from 3.99 at $p = 0.75$ to 18.21 at $p = 0.99$. Since $\theta(N)$ ranges over the whole of $[\theta_{\text{min}}, \theta_{\text{max}}]$ as the nonce varies, the v7 ciphertext length for one fixed padding bucket spans a factor of roughly 4.6 even before the uncapped tail is counted.

This is demonstrable from the delivered artifacts without executing anything. `self_test.rs` embeds a v7 reference ciphertext of 4,784 bytes for the one-codepoint message `"A"`. That gives $N_{\text{tok}} = (4784 - 48)/8 = 592$ tokens for a padded block of $R = 18$ codepoints — 32.9 tokens per padded codepoint, where the cap permits at most 20. And 592 is not a multiple of 20, so no ceiling was applied. Under the specification and under v8 the same message yields exactly 360 tokens and 2,928 bytes. The vector is itself the evidence that the v7 path implements neither bound.

The absent cap also makes the loop unbounded in principle. $\theta(N)/2^{64} \in [0.75, 0.99]$, so a run has geometric length with mean up to 99, and there is no upper limit on any individual run. At the top of the range the expected expansion is roughly 800 bytes per plaintext codepoint against the specification's 160, a 65,535-codepoint message drives the token vector past six million entries, and individual runs in the thousands occur with non-negligible probability. The loop terminates with probability one but has no bound, which makes memory consumption on the encrypt path a function of a value the caller cannot inspect.

Finally, an impact note drawn from code **outside the reviewed surface**, offered as evidence of reach rather than as part of the finding's root cause, which is wholly inside `src/lib.rs`: the crate's own `src/ot_frame.rs` builds its protocol data unit with `encrypt_raw`, and `src/bin/sts.rs` — the generator for the Section 9 SP 800-22 evidence — calls `encrypt_bytes`. Both are v7 entry points with the unbounded loop, so the v7 path is not a dormant compatibility shim in the delivered tree.

Recommendation

Apply both bounds to the legacy loop, or withdraw the legacy encryptors. No change is required on the v8 path.

The mechanical fix is to lift the v8 loop into a shared helper and call it from all five encryptors, differing only in the derivation key and the AAD prefix width. That removes four copies of a loop that has now diverged from its v8 twin in two security-relevant ways, and it is the divergence rather than either bound that made this possible. The helper needs the `noise_run` counter in the loop condition and the ceiling top-up after it; nothing else about the v7 derivations has to change, so v7 ciphertexts stay byte-compatible in shape while becoming length-deterministic.

Note that this is a wire-format change for v7: existing v7 ciphertexts will not decrypt under a bounded decoder and vice versa, because `decrypt_core` classifies every position independently and consumes the whole token slice, whereas a ceiling-aware decoder must derive the real-token count from the length. Since v7 and v8 are already declared non-interoperable and format selection is already out of band, the cleaner option is the second one: deprecate the v7 encryptors rather than issue a third format. The paper specifies only v8, `Section 8.2` describes the Rust port purely in terms

of `generate_v8_key`, `encrypt_bytes_v8` and `decrypt_bytes_v8`, and nothing in the specification requires v7 to exist. Retaining `decrypt_bytes`/`decrypt_raw` for reading archived ciphertexts while removing the v7 *encryptors* from the public API would close this finding without a migration.

Whichever route is taken, two things should change in the documentation. The `MAX_NOISE_RUN` comment currently explains the V2-CVF2 attack and then notes in passing that the cap is "never applied to v7" without connecting the two; it should state plainly that the v7 format is exposed to the attack the following paragraph describes. And if any v7 surface remains public, Section 8.2 should say that the crate exports a second, unspecified wire format whose ciphertext length is not deterministic, so that a reader does not carry Section 4.3's leakage bound over to it.

Finally, if the v7 encryptors are kept, cap the run regardless of format compatibility. An unbounded loop whose iteration count is set by a keyed derivation and unobservable to the caller is worth bounding on robustness grounds alone, independently of the length leak.

Client Comment

/src/lib.rs

```
479     for &c in &padded {
480         loop {
481             if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
482                 let k = key[(real_idx % kk) as usize];
483                 let noise_c = derive_noise_char(&kb, &nonce, ct_pos);
484                 let noise_add = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
485                 cypher.push(noise_c * k + noise_add);
486                 ct_pos += 1;
487             } else {
488                 let k = key[(real_idx % kk) as usize];
489                 let addend = derive_addend(&kb, &nonce, real_idx, k);
490                 cypher.push(c as u64 * k + addend);
491                 ct_pos += 1;
492                 real_idx += 1;
493                 break;
494             }
495         }
496     }
```

same file

```
743     for &c in &padded {
```

```
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
```

```
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
/// Hard cap on consecutive noise tokens emitted before a real token
in the
/// v8 token-emission loop (never applied to v7, matching `napqes.py`
/
/// `C/napqes.c`). As of the V2-CVF2 fix, this cap also fixes the
*total*
/// per-real-token budget: every v8 ciphertext is padded with
additional
/// filler tokens up to a deterministic per-bucket ceiling of
/// `real_token_count * (MAX_NOISE_RUN + 1)` tokens (see
/// `encrypt_bytes_v8` / `decrypt_core_v8`). Without this, the
*natural*
/// token count varies with the message-derived synthetic nonce even
for a
/// fixed padding bucket, letting an observer who collects several
/// ciphertexts of one message under varying AAD average out the
noise and
/// reliably recover the padding bucket (`docs/CAVEATS.md`, V2-CVF2).
After
/// this fix, v8 ciphertext length is a deterministic function of the
/// padding bucket alone, at the cost of always paying the worst-case
~20x
/// expansion instead of the ~13.4x average case.
pub const MAX_NOISE_RUN: u64 = 19;
```

same file

```
    for &c in &padded {
        let mut noise_run: u64 = 0;
        loop {
```

```
1160            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt,
&nonce, ct_pos, noise_theta) {
```

same file

```
1177    // V2-CVF2 fix: pad up to the fixed, bucket-only ceiling so
ciphertext
1178    // length never depends on the message-derived nonce's noise
realisation.
1179    let ceiling = (padded.len() as u64) * (MAX_NOISE_RUN + 1);
1180    while (cypher.len() as u64) < ceiling {
1181        let k = primes[(real_idx % kk) as usize];
1182        let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
1183        let na = derive_noise_token_addend(&sk_fmt, &nonce, ct_pos,
k);
1184        cypher.push(nc * k + na);
1185        ct_pos += 1;
1186    }
```

/src/self_test.rs

```
40 // Reference ciphertext hex, v7 wire format (fixed-width tokens, CVF1
fix),
41 // regenerated under the unified domain-first HMAC derivation layout
42 // introduced by the CVF2 fix (see SPEC.md §3 and
docs/napseq-eprint-preprint.tex
43 // rem:domsep). Every derivation input changed shape (`d || N || ctx`
instead
44 // of the previous mixed nonce-first/domain-first layouts), so this
constant
45 // is not byte-compatible with any pre-CVF2 vector for the same
key/nonce/message.
46 // Decoded once at test time; storing as hex avoids large byte
literals.
47 const KAT_CIPHERTEXT_HEX: &str = concat!(
```

CVF-20: fixed Category: **algorithm**

Key generation ignores the concrete sampling algorithm Remark 3.2 specifies and reintroduces a modulo bias in the one derivation the paper singles out as bias-free, and it aborts the process rather than returning an error when sampling fails.

Remark 3.2 specifies the sampling procedure in unusual detail, and specifies it precisely to eliminate modulo bias. $\mathcal{P}$ is fixed as a public canonical ascending enumeration, and index sampling draws a uniform 32-bit x, sets $L = \lfloor 2^{32}/|\mathcal{P}| \rfloor \cdot |\mathcal{P}|$, redraws while $x \geq$

L, and returns $idx = x \bmod |\mathcal{P}|$, "which is then exactly uniform over $\{0, \dots, |\mathcal{P}| - 1\}$ with no modulo bias". Distinctness is obtained by redrawing collisions, and the remark closes by tying the procedure to the entropy claim: "This realises exactly the uniform distribution over ordered tuples of K distinct elements of $\mathcal{P}$ assumed by the $H_\infty(\mathbf{k})$ computation below."

The implementation does something else:

```
let span = max_val - min_val + 1;
let max_attempts = span.saturating_mul(4);
...
while primes.len() < count && attempts < max_attempts {
    let num = min_val + (rng.next_u64() % span);
    if is_prime(num) && !primes.contains(&num) {
        primes.push(num);
    }
    attempts += 1;
}
```

Two differences, of which only the second is a defect.

It samples a uniform integer from the interval and rejects composites, rather than sampling a uniform index into an enumeration of $\mathcal{P}$. Distributionally the two agree — rejection sampling over an interval, conditioned on landing on a prime, is uniform over the primes in that interval — so the entropy claim survives and this is not a security difference. It is worth noting only because it means the implementation never enumerates $\mathcal{P}$ and therefore never has occasion to check that $|\mathcal{P}| = 579{,}947$ or that the interval is the normative one; the canonical enumeration that `Remark 3.2` requires to be "identical across implementations" does not exist in this port, and cross-port agreement on $\mathcal{P}$ rests on the two interval constants alone.

The defect is `rng.next_u64() % span`. That is precisely the modulo reduction `Remark 3.2` exists to avoid, applied without the rejection step. With `span` $= 8{,}900{,}001$ for the normative interval, 2^{64} is not a multiple of `span`, so the low 2^{64} mod span residues each receive one extra preimage. The magnitude is negligible — about 2.07×10^{12} complete cycles fit in 2^{64}, giving a relative deviation near 5×10^{-13} and a total variation distance from uniform of roughly 2.4×10^{-13} — and no security claim is measurably affected. We file it because the paper elects to specify the exact sampler and to state that the result carries "no modulo bias", and the implementation then declines to implement it in the one place where the paper made bias-freeness a normative requirement. A reader comparing the two would reasonably expect the rejection loop to be present, and its absence is invisible without a line-by-line comparison.

Two lesser points in the same function. It aborts rather than reporting failure:

```
if primes.len() < count {
    panic!("Could not find {} distinct primes in [{}, {}] - widen the
  range.", ...);
}
```

together with `assert!(max_val > min_val)` at the head, so a public key-generation entry point has two panic paths where its callers — `generate_v8_key` and application

code — could handle an error. See CVF-15 for the same pattern across the legacy API. And the attempt cap is $4 \cdot$span, which for the normative interval is 35.6 million trial-division primality tests before the panic fires; the cap is never approached in normal use, since prime density in $[10^6, 9.9 \times 10^6]$ is about $1/15.4$ and ten primes take roughly 154 draws, but a caller who passes a narrow prime-free range pays the full cost before learning it failed.

Reconciliation note (round 4). The independent audit reported the modulo bias at the same line as a separate finding, id 1131, which is excluded as a duplicate of this one. Its recommendation — reach for `rand::distr::Uniform` rather than hand-rolling the rejection loop — is folded into the recommendation below. Its severity was medium against this finding's low; on the user's direction that a duplicate takes the independent auditor's value, this finding is raised to **medium**. Our own assessment is unchanged and is recorded here for context: the recomputed total variation distance of 2.4×10^{-13} supports low on impact, and the case for filing at all rests on spec conformance rather than on measurable bias.

Recommendation

Implement the sampler `Remark 3.2` specifies, or amend the remark to describe what the implementations do.

Implementing it is the better route, because the remark's canonical enumeration is what makes $\mathcal{P}$ identical across ports and that guarantee is currently absent. Precompute the sieve table once, then:

```
fn sample_index(rng: &mut impl RngCore, n: u32) -> u32 {
    let l = (u32::MAX / n) * n;          // floor(2^32 / n) * n
    loop {
        let x = rng.next_u32();
        if x < l { return x % n; }
    }
}
```

and draw key elements as `P[sample_index(&mut rng, P.len() as u32) as usize]`, redrawing on a collision with an already-chosen element. This is exactly uniform, removes the bias, removes the attempt cap and the panic that depends on it — the loop cannot fail, since every draw eventually lands below L and $|\mathcal{P}| \gg K$ — and gives the port a concrete $|\mathcal{P}|$ it can assert equals 579,947, which is a cheap check on the interval constants that nothing currently makes.

If instead the interval-rejection approach is kept, then at minimum remove the modulo bias. Prefer the library primitive over a hand-rolled loop: `rand::distr::Uniform::new_inclusive(min_val, max_val)` performs the rejection internally and is harder to get subtly wrong than an open-coded variant, and it removes the `span` arithmetic entirely.

```
use rand::distr::{Distribution, Uniform};
let dist = Uniform::new_inclusive(min_val, max_val)
    .map_err(|e| format!("invalid prime interval: {e}"))?;
let num = dist.sample(&mut rng);
```

If a dependency-free form is wanted, the equivalent open-coded version is:

```
let l = (u64::MAX / span) * span;
let num = min_val + loop {
    let x = rng.next_u64();
    if x < l { break x % span; }
};
```

In either case amend `Remark 3.2` to say that implementations may sample uniformly from the interval and reject composites, noting that this is distributionally equivalent, so that the paper and the code describe the same procedure. In that case also state that the canonical enumeration is not required by the interval-rejection variant, since a reader is currently told it is.

Separately, change the signature to `Result<Vec<u64>, String>` and replace both panics with errors, so that key generation matches the error discipline of the v8 entry points. `generate_v8_key` should propagate. And if the attempt cap survives, scale it to the expected number of draws — $O(K \log |\mathcal{P}|/\text{density})$ — rather than to $4 \cdot \text{span}$, so a prime-free range is diagnosed in milliseconds instead of after tens of millions of primality tests.

Note finally that the `!primes.contains(&num)` collision check on the same line is CVF-41's item 2, and that the sieve table this recommendation asks for overlaps CVF-41's item 1; if both are implemented, build the table once and use it for generation and validation alike.

Client Comment

/src/lib.rs

```
101 /// Generate `count` distinct primes drawn uniformly from `[min_val,
    max_val]`.
102 pub fn generate_prime_numbers(count: usize, min_val: u64, max_val:
    u64) -> Vec<u64> {
103     assert!(max_val > min_val);
104     let mut rng = rand::thread_rng();
105     let span = max_val - min_val + 1;
106     let max_attempts = span.saturating_mul(4);
107     let mut primes: Vec<u64> = Vec::with_capacity(count);
108     let mut attempts: u64 = 0;
109     while primes.len() < count && attempts < max_attempts {
```

same file

```
110         let num = min_val + (rng.next_u64() % span);
```

same file

```
111         if is_prime(num) && !primes.contains(&num) {
```

```
            primes.push(num);
        }
        attempts += 1;
    }
```

same file

```
    if primes.len() < count {
        panic!(
            "Could not find {} distinct primes in [{}, {}] — widen the
range.",
            count, min_val, max_val
        );
    }
    primes
}
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) ->
(Vec<u64>, [u8; SK_SIZE]) {
    let primes = generate_prime_numbers(count, min_val, max_val);
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    (primes, sk)
}
```

CVF-22: open Category: **procedural**

The delivered Rust fuzz harness targets the legacy v7 decryptor rather than the `decrypt_bytes_v8` that Section 11 names, and because it feeds raw bytes to a function that verifies the tag first, no input it can generate reaches any code past the tag comparison — so the "no panics across 10^6 corpus entries" result is a statement about the length floor and the HMAC, and about nothing else.

Section 11 states:

> The Rust harness (`rust/fuzz/fuzz_targets/decode_bytes.rs`, using `cargo-fuzz`) targets the Rust `decrypt_bytes_v8` entry point with multiple key sizes and AAD values. No panics were observed across 10^6 corpus entries in either harness.

The harness is now delivered and it does not target `decrypt_bytes_v8`. Its only import from the crate is `use napqes::decrypt_bytes;`, and the body makes three calls, all of them to `decrypt_bytes` — the v7 entry point. `decrypt_bytes_v8` appears nowhere in the file. The harness's own header describes it as a target "for NAPSEQ v6 decoder

paths" and reasons about "a valid NAPSEQ v6 ciphertext", so the file is internally consistent with itself and inconsistent with the paper. The scheme the paper specifies has no Rust fuzz coverage at all.

The more consequential problem is what the harness can reach, which is a structural property of its design rather than of its target. `decrypt_bytes` verifies the authentication tag before it touches the payload, so every input has exactly two available exits:

- the parse floor, `if ciphertext.len() < NONCE_SIZE + TAG_SIZE`, for inputs shorter than 48 bytes; and
- the authentication failure, `if !ct_eq_bytes(recv_tag, calc_tag.as_ref())`, for everything else.

Passing the second requires producing a payload whose HMAC-SHA256 tag under the fixture key matches — that is, forging a tag. Everything after that line is therefore unreachable: `varint_keystream`, `fixed_decode_tokens`, `decrypt`, `decrypt_core`, and `unpad_message` are never entered by any input libFuzzer can construct. The harness's header states this as a reassurance rather than as a limitation:

```
//! Security contract:
//!   `decrypt_bytes` MUST never panic.  Any malformed or adversarial
  input
//!   must produce `Err(...)`.  A successful `Ok(plaintext)` is only
  possible
//!   when the fuzzer discovers a valid NAPSEQ v6 ciphertext, which
  requires
//!   forging an HMAC-SHA256 tag — computationally infeasible.
```

The infeasibility that makes the ciphertext unforgeable is the same infeasibility that makes the decoder unfuzzable by this design. The three targets the header claims — "full decode pipeline: auth check → LEB128 varint decode → plaintext recovery" — comprise one stage that is exercised and two that cannot be, and the second of those is named for the LEB128 encoding that the CVF1 fix retired; the pipeline is fixed-width now, and `b128_decode_tokens` is dead code.

The three calls per input are near-duplicates. They vary only the key fixture and the AAD, and all three take the same two exits, so the effective input space explored is the parse floor and a tag mismatch, three times over. The guard `if data.len() < 2 { return; }` discards only inputs of length 0 and 1; at length 2 the payload `&data[2..]` is empty, so the empty-ciphertext short-circuit at the head of `decrypt_bytes` — the behaviour CVF-13 reports — is in fact reached, but because it returns `Ok(String::new())` rather than panicking it is invisible to this panic-only harness, and the 10^6-entries result says nothing about it.

The sentence's other half fares no better. Section 11 says the Python harness "targets `decrypt_bytes_v8` and the fixed-width token decoder, verifying that no exception other than `ValueError` escapes the public API", and closes with "no panics were observed across 10^6 corpus entries in either harness". Read as the oracle for what the Rust claim is being compared against — the Python suite is not itself an audit target here — `tests/fuzz_atheris.py` targets the v7 `decrypt_bytes` and the retired LEB128 parser `_b128_decode_tokens`, never `decrypt_bytes_v8` and never a fixed-width decoder;

and it does not enforce the stated exception contract, whitelisting `IndexError` on the varint target and `UnicodeDecodeError` and `OverflowError` alongside `ValueError` on the legacy-flag target, which is the one that goes through the public API. Its own default run is 10,000 iterations and the coverage-guided invocation its docstring recommends is 10^5; the figure 10^6 appears in neither harness, in no run log, and in no committed corpus. So the claim misidentifies the target of both harnesses, overstates the exception contract of one, and cites a corpus size that nothing in the delivery accounts for.

This bears directly on CVF-15 and corrects part of it. CVF-15 was written without this file and reasoned that the paper's scoping to `decrypt_bytes_v8` was exact, and that the legacy decryptors were consequently unfuzzed. The first half is wrong: the harness targets the legacy decryptor. The second half is right in substance for a different reason: the legacy decryptor is nominally fuzzed but its panicking code is not reachable. None of the five panics CVF-15 identifies could have been found by this harness:

- the `unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")` abort and the unchecked `token - addend` subtraction both sit inside `decrypt_core`, past the tag check;
- the empty-key division by zero at `key[(real_idx % kk) as usize]` requires a zero-length key, and `key_for` returns tuples of 2, 1 and 10 elements, never empty;
- the division by zero for a key element of 0 or 1 requires such an element, and every fixture element is a prime above 10^6; and
- the `pub fn decrypt`, which takes a raw token slice with no tag verification and is the one entry point where the abort is reachable by an arbitrary caller, is never called by the harness.

So the crate's five reachable panics are precisely the five the harness is constructed not to find, and the 10^6-entry result does not speak to any of them.

Finally, the run is not reproducible. `fuzz/Cargo.toml` says the corpus is "created on first run"; `decode_bytes.rs` says corpus seeds are "checked in so that subsequent runs benefit from prior coverage discoveries". These contradict each other, and neither is satisfied in the delivered tree: there is no `fuzz/corpus/` directory. `.gitignore` contains only `/target` and `Cargo.lock`, so the corpus is absent because it was never committed, not because it was excluded. No run log, coverage figure, commit hash or seed count accompanies the 10^6 claim, so a reader cannot tell which build produced it, over which corpus, or against which of the two wire formats. With `Cargo.lock` ignored as well (CVF-23), the claim is not attributable even to a fixed dependency set.

Recommendation

Make the paper and the harness agree, then make the harness able to reach the code the claim is about. These are separate jobs and the second is the one that matters.

For the first: either correct Section 11 to say that the Rust harness targets `decrypt_bytes` and the v7 wire format, or retarget the harness. Retargeting is preferable, and adding a second target rather than switching is better still, since the v7 surface is the one with the reachable panics and the v8 surface is the one the paper's claims are about. Two `[[bin]]` entries in `fuzz/Cargo.toml` cost nothing. Correct the Python half of the same sentence in step: name the entry point `fuzz_atheris.py` actually

drives, list the exception types it actually admits, and replace 10^6 with the figure the recorded run produced.

For the second: a raw-bytes target aimed at a function that authenticates first can only ever test the authenticator. To exercise a decoder, derive the *plaintext side* from the fuzzer input and construct a valid ciphertext, then mutate under a recomputed tag:

```
fuzz_target!(|data: &[u8]| {
    if data.len() < 4 { return; }
    let key = key_for(data[0]);
    let sk  = sk_for(data[1]);
    let aad = aad_for(data[2]);
    let msg = String::from_utf8_lossy(&data[3..]);

    // Round-trip: exercises the full encode and decode pipeline.
    if let Ok(ct) = encrypt_bytes_v8(&msg, &key, &sk, aad) {
        let _ = decrypt_bytes_v8(&ct, &key, &sk, aad);

        // Structure-aware mutation: flip inside the blob, then
  re-tag, so
        // the input authenticates and reaches the
  post-authentication checks.
        let mut bad = ct.clone();
        let i = NONCE_SIZE + (data[3] as usize % (bad.len() -
  NONCE_SIZE - TAG_SIZE));
        bad[i] ^= 1;
        retag_in_place(&mut bad, &sk, aad);   // test-only helper
        let _ = decrypt_bytes_v8(&bad, &key, &sk, aad);
    }
});
```

The re-tagging helper is the essential piece: without it the post-authentication structural checks added in v4 — the token-count multiple, the bucket check and the padded length prefix, which are `Dec` step (3) and the subject of `Remark 3.13` — have no fuzz coverage at all, and they are new code. The negative vectors `W-N06`–`W-N08` in the KAT corpus are exactly these cases and show that the client already knows how to construct them; the harness should generate them rather than relying on three fixed examples.

Add a separate target for the unauthenticated surface, where a raw-bytes target *is* effective: `pub fn decrypt` takes a token slice with no tag check, and `decrypt_raw`, `decrypt_str` and `b128_decode_tokens` are all reachable with attacker-shaped input in a way the current target's exits prevent. This is the target that would have found CVF-15's aborts.

Extend `key_for` to include the key material that panics rather than only well-formed keys. An empty tuple, `[0]`, `[1]`, a tuple with a repeated element, and an element above `MAX_KEY_PRIME` are each a panic or a silent overflow per CVF-15 and CVF-16, and none of the five is in the fixture set. A key-validation fuzz target over `validate_`

`key` would be cheap and would cover all of them.

Commit the corpus and record the run. Reconcile the two contradictory comments, check in a seed corpus that at minimum includes one valid ciphertext per KAT vector, and publish alongside the iteration figure the target name, the commit, the total executions, the corpus size and the reached-coverage percentage, so the claim is checkable. Add `fuzz/target/` and `fuzz/artifacts/` to `.gitignore`, and stop ignoring `Cargo.lock` — see CVF-23, since a fuzz result is attributable to a build only if the build is pinned.

Finally, correct the header's stale references in step with CVF-21: the file is not a v6 target, and the decoder it names is not LEB128.

Client Comment

/.gitignore

```
/target
Cargo.lock
```

/fuzz/Cargo.toml

```
# Cargo-fuzz requires nightly.  Build with:
#   cargo +nightly fuzz run decode_bytes -- -max_total_time=300
# Corpus is stored in fuzz/corpus/decode_bytes/ (created on first
run).
```

/fuzz/fuzz_targets/decode_bytes.rs

```
//! Cargo-fuzz target for NAPSEQ v6 decoder paths.
```

same file

```
//! Targets:
//!   * `decrypt_bytes` — full decode pipeline: auth check → LEB128
varint
//!     decode → plaintext recovery.  Must return `Ok(String)` or
`Err(String)`;
//!     must never panic regardless of input.
//!   * Additional AAD variants to exercise the AAD-binding path.
```

same file

```
//! Corpus seeds are stored in `fuzz/corpus/decode_bytes/` and
checked in
//! so that subsequent runs benefit from prior coverage discoveries.
```

same file

```
//! Security contract:
```

```
//!   `decrypt_bytes` MUST never panic.  Any malformed or adversarial input
//!   must produce `Err(...)`.  A successful `Ok(plaintext)` is only possible
//!   when the fuzzer discovers a valid NAPSEQ v6 ciphertext, which requires
//!   forging an HMAC-SHA256 tag — computationally infeasible.
```

same file

```
use napqes::decrypt_bytes;
```

same file

```
/// Key fixtures — three representative sizes selected by fuzzer byte[0].
fn key_for(selector: u8) -> Vec<u64> {
    match selector % 3 {
        0 => vec![1_000_003, 1_000_033],
        1 => vec![7_999_993],
        _ => vec![
            1_000_003, 1_000_033, 1_000_037, 1_000_039,
            1_000_081, 1_000_099, 1_000_117, 1_000_121,
            1_000_133, 1_000_151,
        ],
    }
}
```

same file

```
    if data.len() < 2 {
        return;
    }
```

same file

```
    let _ = decrypt_bytes(payload, &key, aad);
```

same file

```
    let _ = decrypt_bytes(payload, &key, b"");
```

same file

```
        let _ = decrypt_bytes(payload, &bad_key, aad);
```

/src/lib.rs

```
    if ciphertext.is_empty() {
        return Ok(String::new());
    }
```

same file

```
    if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
```

same file

```
847     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
848         return Err("Authentication failed: invalid HMAC tag.".into());
849     }
```

CVF-23: open Category: **architecture**

The manifest declares no feature gates and no build profile, so a consumer who wants only the AEAD links a full Tokio runtime, a CLI parser, a syslog client and a post-quantum KEM into what the crate calls a FIPS 140-3 cryptographic module; and because `Cargo.lock` is excluded from version control, the build that the self-test and KAT-parity results attest to cannot be identified.

`Cargo.toml` is now delivered. It is 42 lines and contains a `[package]`, a `[dependencies]` block of thirteen entries, one `[dev-dependencies]`, four `[[bin]]` targets and a `[lib]`. It contains no `[features]` section and no `[profile]` section, and `cfg(feature = ...)` appears nowhere in `src/`, `examples/` or `fuzz/`. There is therefore no conditional compilation anywhere in the crate.

The module boundary is the first consequence. `lib.rs` declares six public modules unconditionally:

```
pub mod self_test;
pub mod kem;
pub mod kem_exchange;
pub mod ot_frame;
pub mod protocols;
pub mod vale;
```

and each drags its dependencies in with it. `tokio` is declared with `features = ["full"]` and is used only by `kem_exchange` and the `ls_gateway` binary; `clap` and `syslog` only by `ls_gateway`; `hkdf`, `pqcrypto-frodo` and `pqcrypto-traits` only by `kem`. A downstream crate that wants nothing but `encrypt_bytes_v8` and `decrypt_bytes_v8` — the scheme this paper specifies — nevertheless compiles and links all of it, including Tokio's full feature set with its multi-threaded scheduler, process spawning, signal handling and filesystem APIs.

That matters here more than it would in an ordinary library, because `self_test.rs` opens by presenting the crate as a regulated artifact with a hard pre-operation contract:

> FIPS 140-3 power-on self-tests for the NAPQES Cryptographic Module.
>
> Call `run_power_on_self_tests()` before using the module in production.

A cryptographic module is defined by its boundary, and every approved algorithm inside that boundary has to be covered by the power-on tests. Neither condition is met.

Nothing in the manifest or the source distinguishes the algorithm from its surroundings, so the boundary is whatever the compiler happens to include: an AEAD, a FrodoKEM key exchange, an oblivious-transfer frame layer, two industrial protocol codecs in `protocols::dnp3` and `protocols::modbus`, and four binaries. Meanwhile `run_power_on_self_tests()` covers three known-answer tests over the legacy v7 AEAD and one version-string comparison, and nothing else — see CVF-18. The module as compiled contains a post-quantum KEM with no self-test at all.

The absent `[profile]` section is the second consequence, and it settles a question CVF-16 left open. Cargo's defaults for the release profile are `overflow-checks = false` and `debug-assertions = false`, and nothing here overrides them. So in the configuration that would actually be deployed:

- the token multiplication `c as u64 * k + addend` wraps silently rather than panicking when a key element large enough to overflow `u64` passes `validate_key`, which is the branch CVF-16 identifies as the corrupting one; and
- the `debug_assert_eq!(a.len(), b.len())` that is the only length precondition on `ct_eq_bytes` is compiled out, leaving a tag comparator that returns `true` for a matching prefix on operands of unequal length — see CVF-12.

Both of the crate's two "this is safe in debug builds" arguments therefore hold only in the configuration nobody ships. The `examples/dudect_harness.rs` harness makes the point concrete: its own instructions require `--release`, so the timing evidence for the tag comparator is gathered in exactly the build where the comparator's length guard is gone.

Two smaller items in the same file. `subtle = "2"` is declared and no call site in the crate invokes it; Cargo does not warn on an unused dependency, so nothing in a normal build surfaces this, and a reader checking the manifest against the paper's Section 8.4 claim finds the claim apparently confirmed — this is the subject of CVF-12. And no `rust-version` is declared, so there is no minimum supported toolchain, even though `ct_eq_bytes`'s correctness argument is an argument about what a particular LLVM will not do to a volatile loop, and `fuzz/Cargo.toml` separately requires nightly.

Finally, `.gitignore` is two lines:

```
/target
Cargo.lock
```

Excluding the lockfile means the thirteen caret-range requirements are re-resolved on every checkout, so no two builds are guaranteed to use the same dependency versions. Cargo's own convention is to commit the lockfile for any package that produces binaries, and this one produces four. The practical effect is that every empirical result the paper offers — the KAT byte-parity of Section 10, the fuzz run of Section 11, the statistical study of Section 9, the power-on self-test — is attributed to a build that cannot be reconstructed. It also means the version of `subtle` that Section 8.4's claim refers to is unrecorded, which would matter if that claim were ever made true.

Recommendation

Define the module boundary in the manifest, and make everything outside it optional.

The AEAD is the thing the paper specifies and the thing the self-test attests to; it should be the default, and it should be reachable without the rest. Gate each of the other five modules behind a feature and mark its dependencies optional:

```
[features]
default = []
kem      = ["hkdf", "pqcrypto-frodo", "pqcrypto-traits"]
net      = ["tokio", "serde", "serde_json"]
gateway  = ["net", "clap", "syslog"]

[dependencies]
hmac = "0.12"
sha2 = "0.10"
rand = "0.8"
base64 = "0.22"
subtle = "2"
tokio  = { version = "1", features = ["rt", "net", "io-util"],
  optional = true }
clap   = { version = "4", features = ["derive"], optional = true }
syslog = { version = "6", optional = true }
hkdf   = { version = "0.12", optional = true }
```

with `#[cfg(feature = "kem")] pub mod kem;` and so on in `lib.rs`, and `required-features` on the three binaries that need them. While doing this, narrow `tokio`'s feature list: `features = ["full"]` is a development convenience and enables far more than `kem_exchange` and `ls_gateway` use.

Then state the boundary in prose as well. If the FIPS framing is to be kept, the module needs a named boundary — which functions and which files are inside it — and the power-on suite needs to cover every approved algorithm inside it, which today it does not for the KEM. If the framing is aspirational, say so where an operator will read it, in `self_test.rs`'s header rather than only in the comment above INT-1.

Add an explicit release profile rather than relying on Cargo's defaults, and turn overflow checking on:

```
[profile.release]
overflow-checks = true
debug-assertions = false
```

`overflow-checks = true` costs little on this workload and converts CVF-16's silent wrap into an abort. It is not a substitute for the `checked_mul` that CVF-16 recommends — a profile setting is not a precondition, and a downstream crate can override it — but it removes the case where the arithmetic silently produces a wrong codepoint. Independently, replace the `debug_assert_eq!` in `ct_eq_bytes` with a real length check, since no profile setting should be load-bearing for a tag comparator (CVF-12).

Declare an MSRV with `rust-version`, and pin the toolchain with a `rust-toolchain.toml`, so that the constant-time argument and the fuzz results are attributable to a compiler.

Commit `Cargo.lock`. Remove it from `.gitignore` and add the entries that belong

there instead — `fuzz/target/` and `fuzz/artifacts/`, neither of which is covered by the root-anchored `/target`. Publish the lockfile's hash, or the commit, alongside each empirical result the paper reports, so that Sections 9 through 11 name a build.

Finally, either use `subtle` or remove it, per CVF-12; a declared dependency with no call site is worse than an absent one, because the manifest reads as corroboration.

Client Comment

/.gitignore

```
Cargo.lock
```

/Cargo.toml

```
[dependencies]
hmac = "0.12"
sha2 = "0.10"
base64 = "0.22"
rand = "0.8"
subtle = "2"
hkdf = "0.12"
pqcrypto-frodo = "0.4"
pqcrypto-traits = "0.3"
serde = { version = "1", features = ["derive"] }
serde_json = "1"
tokio = { version = "1", features = ["full"] }
clap = { version = "4", features = ["derive"] }
syslog = "6"
```

same file

```
subtle = "2"
```

same file

```
pqcrypto-frodo = "0.4"
pqcrypto-traits = "0.3"
```

same file

```
tokio = { version = "1", features = ["full"] }
```

same file

```
clap = { version = "4", features = ["derive"] }
```

same file

```
syslog = "6"
```

same file

```
[dev-dependencies]
dudect-bencher = "0.6"
```

same file

```
[lib]
name = "napqes"
path = "src/lib.rs"
```

/examples/dudect_harness.rs

```
//! Run (release mode required for meaningful timings):
//!   cargo run --example dudect_harness --release -- --continuous bench_tag_comparison
```

/src/lib.rs

```
pub mod self_test;
pub mod kem;
pub mod kem_exchange;
pub mod ot_frame;
pub mod protocols;
pub mod vale;
```

CVF-24: fixed Category: **documentation**

The README documents a five-item, v6-era API out of a public surface of about thirty-three, omits the entire v8 format that the paper specifies, states the retired varint wire format as current, and its one usage example does not compile — so the only entry-point guidance the crate offers names exactly the legacy functions that CVF-13, CVF-14 and CVF-15 find defective and none of the ones that fix them.

The scope file lists this file against the check "stated API surface vs. actual `pub fn` set". Carried out, the gap is large and it runs in one direction.

The README's opening line is `Rust port of the v6 authenticated EpiCypher in ../napqes.py`. Both the version and the product name are stale in the way CVF-21 records for `lib.rs`'s module header: the file implements v7 and v8, and "EpiCypher" appears nowhere in the paper, which names the scheme NAPQES throughout. The referenced `../napqes.py` has not been delivered in any round of this engagement.

The documented API is:

```
use napqes::{encrypt_str, decrypt_str, generate_prime_numbers};
```

plus `MIN_KEY_PRIME` and `MAX_KEY_PRIME` used in the example, and a "Scope" section that names `encrypt`/`decrypt`, `encrypt_bytes`/`decrypt_bytes`, `encrypt_`

`str`/`decrypt_str` and prime generation. Against that, `lib.rs` exports six public modules, ten public constants, one public enum with a public method, and sixteen public functions. Nothing in the README mentions:

- `generate_v8_key`, `encrypt_bytes_v8`, `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8` — the entire wire format the paper specifies;
- `PadProfile`, `PadProfile::block_size`, `PAD_MIN_EXP` and `PAD_MAX_EXP`, the padding-profile mechanism that Section 6 and the length-hiding results are built on;
- `SK_SIZE`, `MAX_NOISE_RUN`, `FORMAT_BLOCK_V8` and `FORMAT_STREAM_AE_V8`;
- `encrypt_raw` and `decrypt_raw`, the byte-oriented pair;
- `zeroize_sk` and `zeroize_key`, the only key-hygiene functions in the crate;
- `is_prime`, `NONCE_SIZE`, `TAG_SIZE`; or
- the six public modules `self_test`, `kem`, `kem_exchange`, `ot_frame`, `protocols` and `vale`, one of which is the FIPS power-on self-test that the crate's own contract says a caller MUST invoke before any cryptographic operation.

The stated wire format is the retired one:

```
nonce(16) || varint_blob || hmac_sha256_tag(32)
```

The blob has been fixed-width since the CVF1 fix; `lib.rs` says so itself, that the LEB128 encoder "is removed since nothing in this crate produces LEB128 ciphertexts anymore". The description is also v7-shaped in a way that cannot be made to fit v8, where the nonce is synthetic rather than a supplied field, so the accompanying claim that the format is "interoperable with Python … when the same key, nonce, and AAD are used" has no v8 reading at all. The "Scope" section's closing claim that the named functions are "all byte-compatible with the Python reference" is the claim CVF-25 examines; for the v7 functions the corpus that pins it was produced by the generator's own re-implementation rather than by the public Python API.

"Out of scope: streaming API" sits alongside a `pub const FORMAT_STREAM_AE_V8` and a KAT corpus carrying six `api: "stream_ae"` vectors, which is the dangling-format observation CVF-21 records.

The example does not compile. In four lines it has three errors: `MIN_KEY_PRIME` and `MAX_KEY_PRIME` are used but absent from the `use` declaration; `encrypt_str` returns `Result<String, String>` and its value is bound without `?` or `.unwrap()`; and `decrypt_str` is then handed `&ct`, a `&Result<String, String>`, where it expects `&str`. `lib.rs` does not pull the README in with `#![doc = include_str!("../README.md")]`, so nothing compiles this fence and no test would have caught it — the crate's only usage example has never been exercised.

The consequence is entry-point selection, and this is why the finding is not cosmetic. The functions the README names are, precisely and exhaustively, the ones the three preceding findings are about:

- CVF-13 reports that six public v7 entry points short-circuit the empty input to an empty output, diverging from the specified behaviour that an empty message produces a

full authenticated ciphertext. `encrypt_str`, `decrypt_str`, `encrypt_bytes` and `decrypt_bytes` are four of the six, and they are four of the seven functions the README names.

- CVF-14 reports that the crate realises both competing message-space encodings. The README documents the codepoint-oriented `str` pair and does not mention the byte-oriented `raw` pair at all, so a reader cannot discover that the choice exists, let alone which one the paper specifies.
- CVF-15 reports that `validate_key` is called from exactly two places, `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8`, so no legacy entry point validates its key and an empty key or an element of 0 or 1 aborts the process. Neither of the two validating functions appears in the README.

So every defect those findings identify lies inside the surface the README documents, and every entry point that avoids them lies in the surface it omits. A developer who integrates this crate from its README will pick the empty-input-diverging, unvalidated-key, legacy-format API in every case, and will have no indication that a misuse-resistant one exists.

Recommendation

Rewrite the README around the format the paper specifies, and mark the other one as legacy.

The v8 pair should be what a reader meets first, with the key-generation call that produces both components and the padding profile that governs length leakage:

```
use napqes::{generate_v8_key, encrypt_bytes_v8, decrypt_bytes_v8,
             zeroize_key, zeroize_sk, MIN_KEY_PRIME, MAX_KEY_PRIME};

let (mut primes, mut sk) = generate_v8_key(10, MIN_KEY_PRIME,
  MAX_KEY_PRIME);
let ct = encrypt_bytes_v8("hello", &primes, &sk, b"aad")?;
let pt = decrypt_bytes_v8(&ct, &primes, &sk, b"aad")?;
zeroize_sk(&mut sk);
zeroize_key(&mut primes);
```

Compile whatever example you settle on. Adding `#![doc = include_str!("../README.md")]` to `lib.rs` makes every fence in this file a doctest, so the three errors above become build failures and stay fixed; if that is unwanted, mark the fences `no_run` but still ensure they type-check. Either way the current example needs the two missing imports and correct `Result` handling.

Document the full public surface, or narrow the surface to what you want to document. Thirty-three public items with five described is a gap best closed from both ends: gate and privatise what is not meant for consumers — the feature work recommended in CVF-23 does most of this — and enumerate what remains, including the six public modules and, prominently, that `self_test::run_power_on_self_tests()` is a required pre-operation call.

Mark the v7 surface deprecated rather than merely undocumented. This is the change that closes the entry-point half of CVF-13, CVF-14 and CVF-15 at once, and it is cheaper than fixing each:

```rust
#[deprecated(since = "0.2.0", note = "legacy v7 format; use
  encrypt_bytes_v8")]
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
  Result<Vec<u8>, String> { ... }
```

applied to `encrypt`, `decrypt`, `encrypt_bytes`, `decrypt_bytes`, `encrypt_str`, `decrypt_str`, `encrypt_raw` and `decrypt_raw`. A compiler warning at every call site reaches integrators that a README section cannot. It does not substitute for the fixes those three findings recommend, since the functions remain callable, but it stops new callers selecting them by default.

Correct the stale text in step with CVF-21, so the two files do not have to be fixed twice: name NAPQES rather than EpiCypher, drop the v6 label, describe the blob as fixed-width, state the two wire formats separately and say which one the paper specifies, and qualify or remove the byte-compatibility sentence — it holds for v7 with a supplied nonce and has no v8 reading. Resolve "Out of scope: streaming API" against the public `FORMAT_STREAM_AE_V8` and the six `stream_ae` corpus vectors, and against the `CAV-001` caveat that CVF-21 raises.

Finally, replace the `../napqes.py` pointer with a reference to the paper by version and date, since the Python reference has not been delivered and a README citing an unavailable file gives a reader nothing to check.

Client Comment

/README.md

```
# napqes (Rust)

Rust port of the v6 authenticated EpiCypher in `../napqes.py`.
```

same file

```
## API

```rust
use napqes::{encrypt_str, decrypt_str, generate_prime_numbers};

let key = generate_prime_numbers(10, MIN_KEY_PRIME, MAX_KEY_PRIME);
let ct  = encrypt_str("hello", &key, b"");
let pt  = decrypt_str(&ct, &key, b"").unwrap();
```
```

same file

```
Wire format (interoperable with Python):

```

```
nonce(16) || varint_blob || hmac_sha256_tag(32)
```

String form is the base64 encoding of the above.

same file

Scope

Implements: prime generation, `encrypt` / `decrypt`, `encrypt_bytes` / `decrypt_bytes`, `encrypt_str` / `decrypt_str`, all HMAC derivations (positions, addends, noise chars, noise probability, auth tag) — all byte-compatible with the Python reference.

same file

Out of scope: streaming API, legacy v2/v3/v5 ciphertexts.

/src/lib.rs

```
pub mod self_test;
pub mod kem;
pub mod kem_exchange;
pub mod ot_frame;
pub mod protocols;
pub mod vale;
```

same file

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) ->
(Vec<u64>, [u8; SK_SIZE]) {
```

same file

```
pub fn encrypt_bytes_v8(
```

same file

```
pub fn encrypt_bytes_v8_with_profile(
```

same file

```
pub fn decrypt_bytes_v8(
```

same file

```
1240 pub fn zeroize_sk(sk: &mut [u8; SK_SIZE]) {
```

same file

```
1255 pub fn zeroize_key(key: &mut [u64]) {
```

CVF-25: fixed Category: **procedural**

With both corpora now delivered, the byte-parity claim of Section 10 holds for three of the six tests and is a statement about printable ASCII: every message in the v8 corpus is ASCII and only one character in either corpus exceeds U+00FF, so the encoding divergences of CVF-3 and CVF-14 are invisible to it; three tests consume the corpus the paper disclaims; and the one vector written to catch CVF-13's empty-input divergence is skipped by one test and asserted to its inverse by another.

Section 10 states that `rust/src/kat_cross_check.rs` consumes `tests/kat/v8_vectors.json`, that "conforming implementations must produce byte-identical ciphertext for every positive vector and reject every negative vector", and that the second corpus `tests/kat/v6_vectors.json` "is retained in the repository for the earlier v7 wire format, which ... is *not* the scheme analysed here; its vectors are outside the scope of this paper". The prior round could evaluate none of this because all six tests aborted at load with no corpus present. Both corpora are now delivered.

Start with what holds. The three v8 tests are real byte-parity tests. `v8_positive_encrypt_matches_python` calls the public `encrypt_bytes_v8` and compares the full result to the stored `ciphertext_hex`; `v8_positive_decrypt_roundtrip` decrypts the stored bytes back to the stored message; `v8_negative_returns_err` requires an `Err` for each tampered vector. All three end in `assert!(tested > 0, ...)`, so none can pass vacuously — the defect CVF-19 records is confined to the v7 tests. And the corpus is arithmetically consistent with the delivered encryptor, which is checkable without running anything: under `PadProfile::Bucket` the token ceiling is $(B + 2)(\text{MAX_NOISE_RUN} + 1)$ fields of `TOKEN_WIDTH = 8` bytes, plus 48 for nonce and tag, and every positive vector matches — 2,928 bytes for the messages of 0 to 15 codepoints where $B = 16$, 5,488 at 16 codepoints where $B = 32$, and 10,608 at 32 codepoints where $B = 64$. The v8 positives were also generated through the public `napqes.encrypt_bytes_v8`, so for v8 the parity is parity with the shipped Python API.

Four qualifications follow, and then the coverage limits.

The oracle is not in the audit. Parity is with `tests/gen_kats_v8.py`, which begins by putting the repository root on `sys.path` and importing a module `napqes`; that module has not been delivered in any round of this engagement. We can confirm that the Rust port reproduces the bytes stored in the corpus. We cannot confirm that those bytes are what the Python reference produces, because the Python reference is not here. The paper's accompanying sentence overstates what the generator does: "The generator `tests/gen_kats_v8.py` regenerates the file and verifies parity on any port." Its `--check` mode reruns the Python generator and string-compares the result to the checked-in JSON. There

is no cross-language code in the file — no subprocess invocation, no reference to the Rust or C ports. It verifies that the Python output is reproducible from the Python module, which is a useful property and not parity with any port.

For v7 the oracle is weaker still: a test-only re-implementation. `gen_kats.py` does not call the public `napqes.encrypt_bytes`. It rebuilds encryption out of private primitives in a local `_encrypt_with_nonce`, and that is the function whose output the stored v7 ciphertexts are. So the three v7 tests compare the Rust port against the generator's re-implementation of the Python reference rather than against the reference's own public entry point, and a divergence between those two would be invisible to both sides.

Three of the six tests consume the corpus the paper places outside its scope. `positive_decrypt_roundtrip`, `positive_encrypt_bytes_deterministic` and `negative_returns_err` all read `v6_vectors.json`, and the file's own opening line calls itself the "Rust KAT cross-check for NAPSEQ v7". So the majority of the harness tests a wire format Section 10 explicitly disclaims, and the paper's characterisation of what `kat_cross_check.rs` consumes covers three of its tests.

The v6 corpus's label contradicts its contents, and the generator says so. `spec_version` is `"v6"`, but every one of the 22 non-streaming positive vectors is a fixed-width v7 encoding: for each, the ciphertext length minus 48 is an exact multiple of 8, which the retired LEB128 encoding would not produce. `gen_kats.py`'s docstring concedes it — block-mode vectors "use the v7 fixed-width token wire format", and "The output file is still named `v6_vectors.json` for path stability ... even though block-mode vectors are now v7" — while the six `api: "stream_ae"` vectors in the same document are still LEB128. One file therefore carries two wire formats under a label matching neither. This also settles, in the negative, the staleness concern the scope file raises about the corpus: it was regenerated after the CVF1 fix. See CVF-18 for the same question applied to the self-test's embedded vector.

The empty-message vector is skipped by one test and inverted by the other. V001's description in the corpus is `Empty message produces authenticated ciphertext (not empty bytes)`, and it carries a 16-byte `nonce_hex` and a 1,776-byte `ciphertext_hex`. The deterministic-encrypt test asserts the opposite:

```
// Empty-message vector: nonce_hex is "" and ciphertext_hex is ""
if message.is_empty() {
    let ct = encrypt_bytes_with_nonce(message, &[], [0u8;
  NONCE_SIZE], b"");
    assert!(ct.is_empty(), "[{}] empty message should produce empty
  ciphertext", id);
    eprintln!("[PASS] {} — empty message", id);
    tested += 1;
    continue;
}
```

The comment's premise is false for the delivered corpus. The assertion is made with an empty key and an all-zero nonce, discarding the vector's own key and nonce, so it says nothing about V001; and because `encrypt_bytes_with_nonce` returns `Vec::new()` at its own empty-message short-circuit before it reads either argument, the assertion holds for every possible argument — including the empty key that CVF-15 shows would

otherwise divide by zero. It then reports `[PASS]` and increments `tested`. Its sibling `positive_decrypt_roundtrip` skips the same vector with `[SKIP] … empty message, no ciphertext`. So the one vector the corpus contains specifically to pin the behaviour CVF-13 reports as divergent is neutralised in one test and certified backwards in the other. The Python suite pins the correct behaviour for the v8 format — `test_v8_empty_message_is_not_empty_ciphertext` asserts a 2,928-byte result — and the Rust `encrypt_bytes_v8` has no such short-circuit, so the divergence is v7-only, which is consistent with CVF-13's scoping. This is a different defect from CVF-19: there a guard fails to fire on an empty run, here an assertion fires and is wrong.

What byte-parity structurally cannot cover. Every message in the v8 corpus is printable ASCII; its maximum codepoint is U+0079. Across both corpora exactly one character exceeds U+00FF — an em dash, U+2014, in V016 — and nothing anywhere exceeds U+FFFF. Consequently:

- CVF-3's divergence between the C reference's byte-oriented message space and the specified codepoint space cannot be detected at all, because below U+0080 the two encodings coincide. The corpus is drawn entirely from the region where the disagreement is invisible.
- CVF-14's two competing Rust encodings agree on every vector in either corpus: the codepoint path in `decrypt_bytes` and the `(c & 0xFF) as u8` truncation in `decrypt_raw` return identical results for every message present.
- The v7 decrypt path's `filter_map(char::from_u32)`, which silently drops values that are not Unicode scalars and returns a shorter plaintext with no error, is unreachable from any vector.
- The v8 length guard `if codepoints.len() > 0xFFFF` has nothing near it; the longest message in either corpus is 1,000 codepoints.

So the parity result is a statement about printable ASCII. Every encoding question the audit has raised lives above U+007F, which is precisely where no vector goes.

Public entry points with no vector coverage. The six tests exercise `decrypt_bytes`, `encrypt_bytes_with_nonce`, `encrypt_bytes_v8` and `decrypt_bytes_v8`. Nothing exercises `encrypt_bytes`, the random-nonce production path; `encrypt_str` or `decrypt_str`, the base64 wrappers a caller is most likely to use, and the two the README documents; `encrypt_raw` or `decrypt_raw`, the byte-oriented pair CVF-14 is about; `encrypt_bytes_v8_with_profile` under any profile other than the default `Bucket`; or `PadProfile::Frame`, the zero-leakage profile that Section 6 recommends and CVF-5 discusses.

Negative vectors are asserted rather than verified. In both generators the negative-vector builders are pure dictionary constructors that never attempt a decryption, so nothing establishes that a `tampered_hex` value actually fails, or fails for the reason its `expected_exception` field records; three of the eight v8 negatives are manufactured by monkey-patching the reference's padding function. On the Rust side both negative tests check only `result.is_err()` and never which error, so a vector intended to reach the post-authentication bucket check and in fact rejected at the tag check passes identically. `W-N06` through `W-N08` exist specifically to reach the structural checks added at `Dec` step (3) in v4 — the new code — and no test confirms that they get there.

On the corpus path, stated carefully. Both loaders build the path by taking `CARGO_MANIFEST_DIR`, stepping up one level, and joining `tests/kat/…`, with the comment "repo root / rust/". The Python cross-language driver assumes the matching layout, resolving the crate at `/ "rust"` and the corpora at `/ "tests" / "kat"`. The two sides agree with each other, so in the client's own repository the paths presumably resolve. In the delivered snapshot `Cargo.toml` and `tests/` are siblings at one level, and in that layout `CARGO_MANIFEST_DIR/../tests/kat/` does not exist and `load_vectors()` panics on `Cannot read …`. We read this as an artifact of how the snapshot was packaged rather than as a defect, and we could not distinguish the two, since the enclosing repository was not delivered. What is a finding independently of the layout is the brittleness: the location is a hard-coded relative literal behind `.parent().unwrap()`, with no fallback and no search; the corpora sit outside the crate's package root, so `cargo package` cannot carry them and `cargo test` against a published crate cannot find them; and the failure mode is a panic inside a helper shared by three tests rather than a diagnostic skip.

Recommendation

Take the empty-message inversion first, since it is the only item here that certifies a known divergence as correct.

Delete the special case and test the vector as a vector, using its own key, nonce and expected ciphertext like every other positive:

```
let got_ct = encrypt_bytes_with_nonce(message, &key, nonce, &aad);
assert_eq!(got_ct, expected_ct, "[{}] ciphertext differs from the
  Python KAT", id);
```

For V001 that will fail against the delivered corpus until the v7 empty-input short-circuit is removed, which is the point: the test should fail, and CVF-13 is the fix. If the divergence is instead going to be kept, then V001 has to be removed from the corpus and the paper's claim that conforming implementations produce byte-identical output for every positive vector has to be qualified — but the current arrangement, which keeps the vector and asserts its negation, leaves both the corpus and the harness reporting success while they contradict each other. Remove the corresponding `[SKIP]` in `positive_decrypt_roundtrip` at the same time, and with it the `skipped` counter that CVF-19 shows disables that test's only guard.

Extend the corpus above U+007F. This is the single change that would give the parity claim reach, and it is cheap, since the generator already produces vectors from a message list. At minimum add, to the v8 corpus: a message in U+0080–U+00FF, where the codepoint and byte encodings first diverge and CVF-3 and CVF-14 become observable; one in the BMP above U+00FF; one above U+FFFF, which is where a UTF-16-based port would break; a lone surrogate or an out-of-range value on the decrypt side, to reach the `filter_map(char::from_u32)` path that currently drops silently; and a message at the `0xFFFF` codepoint boundary. Until such vectors exist, state in Section 10 that the corpus is ASCII-only and that cross-implementation agreement is claimed only over that range — a reader currently has no way to know it.

Cover the untested public surface. Add v8 vectors under `PadProfile::Frame` as well as `Bucket`, since `Frame` is the profile the caveats section recommends and it has no coverage at all, and add round-trip tests through `encrypt_str`/`decrypt_str` and `encrypt_raw`/`decrypt_raw`. The `raw` pair matters most: it is the one whose

message space differs, and it is currently exercised by nothing.

Make the negative vectors verify what they claim. Have both generators attempt the decryption of each `tampered_hex` and assert that it raises, and that the message matches `expected_exception`, so the corpus records a checked property rather than an intention. On the Rust side, assert the error text rather than just `is_err()`:

```
let err = decrypt_bytes_v8(&ct, &key, &sk, &aad).unwrap_err();
let want = vec["expected_exception"].as_str().unwrap();
assert!(err.contains(want), "[{}] rejected for the wrong reason:
  {}", id, err);
```

Without this, `W-N06`–`W-N08` cannot distinguish rejection by the new structural checks from rejection by the tag check, which defeats the reason they were written.

Retarget the harness at the format the paper specifies. Section 10 disclaims the v6 corpus, so a majority of the tests in this file are testing something the paper does not analyse. Either move the three v7 tests behind a clearly named `legacy_v7` module so a reader can see which results bear on the paper, or retire them along with the v7 format. Correct the file's opening line either way: it announces itself as a v7 cross-check while the paper's claim is about v8.

Relabel the v6 corpus. `spec_version: "v6"` on a document whose block vectors are v7 and whose streaming vectors are LEB128 is a trap of the same kind the scope file warned about for the self-test constant. Split the two formats into separate files with accurate `spec_version` fields, or add a per-vector `wire_format` field; "path stability" is not worth a mislabelled cryptographic test corpus.

Make the corpus path robust and the tests hermetic. Resolve the corpus by searching upward from `CARGO_MANIFEST_DIR` for a `tests/kat/` directory, or better, move the corpora inside the crate and embed them with `include_str!`, which removes the filesystem dependency entirely, makes the vectors travel with `cargo package`, and turns a missing corpus into a compile error rather than a runtime panic. Replace `.parent().unwrap()` with a handled error either way.

Finally, correct Section 10's description of the generator: `--check` verifies that the Python output is reproducible, not that any port agrees. If a genuine cross-port parity check is wanted, it needs to invoke the ports and compare bytes on the Python side; `test_cross_lang.py` currently reads a self-reported pass count out of the Rust harness's stdout behind an `if` that makes the assertion optional, so the byte comparison happens inside the artifact under test. And since the Python reference module itself has not been delivered in any round, submitting it is what would let the parity claim be checked rather than taken on report.

Client Comment

/src/kat_cross_check.rs

```
//! Rust KAT cross-check for NAPSEQ v7 (Phase 0, workstream 0.1 / ROADMAP §2.B8).
```

same file

```
//! Reads `tests/kat/v6_vectors.json` from the repo root, exercises both the
//! deterministic `encrypt_bytes_with_nonce` API (exact byte comparison) and
//! the `decrypt_bytes` API for positive vectors, and asserts that negative
//! vectors return an `Err`.
```

same file

```
fn load_vectors() -> Vec<Value> {
    // Path is relative to the Cargo workspace root (repo root / rust/).
    let manifest_dir = env!("CARGO_MANIFEST_DIR");
    let path = Path::new(manifest_dir)
        .parent()              // repo root
        .unwrap()
        .join("tests/kat/v6_vectors.json");
```

same file

```
    for vec in vectors
        .iter()
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        if message.is_empty() {
            skipped += 1;
           eprintln!("[SKIP] {} — empty message, no ciphertext", id);
            continue;
        }
```

same file

```
    for vec in vectors
        .iter()
```

```
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        // Empty-message vector: nonce_hex is "" and ciphertext_hex is ""
        if message.is_empty() {
            let ct = encrypt_bytes_with_nonce(message, &[], [0u8; NONCE_SIZE], b"");
            assert!(ct.is_empty(), "[{}] empty message should produce empty ciphertext", id);
            eprintln!("[PASS] {} — empty message", id);
            tested += 1;
            continue;
```

same file

```
        if nonce_hex.is_empty() || ct_hex.is_empty() {
            eprintln!("[SKIP] {} — missing nonce_hex or ciphertext_hex", id);
            continue;
        }
```

same file

```
    for vec in vectors
        .iter()
        .filter(|v| v["kind"] == "negative" && v["api"] != "stream_ae")
```

same file

```
        assert!(
            result.is_err(),
            "[{}] expected Err but got Ok({:?})",
            id,
            result.unwrap()
        );
```

same file

```
fn load_v8_vectors() -> Vec<Value> {
    let manifest_dir = env!("CARGO_MANIFEST_DIR");
    let path = Path::new(manifest_dir)
        .parent()
        .unwrap()
        .join("tests/kat/v8_vectors.json");
```

same file

```
        let got_ct = encrypt_bytes_v8(message, &key, &sk, &aad)
```

same file

```
300         assert!(
301             got_ct == expected_ct,
302             "[{}] v8 ciphertext differs from the Python KAT (first differing byte at {:?})",
303             id,
304             got_ct.iter().zip(expected_ct.iter()).position(|(a, b)| a != b)
305         );
```

same file

```
350         let result = decrypt_bytes_v8(&ct, &key, &sk, &aad);
351         assert!(
352             result.is_err(),
353             "[{}] expected Err but got Ok({:?})",
354             id,
355             result.unwrap()
356         );
```

/src/lib.rs

```
791     if message.is_empty() {
792         return Vec::new();
793     }
```

same file

```
857     let s: String = codepoints.into_iter().filter_map(char::from_u32).collect();
```

same file

```
886 pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
```

same file

```
933 pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
```

same file

```
1145     let codepoints: Vec<u32> = message.chars().map(|c| c as u32).collect();
1146     if codepoints.len() > 0xFFFF {
1147         return Err("Message too long for 2-byte length prefix.".into());
```

/tests/kat/v6_vectors.json

```
2   "spec_version": "v6",
```

same file

```
5       "id": "V001",
6       "kind": "positive",
```

```
7       "description": "Empty message produces authenticated ciphertext
(not empty bytes)",
```

same file

```
20      "id": "V002",
```

/tests/kat/v8_vectors.json

```
2   "spec_version": "v8",
```

same file

```
5       "id": "W001",
6       "kind": "positive",
7       "description": "Empty message produces a full authenticated
ciphertext",
```

same file

```
269     "id": "W-N06",
```

same file

```
284     "id": "W-N07",
```

same file

```
299     "id": "W-N08",
```

CVF-27: open Category: **algorithm**

`Remark 3.13`'s central claim is false: steps (6) and (8) do not make `Dec` step (7) total, and the Rust port silently carries two rejection rules the specification does not contain — a port written from the paper alone inherits an integer underflow on a validly tagged ciphertext.

> *(5) if $T \neq T'$ (compared in constant time), output $\perp$; (6) if $R \notin \{B + 2 : B \in \{2^4, \ldots, 2^{16}\}\}$, output $\perp$ (Remark 3.13); (7) else unmask $B = \widetilde{B} \oplus \mathrm{ks}(N)$, deserialise the N_{tok} fixed-width tokens, and invert the token-emission loop using the same (sk, N)-derived noise-position oracle: skip at most* MAX_NOISE_RUN *consecutive noise positions, then read one real token t and recover the padded codepoint as $(t - a)/k_{(\mathrm{real_idx} \bmod K)+1}$, where a is the* real_idx*-th addend $\mathrm{Derive}_{0x01}(\mathsf{sk}, N, \mathrm{real_idx})$ and* real_idx *counts real tokens from 0 — exactly the prime and addend* Enc *used at that index; stop once R real tokens have been extracted, discarding any*

Reference PDF p. 11 — `Dec` *step (7) recovers each padded codepoint as $(t - a)/k_{(\mathrm{real_idx} \bmod K)+1}$, with no guard requiring $t \geq a$ and no divisibility condition on $t - a$.*

> *addend and used at that index, stop once R real tokens have been extracted, discarding any trailing filler; (8) read the 2-codepoint big-endian length prefix n and, if $n > R - 2$, output $\perp$ (Remark 3.13); (9) output the n codepoints following the prefix.*
>
> **Remark 3.13** (The post-authentication structural checks)**.** *Steps (6) and (8) are performed* after *the tag comparison of step (5), so they are never reachable on an input an adversary without* sk *could construct, and they play no part in the INT-CTXT argument. They are specified nonetheless because step (7) is not total without them: R is recovered from $|C|$ and n from the decrypted buffer, so a malformed but correctly tagged ciphertext — which a key holder can trivially produce — would otherwise drive an out-of-range read or an allocation sized by a field the caller did not choose. Divisibility by* MAX_NOISE_RUN $+ 1$ *alone does not imply step (6): $R = 20$ passes step (3) and fails step (6). All three reference implementations enforce both checks, and vectors W-N06–W-N08 of Section 10 pin them.*

Reference PDF p. 11 — `Remark 3.13` justifies the two structural checks by asserting that step (7) "is not total without them". Steps (6) and (8) bound only R and the length prefix n; neither constrains a token value, so the totality claim is false.

`Remark 3.13` exists to justify the two structural checks, and it justifies them with a totality argument: steps (6) and (8) "are specified nonetheless because step (7) is not total without them: R is recovered from $|C|$ and n from the decrypted buffer, so a malformed but correctly tagged ciphertext — which a key holder can trivially produce — would otherwise drive an out-of-range read or an allocation sized by a field the caller did not choose."

The premise is right and the conclusion is too weak. Steps (6) and (8) are **not sufficient**. Step (6) constrains R, which is derived from $|C|$; step (8) constrains n, which is read from the recovered prefix. Neither constrains the **token values**. `Dec` step (7) specifies only that the padded codepoint is recovered as $(t - a)/k_{(real_idx \bmod K)+1}$, and a key holder — precisely the adversary `Remark 3.13` invokes and `Remark 3.15` describes — can produce a validly tagged ciphertext with $|C| = 2928$, so $R = 18$ passes step (6), whose masked blob unmasks to arbitrary 64-bit tokens. Three failure modes then survive both structural checks:

- $t < a$. The specification computes $t - a$ over the naturals, where it is undefined. A 64-bit implementation underflows.
- $(t - a) \bmod k \neq 0$. The quotient is not an integer and the specification does not say what to do.
- The quotient exceeds `0x10FFFF` or lands in the surrogate range, i.e. outside $\mathcal{M}$, and step (9) is then asked to output a value that is not a codepoint.

Step (8) cannot help with any of these: it is specified *after* step (7) and constrains only the slice taken in step (9).

The implementers found the gap and the paper did not record it. `decrypt_core_v8` carries two rejections that appear nowhere in `Definition 3.10`, and the comment above the first states the reason in as many words — "the bare subtraction panics here in debug builds and wraps in release builds and in C":

```
if token < addend || (token - addend) % k != 0 {
    return Err(/* not of the form codepoint * k + addend */);
```

```
}
let cp = (token - addend) / k;
if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
    return Err(/* not a Unicode scalar value */);
}
```

The first rejection closes the first two failure modes; the second closes the third and is stricter than `Definition 3.10`, which says the arithmetic layer "is indifferent to which subset of $[0, 2^{21})$" the values range over.

There is **no forgery consequence**. All three conditions are post-authentication, unreachable without sk, so `Theorem 4.18` is untouched and no outside adversary can drive any of them. The consequences are correctness and interoperability:

- A conforming implementation written from the paper alone carries the panic. In a Rust debug build the bare subtraction panics; in a Rust release build with no `overflow-checks` — which is the delivered configuration, since `Cargo.toml` declares no `[profile.release]` section — it wraps silently; in C it wraps silently. So the paper as written specifies a decryption routine that a key holder can crash or drive to a wrong answer, which is exactly the class `Remark 3.13` was written to close.
- Three implementations of the same specification disagree on this input class: the Rust port returns an error, a port that uses floor division returns a plaintext, and a port that follows the specification's literal subtraction underflows. Section 10 asks for byte-identical behaviour on a corpus that contains no vector in this class, so nothing in the delivered evidence would catch the divergence.

`Proposition 3.14` is unaffected, because correctness quantifies only over honest `Enc` outputs. The defect is in the totality claim, not in the correctness proof.

This is distinct from the addend-notation defect recorded under CVF-7, which concerns how step (7) *writes* the addend rather than whether step (7) is *total*, and from the missing-upper-bound overflow of CVF-16, which is an encrypt-side multiplication rather than a decrypt-side subtraction.

Three of the four conformance regions in this pass reached the same conclusion independently; it is filed once.

Recommendation

Extend `Definition 3.10` step (7) with the two rejections the ports already perform, and correct `Remark 3.13` so that its totality claim matches.

In step (7), before the division, specify: if $t < a$ or $(t - a) \not\equiv 0 \pmod{k_{(real_idx \bmod K)+1}}$, output $\perp$. State it as a precondition on the division rather than as prose, so that a reader implementing from the paper cannot reach the subtraction without the guard. This is the same device `Definition 3.4` uses for the tag input and the same device recommended for the addend under CVF-7: name the constraint once, in the algorithm, so it cannot be dropped in translation.

Decide explicitly what to do about the third check. The Rust port rejects a recovered value that is not a Unicode scalar value, which is stricter than the message space paragraph of `Definition 3.10` allows for. Either add that rejection to step (7) and delete the "indifferent to which subset of $[0, 2^{21})$" sentence, or remove the check

from the ports. It should not stay in the code and out of the specification, because it is a rejection an interoperating port can observe.

Then rewrite `Remark 3.13`'s justification. As written it says steps (6) and (8) are what make step (7) total; they are not. The accurate statement is that step (7)'s own token-form check makes the arithmetic total, while steps (6) and (8) bound the two length fields, and all of them are post-authentication so none affects the integrity argument. Keeping the remark's structure and correcting only the claim is enough — the remark is otherwise the right place for this material.

Finally, add a negative known-answer vector for this class: a validly tagged, key-holder-produced ciphertext of 2,928 bytes whose blob unmasks to a token with $t < a$ at a known position. That is the only one of the three delivered structural negatives (`W-N06`–`W-N08`) that the corpus is missing, and without it a port that omits the guard passes the whole suite. Note that the vector must be generated with the key, since the class is unreachable without sk — the same construction the existing "re-tagged" negatives already use.

Client Comment

/Cargo.toml

```toml
[package]
name = "napqes"
version = "0.1.0"
edition = "2021"
```

/docs/napseq-eprint-v4.tex

```
       exactly the set of strings that admit a UTF-8 encoding. Nothing
        weaker is needed and nothing stronger is assumed: the
arithmetic
        layer itself operates on the integer codepoint values and is
        indifferent to which subset of $[0,2^{21})$ they range over.
  \item Ciphertext space $\mathcal{C}=\{0,1\}^{*}$: every output of
```

same file

```
(Remark~\ref{rem:dec-structural});
(7) else unmask $B=\widetilde{B}\oplus\mathrm{ks}(N)$, deserialise
the
$N_{\mathrm{tok}}$ fixed-width tokens, and invert the token-emission
loop
using the same $(\mathsf{sk},N)$-derived noise-position oracle: skip
at most
$\mathrm{MAX\_NOISE\_RUN}$ consecutive noise positions, then read one
real
```

```
token $t$ and recover the padded codepoint as
$(t-a)/k_{(\mathrm{real\_idx}\bmod K)+1}$, where $a$ is the
$\mathrm{real\_idx}$-th addend
$\mathrm{Derive}_{0x01}(\mathsf{sk},N,\mathrm{real\_idx})$ and
$\mathrm{real\_idx}$ counts real tokens from $0$ — exactly the prime
and
addend $\mathsf{Enc}$ used at that index; stop once $R$ real tokens
have
```

same file

```
\begin{remark}[The post-authentication structural checks]
\label{rem:dec-structural}
Steps (6) and (8) are performed \emph{after} the tag comparison of
step (5),
so they are never reachable on an input an adversary without
$\mathsf{sk}$
could construct, and they play no part in the INT-CTXT argument. They
are
specified nonetheless because step (7) is not total without them: $R$
is
recovered from $|C|$ and $n$ from the decrypted buffer, so a
malformed but
correctly tagged ciphertext — which a key holder can trivially
produce —
would otherwise drive an out-of-range read or an allocation sized by
a field
the caller did not choose. Divisibility by
$\mathrm{MAX\_NOISE\_RUN}+1$
alone does not imply step (6): $R=20$ passes step (3) and fails step
(6).
All three reference implementations enforce both checks, and vectors
\texttt{W-N06}--\texttt{W-N08} of Section~\ref{sec:kats} pin them.
\end{remark}
```

/src/lib.rs

```
        // A genuine real token is exactly c * k + addend with addend
in
        // [1, k - 1]. Checking that explicitly, rather than
subtracting and
        // dividing, keeps the three ports in lock-step: the bare
subtraction
       // panics here in debug builds and wraps in release builds and
in C.
```

same file

```
        if token < addend || (token - addend) % k != 0 {
            return Err(format!(
```

```
                "Malformed v8 ciphertext: token at position {} is not
of the form \
                 codepoint * k + addend for key element {}.",
                ct_pos, k
            ));
        }
```

same file

```
        let cp = (token - addend) / k;
```

same file

```
        if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
            return Err(format!(
                "Malformed v8 ciphertext: recovered value {} at
position {} is not a \
                 Unicode scalar value.",
                cp, ct_pos
            ));
        }
```

CVF-28: open Category: **architecture**

The public v7 surface still runs the single-secret key schedule that the entire v8 redesign was built to eliminate, uses a 4-byte AAD length prefix where `Definition 3.4` fixes 8, and is described in Section 10 as a nonce-taking test format when in fact every public v7 entry point generates its own nonce — so the crate ships, undeprecated, a format whose defects the crate itself documents at length.

Three divergences on the same surface, filed together because one decision resolves all three.

The key schedule is the discarded one. Every v7 derivation is keyed by `kb = key_bytes(key)`, the five-byte-per-element serialisation of the prime tuple, at seven call sites: `encrypt`, `decrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce`, `decrypt_bytes`, `encrypt_raw` and `decrypt_raw`. That is exactly the structured, low-min-entropy HMAC key that `Remark 3.3` and `Assumption 4.2` were rewritten to eliminate, and the crate says so itself. The module comment introducing the v8 schedule enumerates three prior findings that "trace back to that one design choice": that a repeated nonce under it permits solving $k = (t_1 - t_2)/(c_1 - c_2)$ from a single known plaintext codepoint — "catastrophic key recovery, not merely a confidentiality loss"; that `key_bytes(primes)` carries only $H_\infty(\mathbf{k})$ bits of min-entropy and is therefore "a structured, non-uniform HMAC key, which is a different (and non-standard) hypothesis from the textbook uniform-key HMAC-SHA256 PRF assumption every theorem bound otherwise

cites"; and that the PRF hop in the reductions is vacuous because the reduction must already know the primes that constitute the supposedly hidden oracle key.

Every word of that is a description of the code that is still exported. `Assumption 4.2` is stated against a uniform 256-bit key and does not reach `key_bytes(primes)`; `Remark 3.3` is explicit that "sk is the *only* secret that ever keys an HMAC call in this construction", which is true of v8 and false of v7. None of `Theorem 4.8`, `Theorem 4.12`, `Theorem 4.18` or `Theorem 4.20` covers a v7 ciphertext. Nothing in the crate, the README or the paper says so; the v7 functions carry no `#[deprecated]` attribute and no warning in their doc comments about the key schedule.

This is distinct from the two v7 findings already on file. CVF-17 is about the emission loop lacking the noise-run cap and the ceiling; CVF-22 is about the fuzz harness targeting the v7 decoder. Neither states that the v7 *key schedule* is the one the v8 redesign was built to replace.

The AAD length prefix is 4 bytes, not 8. `Definition 3.4` introduces the length prefix for one stated purpose — so that "encryptor and decryptor cannot diverge in the width of the length prefix" — and fixes it at be8$(|A|)$. The v8 path honours that with `AAD_LEN_WIDTH_V8 = 8`. The v7 path uses `AAD_LEN_WIDTH_V7 = 4` at all five of its tag sites. v7 is internally symmetric, so encryptor and decryptor do not in fact diverge, and v7 is not the specified scheme — but the helper that writes the prefix, `be_len_prefix(n, width)`, takes the **low** `width` bytes with no range check, so at width 4 it silently encodes $|A|$ mod 2^{32}. Two associated-data values whose lengths differ by a multiple of 2^{32} therefore produce identical tag inputs. Reaching that requires a 4 GiB AAD, so there is no practical consequence, but it is a silent truncation where an error is the correct behaviour, and it sits in the one construct `Definition 3.4` introduced specifically to make length encoding unambiguous.

Section 10 mischaracterises v7 as nonce-taking. The paper says `v6_vectors.json` "is retained in the repository for the earlier v7 wire format, which takes an externally supplied nonce and is *not* the scheme analysed here". In the delivered crate the only nonce-taking function is `encrypt_bytes_with_nonce`, which is `pub(crate)` and, by its own doc comment, "reachable only from the self-test module and the in-crate KAT cross-check, never from external consumers of this crate". Every *public* v7 entry point generates its own nonce internally: `encrypt` draws one directly from `rand::thread_rng()`, and `encrypt_bytes` and `encrypt_raw` go through `generate_nonce_with_crng_check`. So the sentence describes v7 as a vestigial, caller-driven test format when it is in fact the crate's most prominent production API — the one `encrypt_str`, `encrypt_bytes` and `encrypt_raw` present to every consumer. That matters for how a reader scopes the paper's security claims against the delivered code: told that v7 takes an external nonce, a reader concludes it is a harness artefact and stops there.

Reach, from code outside the reviewed surface. Offered as impact only; the root cause of all three items is inside `src/lib.rs`. The crate's own `src/ot_frame.rs` seals and opens its protocol data units with `encrypt_raw` and `decrypt_raw`, and the DNP3 and Modbus protocol modules and the `ls_gateway` binary sit on top of that. The delivered transport path therefore runs the v7 key schedule end to end, so the shipped industrial-protocol gateway is carrying traffic that no theorem in the paper covers.

Recommendation

Decide whether v7 is a supported format or an archive-reading shim, and make the code and the paper say the same thing.

The cleanest resolution is to withdraw the v7 encryptors. Nothing in the specification requires v7 to exist: the paper specifies only v8, Section 8.2 describes the Rust port purely in terms of `generate_v8_key`, `encrypt_bytes_v8` and `decrypt_bytes_v8`, and the crate's own module comment sets out three reasons why the v7 key schedule was replaced. Keeping `decrypt_bytes` and `decrypt_raw` for reading archived ciphertexts, while removing `encrypt`, `encrypt_bytes`, `encrypt_str` and `encrypt_raw` from the public API, closes this finding without a migration path for stored data, and it closes CVF-17 at the same time. It also requires migrating `ot_frame::wrap_pdu` to the v8 entry points, which is the substantive part of the work and is worth doing on its own merits, since the transport layer is the one consumer for which the theorems ought to hold.

If v7 must stay public, then three things need saying, in the code and in the paper:

- Mark every v7 entry point `#[deprecated]` and state in each doc comment that the format keys its derivations by `key_bytes(primes)` rather than by sk, that this is the schedule the module comment's CVF3/CVF8/CVF13 discussion describes, and that consequently none of the paper's security theorems applies to a v7 ciphertext. At present a caller must read the v8 rationale block and then notice, by absence, that v7 was left behind.
- Make `be_len_prefix` return an error rather than truncate when `n` does not fit in `width` bytes. The v8 path is unaffected, since $|A| < 2^{64}$ always fits in eight bytes; the change only removes a silent wraparound on the v7 path. If the four-byte v7 prefix is retained for wire compatibility, add the explicit range check beside it.
- Correct Section 10's description of `v6_vectors.json`. Replace "which takes an externally supplied nonce" with an accurate statement: the v7 format draws a fresh CSPRNG nonce per message in every public entry point, keys its derivations by the serialised prime tuple rather than by sk, and is not covered by any result in Section 4. The current wording understates what is still shipped.

  In either case, add one sentence to Section 8.2 recording that the Rust crate exports a second, unspecified wire format alongside the one this paper defines, so that a reader does not carry Section 4's bounds across to it.

Client Comment

/docs/napseq-eprint-v4.tex

```
312 $\mathsf{sk}$ is the \emph{only} secret that ever keys an HMAC call
    in this
313 construction (Section~\ref{sec:domains}); $\mathbf{k}$ is used
    exclusively,
```

```
directly, and un-keyed by the arithmetic token map $c \mapsto c\cdot
k_j+a$
(Section~\ref{sec:token-construction}). Consequently, none of the
security
theorems of Section~\ref{sec:security} depend in any way on
$H_\infty(\mathbf{k})$: the PRF assumption below is stated, and used,
against the standard \emph{uniform}-key distribution of $\mathsf{sk}$
only.
```

same file

```
\begin{definition}[Tag input]\label{def:taginput}
The authentication tag is always computed over the single quantity
\[
  \mathrm{TagIn}(A, \widetilde{B}) \;=\; \mathrm{be8}(|A|) \,\|\, A
\,\|\, \widetilde{B},
\]
so that the complete domain-$\texttt{0x03}$ HMAC input is
$\texttt{0x03}\|N\|\mathrm{TagIn}(A,\widetilde{B})$. Every occurrence
of the
tag in this paper — Table~\ref{tab:domains}, the wire format
(Section~\ref{sec:wire-format}), both $\mathsf{Enc}$ and
$\mathsf{Dec}$
(Definition~\ref{def:aead-triple}), and the integrity proof
(Theorem~\ref{thm:int-ctxt}) — denotes this one quantity, so
encryptor and
decryptor cannot diverge in the width of the length prefix.
\end{definition}
```

same file

```
\begin{assumption}[Uniform-key HMAC-SHA256 PRF]\label{assum:hmac-prf}
$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-
}SHA256}}(\mathcal{B})$
is negligible in the security parameter for every PPT $\mathcal{B}$,
for $\kappa$ drawn uniformly from $\{0,1\}^{256}$ (matching how
$\mathsf{sk}$ is sampled, Definition~\ref{def:keygen}).
\end{assumption}
```

same file

```
\subsection{Rust Implementation}

A Rust port (\texttt{rust/src/lib.rs}) mirrors the Python
implementation
(\texttt{generate\_v8\_key}, \texttt{encrypt\_bytes\_v8},
\texttt{decrypt\_bytes\_v8}). Unit tests cover round-trip
correctness,
tamper/AAD-mismatch rejection, same-message determinism (the
disclosed
```

```
MRAE trade-off), distinct-message nonce distinctness, and
ciphertext-length determinism across varied AAD and keys
(Lemma~\ref{lem:length-det}).
```

same file

```
A second corpus, \texttt{tests/kat/v6\_vectors.json} (37 vectors), is
retained in the repository for the earlier v7 wire format, which
takes an
externally supplied nonce and is \emph{not} the scheme analysed here;
its
vectors are outside the scope of this paper and none of the results of
Section~\ref{sec:security} are asserted of them.
```

/src/lib.rs

```
const AAD_LEN_WIDTH_V7: usize = 4;
```

same file

```
fn key_bytes(key: &[u64]) -> Vec<u8> {
    let mut out = Vec::with_capacity(key.len() * 5);
    for &k in key {
        let b = k.to_be_bytes(); // 8 bytes
        out.extend_from_slice(&b[3..8]); // low 5 bytes, big-endian
    }
    out
}
```

same file

```
fn be_len_prefix(n: usize, width: usize) -> Vec<u8> {
    (n as u64).to_be_bytes()[8 - width..].to_vec()
}
```

same file

```
pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
Vec<u64>) {
    let mut nonce = [0u8; NONCE_SIZE];
    rand::thread_rng().fill_bytes(&mut nonce);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
```

same file

```
pub(crate) fn encrypt_bytes_with_nonce(
    message: &str,
    key: &[u64],
    nonce: [u8; NONCE_SIZE],
    aad: &[u8],
) -> Vec<u8> {
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
```

same file

```
// ─── V8 key schedule + synthetic nonce (CVF3 / CVF8 / CVF13 fix)
─────────────
//
// v7 (and every earlier wire format) keys *every* domain derivation
with
// `kb = key_bytes(primes)` — the serialisation of the same prime
tuple used
// for the token arithmetic (`c*k+a`) — and draws the nonce from a
CSPRNG
// independent of the message. Three audit findings trace back to
that one
```

same file

```
//   - **CVF8.** `key_bytes(primes)` has only `H_inf(k) ≈ log2(|P|!/(|P|-K)!)`
//     bits of min-entropy — a structured, non-uniform HMAC key, which is a
//     different (and non-standard) hypothesis from the textbook uniform-key
//     HMAC-SHA256 PRF assumption every theorem bound otherwise cites.
//   - **CVF13.** The IND-CPA/INT-CTXT reductions must simulate the token
```

same file

```
//     in place of `key_bytes(primes)`.
```

CVF-41: open Category: **efficiency**

Four avoidable inefficiencies in the prime-generation and key-validation path, one of which would also close the timing channel of CVF-29

The crate establishes primality and distinctness of key material by the most direct algorithm available in each case. None of the four choices below is incorrect, and at the default `K = 10` none is individually expensive. They are filed together because `validate_key` sits on the per-message path of both v8 entry points (`lib.rs:1149`, `lib.rs:1212`), so the cost is paid on every encrypt and every decrypt rather than once per key, and because the first item has a remedy that is also the cheapest available fix for a security finding.

1. `is_prime` trial-divides by every odd integer up to the square root when a fixed table of 446 primes would do. The loop is

```
let mut i: u64 = 3;
while i.saturating_mul(i) <= n {
    if n % i == 0 { return false; }
    i += 2;
}
```

Sieving confirms the count Mikhail Vladimirov gives:

$$\lfloor \sqrt{\text{MAX_KEY_PRIME}} \rfloor = \lfloor \sqrt{9{,}900{,}000} \rfloor = 3146$$

, and there are exactly **446 primes** at or below 3,146, of which 445 are odd. Trial division by that fixed table replaces up to 1,572 divisions with 445, and removes the per-iteration `i.saturating_mul(i)` bound recomputation as well, since the table is traversed by index rather than by a multiply-and-compare.

The consequence that matters is not the speed. Because the table is of **fixed length**, running it to completion unconditionally makes the work performed by `is_prime` independent of `n` for every `n` in the normative interval — which is exactly the property CVF-29 reports as missing. CVF-29 measures the current loop at 499 iterations for $k = 1{,}000{,}003$

and 1,572 for $k = 9{,}899{,}999$, a $3.15\times$ spread that is a monotone function of each secret prime and is re-leaked on every message. A fixed table run to completion removes that spread, so this item is simultaneously the cheapest fix for CVF-29 and the reason CVF-29's recommendation against "a constant-time primality test" should be read narrowly: a general-purpose constant-time primality test is the wrong answer, but a fixed small-prime table over a bounded interval is not one, and it is cheaper than what is there now.

This is sound only once CVF-16 is fixed, and it must not be applied before. `validate_key` currently enforces `k >= MIN_KEY_PRIME` and deliberately omits any upper bound, so it accepts key elements far above `MAX_KEY_PRIME` — that omission is CVF-16. A table capped at $\sqrt{\text{MAX_KEY_PRIME}}$ would declare such an element prime without having tested the divisors above 3,146, turning a validation gap into a validation **error**: a composite key element above $3146^2 = 9{,}897{,}316$ would be accepted. The table optimisation and CVF-16's upper bound are therefore one change, not two, and shipping the first without the second is worse than shipping neither.

2. `generate_prime_numbers` rejects duplicates by linear scan of a growing vector (`lib.rs:111`, `!primes.contains(&num)`). At $K = 10$ this is at most ten comparisons per accepted candidate and is not worth changing on its own; it is recorded because it shares its remedy with item 3.

3. `validate_key` checks distinctness in $O(K^2)$ (`lib.rs:243`, `key[..i].contains(&k)`). At $K = 10$ that is 45 comparisons. `validate_key` places **no upper bound on `K`**, however — it rejects only the empty tuple — so a caller-supplied key of length L costs $L(L-1)/2$ comparisons *and* L trial divisions on every message. This is a second axis of the denial-of-service exposure CVF-16 describes: CVF-16 concerns the unbounded **magnitude** of a single key element, this concerns the unbounded **length** of the tuple, and a key of 10^6 elements costs roughly 5×10^{11} comparisons per call before the trial divisions are even counted.

4. The square-root bound is recomputed on every iteration. `i.saturating_mul(i) <= n` performs a multiplication and a comparison per step where a single `let bound = n.isqrt();` before the loop would serve. Subsumed entirely by item 1 if the table is adopted.

Recommendation

Treat items 1 and 3 as part of the key-validation rework CVF-16 and CVF-29 already call for, rather than as standalone micro-optimisations.

Adopt the fixed prime table **together with** CVF-16's upper bound, in that order of dependency:

```
/// All 446 primes <= isqrt(MAX_KEY_PRIME) = 3146.
const SMALL_PRIMES: [u64; 446] = [2, 3, 5, /* ... */ 3137,
  3163_u64 /* excluded */];

fn is_prime_bounded(n: u64) -> Result<bool, String> {
    if n > MAX_KEY_PRIME {
        return Err(format!("{} exceeds MAX_KEY_PRIME ({})", n,
  MAX_KEY_PRIME));
```

```
    }
    if n < 2 { return Ok(false); }
    let mut composite = false;
    for &p in SMALL_PRIMES.iter() {
        // Run the whole table unconditionally: the work is then
  independent
        // of n, which is what closes the CVF-29 timing channel.
        composite |= (n % p == 0) & (n != p);
    }
    Ok(!composite)
}
```

The unconditional traversal is the point — an early `return false` on the first divisor would restore a data-dependent iteration count for composite inputs. Keep `is_prime`'s existing unbounded form available under its current name if any caller outside key validation needs it, and route `validate_key` at `lib.rs:229` through the bounded version only.

Bound `K` in `validate_key`. The paper's key space is ordered K-tuples of distinct elements of $\mathcal{P}$ with a stated default of $K = 10$; the crate accepts any non-empty length. Adding an explicit ceiling — `MAX_KEY_ELEMENTS`, named as a `pub const` alongside `MIN_KEY_PRIME` and `MAX_KEY_PRIME` — closes the length axis of the denial-of-service exposure and makes the $O(K^2)$ distinctness check unconditionally cheap, at which point items 2 and 3 need no algorithmic change at all. If no ceiling is acceptable, sort a scratch copy and scan once, as originally suggested, which is $O(K \log K)$; but bounding K is the better fix, because an unbounded key length is a problem for reasons beyond the comparison count.

Move the whole check off the per-message path, as CVF-29 and CVF-16 both recommend, by validating once into a `NapqesKey` newtype. With validation performed once at key construction, items 1 to 4 stop being hot-path concerns entirely and the timing channel closes for a second, independent reason. The table remains worth adopting even then, because it is the thing that makes a single validation cheap enough to be unremarkable.

Item 4 needs no separate action: `n.isqrt()` hoisted out of the loop is the right change if the table is not adopted, and is moot if it is.

Client Comment

/src/lib.rs

```
pub fn is_prime(n: u64) -> bool {
    if n < 2 {
        return false;
    }
```

```
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
```

same file

```
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```
        if is_prime(num) && !primes.contains(&num) {
```

same file

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
```

same file

```
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not
prime.", i, k));
        }
```

same file

```
        if key[..i].contains(&k) {
            return Err(format!(
                "Key element {} at index {} is a duplicate; all
elements must be distinct.",
                k, i
            ));
```

same file

```
    validate_key(primes)?;
```

same file

```
    validate_key(primes)?;
```

CVF-43: fixed Category: **flaw**

Length-prefix reconstruction in unpad_message treats two u32 codepoints as bytes without range-checking them, so the padded buffer has a non-canonical encoding and a mis-parsed length is returned rather than rejected

unpad_message recovers the message length from the first two entries of the decoded codepoint buffer:

```
let n = ((padded[0] as usize) << 8) | (padded[1] as usize);
```

This reconstruction assumes padded[0] and padded[1] are byte-valued (each in $[0, 255]$), mirroring the encoder in pad_to_block, which writes the length as a two-codepoint big-endian prefix and masks each half:

```
out.push(((n >> 8) & 0xFF) as u32);
out.push((n & 0xFF) as u32);
```

But padded holds u32 codepoints recovered by the decryptor, and neither prefix slot is constrained to a byte. On the v7 path the recovered codepoint is pushed with no range check at all — padded.push(((token - addend) / k) as u32) at lib.rs:513. On the v8 path it is bounded only to a valid Unicode scalar, cp > 0x10FFFF || char::from_u32(cp as u32).is_none() at lib.rs:606. In both cases padded[0] and padded[1] can exceed 0xFF.

The reconstruction is not injective, which we confirmed by direct computation. Because the shifted high slot and the low slot overlap through the |, distinct codepoint pairs collapse onto the same n. The pairs $(1, 0)$, $(0, 256)$ and $(1, 256)$ all yield $n = 256$: the first is the canonical encoding the encoder emits, and the other two are non-canonical spellings of it that unpad_message accepts as equivalent. So the padded buffer does not have a unique reading, and an authenticated ciphertext does not determine its plaintext through this function alone.

The length guard does not catch it. The subsequent check 2 + n > padded.len() rejects an oversized n, but an in-range yet mis-parsed n passes and unpad_message returns a wrong-length slice padded[2..2 + n]. The boundary case is exactly reachable rather than merely conceivable: at the largest padded block $B = 2^{16}$ the buffer length is $B + 2 = 65{,}538$, and the pair $(0\text{x}100, 0)$ gives $n = 65{,}536$ with $2 + n = 65{,}538 \leq 65{,}538$, so the guard passes and the function returns the entire buffer — prefix slots included — as though it were the message.

What the exposure is, stated precisely. On the authenticated wrappers the tag is verified before decryption, so a network attacker cannot drive the prefix out of range without forging the tag, and the practical consequence is confined to two cases.

- A sender holding the key can emit an authentic ciphertext whose padded prefix is non-canonical, and the receiver will decode it to a wrong-length plaintext while the tag verifies. Two distinct valid ciphertexts therefore decrypt to the same plaintext, and one ciphertext has a reading the encoder never intended. For a scheme whose Section 10 claim is that "conforming implementations must produce byte-identical ciphertext for every positive vector", an ambiguous decode is a cross-implementation hazard: two conforming ports could disagree about what a given authentic ciphertext means.

- On the unauthenticated low-level surface, `pub fn decrypt` calls `decrypt_core` with no tag check, and `decrypt_core` finishes with `unpad_message(&padded).expect(...)` at `lib.rs:520`, so a prefix that *does* trip the length guard aborts the process there. That abort is one of the five panics CVF-15 already reports and is not re-filed here; it is noted because it is the same missing check seen from the other side, and the fix below closes both.

The gap is also inconsistent with the function's own defensive posture. `unpad_message`'s doc comment is explicit that the length prefix "is attacker-chosen in the sense that it is recovered from the decrypted blob, so it must be validated against the buffer actually present" — and it then validates `n` against the buffer length while never validating the byte-range of the two codepoints `n` is built from. Separately, CVF-40 item 8 records that Section 12 of the paper calls this prefix "2-byte" where `Definition 3.10` and `Dec` step (8) correctly call it two codepoints; the code has taken the paper's looser reading at exactly the point where the distinction matters.

Recommendation

Validate the two prefix codepoints before reconstructing the length, returning the `Err` contract `unpad_message` already uses:

```
if padded[0] > 0xFF || padded[1] > 0xFF {
    return Err(format!(
        "Malformed length prefix: codepoints ({}, {}) are not
  byte-valued.",
        padded[0], padded[1]
    ));
}
let n = ((padded[0] as usize) << 8) | (padded[1] as usize);
```

Rejecting is preferable to masking with `& 0xFF`. Masking would accept a non-canonical buffer as a valid encoding of some `n` and so would leave the encoding ambiguous; an explicit check keeps the length field canonical and matches the encoder's intent, which is what a byte-parity claim across ports needs.

Apply the same guard in `napqes.py::_unpad_message` and in `C/napqes.c`, so the three ports agree on out-of-range input as well as on well-formed input. At present the ports agree only where the encoder's output is concerned; this is one of the places where they have never been compared because no test vector reaches it.

Add a negative KAT vector for it. The corpus already carries `W-N06` to `W-N08` for the post-authentication structural checks added in v4, and a vector whose padded prefix is non-canonical belongs in the same group — it is the one remaining post-authentication check with no vector. CVF-25 records that the negative vectors are currently asserted rather than verified and that the Rust side checks only `is_err()`; a vector added here should assert the error text, or it will be indistinguishable from a rejection at the tag check.

Finally, note the interaction with CVF-15: promoting `decrypt_core`'s `expect` to a propagated error is the other half of this fix. With the range check in place the `expect` becomes unreachable through the v8 path but remains reachable through `pub fn decrypt`, so both changes are needed.

Client Comment

/src/lib.rs

```
    let mut out = Vec::with_capacity(2 + block_size);
    out.push(((n >> 8) & 0xFF) as u32);
    out.push((n & 0xFF) as u32);
```

same file

```
fn unpad_message(padded: &[u32]) -> Result<Vec<u32>, String> {
    if padded.len() < 2 {
        return Err("Padded message too short to contain length prefix.".into());
```

same file

```
    let n = ((padded[0] as usize) << 8) | (padded[1] as usize);
```

same file

```
    if 2 + n > padded.len() {
        return Err(format!(
            "Length prefix ({}) exceeds available data ({} codepoints).",
            n,
            padded.len() - 2
        ));
    }
```

same file

```
            padded.push(((token - addend) / k) as u32);
```

same file

```
    unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")
```

same file

```
        if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
```

CVF-45: fixed Category: **flaw**

The constant-time tag comparator returns `true` when its two arguments have different lengths, and the only thing preventing that is a `debug_assert!` the shipped release profile compiles out

`ct_eq_bytes` is the crate's equality primitive for authentication tags. Its contract, as its doc comment states it, is a comparison of "two equal-length byte slices in constant time", and the comment closes with a specific claim about the generated code:

> The resulting assembly is a fixed-count loop: `jne` branches only on the loop counter (always 32 iterations), never on the tag data.

The signature accepts two slices of arbitrary and independent length, and the body enforces equality of those lengths only in a debug build:

```
#[inline(never)]
fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    debug_assert_eq!(a.len(), b.len());
    let mut diff = 0u8;
    for (x, y) in a.iter().zip(b.iter()) {
        ...
    }
    diff == 0
}
```

`Iterator::zip` stops at the shorter of the two operands. So on a length mismatch the function compares the common prefix and ignores the remainder of the longer slice, and `diff` is left at its initial `0` when the prefix matches — including the degenerate case where one slice is empty. `ct_eq_bytes(&[], &tag)` returns **`true`** for every `tag`. For a function whose result is the sole gate on `Err("Authentication failed: invalid HMAC tag.")` at three call sites, "returns true for an empty candidate tag" is the wrong failure mode: a comparator of this kind must fail closed on any input it was not designed for.

The guard does not survive the build the client ships. `debug_assert_eq!` is active only when `debug_assertions` is on, and `Cargo.toml` declares no `[profile.release]` section at all — the omission CVF-23 reports and CVF-31 relies on for its own analysis. In the delivered configuration a release build therefore contains no length check whatsoever, and a mismatch is silent rather than fatal.

The documented invariant is already false at one live call site. The comment's "always 32 iterations" holds for the three tag comparisons but not for `generate_nonce_with_crng_check`, which calls the same function on two 16-byte nonces:

```
if ct_eq_bytes(&nonce, &prev_nonce) {
```

That call is correct — both operands are `[u8; NONCE_SIZE]` — but it makes the loop run 16 times, so the one concrete property the doc comment asserts about the emitted code is not a property of the function as used. CVF-35 records this call site for a different reason; it is cited here because it is the existing counterexample to the comment.

Reachability, stated plainly, because it bounds the severity. No current call site can trigger the mismatch. At `lib.rs:847`, `lib.rs:949` and `lib.rs:1218` the received tag

is `&ciphertext[split..]` with `split = ciphertext.len() - TAG_SIZE`, taken after a length floor that guarantees `ciphertext.len() >= NONCE_SIZE + TAG_SIZE`, so it is exactly 32 bytes; and the computed tag is a `[u8; 32]`. At `lib.rs:56` both operands are `[u8; NONCE_SIZE]`. This is therefore a latent defect and not a live vulnerability.

It is filed at medium rather than low for three reasons. The function is `fn`, not `pub fn`, but it is the crate's designated constant-time primitive and the natural thing for a future call site to reuse — a variable-length MAC, a KEM shared secret, a caller-supplied tag arriving from a protocol layer such as `ot_frame` or `protocols`. The type system was available and was not used: `&[u8; 32]` would have made the invariant unbreakable at no cost. And the defence that was chosen is the one class of check the crate's own build configuration removes, which is the same root cause CVF-31 identifies in `pad_to_block`'s `debug_assert!(block_size > n)` — two security-relevant preconditions in this crate are now guarded by assertions that do not exist in the shipped binary.

Recommendation

Make the length invariant a type-level one, which removes the failure mode rather than detecting it:

```
#[inline(never)]
fn ct_eq_tag(a: &[u8; TAG_SIZE], b: &[u8; TAG_SIZE]) -> bool { ...
  }
```

All three tag call sites can supply that type directly: `calc_tag` is already `[u8; 32]`, and `recv_tag` becomes `ciphertext[split..].try_into().map_err(...)?` — which is a genuine improvement, because it replaces an invariant maintained by arithmetic at three separate sites with one the compiler checks. Give the nonce comparison in `generate_nonce_with_crng_check` its own `[u8; NONCE_SIZE]`-typed comparator, or, since both of its operands are public values that appear in the clear on the wire, a plain `==`; CVF-35 makes the same point and notes that using the constant-time comparator there obscures which comparisons are security-relevant.

If the slice signature must be retained for generality, then fail closed before the loop and do so in every build:

```
if a.len() != b.len() {
    return false;
}
```

An early `return false` on a length mismatch leaks only the lengths, which are public in every current and foreseeable use, and it is the standard shape — `subtle`'s `ConstantTimeEq` for slices does exactly this. Do not leave the `debug_assert!` as the only check: keep it if wanted, but it must not be load-bearing.

Correct the doc comment in the same change. "Always 32 iterations" should say that the iteration count is the fixed length of the tag type, and the comment should state explicitly that the comparison is constant-time in the tag *contents* and not in the tag *length*, since that is the distinction a reader needs before reusing the function.

More broadly, CVF-12 already recommends replacing the hand-rolled `read_volatile`/`write_volatile` construction with `subtle`'s `ConstantTimeEq`. Doing so resolves this finding as a side effect, because `subtle`'s slice implementation length-checks and fails closed. If that migration is planned, this finding can be closed by it rather than fixed separately — but until it lands, the `[u8; TAG_SIZE]`

signature is a two-line change that removes the hazard now.

Finally, enable `[profile.release]` with `debug-assertions` or at minimum `overflow-checks`, per CVF-23. That does not fix this finding, but it is what would have turned both this and CVF-31's `pad_to_block` precondition from silent into loud.

Client Comment

/Cargo.toml

```
6 [dependencies]
7 hmac = "0.12"
8 sha2 = "0.10"
```

/src/lib.rs

```
56         if ct_eq_bytes(&nonce, &prev_nonce) {
```

same file

```
622 /// Compare two equal-length byte slices in constant time.
```

same file

```
635 /// The resulting assembly is a fixed-count loop: `jne` branches only
on the
636 /// loop counter (always 32 iterations), never on the tag data.
```

same file

```
637 #[inline(never)]
638 fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
639     debug_assert_eq!(a.len(), b.len());
```

same file

```
640     let mut diff = 0u8;
641     for (x, y) in a.iter().zip(b.iter()) {
642         unsafe {
643             diff |= std::ptr::read_volatile(x) ^
std::ptr::read_volatile(y);
```

same file

```
649     diff == 0
```

same file

```
847     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
949     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

same file

```
1218     if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
```

CVF-46: open Category: **architecture**

The token-emission loop is hand-copied into five encryption entry points and has already diverged between the copies, and within each copy both branches repeat the same two statements

Five public or crate-visible encryptors each contain their own copy of the NAPQES emission loop: `encrypt` at `lib.rs:467`, `encrypt_bytes` at `lib.rs:730`, `encrypt_bytes_with_nonce` at `lib.rs:785`, `encrypt_raw` at `lib.rs:886`, and `encrypt_bytes_v8_with_profile` at `lib.rs:1136`. The four v7 copies are line-for-line identical apart from two local variable names:

```
for &c in &padded {
    loop {
        if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
            let k = key[(real_idx % kk) as usize];
            let nc = derive_noise_char(&kb, &nonce, ct_pos);
           let na = derive_noise_token_addend(&kb, &nonce, ct_pos, k);
            cypher.push(nc * k + na);
            ct_pos += 1;
        } else {
            let k = key[(real_idx % kk) as usize];
            let addend = derive_addend(&kb, &nonce, real_idx, k);
            cypher.push(c as u64 * k + addend);
            ct_pos += 1;
            real_idx += 1;
            break;
        }
    }
}
```

Three of the four are followed by an identical sealing sequence as well — `fixed_encode_tokens`, `varint_keystream`, the XOR mask, the `nonce || masked` payload and `compute_auth_tag(..., AAD_LEN_WIDTH_V7)` — at `lib.rs:761-769`, `lib.rs:820-828` and `lib.rs:917-925`.

The copies have already diverged, and the divergence is the substance of CVF-17. The fifth copy, in `encrypt_bytes_v8_with_profile`, is the same loop with a noise-run counter added:

```
if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt, &nonce,
  ct_pos, noise_theta) {
```

and a `noise_run += 1` in the noise branch. The four v7 copies have neither, which is precisely what CVF-17 reports: the legacy emission loop applies neither the noise-run cap nor the token ceiling, so its length is a realisation of the noise schedule rather than a fixed function of the padded block. That defect did not arise from a wrong line of reasoning about v7. It arose because a fix was applied to one of five copies of the same code, which is the failure mode this finding is about. `derive_noise_char` and `derive_noise_token_addend` are shared, and the loop that calls them is not.

Within each copy, the two branches repeat the same work. `let k = key[(real_idx % kk) as usize];` is the first statement of both the noise branch and the real branch, and `ct_pos += 1;` is the last operation both perform before diverging — `lib.rs:482` and `lib.rs:488`, `lib.rs:486` and `lib.rs:491`, with the same pairing in each of the other four copies. Both can be hoisted, which shortens every copy by two lines and removes two more places where a future edit can reach one branch and not the other.

The loop's shape does not say what it does. Written as an unconditional `loop` with a `break` in the `else` arm, it reads as "emit either a noise token or a real token", whereas the actual behaviour is "emit zero or more noise tokens, then exactly one real token". The distinction is not cosmetic here: the number of noise tokens per real token is the quantity CVF-17 is about, the quantity the v8 cap bounds, and the quantity the ceiling loop at `lib.rs:1179-1186` afterwards pads out. A reader checking whether the cap is applied has to establish the loop's trip structure first, in each of five places.

No ciphertext is wrong today as a result of any of this — the four v7 copies agree with each other, and the KAT corpus covers two of the five entry points. It is filed because the crate's Section 10 claim is byte-parity across three language ports, and a construction maintained as five hand-synchronised copies in one port is the weakest link in that claim: the next fix applied to the wrong subset of them produces a silent wire-format divergence between two entry points of the same library, exactly as the v7/v8 noise cap already has.

Recommendation

Extract the emission loop once and call it from all five entry points.

The five sites differ in four parameters only — the derivation key (`kb` for v7, `sk_fmt` for v8), the noise predicate (`is_noise_pos` against an `f64` probability, or `is_noise_pos_v8` against an integer threshold), whether a noise-run cap applies, and whether the ceiling pad follows. That is one function with a small parameter object, not five functions:

```
struct EmitParams<'a> {
    kb: &'a [u8],
    nonce: &'a [u8],
    key: &'a [u64],
    noise: NoiseRule,          // V7 { p: f64 } | V8 { theta: u64,
  cap: u64 }
}

fn emit_tokens(padded: &[u32], p: &EmitParams) -> Vec<u64> {
    let kk = p.key.len() as u64;
    let (mut real_idx, mut ct_pos) = (0u64, 0u64);
    let mut cypher = Vec::new();
```

```
    for &c in padded {
        let mut noise_run = 0u64;
        // Hoisted: identical in both former branches.
        while p.noise.is_noise(p, ct_pos, noise_run) {
            let k = p.key[(real_idx % kk) as usize];
            let nc = derive_noise_char(p.kb, p.nonce, ct_pos);
            let na = derive_noise_token_addend(p.kb, p.nonce,
  ct_pos, k);
            cypher.push(nc * k + na);
            ct_pos += 1;
            noise_run += 1;
        }
        let k = p.key[(real_idx % kk) as usize];
        let addend = derive_addend(p.kb, p.nonce, real_idx, k);
        cypher.push(c as u64 * k + addend);
        ct_pos += 1;
        real_idx += 1;
    }
    cypher
}
```

Note that the `while` form makes the trip structure explicit — zero or more noise tokens, then exactly one real token — so the restructure and the deduplication are the same edit, and the hoisted `let k` and `ct_pos += 1` fall out of it. Extract the v7 sealing tail (`fixed_encode_tokens` through `compute_auth_tag`) as a second helper on the same principle; it is three identical copies.

Sequence this against the v7 retirement rather than duplicating the work. CVF-17 and CVF-28 both recommend withdrawing the v7 encryptors, and CVF-22 notes that the v7 surface is the one carrying the reachable panics. If v7 is being retired in this release, deleting four of the five copies is a better answer than unifying them, and this finding closes with CVF-17. If v7 is being retained for byte compatibility, the extraction should happen first, because the noise-cap divergence shows that keeping five copies synchronised by hand is not something this codebase has managed to do.

Whichever route is taken, add a test that pins the v7 entry points against each other on a shared input — `encrypt_bytes`, `encrypt_raw` on the same bytes, and `encrypt_bytes_with_nonce` on a fixed nonce must produce the same token stream. CVF-25 records that `encrypt_raw` and `encrypt_str` have no KAT coverage at all, so there is currently nothing that would notice if one copy were edited and the others were not.

Client Comment

/src/lib.rs

```
pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
Vec<u64>) {
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let noise_c = derive_noise_char(&kb, &nonce, ct_pos);
               let noise_add = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(noise_c * k + noise_add);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
```

```
        }
    }
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
pub(crate) fn encrypt_bytes_with_nonce(
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
```

```
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    payload
```

same file

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
    for &c in &padded {
        loop {
            if is_noise_pos(&kb, &nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(&kb, &nonce, ct_pos);
                let na = derive_noise_token_addend(&kb, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(&kb, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
pub fn encrypt_bytes_v8_with_profile(
```

same file

```
    for &c in &padded {
        let mut noise_run: u64 = 0;
```

```
        loop {
            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(&sk_fmt, &nonce, ct_pos, noise_theta) {
                let k = primes[(real_idx % kk) as usize];
                let nc = derive_noise_char(&sk_fmt, &nonce, ct_pos);
                let na = derive_noise_token_addend(&sk_fmt, &nonce, ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
                noise_run += 1;
            } else {
                let k = primes[(real_idx % kk) as usize];
                let addend = derive_addend(&sk_fmt, &nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
```

CVF-47: open Category: **procedural**

The FIPS power-on self-test gate is documentation only: nothing records whether the self-tests ran, and every cryptographic entry point works identically if they never did or if they failed

The self-test module opens with an operator contract stated as a requirement:

> Call `run_power_on_self_tests()` before using the module in production. It returns `Ok(())` only if all tests pass. On any failure it returns `Err(SelfTestError)` — the caller MUST NOT perform any cryptographic operations after a self-test failure.

and closes with the citation it is written against, NIST SP 800-140B §4.9.1. The contract has no implementation behind it. `run_power_on_self_tests()` is a free function that returns a `Result` and touches no shared state; `pub mod self_test;` at `lib.rs:19` is the module's only connection to the rest of the crate, and no entry point references it. We checked every public encrypt and decrypt path: `encrypt_bytes`, `encrypt_str`, `encrypt_raw`, `encrypt_bytes_v8`, `encrypt_bytes_v8_with_profile` and their decrypt counterparts each proceed identically in all four of the states the contract distinguishes — self-tests never called, called and passed, called and failed, and called and failed with the error discarded by the caller.

Three consequences follow.

The MUST is unenforceable by the party it binds. A caller who follows the instruction and a caller who never discovers it obtain the same behaviour from the same API, and nothing in any signature, type or doc comment on the entry points themselves records that a precondition exists. The requirement lives in the documentation of a module a consumer has no reason to open.

A failed self-test does not stop the module. This is the part that matters under the standard the module cites. FIPS 140-3 requires a module that fails a self-test to enter an error state in which all cryptographic operations and data output are inhibited; it does not permit the module to continue operating while reporting the failure to a caller who may ignore it. As implemented, a `SelfTestError` is an ordinary return value. A deployment that logs it and carries on is indistinguishable, from the library's side, from one that halts — and the KAT-1 and KAT-2 failures the suite is designed to catch are exactly the conditions under which continuing to encrypt is most harmful.

The gap compounds with what the self-tests actually cover. CVF-18 establishes that all three known-answer tests exercise the legacy v7 format and none touches the v8 scheme the paper specifies, and that INT-1 is a version-string comparison rather than an integrity check. Those findings concern the *content* of the gate. This one concerns its *existence*: even a complete and correct self-test suite would not gate anything here. The two are independent — fixing either leaves the other — and both are needed before the module's SP 800-140B claim holds. The same applies to `pub mod kem`, which CVF-18 notes has no self-test of its own and which would likewise not be gated by one if it had.

Nothing in the paper's security argument depends on the self-tests, and no theorem is affected. The finding is against the module's own advertised FIPS posture, which is asserted in the crate's documentation and is the basis on which an operator would deploy it in a regulated setting.

Recommendation

Make the gate real by holding the self-test outcome in module state and consulting it on the cryptographic paths.

```
use std::sync::atomic::{AtomicU8, Ordering};

const POST_NOT_RUN: u8 = 0;
const POST_PASSED:  u8 = 1;
const POST_FAILED:  u8 = 2;

static POST_STATE: AtomicU8 = AtomicU8::new(POST_NOT_RUN);

pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
    let r = run_all();
    POST_STATE.store(
        if r.is_ok() { POST_PASSED } else { POST_FAILED },
        Ordering::SeqCst,
    );
    r
```

```
}

/// Returns `Err` unless the power-on self-tests have run and
  passed.
pub(crate) fn require_post() -> Result<(), String> {
    match POST_STATE.load(Ordering::SeqCst) {
        POST_PASSED => Ok(()),
        POST_FAILED => Err("Module is in the self-test error state;
  \
                            cryptographic operations are
  inhibited.".into()),
        _           => Err("Power-on self-tests have not been
  run.".into()),
    }
}
```

with `require_post()?` as the first line of each public encrypt and decrypt entry point. Once `POST_FAILED` is latched it must not be clearable except by a fresh `run_power_on_self_tests()`, so that the error state is sticky in the way the standard requires.

Decide deliberately between inhibiting and warning, and say which in the documentation. Inhibiting — returning `Err` — is what SP 800-140B §4.9.1 calls for and is the right default for a module that advertises conformance. Warning is weaker than the standard requires and should be chosen only if the crate is also going to stop claiming §4.9.1; the one option that should not survive is the present arrangement, where the documentation states the strong requirement and the code implements neither.

Two secondary points. If inhibiting by default would break existing embedders, gate it behind a `fips` Cargo feature that is on by default — CVF-23 notes the manifest declares no feature gates at all, and this is a natural first one; a crate compiled without the feature should then not carry the SP 800-140B text. And whichever route is taken, `pub mod kem` needs the same treatment as the AEAD entry points, since it is inside the module boundary the self-test documentation describes.

This change is worth sequencing after CVF-18, not before. Turning on a gate whose three KATs all exercise the legacy v7 format would inhibit the module on the strength of tests that say nothing about the v8 scheme, and INT-1 in its current form would inhibit it outright at the first version bump past `0.1.0`.

Client Comment

/src/lib.rs

```
19 pub mod self_test;
```

same file

```
730 pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
886 pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

same file

```
1120 pub fn encrypt_bytes_v8(
```

same file

```
1136 pub fn encrypt_bytes_v8_with_profile(
```

/src/self_test.rs

```
1 //! FIPS 140-3 power-on self-tests for the NAPQES Cryptographic
Module.
2 //!
3 //! Call `run_power_on_self_tests()` before using the module in
production.
4 //! It returns `Ok(())` only if all tests pass.  On any failure it
returns
5 //! `Err(SelfTestError)` — the caller MUST NOT perform any
cryptographic
6 //! operations after a self-test failure.
```

same file

```
8 //! Tests performed:
9 //!   KAT-1  Encrypt a known plaintext → compare to reference
ciphertext.
10 //!   KAT-2  Decrypt the reference ciphertext → compare to original
plaintext.
11 //!   KAT-3  Decrypt a tampered ciphertext → confirm authentication
failure.
12 //!   INT-1  Software integrity check via embedded compile-time build
hash.
```

same file

```
27 //! Reference: NIST SP 800-140B §4.9.1 (power-on self-tests).
```

5.3. Minor

CVF-19: fixed Category: **procedural**

Four tests in the audited surface cannot fail: two KAT-parity tests report success when they have exercised zero vectors, one of them by explicit design, and two unit tests make assertions that are true independently of the code under test.

Section 10 offers the known-answer corpus as the evidence that the ports agree, and Section 8.2 lists the Rust unit tests as evidence of implementation care. Read as source, four of those tests are incapable of reporting a failure.

The positive-decrypt KAT guards against a vacuous run with a conjunction that defeats the guard:

```
    // Fail if ZERO vectors were tested successfully (indicates a
build problem)
    if tested == 0 && skipped == 0 {
        panic!("No positive vectors were tested — check vector file
path");
    }
```

`skipped` is incremented for every vector whose message is empty. So if the corpus consists entirely of empty-message vectors — or if a field rename makes `v["message"].as_str().unwrap_or("")` yield `""` for every entry, which is the realistic failure mode, since the code falls back to the empty string rather than erroring on a missing field — then `skipped > 0`, `tested == 0`, the guard does not fire, and the test passes having decrypted nothing. Its sibling `positive_encrypt_bytes_deterministic` gets this right with a plain `assert!(tested > 0, "No vectors were tested")`, so the two guards disagree and the weaker one protects the decrypt direction.

The negative KAT declines to guard at all, and says so:

```
    if tested == 0 {
        // Not a hard failure: negative vectors may also hit
SKIP-PHASE2 paths
        eprintln!("PARITY-NOTE: no negative vectors exercised");
    }
```

This is the test that checks tampered ciphertexts are rejected. As written it passes when no tampered ciphertext was ever submitted, emitting a note to stderr that no CI system

will fail on. Between this and the previous case, the v7 parity suite can report a clean run having neither decrypted a valid vector nor rejected an invalid one.

Two unit tests in `lib.rs` assert propositions that hold regardless of the implementation. The first purports to check that the two v8 key components are independently sampled:

```
#[test]
fn v8_key_generation_is_independent() {
    let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
  MAX_KEY_PRIME);
    assert_eq!(primes.len(), 10);
    // sk must not be derivable from key_bytes(primes) via the v7 KDF
  —
    // spot-check it does not equal the v7 HMAC key material shape.
    assert_ne!(sk.to_vec(), key_bytes(&primes));
}
```

`sk` is `SK_SIZE = 32` bytes and `key_bytes` returns $5K$ bytes, here 50. `Vec<u8>` equality compares lengths first, so the assertion is true for every K — $5K = 32$ has no integer solution — and would remain true if `generate_v8_key` were changed to derive `sk` directly from the primes, which is exactly the property the test is named for. The independence of the two draws, which the CVF8/CVF13 security argument rests on, is untested.

The second is `empty_message_roundtrip`, which is discussed under CVF-13: both calls return through their empty-input short-circuits, so the assertion compares `""` to `""` without executing any cryptographic operation.

Two lesser gaps in the same area, noted for completeness rather than filed separately. `primes_are_prime` checks that generated elements are prime but not that they fall below `MAX_KEY_PRIME` and not that they are distinct, which are the two properties CVF-16 and the key-space entropy claim depend on. And both tamper tests flip the final byte, which is always a tag byte; no test tampers the nonce or the masked blob, so the tag's coverage of the blob is never exercised.

The v8 parity tests are, by contrast, correctly guarded — all three end in `assert!(tested > 0, ...)` — and `v8_positive_encrypt_matches_python` compares the full ciphertext byte-for-byte. The defect is confined to the v7 suite and the two unit tests above.

Recommendation

Make each of the four tests capable of failing.

For the two parity guards, replace both with the unconditional form the sibling test already uses, and assert an expected count rather than merely a non-zero one, so that a corpus which silently loses vectors fails:

```
assert!(tested > 0, "No positive vectors were tested");
assert_eq!(tested + skipped, expected_vector_count, "corpus size
  changed unexpectedly");
```

Remove the `skipped` term from the positive-decrypt guard entirely; a skip is not evidence. If the empty-message vector is retained at all, it should be handled by an

explicit assertion about the intended behaviour rather than by a `continue` that increments a counter which then suppresses the guard — and see CVF-13, which recommends removing that behaviour and the vector with it.

For the negative suite, promote the note to a failure. A test asserting that forgeries are rejected must fail when it has rejected nothing; if some negative vectors are legitimately out of scope for the Rust port, filter them explicitly by an `api` or `phase` field and assert a non-zero count of the remainder, which is what the `stream_ae` filter already does elsewhere in the same file.

For `v8_key_generation_is_independent`, test the property rather than the shape. Comparing a 32-byte value to a 50-byte one cannot detect the failure it is aimed at. A test that would: derive what the v7 schedule would have produced and confirm `sk` is not it, over a fixed seed —

```
let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
  MAX_KEY_PRIME);
// sk must not be any HMAC of the prime material under the v7 or
  subkey schedules
assert_ne!(sk, hmac_digest(&key_bytes(&primes), &[]));
assert_ne!(sk, derive_format_subkey(&key_bytes(&primes),
  FORMAT_BLOCK_V8));
```

or, more usefully, a statistical test that two calls with the same `primes` yield different `sk`, which is the observable consequence of independent sampling and would catch an accidental derivation directly.

For `empty_message_roundtrip`, see CVF-13: assert the ciphertext length is the specified 2,928 bytes so the test exercises the tag path.

Two additions worth making while in this file: strengthen `primes_are_prime` to assert `p <= MAX_KEY_PRIME` and pairwise distinctness, and add a tamper test that flips a byte inside the masked blob and one that flips a nonce byte, so that the tag's coverage of both is tested rather than assumed.

Client Comment

/src/kat_cross_check.rs

```
68          let message = vec["message"].as_str().unwrap_or("");
```

same file

```
73          if message.is_empty() {
74              skipped += 1;
75              eprintln!("[SKIP] {} — empty message, no ciphertext", id);
76              continue;
77          }
```

same file

```
    // Fail if ZERO vectors were tested successfully (indicates a build problem)
    if tested == 0 && skipped == 0 {
        panic!("No positive vectors were tested — check vector file path");
    }
```

same file

```
    eprintln!("\nRust KAT deterministic encrypt: {} passed", tested);
    assert!(tested > 0, "No vectors were tested");
```

same file

```
    eprintln!("\nRust KAT negative: {} passed", tested);
    if tested == 0 {
        // Not a hard failure: negative vectors may also hit SKIP-PHASE2 paths
        eprintln!("PARITY-NOTE: no negative vectors exercised");
    }
```

/src/lib.rs

```
        let last = ct.len() - 1;
        ct[last] ^= 0x01;
```

same file

```
    #[test]
    fn empty_message_roundtrip() {
        let k = test_key();
        assert_eq!(decrypt_str(&encrypt_str("", &k, b"").unwrap(), &k, b"").unwrap(), "");
    }
```

same file

```
    #[test]
    fn primes_are_prime() {
        let ps = generate_prime_numbers(10, MIN_KEY_PRIME, MAX_KEY_PRIME);
        assert_eq!(ps.len(), 10);
        for p in &ps {
            assert!(is_prime(*p));
        }
    }
```

same file

```
        let last = ct.len() - 1;
        ct[last] ^= 0x01;
```

same file

```
    #[test]
```

```
    fn v8_key_generation_is_independent() {
        let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
MAX_KEY_PRIME);
        assert_eq!(primes.len(), 10);
        // sk must not be derivable from key_bytes(primes) via the v7
KDF —
       // spot-check it does not equal the v7 HMAC key material shape.
        assert_ne!(sk.to_vec(), key_bytes(&primes));
    }
```

CVF-21: fixed Category: **documentation**

The crate header documents a superseded version and a retired serialisation, and every normative pointer in the file cites a round-1 or round-3 document rather than this paper.

One preliminary, because an earlier revision of this finding drew a conclusion the evidence does not support. The module declaration list ends with

```
pub mod vale;
```

and no `src/vale.rs` and no `src/vale/` directory is present in the delivered snapshot. Three delivered artifacts nonetheless expect the module: the declaration itself, `Cargo.toml`'s `[[bin]] name = "vale_demo"` target pointing at `src/bin/vale_demo.rs`, and that binary's own `use napqes::vale::{...}`. We therefore cannot distinguish a module genuinely missing from the client's source tree from a file lost when the snapshot was assembled, and the balance of evidence favours the second: a tree lacking `vale.rs` could not have built the `vale_demo` binary that was delivered and declared, and this submission is independently known to have been flattened out of a larger repository — both the Rust KAT loaders and the Python cross-language driver resolve paths relative to a `rust/` subdirectory and an enclosing repository root that are not present (CVF-25). We make no claim that the crate fails to compile in the client's own tree, and `src/vale.rs` is out of scope for this review. What remains worth recording is only that the module was not submitted, so nothing in it has been read, and whatever `pub mod vale` exposes is part of the crate's public surface and is unreviewed.

The file header describes an earlier version of the scheme:

> Rust port of `napqes.py` — v6 authenticated EpiCypher.
>
> Wire format (binary): nonce(16) || masked_blob || hmac_sha256_tag(32) where masked_blob = varint_blob XOR varint_keystream(key_bytes, nonce, len).

Three things here are stale. The version is given as v6 where the file implements v7 and v8. "EpiCypher" is a product name that appears nowhere in the paper, which names the scheme NAPQES throughout. And the serialisation is described as a "varint_blob" masked by a "varint_keystream" — the LEB128 encoding that the CVF1 fix retired, and

which the code itself states is gone: the encoder "is removed since nothing in this crate produces LEB128 ciphertexts anymore". The blob is now fixed-width, `fixed_encode_tokens`, and only the function name `varint_keystream` still carries the old term. The header also states that derivations "match the reference Python implementation byte-for-byte, so ciphertexts are interoperable between languages when the same key, nonce, and AAD are used", which no longer describes v8, where the nonce is derived rather than supplied. The `README.md` now delivered repeats all three defects verbatim, which is CVF-24.

More consequentially, every normative pointer in the file cites a document other than this paper:

- `docs/napseq-eprint-v3.tex` is cited four times, for the notation section's prime interval, the noise-probability section, the padding-profiles section, and the padding-profile API.
- `docs/napseq-eprint-preprint.tex` — the round-1 document — is cited as the specification of the V8 key schedule and synthetic nonce, which is the material CVF-11 shows is absent from v4.
- `docs/CAVEATS.md` is cited four times and `docs/SPEC.md` once.

None of these was delivered, and none is `napseq-eprint-v4.tex`. So the code's own account of where its specification lives points two rounds back, and a reader reconciling code against paper has no in-source pointer to the paper. This is the mechanism behind CVF-11: the `0x0B` layer is documented, at length and accurately, in a comment that directs the reader to a superseded document instead of to the current specification, which is how it came to be absent from the latter. The same pattern recurs in the two evidence artifacts delivered this round: `examples/dudect_harness.rs` cites "ROADMAP §5 NF-6" for its pass threshold and `fuzz/fuzz_targets/decode_bytes.rs` describes itself as a v6 target, neither of which can be resolved against anything submitted.

Two dangling format artifacts round this out. `FORMAT_STREAM_AE_V8: u8 = 0x02` is declared `pub` beside the block-mode identifier and is never referenced anywhere in the crate, while `kat_cross_check.rs` filters corpus vectors on `api != "stream_ae"` in three places. The delivered corpus settles what those filters are for: `tests/kat/v6_vectors.json` contains six `api: "stream_ae"` vectors, four positive and two negative, and they are the only vectors in either corpus still carrying the retired LEB128 token encoding. Taken together these confirm that the streaming-AE format still exists in the implementation family and the KAT corpus. That bears on an item this engagement has carried as deferred — the unauthenticated-streaming caveat `CAV-001`, which v3 dropped along with the streaming format and which v4 neither addresses nor contradicts. The code and the corpus both indicate the format did not go away when the caveat did.

And `b128_decode_tokens` remains as `#[allow(dead_code)]`, a decoder for the retired LEB128 format kept "only for potential legacy-format tooling", unreferenced by the public API. Between it, the v7 format and the streaming identifier, the crate carries three format-shaped artifacts that the paper does not specify.

One smaller item: `TOKEN_WIDTH`'s doc comment justifies the 8-byte field as "sized to comfortably hold the largest realistic token", which was the right level of precision before `Remark 3.11` supplied an exact bound. The bound is now $2^{43.33}$ with a margin of over

1.6×10^6, and stating it here would connect the constant to the guarantee it depends on — see CVF-16, where the absence of the corresponding upper bound on key elements makes that margin unenforced.

For balance: `MAX_KEY_PRIME`'s doc comment is accurate and current, giving $|\mathcal{P}| = 579{,}947$ "verified by sieve" and the derived $2^{191.46}$ and $2^{95.73}$ figures, all of which match v4's exact values.

Recommendation

On the `vale` module the action is a delivery one rather than a code one: include `src/vale.rs` in the next submission, or confirm that no such file exists in the source tree, so that the question is settled rather than inferred. Because `pub mod vale` is a public module of the audited crate and `vale_demo` is a declared binary target, whatever the module exposes is part of the crate's public surface and is at present unreviewed.

Retarget every specification pointer at the current paper. Replace the `docs/napseq-eprint-v3.tex` and `docs/napseq-eprint-preprint.tex` citations with `napseq-eprint-v4.tex` and, where the cited material has moved, with the current section or environment label. The pointer at line 1042 is the important one: it directs the reader to the round-1 document for the V8 key schedule, and once CVF-11 is fixed it should name the new `0x0B` definition in the paper instead. Consider a single `//! Specification:` line in the module header naming the paper, version and date, so that the correspondence is stated once rather than inferred from eleven scattered citations. `docs/CAVEATS.md` and `docs/SPEC.md` should either be delivered alongside the code or replaced by paper references, since a comment citing an undelivered document is not checkable. Apply the same treatment to `ROADMAP §5 NF-6` in the dudect harness (CVF-26).

Rewrite the module header to describe what the file implements. It should name NAPQES rather than EpiCypher, state that the file contains two wire formats and which one the paper specifies, describe the blob as fixed-width rather than varint, and drop or qualify the byte-for-byte interoperability sentence, which holds for v7 with a supplied nonce but not for v8. Fix `README.md` in the same change rather than separately — it carries the same three defects, and CVF-24 covers the rest of it.

Resolve the two dangling format artifacts explicitly rather than leaving them as constants. If the streaming-AE format is still supported, it needs a specification — and the `CAV-001` caveat that v3 dropped needs restoring or superseding, since the format it applied to has demonstrably survived the caveat's removal, in the code as a public constant and in the corpus as six live vectors. If it is not supported, remove `FORMAT_STREAM_AE_V8` and the three `stream_ae` filters, and drop the corresponding vectors from the corpus. Similarly, delete `b128_decode_tokens` unless a named tool consumes it; a dead decoder for a retired format is a liability in a cryptographic crate, and `#[allow(dead_code)]` suppresses the warning that would otherwise prompt its removal.

Finally, restate `TOKEN_WIDTH`'s justification in terms of `Remark 3.11`'s bound rather than "the largest realistic token", and cross-reference the key-element upper bound that the guarantee requires.

Client Comment

/Cargo.toml

```
[[bin]]
name = "vale_demo"
path = "src/bin/vale_demo.rs"
```

/README.md

```
# napqes (Rust)

Rust port of the v6 authenticated EpiCypher in `../napqes.py`.
```

same file

```
Wire format (interoperable with Python):

```

nonce(16) || varint_blob || hmac_sha256_tag(32)
```
```

/src/kat_cross_check.rs

```
        .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "positive" && v["api"] !=
"stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "negative" && v["api"] !=
"stream_ae")
```

/src/lib.rs

```
//! Rust port of `napqes.py` — v6 authenticated EpiCypher.
//!
//! Wire format (binary): nonce(16) || masked_blob ||
hmac_sha256_tag(32)
//! where masked_blob = varint_blob XOR varint_keystream(key_bytes,
nonce, len).
//! String wrapper: base64(binary).
//!
//! All HMAC derivations (noise positions, real-token addends, noise
chars,
```

```
//! noise-token addends, noise probability, padding codepoints, varint
//! keystream, and auth tag) match the reference Python implementation
//! byte-for-byte, so ciphertexts are interoperable between languages when
//! the same key, nonce, and AAD are used.
```

same file

```
pub mod vale;
```

same file

```
/// `docs/napseq-eprint-v3.tex` §Notation. `P` contains exactly 579_947
```

same file

```
/// integers (docs/napseq-eprint-v3.tex, Section "Noise Probability").
```

same file

```
/// (docs/napseq-eprint-v3.tex, Section "Padding Profiles").
```

same file

```
// ─── Base-128 varint (retired for new ciphertexts — see CVF1 fix below) ──────
// The variable-length LEB128 encoding gave each token a byte-length that grew
// with its magnitude (token = codepoint * key_element + addend), so the
// serialised blob length leaked plaintext content even between messages of
// equal padded length, breaking the IND-CPA hiding argument (audit finding
// CVF1). `encrypt_bytes`/`encrypt_raw` now use `fixed_encode_tokens` instead.
// The encoder is removed since nothing in this crate produces LEB128
// ciphertexts anymore; the decoder is kept only for potential legacy-format
// tooling and is currently unused by the public API.
```

same file

```
#[allow(dead_code)]
fn b128_decode_tokens(data: &[u8]) -> Result<Vec<u64>, String> {
```

same file

```
/// Width in bytes of each fixed-width token field, sized to comfortably hold
/// the largest realistic token (codepoint * key_element + addend).
const TOKEN_WIDTH: usize = 8;
```

same file

```
// See `docs/napseq-eprint-preprint.tex` (new subsection, "V8 Key Schedule
// and Synthetic Nonce") and `docs/CAVEATS.md` (CVF3/CVF8/CVF13 follow-ups)
// for the full specification and updated security argument.
```

same file

```
pub const FORMAT_BLOCK_V8: u8 = 0x01;
/// Domain `0x0B` format-subkey identifier for the v8 streaming-AE format.
pub const FORMAT_STREAM_AE_V8: u8 = 0x02;
```

same file

```
/// (docs/napseq-eprint-v3.tex, Section "Padding Profiles").
```

/tests/kat/v6_vectors.json

```
  "spec_version": "v6",
```

CVF-26: open Category: **procedural**

Section 8.4 reports a dudect study of the C port in an undelivered attestation file and does not mention the Rust harness that was delivered, so the constant-time claim for the one port with a hand-rolled comparator rests on no reported measurement; and the harness, though carefully built, times the whole `decrypt_bytes` call, burying a tens-of-cycles signal inside an HMAC over several kilobytes.

Section 8.4 closes its treatment of timing with:

> A dudect-style timing study of the C port is reported in `docs/DUDECT_ATTESTATION.md`; it covers the tag comparison only.

The delivered artifact is not that. `examples/dudect_harness.rs` is a Rust harness against `napqes::decrypt_bytes`, and the paper mentions no Rust timing study anywhere; `docs/DUDECT_ATTESTATION.md` has not been delivered. So the paper cites a measurement of one port that we cannot see, and the delivery contains a harness for another port that the paper does not cite and whose results are recorded nowhere — no t-statistic, no sample count, no platform, no attestation. Since the Rust comparator is the hand-rolled one (CVF-12), the port whose constant-time property most needs measuring is the port with no measurement on the record.

The harness itself is well constructed, and that is worth saying plainly. `Class::Left` flips the first tag byte and `Class::Right` the last, so an early-exit comparator would return after one byte in one class and thirty-two in the other — the right discriminator for the property being tested. Both classes fail authentication, so `decrypt_bytes` returns at the

same `return Err` immediately after `ct_eq_bytes` and the two measurement windows are structurally identical up to the comparison. The class is drawn before `run_one`, and the doc comment reasons explicitly about the memory-layout confound, the within-call measurement-order bias and the branch-predictor artefact of the selecting branch. This is more careful than most such harnesses.

Its limits are nonetheless severe, and none is stated.

The measurement window is dominated by work that is not under test. `run_one` wraps the entire `decrypt_bytes` call, which before reaching the comparison performs `key_bytes(key)` and then `compute_auth_tag` — an HMAC-SHA256 over the whole payload. The base ciphertext is produced by `encrypt_bytes` over a 42-character message under a ten-element key, and the v7 path applies no token ceiling, so the payload is several kilobytes. The signal an early-exit comparator would emit is on the order of thirty-one byte comparisons, tens of cycles, inside a window of tens of thousands of cycles that carries its own variance. The harness's stated criterion, $|t| < 4.5$, is therefore satisfiable by a genuinely leaky comparator at any realistic sample count, which makes a null result weak evidence rather than strong. The note that batches below 200 k are noisy and that 2 M measurements should be accumulated addresses sample size, but the problem here is the signal-to-noise ratio, which more samples improve only as $\sqrt{n}$ against a constant-factor handicap of three or four orders of magnitude.

It cannot reach any behaviour Section 8.4 concedes is not constant-time. That section's "What is not" list has five items: loop trip counts that vary with the realised noise pattern, data-dependent division latency by a key element k, a padding loop whose cost depends on the exact codepoint count n, the Python reference's absence of any guarantee, and the structural checks at `Dec` steps (6) and (8) which "reject at different points and in different time". Because both of the harness's classes fail authentication, the call returns before `varint_keystream`, `fixed_decode_tokens` and `decrypt_core` are entered, so no token-inversion work and no division by k is ever inside the window; and the padding loop is on the encryption side, which the harness does not time at all. The harness measures exactly the one operation the paper already asserts is constant-time and none of the five it concedes are not.

It targets the legacy format. `decrypt_bytes` is the v7 entry point. `decrypt_bytes_v8` calls the same `ct_eq_bytes`, so the comparator result carries over, but the v8 path is the one whose post-authentication checks Section 8.4's fifth item names, and none of that is measured. This is the same misdirection CVF-22 records for the fuzz harness: both of the Tier-2 evidence artifacts in this submission target the wire format the paper disclaims, and neither exercises the one it specifies.

It measures a build in which the comparator's only length check is absent. The harness requires `--release`, and with no `[profile]` section in the manifest that build has `debug-assertions = false` (CVF-23), so the `debug_assert_eq!(a.len(), b.len())` in `ct_eq_bytes` is not in the binary under measurement. That is the right build to measure, and the consequence is worth recording: the harness confirms the equal-length timing behaviour of a comparator whose unequal-length behaviour is unchecked and, per CVF-12, fails open.

Two smaller points. The threshold is cited to "ROADMAP §5 NF-6", another document not delivered, which is the class of unresolvable pointer CVF-21 records throughout the crate.

And the harness is an `example` rather than a test or a bench, so nothing gates a build on $|t| < 4.5$; the criterion exists in a comment and is enforced by whoever remembers to run it.

Recommendation

Narrow the measurement window to the operation under test. The property being checked is that comparison timing does not depend on the position of the first differing byte, and that is a property of thirty-two bytes of work; timing an HMAC over several kilobytes around it discards most of the resolution. Expose the comparator to the harness behind a test-only gate and measure it directly:

```
#[doc(hidden)]
#[cfg(feature = "ct-bench")]
pub fn __ct_eq_bytes_for_bench(a: &[u8], b: &[u8]) -> bool {
  ct_eq_bytes(a, b) }
```

with the two classes differing only in which byte of a 32-byte operand is flipped. Keep the existing end-to-end harness as a second, coarser check if you like — it does test that nothing else in `decrypt_bytes` leaks the tag position — but the primary measurement should be the one with the signal in it.

Report a result. A harness with no attestation is not evidence. Publish, for each class pair, the platform and CPU, the compiler and flags, the sample count, the observed t and the decision against the $|t| < 4.5$ threshold, and cite that in Section 8.4 in place of, or alongside, the C attestation. Deliver `docs/DUDECT_ATTESTATION.md` as well: it is the document the paper's only timing claim rests on, and it is not in the submission.

Correct Section 8.4 to describe what exists. It should say that a Rust dudect harness covering the tag comparison is provided, give its result, and state that the C study is separate — as written, a reader concludes that only the C port was measured and that the Rust port's comparator, which the same section wrongly attributes to `subtle` (CVF-12), needed no study. State the harness's scope explicitly too: that it covers the tag comparison on the v7 decryption path, that both classes fail authentication and so the token-inversion and division behaviour of the "What is not" list is outside the measurement, and that encryption-side timing including the padding loop is not measured at all.

Add a v8 harness, or say why the v7 one suffices. The comparator is shared, so a short sentence explaining that the tag-comparison result transfers would do — but the structural checks at `Dec` steps (6) and (8) are v8-only, new in v4, and named by Section 8.4 as rejecting in different time. If the argument that this is not an outside adversary's oracle is to be relied on, it should be made where the harness's scope is stated, not left to the reader.

Move the harness where a build can enforce it. As a `#[bench]`, or an example invoked by a CI job that fails on $|t| \geq 4.5$, the threshold becomes a gate; as an example with the threshold in a comment, it is documentation. Given the crate's FIPS framing, a regression in the comparator is exactly the kind of change that should not pass silently.

Finally, replace the "ROADMAP §5 NF-6" citation with a reference to the paper's own Section 8.4, or deliver the ROADMAP, in step with the pointer cleanup CVF-21 recommends.

Client Comment

/examples/dudect_harness.rs

```
1  //! Constant-time timing analysis for `napqes::decrypt_bytes`.
```

same file

```
3  //! Run (release mode required for meaningful timings):
4  //!   cargo run --example dudect_harness --release -- --continuous
   bench_tag_comparison
```

same file

```
6  //! The test compares decryption timing when the HMAC-SHA256 tag
   differs at the
7  //! FIRST byte (Class::Left) versus the LAST byte (Class::Right).  A
   naive
8  //! byte-by-byte comparator would exit early on Left (faster),
   producing a
9  //! detectable t-statistic.  A correct constant-time implementation
   scans all 32
10 //! bytes unconditionally, so both classes should be
   timing-indistinguishable.
```

same file

```
12 //! Target: |t| < 4.5 (TVLA threshold per ROADMAP §5 NF-6).
```

same file

```
14 //! Note: allow the benchmark to accumulate at least 2 M measurements
   before
15 //! drawing conclusions.  Early batches (n < 200 k) are noisy.
```

same file

```
32 use napqes::{decrypt_bytes, encrypt_bytes, TAG_SIZE};
```

same file

```
47 /// A single valid ciphertext produced once at startup.  Both classes
   derive
48 /// from this by cloning (allocator reuses the same heap slot between
   calls)
49 /// and corrupting one tag byte in-place.
50 fn base_ct() -> &'static Vec<u8> {
```

same file

```
53         encrypt_bytes(
```

```
            "napqes dudect constant-time timing harness",
            fixed_key(),
            b"",
        )
```

same file

```
fn bench_tag_comparison(runner: &mut CtRunner, rng: &mut BenchRng) {
    let key = fixed_key();
    let base = base_ct();
    let n = base.len();
```

same file

```
    let mut ct = base.clone();
```

same file

```
    if rng.gen::<bool>() {
        // Left: tag byte 0 wrong (early-exit comparator fails immediately)
        ct[n - TAG_SIZE] ^= 0xFF;
        runner.run_one(Class::Left, || decrypt_bytes(&ct, key, b""));
```

same file

```
    } else {
        // Right: tag byte 31 wrong (early-exit comparator scans all 31 bytes)
        ct[n - 1] ^= 0xFF;
        runner.run_one(Class::Right, || decrypt_bytes(&ct, key, b""));
    }
```

/src/lib.rs

```
#[inline(never)]
fn ct_eq_bytes(a: &[u8], b: &[u8]) -> bool {
    debug_assert_eq!(a.len(), b.len());
```

same file

```
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
```

CVF-29: open Category: **flaw**

`validate_key` re-tests every key element for primality by trial division on every v8 encrypt and decrypt call, so per-message timing is a monotone function of each secret

prime's magnitude — and Section 8.4's inventory of what is not constant-time omits key validation entirely.

`validate_key` runs on **every** call to `encrypt_bytes_v8_with_profile` and `decrypt_bytes_v8`, and for each key element it calls `is_prime`, which is trial division:

```
let mut i: u64 = 3;
while i.saturating_mul(i) <= n {
    if n % i == 0 { return false; }
    i += 2;
}
```

For a genuine prime the loop never returns early, so it runs to completion and the iteration count is $\lfloor(\lfloor\sqrt{k}\rfloor - 3)/2\rfloor + 1$ — a direct, monotone function of the secret key element. Across the normative interval that is **499 iterations at** $k = 1{,}000{,}003$ **and 1,572 at** $k = 9{,}899{,}999$, a $3.15\times$ spread, and at the default $K = 10$ the total ranges over roughly **4,990 to 15,720** 64-bit division instructions per message. An attacker who can time encryption or decryption therefore learns approximately $\sum_i \sqrt{k_i}$ once per message, for free: no ciphertext manipulation is needed, the leak is present on the encrypt side as well as the decrypt side, and it is independent of everything the noise loop does.

The divergence is with **Section 8.4** rather than with Section 3. Section 8.4's "What is not constant-time" paragraph is an explicit inventory, and it is careful and honest as far as it goes — it lists the token-emission and token-inversion loop counts, the integer division by a key element, the padding loop's dependence on the exact codepoint count, the Python port, and the differing rejection points of `Dec` steps (6) and (8). **Key validation is not among the five**, and it is not in Section 12's timing caveat either. So the paper's stated inventory of non-constant-time operations is incomplete in the one place where the leaked quantity is key material rather than a public or nonce-derived value. Every item the paper does list leaks something about (sk, N) or about n; this one leaks **k** directly.

Stated honestly, the security consequence is bounded. Recovering $\lfloor\sqrt{k_i}\rfloor$ for each element would cut $H_\infty(\mathbf{k})$ from 191.46 bits to roughly the log of the number of primes per $\sqrt{\cdot}$ bucket summed over the tuple — a large reduction — but `Remark 3.3` and Section 6 are explicit that no theorem depends on $H_\infty(\mathbf{k})$, and Section 12 concedes that **k**'s secrecy "adds **no** independent confidentiality margin once sk is known". The channel does not touch sk. So this is primarily a documentation-completeness failure sitting on a real but non-catastrophic channel.

It is also gratuitous. Re-deriving the primality of a fixed, caller-supplied key on every single message is pure waste: the property is immutable for the lifetime of the key, and the check costs between five thousand and sixteen thousand divisions per message to re-establish something that was already true.

CVF-16 recommends the right remedy for a different reason — it asks for validation to move off the hot path into a validated key type, motivated by the denial-of-service cost of trial-dividing a large caller-supplied prime. Neither CVF-16 nor any other finding identifies the timing channel or the Section 8.4 omission, and the corrective action on the paper side (extend the Section 8.4 list) is separate from the corrective action on the code side, which is why this is filed rather than folded in.

Recommendation

Validate once, at key construction, and record the channel in Section 8.4 in the meantime.

On the code side, introduce a validated key type so that primality is established when the key enters the system and never again:

```
pub struct NapqesKey(Vec<u64>);

impl NapqesKey {
    pub fn new(primes: Vec<u64>) -> Result<Self, String> {
        validate_key(&primes)?;
        Ok(NapqesKey(primes))
    }
}
```

with `encrypt_bytes_v8*` and `decrypt_bytes_v8` taking `&NapqesKey` instead of `&[u64]`. That removes the per-message trial division entirely, which closes this channel, removes the per-message cost, and is the same change CVF-16 asks for on denial-of-service grounds — so one refactor closes both. If the API cannot change in this release, at minimum hoist `validate_key` out of the per-message path in the internal call graph and document that callers must validate once.

Note that a constant-time primality test is *not* the right fix here and should not be attempted. The correct answer is not to test in constant time but not to test repeatedly: the cheapest constant-time primality test is still far more expensive than doing the check once, and a Miller-Rabin variant would introduce a probabilistic failure mode into what is currently a total function.

On the paper side, add a sixth item to Section 8.4's "What is not constant-time" list, in the same register as the existing five: key validation performs trial division to $\sqrt{k}$ on each key element on every encrypt and decrypt call, so the iteration count is a monotone function of each secret prime and an attacker with a timing channel learns approximately $\sum_i \sqrt{k_i}$ per message. State the mitigating context the paper already establishes elsewhere — that no theorem depends on $H_\infty(\mathbf{k})$ and that Section 12 concedes **k** adds no confidentiality margin once sk is known — so that a reader can size the leak. The list is otherwise the most candid part of the paper, and this is the only omission from it we found; leaving it out is what makes the omission conspicuous. If the validated-key-type change lands first, the item can instead say that key validation is performed once at key construction and is therefore off the per-message path.

Client Comment

/docs/napseq-eprint-v4.tex

```
\paragraph{What is not.} The token-emission and token-inversion loops are
\emph{not} constant-time, and are not intended to be:
\begin{itemize}
```

same file

```
  \item The number of loop iterations depends on the realised noise
        pattern, which is a deterministic function of $(\mathsf{sk},N)$.
        The total ciphertext length is padded to a fixed ceiling
        (Lemma~\ref{lem:length-det}) and so leaks nothing, but the
        \emph{time} to produce or consume it varies with the number of
        noise tokens actually emitted before the run cap engages.
```

same file

```
  \item Both directions use arbitrary-precision or 64-bit integer
        multiplication and division by a key element $k$. Division latency
        is data-dependent on many microarchitectures.
```

same file

```
  \item The padding loop of Section~\ref{sec:padding} performs $B-n$
        derivations, so encryption time depends on the \emph{exact}
        codepoint count $n$, not merely on its padding bucket $B$. The
        padding-bucket caveat below concedes a $\log_2 13\approx3.70$-bit
        length leak through ciphertext \emph{length}; a timing channel can
        in principle recover more than that.
```

same file

```
  \item The Python reference implementation offers no timing guarantees of
        any kind: it uses \texttt{int}, list resizing, and dictionary
        lookups throughout.
```

same file

```
  \item The structural checks added at $\mathsf{Dec}$ steps (6) and (8)
```

```
        (Remark~\ref{rem:dec-structural}) reject at different points and in
        different time. They are reachable only after the tag has verified,
        i.e.\ only by a party already holding $\mathsf{sk}$, so this
        difference is not an oracle for an outside adversary.
```

same file

```
        noise-dependent number of times and perform integer division by a
        key element, both of which are data-dependent on real hardware, and
        the Python reference implementation offers no timing guarantees at
        all. Section~\ref{sec:constant-time} states precisely what is and is
        not covered. Deployments in which an adversary can measure
        encryption or decryption time on inputs it influences should treat
        this as an unresolved gap; a TVLA or dudect study of the full
        encode/decode path, rather than of the tag comparison alone, remains
        future work.
```

/src/lib.rs

```rust
pub fn is_prime(n: u64) -> bool {
    if n < 2 {
        return false;
    }
    if n == 2 {
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
```

same file

```rust
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
```

same file

```
    validate_key(primes)?;
```

same file

```
    validate_key(primes)?;
```

CVF-30: fixed Category: **behavior**

Section 3.4 states without qualification that "no floating-point representation is used anywhere in this construction", but the v7 noise threshold is computed and compared in IEEE-754 double precision on five public code paths.

Section 3.4 opens the noise-probability definition with an unconditional sentence: "The noise probability is realised entirely with fixed-width integer arithmetic; no floating-point representation is used anywhere in this construction." The paragraph then explains why it matters — $\theta(N)$ is computed "with a single 128-bit intermediate product and one integer division, so that $\theta(N)$ is bit-identical across all conforming implementations regardless of platform or language."

The v8 path honours this exactly. `derive_noise_threshold_v8` widens to `u128`, multiplies, shifts right by 64 and narrows, and `is_noise_pos_v8` compares the leading eight digest bytes as a `u64` against the resulting `u64` threshold. We checked the arithmetic end to end: $\theta_{\text{min}} = 13{,}835{,}058{,}055{,}282{,}163{,}712$ (exactly $3 \cdot 2^{62}$, so the floor is exact), $\theta_{\text{max}} = 18{,}262{,}276{,}632{,}972{,}456{,}099$, both below 2^{64} so the `as u64` casts are lossless; $\tau \cdot \Delta < 2^{128}$ so the product cannot overflow; and `>> 64` is precisely the specified floor division. No divergence.

The v7 path does something else entirely:

```
const TWO_POW_64: f64 = 18446744073709551616.0_f64;

fn derive_noise_p(kb: &[u8], nonce: &[u8]) -> f64 {
    ...
```

```
    let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    0.75 + t * (0.99 - 0.75)
}

fn is_noise_pos(kb: &[u8], nonce: &[u8], ct_pos: u64, noise_p: f64)
  -> bool {
    ...
    let v = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    v < noise_p
}
```

The threshold is an `f64`, the comparand is an `f64`, and the comparison is a float comparison. This is live on every v7 entry point: `encrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce`, `encrypt_raw` and `decrypt_core`.

Two mechanisms make the result implementation-dependent rather than merely imprecise. The `u64 as f64` conversion discards the low 11 bits of a 64-bit value, so roughly one draw in 2^{53} that ought to fall on one side of the threshold falls on the other. And `0.99 - 0.75` is a subtraction of two values neither of which is exactly representable in binary, evaluated at whatever precision and in whatever order the compiler chooses; the literal `0.75` is exact but `0.99` is not, and the fused-multiply-add of `0.75 + t * (0.99 - 0.75)` may or may not be contracted depending on target features. That is precisely the class of platform-dependence the sentence at Section 3.4 rules out.

The consequence is interoperability, not security. The threshold is secret and the noise positions are masked by the keystream, so no observable quantity depends on the float result. But two conforming v7 ports built on different targets, or with different optimisation settings, can disagree about whether a given position is a noise position, and because the v7 decoder classifies every position independently using the same predicate, a disagreement produces a ciphertext that does not round-trip. It is exactly the reproducibility defect that the doc comment above `derive_noise_threshold_v8` says was removed — removed for v8, and left in place on v7.

Section 3.4's sentence is unqualified and is stated about "this construction". The paper's Π is v8, and Section 10 does disclaim the v7 corpus as "not the scheme analysed here", so a charitable reading confines the sentence to v8. But the crate ships the float path as public, undeprecated API, and a reader who takes the sentence at face value and then reads the crate will find it contradicted at the first v7 entry point.

This is distinct from CVF-17, which concerns the missing noise-run cap and token ceiling on the same v7 loop, and from CVF-28, which concerns the v7 key schedule and AAD prefix width. The arithmetic used to compute the threshold is a third, independent divergence on that path.

Recommendation

Make the v7 threshold integer-valued, or retire the v7 encryptors.

If the v7 encryptors are being withdrawn under CVF-17 or CVF-28, this finding closes with them and no separate work is needed — note only that `decrypt_core` must keep whatever arithmetic archived v7 ciphertexts were produced under, so a retained v7

decryptor cannot simply switch to the integer path without breaking the ciphertexts it exists to read.

If v7 stays, replace `derive_noise_p` and `is_noise_pos` with the v8 pair. `derive_noise_threshold_v8` and `is_noise_pos_v8` already take the derivation key as a parameter, so they can be called with `kb = key_bytes(key)` unchanged; the only difference between the two paths would then be the derivation key, which is where the difference belongs. Be explicit that this is a wire-format change for v7 — the set of noise positions moves for a small fraction of positions, so existing v7 ciphertexts will not round-trip under the new predicate — and version it accordingly rather than silently changing behaviour.

Independently of which route is taken, qualify the sentence in Section 3.4. "No floating-point representation is used anywhere in this construction" should say what "this construction" means when the reference crate exports two, for example by naming Π explicitly: the scheme specified in this paper realises the noise probability entirely in fixed-width integer arithmetic. The surrounding paragraph's reasoning — bit-identical across platforms and languages — is the whole point of the requirement and is worth keeping in the foreground, because it is what a second implementer will rely on.

Finally, if any float remains anywhere on a derivation path, add it to Section 8.4's inventory alongside the other implementation-behaviour notes, so that the paper's account of what the ports actually do stays complete.

Client Comment

/docs/napseq-eprint-v4.tex

```
The noise probability is realised entirely with fixed-width integer
arithmetic; no floating-point representation is used anywhere in this
construction. Fix the public integer constants
```

same file

```
The per-message noise threshold is the 64-bit integer
\[
  \theta(N) = \theta_{\min} + \left\lfloor \frac{\tau \cdot
(\theta_{\max} - \theta_{\min})}{2^{64}} \right\rfloor,
\]
computed with a single 128-bit intermediate product and one integer
division, so that $\theta(N)$ is bit-identical across all conforming
implementations regardless of platform or language. Every informal
```

/src/lib.rs

```
const TWO_POW_64: f64 = 18446744073709551616.0_f64;
```

same file

```
fn is_noise_pos(kb: &[u8], nonce: &[u8], ct_pos: u64, noise_p: f64)
-> bool {
    let mut buf = Vec::with_capacity(1 + nonce.len() + 5);
    buf.push(0x00);
    buf.extend_from_slice(nonce);
    buf.extend_from_slice(&be5(ct_pos));
    let d = hmac_digest(kb, &buf);
    let v = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    v < noise_p
}
```

same file

```
fn derive_noise_p(kb: &[u8], nonce: &[u8]) -> f64 {
    let mut buf = Vec::with_capacity(1 + nonce.len());
    buf.push(0x02);
    buf.extend_from_slice(nonce);
    let d = hmac_digest(kb, &buf);
    let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
    0.75 + t * (0.99 - 0.75)
}
```

same file

```
fn derive_noise_threshold_v8(kb: &[u8], nonce: &[u8]) -> u64 {
    let mut buf = Vec::with_capacity(1 + nonce.len());
    buf.push(0x02);
    buf.extend_from_slice(nonce);
    let d = hmac_digest(kb, &buf);
    let tau = u64_from_be8(&d[..8]) as u128;
    THETA_MIN + ((tau * (THETA_MAX - THETA_MIN) as u128) >> 64) as u64
}
```

same file

```
fn is_noise_pos_v8(kb: &[u8], nonce: &[u8], ct_pos: u64, theta: u64)
-> bool {
    let mut buf = Vec::with_capacity(1 + nonce.len() + 5);
    buf.push(0x00);
    buf.extend_from_slice(nonce);
    buf.extend_from_slice(&be5(ct_pos));
    let d = hmac_digest(kb, &buf);
    u64_from_be8(&d[..8]) < theta
}
```

same file

```
    let noise_p = derive_noise_p(&kb, &nonce);
```

same file

```
    let noise_p = derive_noise_p(kb, nonce);
```

same file

```
737     let noise_p = derive_noise_p(&kb, &nonce);
```

same file

```
795     let noise_p = derive_noise_p(&kb, &nonce);
```

same file

```
893     let noise_p = derive_noise_p(&kb, &nonce);
```

CVF-31: fixed Category: **over-underflow**

The public `PadProfile::block_size` returns block sizes outside the 13-element set that its own doc comment and Section 3.6 declare invariant, and the private consumer of that value guards its unchecked subtraction with a `debug_assert!` that release builds compile out.

Section 3.6 makes a specific structural promise about the profile family: "All three profiles take values in the same 13-element set, so the set of legal real-token counts $R = B+2$ — and hence the set of legal ciphertext lengths — is identical for all of them." That promise is what lets a decryptor be profile-agnostic and check R against one fixed set. The crate restates it as an invariant in the doc comment on `PAD_MIN_EXP`/`PAD_MAX_EXP`: "Every padding profile takes values in this same 13-element set, so the set of legal token counts is profile-independent and a decryptor never needs to know which profile the sender used."

`PadProfile::block_size` is `pub`, and it does not enforce the upper end of that set. It clamps only the lower bound:

```
let e = bit_length(n).max(PAD_MIN_EXP);
match self {
    PadProfile::Bucket   => Ok(bucket_block_size(n)),          // 1 <<
  bit_length(n).max(4)
    PadProfile::Coarse(g) => { ...; Ok(1usize << (PAD_MIN_EXP + g *
  steps)) }
    PadProfile::Frame(f)  => { /* range-checks both ends */ }
}
```

Neither `Bucket` nor `Coarse` is capped at `PAD_MAX_EXP`. `PadProfile::Bucket.block_size(70_000)` and `PadProfile::Coarse(1).block_size(70_000)` both return `Ok(131_072)`, i.e. 2^{17}, which is outside $\{2^4, \ldots, 2^{16}\}$ and would give $R = 131{,}074$, a real-token count no conforming decryptor accepts at `Dec` step (6). Only `Frame(F)` range-checks both ends.

The shift itself is also unguarded. `bit_length(n)` is `usize::BITS - n.leading_zeros()`, so for $n \geq 2^{63}$ it is 64, and `1usize << 64` is a shift overflow: it panics in a debug build, and in a release build with `overflow-checks` off — the delivered

configuration, since `Cargo.toml` declares no `[profile.release]` section — the shift amount is masked modulo 64 and the expression evaluates to `1`. That yields a block size *smaller* than the message, inverting the one precondition Section 3.6 places on a profile, "$B(n) > n$ throughout the message space".

The consumer of that value assumes the precondition holds and enforces it with an assertion that does not survive release:

```
fn pad_to_block(msg: &[u32], kb: &[u8], nonce: &[u8], block_size:
  usize) -> Vec<u32> {
    let n = msg.len();
    debug_assert!(block_size > n, "padding block must exceed the
  message");
    let pad_len = block_size - n;
    ...
    for i in 0..pad_len {
```

In release, `debug_assert!` is compiled out and with no `overflow-checks` the `usize` subtraction wraps to a value near 2^{64}, which the following `for i in 0..pad_len` loop then attempts to fill with HMAC-derived codepoints — one `Vec` push and one HMAC-SHA256 per iteration.

Reachability, stated precisely, because it bounds the severity. Neither condition is reachable today. `pad_to_block` is private, so no downstream crate can feed it a bad `block_size` directly; the only public route is `encrypt_bytes_v8_with_profile`, and it rejects `codepoints.len() > 0xFFFF` *before* it calls `pad_profile.block_size(...)`, so `n` never exceeds 65,535 and all three profiles then return a B in the legal set with $B > n$. The v7 path calls `bucket_block_size` through `pad_message`, which carries its own `assert!(n <= 0xFFFF)`. So this is a latent defect, not a live one.

What makes it worth filing anyway is that `PadProfile::block_size` is public API whose doc comment tells a caller the result is confined to a 13-element set, and it is not. A downstream crate that reads that invariant and relies on it — to size a buffer, to precompute the legal R set, or to validate a configuration before handing it to `encrypt_bytes_v8_with_profile` — is relying on something the function does not guarantee, and the guarantee is one the paper also makes. The two halves compound: the API can produce an out-of-range or too-small B, and the internal consumer's only defence against a too-small B evaporates in the build configuration that ships.

This shares a root cause with CVF-16 and CVF-23 — no `overflow-checks` in the manifest, so an arithmetic mistake wraps rather than aborts — but the site and the mechanism are distinct: CVF-16 is an unbounded multiplication in the token map arising from a missing key-validation bound, and this is an unbounded subtraction in the padding path arising from a missing profile clamp.

Recommendation

Enforce the invariant where it is declared, and make the precondition a real check rather than a debug-only one.

Clamp both profiles at the top, so that `block_size` cannot return a value outside the 13-element set:

```
pub fn block_size(self, n: usize) -> Result<usize, String> {
```

```
    if n >= 1usize << PAD_MAX_EXP {
        return Err(format!(
            "message of {} codepoints exceeds the largest reachable
  block size {}.",
            n, 1usize << PAD_MAX_EXP
        ));
    }
    let e = bit_length(n).max(PAD_MIN_EXP);
    ...
}
```

The early return also removes the `1usize << 64` shift-overflow path, because `bit_length(n)` can no longer exceed `PAD_MAX_EXP` by the time it reaches either shift. Applying the guard once at the head of `block_size` covers `Bucket` and `Coarse` and is consistent with what `Frame` already does for its own parameter. Note that the check subsumes the caller-side `codepoints.len() > 0xFFFF` gate in `encrypt_bytes_v8_with_profile`, since

$$2^{16} - 1 = \text{MAX_PLAINTEXT_CODEPOINTS}$$

— which is the right place for it, because it puts the bound next to the invariant it protects rather than in one of the two call sites that happen to precede it today.

Promote `pad_to_block`'s `debug_assert!` to a real check. The function returns `Vec<u32>` rather than a `Result`, so the least disruptive form is `assert!`, which fires in release too; changing the signature to `Result<Vec<u32>, String>` and propagating with `?` at the two call sites is better still and matches the error discipline the v8 entry points already use. Either way the subtraction must not be reachable with `block_size <= n` in a release build.

Enable `overflow-checks` in `[profile.release]`, which is CVF-23's recommendation and would turn this and CVF-16's multiplication from silent wraparound into an abort. That is a backstop, not a substitute for the two changes above.

Finally, add a unit test that pins the invariant across the profile family rather than within one profile. The existing `pad_profiles_share_one_reachable_set` test checks membership over the reachable input range; extend it to assert that `block_size` *errors* for every profile at $n = 2^{16}$ and above, so that the upper clamp is covered the way `Frame`'s bounds already are by `pad_profile_frame_is_constant_and_range_checked`.

Client Comment

/Cargo.toml

```
[dependencies]
hmac = "0.12"
```

```
sha2 = "0.10"
```

/docs/napseq-eprint-v4.tex

```
parameter rather than as a hard-wired constant. A \emph{padding profile} is
any function $B(\cdot)$ from codepoint counts to block sizes satisfying
$B(n)>n$ throughout the message space. This paper specifies three:
```

same file

```
All three profiles take values in the same $13$-element set, so the set of
legal real-token counts $R=B+2$ — and hence the set of legal ciphertext
lengths — is identical for all of them. A decryptor consequently needs
```

/src/lib.rs

```
/// Exponent range of the reachable block sizes `{2^4, ..., 2^16}`.
///
/// Every padding profile takes values in this same 13-element set, so the set
/// of legal token counts is profile-independent and a decryptor never needs to
/// know which profile the sender used.
pub const PAD_MIN_EXP: u32 = 4;
pub const PAD_MAX_EXP: u32 = 16;
```

same file

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum PadProfile {
    /// Default: smallest power of two strictly above `n`, floored at 16.
    /// 13 reachable sizes, leaking at most `log2(13) ~= 3.70` bits of length.
    Bucket,
    /// [`PadProfile::Bucket`] thinned by a stride `g` dividing 12, leaving
    /// `12 / g + 1` reachable sizes.
    Coarse(u32),
    /// Every message padded to the single size `F`, leaking exactly zero bits.
    /// Requires `n < F`.
    Frame(u32),
}
```

same file

```
fn bit_length(n: usize) -> u32 {
```

```
    usize::BITS - n.leading_zeros()
}
```

same file

```
fn bucket_block_size(n: usize) -> usize {
    1usize << bit_length(n).max(PAD_MIN_EXP)
}
```

same file

```
    pub fn block_size(self, n: usize) -> Result<usize, String> {
        let e = bit_length(n).max(PAD_MIN_EXP);
        match self {
            PadProfile::Bucket => Ok(bucket_block_size(n)),
            PadProfile::Coarse(g) => {
                let span = PAD_MAX_EXP - PAD_MIN_EXP;
                if g == 0 || span % g != 0 {
                    return Err(format!(
                        "coarse stride g={} must divide {}.",
                        g, span
                    ));
                }
                let steps = (e - PAD_MIN_EXP + g - 1) / g; // ceil
division
                Ok(1usize << (PAD_MIN_EXP + g * steps))
            }
```

same file

```
fn pad_to_block(msg: &[u32], kb: &[u8], nonce: &[u8], block_size:
usize) -> Vec<u32> {
    let n = msg.len();
    debug_assert!(block_size > n, "padding block must exceed the
message");
    let pad_len = block_size - n;
```

same file

```
    for i in 0..pad_len {
```

same file

```
    if codepoints.len() > 0xFFFF {
        return Err("Message too long for 2-byte length
prefix.".into());
    }
```

same file

```
    let block_size = pad_profile.block_size(codepoints.len())?;
```

same file

```
    fn pad_profiles_share_one_reachable_set() {
        let legal: std::collections::HashSet<usize> =
            (PAD_MIN_EXP..=PAD_MAX_EXP).map(|e| 1usize <<
e).collect();
```

CVF-32: fixed Category: **procedural**

Two of the three normative padding profiles have no known-answer vector and no pinned-byte test of any kind, and the vector schema has no field that could express a profile — so the byte-parity requirement of Section 10 covers only the default profile.

Section 3.6 specifies three padding profiles normatively: `bucket`, `coarse(g)` and `frame(F)`. Section 12 actively recommends the third, directing deployments "requiring zero length leakage" to select frame(F). Section 10 then states the conformance bar for the whole scheme: "Conforming implementations must produce byte-identical ciphertext for every positive vector and reject every negative vector."

The corpus cannot discharge that bar for two of the three profiles. Reading `tests/kat/v8_vectors.json` as data, all twelve positive vectors (`W001`–`W012`) are default-`bucket` ASCII messages of 0, 1, 7, 12, 14, 15, 16 and 32 codepoints, with $K \in \{1, 4, 10\}$ and AAD lengths of 0, 8, 12, 16 and 64 bytes. **No vector exercises `coarse(g)` or `frame(F)`.** More fundamentally, the vector schema could not express one if it wanted to: the union of keys across all twenty vectors is `id`, `kind`, `description`, `key`, `sk_hex`, `message`, `aad_hex`, `ciphertext_hex`, `expected_exception` and `tampered_hex`. There is no profile field, and no vector carries one.

The in-crate tests do not close the gap either, because none of them pins a byte string. `pad_profile_bucket_matches_legacy_ladder`, `pad_profiles_share_one_reachable_set`, `pad_profile_coarse_stride_must_divide_twelve` and `pad_profile_frame_is_constant_and_range_checked` all test `block_size` arithmetic and reachable-set membership — properties of the profile function, not of the ciphertext. `v8_frame_profile_hides_length_and_round_trips` and `v8_frame_profile_rejects_oversized_message` test one port against itself: the first asserts that two messages under `frame` produce equal-length ciphertexts that round-trip, which any self-consistent implementation passes, and the second asserts an error condition. Nothing anywhere fixes a ciphertext byte string for a non-default profile.

So two-thirds of the normative profile space has no cross-implementation anchor at all, and the two disagreements most likely to arise are exactly the ones the corpus cannot catch:

- `coarse(g)`'s exponent is $4 + g\lceil (e(n) - 4)/g \rceil$. The Rust port implements the ceiling division as `(e - PAD_MIN_EXP + g - 1) / g`. A second port that rounds differently at a boundary — or that computes $e(n)$ as $\lceil \log_2(n+1) \rceil$ without the $\max(4, \cdot)$ floor, or with a floating-point `log2` — produces a different B for a small set of n, and therefore a different ciphertext length and different padding codepoints.

- `frame(F)`'s admissibility boundary is $n < F$ in Section 3.6 and `n >= f => Err` in the Rust port. A port implementing $n \leq F$ accepts one message length the reference rejects, and for that length produces $B = F$ with **pad** of length zero — a padded buffer with no padding at all, which is outside the encoder's image everywhere else.

Neither divergence is a security defect. Both are silent interoperability breaks between implementations that each pass the delivered suite, in a profile that Section 12 recommends for the deployments with the strongest traffic-analysis requirements. Since the profile is never transmitted and the decryptor is profile-agnostic by design, a sender-side profile disagreement does not surface as a decryption failure — it surfaces as two deployments that believe they have the same length-leakage posture and do not.

This is adjacent to CVF-25 but not the same finding. CVF-25 concerns whether the byte-parity claim can be checked across the three ports at all, given that two of them were not delivered. This concerns the coverage of the corpus itself: even with all three ports in hand, the corpus would pin only one of the three normative profiles.

Recommendation

Add a profile field to the vector schema and generate positive vectors for the two uncovered profiles.

The schema change is small: an optional `"pad_profile"` key, absent or `"bucket"` for the existing twelve vectors so they are unaffected, and `{"coarse": 3}` / `{"frame": 1024}` for the new ones. `tests/gen_kats_v8.py` regenerates the file, and both consumers — `tests/test_kats.py` and `src/kat_cross_check.rs` — need one branch each to select the profile before encrypting. The Rust harness already dispatches on vector fields, so this follows the existing pattern.

Choose the new vectors to pin the two boundaries that actually differ, rather than adding volume:

- For `coarse(g)`, at least one vector at a stride boundary where the ceiling division rounds up and one just below it — for example `coarse(3)` at $n = 16$ and $n = 17$, which straddle $e(n) = 5 \to 6$ and must both land on $B = 128$, and one at $n = 15$, which must land on $B = 16$. Add a `coarse(12)` vector as well, since it is the extreme case Section 3.6 names explicitly and leaves only $\{2^4, 2^{16}\}$ reachable.
- For `frame(F)`, one positive vector at $n = F - 1$, the largest admissible message, and one negative vector at $n = F$ with `expected_exception` set, which pins the strict inequality that Section 3.6 states and the port enforces. Without the negative vector the boundary is not tested in either direction.

Update Section 10's inventory of what the corpus covers to name the profiles, in the same style as the existing sentence listing the message lengths, key sizes and AAD lengths. As written, a reader is told the corpus covers "the block boundaries at 15, 16 and 32 codepoints" and reasonably infers that the profile space is covered too, since Section 3.6 specifies all three normatively and Section 12 recommends one of them.

If generating cross-port vectors for the non-default profiles is not practical in this release, then say so explicitly in Section 10 — that the corpus pins the `bucket` profile only, and that `coarse(g)` and `frame(F)` conformance is untested across ports — rather than leaving the scope of the byte-parity requirement to be inferred from the vector file.

Client Comment

/docs/napseq-eprint-v4.tex

```
\subsection{Padding Profiles}
\label{sec:pad-profiles}

The map $n\mapsto B$ above is the \emph{bucket} profile, and it is the
normative default. Because it is the sole source of the length-hiding
property proved in Section~\ref{sec:length-hiding}
(Remark~\ref{rem:coarsening-not-inflation}), we specify it as a
replaceable
parameter rather than as a hard-wired constant. A \emph{padding
profile} is
any function $B(\cdot)$ from codepoint counts to block sizes
satisfying
$B(n)>n$ throughout the message space. This paper specifies three:
```

same file

```
  \item[$\mathsf{coarse}(g)$.] $B(n)=2^{\,4+g\lceil(e(n)-4)/g\rceil}$
where
        $e(n)=\max\bigl(4,\lceil\log_2(n+1)\rceil\bigr)$ and $g$
divides
        $12$: the same ladder thinned by a stride $g$, with $12/g+1$
        reachable values. $\mathsf{coarse}(1)=\mathsf{bucket}$, and
        $\mathsf{coarse}(12)$ leaves only $\{2^4,2^{16}\}$.
```

same file

```
  \item[$\mathsf{frame}(F)$.] $B(n)=F$ for a fixed
        $F\in\{2^4,\ldots,2^{16}\}$, on the restricted message space
$n<F$:
        every message pads to one common block size, so exactly one
value is
        reachable.
\end{description}
```

same file

```
The corpus for the scheme specified in this paper is
\texttt{tests/kat/v8\_vectors.json}. It contains 20 vectors: 12
positive
(ids \texttt{W001}--\texttt{W012}) covering the empty message, a
single
```

```
character, the block boundaries at 15, 16 and 32 codepoints, 1-element and
10-element prime tuples $\mathbf{k}$, empty, binary and 64-byte associated
data (the last pinning the 8-byte $\mathrm{be8}$ AAD length prefix of
Definition~\ref{def:taginput}), and mixed-case punctuation; and 8 negative.
Of the negative vectors, \texttt{W-N01}--\texttt{W-N05} exercise
authentication (flipped tag byte, flipped masked-blob byte, wrong $A$,
wrong $\mathsf{sk}$) and the 48-byte parse floor, while
```

same file

```
needed. Conforming implementations must produce byte-identical ciphertext
for every positive vector and reject every negative vector. The generator
```

same file

```
        fixed per bucket, it does not vary with $\mathsf{sk}$, the nonce, or
        the associated data, so no averaging attack across repeated
        encryptions of one message under varying AAD can extract more than
        the bucket itself. Deployments requiring strictly less than this
        should select a coarser padding profile, and those requiring zero
        length leakage should select $\mathsf{frame}(F)$
        (Section~\ref{sec:pad-profiles}), which needs no change to the wire
        format and no cooperation from the decryptor; no external
```

/src/kat_cross_check.rs

```
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "positive" && v["api"] != "stream_ae")
```

same file

```
        .filter(|v| v["kind"] == "negative" && v["api"] != "stream_ae")
```

/src/lib.rs

```
    pub fn block_size(self, n: usize) -> Result<usize, String> {
        let e = bit_length(n).max(PAD_MIN_EXP);
```

```
        match self {
```

same file

```
            PadProfile::Frame(f) => {
                if !f.is_power_of_two()
                    || f.trailing_zeros() < PAD_MIN_EXP
```

same file

```
    fn pad_profiles_share_one_reachable_set() {
        let legal: std::collections::HashSet<usize> =
            (PAD_MIN_EXP..=PAD_MAX_EXP).map(|e| 1usize <<
e).collect();
```

same file

```
    fn pad_profile_coarse_stride_must_divide_twelve() {
        assert!(PadProfile::Coarse(5).block_size(3).is_err());
        assert!(PadProfile::Coarse(0).block_size(3).is_err());
```

same file

```
    fn pad_profile_frame_is_constant_and_range_checked() {
        for n in [0usize, 1, 100, 511] {
            assert_eq!(PadProfile::Frame(512).block_size(n).unwrap(),
512);
```

same file

```
    fn v8_frame_profile_hides_length_and_round_trips() {
        let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
MAX_KEY_PRIME);

```

same file

```
    fn v8_frame_profile_rejects_oversized_message() {
        let (primes, sk) = generate_v8_key(10, MIN_KEY_PRIME,
MAX_KEY_PRIME);
        let msg = "a".repeat(600);
```

CVF-33: fixed Category: **procedural**

Section 9's SP 800-22 result is asserted of "NAPQES ciphertext output", but the bitstream was produced by the legacy v7 encryptor rather than by the scheme Section 3 specifies — and the crate then cites SP 800-22 as the justification for a wire-format decision on a format the tests never saw.

Section 9 states the evidence without qualification: ”We applied the full NIST SP 800-22 Rev. 1a Statistical Test Suite to 10^7 bits of NAPQES ciphertext output generated by encrypting a cycling corpus of printable-ASCII plaintext with a 10-element prime key. All 15 eligible SP 800-22 tests passed at the $\alpha = 0.01$ significance level.”

The generator is `src/bin/sts.rs`, whose ciphertext-producing call is `napqes::encrypt_bytes(msg, &STS_KEY, b"")` — the **v7** entry point. It never calls `encrypt_bytes_v8` or `encrypt_bytes_v8_with_profile`. Its own module header says so plainly: the file is titled ”NIST SP 800-22 Rev 1a Statistical Test Suite — NAPSEQ v6 (Rust)” and the function comment above the generator loop describes the input as ”NAPSEQ v6 ciphertexts with `STS_KEY`”. `src/bin/` is **outside the reviewed artifact surface** for this engagement, so it is cited here as evidence of what produced the bitstream rather than as the defect site; the inaccurate claim is in Section 9, which is in scope.

The bitstream that passed is therefore not produced by the specified scheme. It differs from a v8 bitstream in four ways at once, each of which is one of the divergences recorded elsewhere in this round: the derivations are keyed by `key_bytes(primes)` rather than by $\mathrm{sk}_{\mathrm{fmt}}$ (CVF-28, CVF-11); the emission loop applies neither the noise-run cap nor the token ceiling, so the token count per padded codepoint is unbounded and nonce-dependent rather than fixed at 20 (CVF-17); the noise threshold is an IEEE-754 float rather than the integer $\theta(N)$ (CVF-30); and the tag carries a 4-byte AAD length prefix rather than 8 (CVF-28). The first three change the statistical structure of the byte stream directly — the token population, its length distribution and the noise density all differ.

What makes this more than a generic evidence-scoping complaint is that the paper’s own design rationale leans on it. The reason given in the crate for masking the token blob at all is a statistical one: the comment on `varint_keystream` says that ”XOR-masking the raw LEB128 blob eliminates the 3:1 MSB continuation-bit bias that otherwise causes systematic NIST SP 800-22 failures.” So SP 800-22 is being used to underwrite a wire-format decision — and the format it underwrites is not the format the tests were run on. The LEB128 encoding that produced the 3:1 bias was itself retired; the delivered v8 blob is fixed-width 8-byte tokens, for which the bias argument does not apply in the same form and has not been re-established.

Nothing here suggests the v8 bitstream would fail SP 800-22. It is masked by an HMAC-derived keystream, so it should pass comfortably, and if anything the v8 stream is the better-behaved of the two. The defect is that the paper reports a result about one artifact as a result about another, in an evidence section, when Contribution 8 promises to ”state in each case what the evidence does and does not establish”.

This is the same substitution as CVF-22, where the fuzz harness targets `decrypt_bytes` rather than `decrypt_bytes_v8`, at a second site; and it is a concrete instance of the class CVF-10 describes. It is filed separately because the remedy is specific and different from both: regenerating a bitstream is not retargeting a fuzz harness, and neither is a rewording of the contributions paragraph.

Recommendation

Regenerate the bitstream from the v8 encryptor and re-run the suite, or restate Section 9 to say what was actually tested.

Regenerating is straightforward and is the better outcome, because it makes the evidence match the claim rather than narrowing the claim to match the evidence. The change in the generator is one call site — `napqes::encrypt_bytes_v8(msg, &STS_KEY_PRIMES, &STS_SK, b"")` in place of `napqes::encrypt_bytes(msg, &STS_KEY, b"")`, with a fixed `sk` added to the pinned test constants — plus a corresponding update to the module header, which still describes the file as a v6 artifact. Note one consequence worth anticipating: because the v8 nonce is synthetic, encrypting a *cycling* corpus reproduces identical ciphertexts for repeated plaintext blocks, where the v7 CSPRNG nonce made every block fresh. The corpus must therefore be made non-repeating — vary the plaintext, or the AAD, per chunk — or the bitstream will contain exact duplicate 2,928-byte segments and several SP 800-22 tests will fail for a reason that has nothing to do with the keystream. That is a property of deterministic AEAD, not a defect, but it will surface the moment the entry point is switched.

If the suite is not re-run for this release, then Section 9 must say which format was tested. Something in the register the paper already uses elsewhere: the SP 800-22 results were obtained on the legacy v7 wire format, which differs from the scheme specified here in its key schedule, its emission bounds and its noise-threshold arithmetic; no statistical evidence is offered for the v8 bitstream. That is a considerably weaker statement than the one currently printed, which is the point — Contribution 8 undertakes to make exactly this kind of boundary explicit.

Separately, update the `varint_keystream` doc comment. It justifies the mask by reference to an SP 800-22 failure mode of the LEB128 encoding, and LEB128 was retired; the blob it masks today is fixed-width. Either restate the justification in terms of the current format — the unmasked fixed-width token stream is a sequence of `c*k+a` values with a small key element, so its high-order bytes are far from uniform, which is a sound reason for the mask that does not depend on the retired encoding — or drop the SP 800-22 reference from the comment. As written it points a reader at evidence that concerns neither the current encoding nor the current entry point.

Client Comment

/docs/napseq-eprint-v4.tex

```
181   \item We report NIST SP 800-22 Rev.\ 1a statistical test results on
182         ciphertext output (Section~\ref{sec:sts}), the
cross-implementation
183         KAT corpora
184        (Section~\ref{sec:kats}), and a fuzz harness covering the full
185         decoder pipeline (Section~\ref{sec:fuzz}), and state in each
case
186         what the evidence does and does not establish.
```

same file

```
We applied the full NIST SP 800-22 Rev.\ 1a Statistical Test
Suite~\cite{sp800-22} to $10^7$ bits of NAPQES ciphertext output
generated
by encrypting a cycling corpus of printable-ASCII plaintext with a
10-element prime key. All 15 eligible SP 800-22 tests passed at the
$\alpha=0.01$ significance level. Results are reproducible using
\texttt{tests/sts\_pipeline.py} in the reference implementation
repository.
```

/src/lib.rs

```
/// HMAC-CTR keystream for masking the varint blob — domain byte 0x07.
///
/// Each 32-byte block is `HMAC(key_bytes, 0x07 || nonce ||
uint32_be(block))`.
/// XOR-masking the raw LEB128 blob eliminates the 3:1 MSB
continuation-bit
/// bias that otherwise causes systematic NIST SP 800-22 failures.
```

same file

```
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
```

CVF-34: fixed Category: **behavior**

Section 12's plaintext-cap caveat states unconditionally that "Exceeding the cap raises an error immediately; there is no silent truncation", but on the legacy path an over-long message panics rather than erroring, and the legacy decryptor silently drops recovered values that are not Unicode scalars.

Section 12's first caveat makes two promises about the 65,535-codepoint cap, both unqualified: exceeding it raises an error immediately, and there is no silent truncation. The v8 path keeps both. `encrypt_bytes_v8_with_profile` checks `codepoints.len() > 0xFFFF` and returns `Err("Message too long for 2-byte length prefix.")`, and `decrypt_bytes_v8` rejects any recovered value that is not a Unicode scalar inside `decrypt_core_v8` rather than dropping it, with a comment saying exactly that: "`decrypt_core_v8` has already rejected any non-scalar value, so this maps every recovered codepoint rather than silently dropping some."

Neither promise holds on the legacy path, and the two failures are in opposite directions.

"Raises an error" is a panic. The v7 padding routine enforces the cap with a plain assertion:

```
fn pad_message(msg: &[u32], kb: &[u8], nonce: &[u8]) -> Vec<u32> {
    let n = msg.len();
```

```
    assert!(n <= 0xFFFF, "Message too long for 2-byte length prefix");
```

`assert!` — unlike `debug_assert!` — fires in release builds, so this aborts the process rather than returning an error. It is reached from four public entry points: `encrypt`, `encrypt_bytes`, `encrypt_raw` directly, and `encrypt_str` through `encrypt_bytes`; also from the `pub(crate) encrypt_bytes_with_nonce`. A caller who passes a 70,000-codepoint string to `encrypt_bytes` — which returns `Result<Vec<u8>, String>`, and so advertises that it reports failures as values — gets a panic instead. In a server that is a denial of service reachable from any caller that does not itself bound message length, since the panic unwinds through a function whose signature promised an `Err`.

The same pattern appears once more on the v7 decrypt side, where `decrypt_core` ends with

```
unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")
```

whose own comment concedes the point: "a malformed padded buffer panics here exactly as it did before V3-CVF8. The v8 path (`decrypt_core_v8`) returns the error instead."

"No silent truncation" is false on the decrypt side. `decrypt_bytes` finishes with

```
let s: String =
  codepoints.into_iter().filter_map(char::from_u32).collect();
```

`filter_map` **drops** every recovered value in the surrogate range or above `U+10FFFF` instead of rejecting the ciphertext. So `decrypt_bytes` — and `decrypt_str`, which delegates to it — can return `Ok` with a plaintext strictly shorter than what was encrypted, with no error and no indication that anything was discarded. That is silent truncation, on the decrypt side of the same construction whose caveat says there is none. It is unreachable without a valid tag, so there is no forgery consequence; a key holder or a corrupted-at-rest ciphertext produces it. The v8 path was explicitly changed to avoid exactly this.

The new element here, relative to CVF-15, is that a **normative caveat is stated unconditionally and holds only of v8**. CVF-15 records that the paper's no-panic attestation covers one entry point while several others panic, and it is complete on the panic surface itself. It does not record that Section 12's plaintext-cap paragraph — which is written as a property of "the block padding scheme", not of one port or one format — is false for four public entry points, nor that the crate performs the very silent truncation the caveat denies. Both statements are read by a deployer deciding how to bound input, which is the audience Section 12 is written for.

Recommendation

Make the two v7 sites behave the way the caveat says, or qualify the caveat.

On the encrypt side, the difficulty is that `pad_message` returns `Vec<u32>` and cannot report a failure. The clean fix is to hoist the length check into the four public entry points, where the signature already permits an error, and delete the assertion:

```
if codepoints.len() > 0xFFFF {
    return Err("Message too long for 2-byte length
  prefix.".into());
```

```
}
```

That is the same check `encrypt_bytes_v8_with_profile` already performs, and it should use a shared named constant rather than a repeated `0xFFFF` literal — see the separate finding on `MAX_PLAINTEXT_CODEPOINTS`, which is a named parameter in the paper and a bare literal at both existing sites, which is why the two sites diverged into `assert!` and `Err` in the first place. `encrypt` returns `([u8; NONCE_SIZE], Vec<u64>)` rather than a `Result`, so it needs a signature change or a documented panic; given CVF-17 and CVF-28 both recommend withdrawing the v7 encryptors, changing the signature is only worth doing if they are being kept.

On the decrypt side, replace the `filter_map` with a rejecting map, so that a non-scalar value is an error rather than a deletion:

```
let s: String = codepoints
    .into_iter()
    .map(|c| char::from_u32(c).ok_or_else(|| format!(
        "Malformed v7 ciphertext: recovered value {} is not a
  Unicode scalar value.", c)))
    .collect::<Result<String, String>>()?;
```

`decrypt_bytes` already returns `Result<String, String>`, so this needs no signature change and brings the v7 path into line with the v8 one.

Replace `unpad_message(&padded).expect(...)` in `decrypt_core` with error propagation. That requires `decrypt_core` to return a `Result`, which is a larger change and is squarely CVF-15's territory; it is noted here because it is the third instance of the same substitution of a panic for the error the caveat promises.

If none of the v7 changes is made — for instance because the v7 encryptors are being withdrawn instead — then qualify Section 12's caveat. It should say that the cap is enforced by an error on the scheme specified in this paper, and name the legacy behaviour rather than leaving a deployer to infer it. A caveats section that overstates the robustness of the shipped code is worse than one that says less.

Client Comment

/docs/napseq-eprint-v4.tex

```
2031   \item[Plaintext length cap.] The block padding scheme stores the
2032         plaintext length as a 2-byte big-endian integer, capping
messages
2033         at 65{,}535 codepoints. Exceeding the cap raises an error
2034         immediately; there is no silent truncation. Callers requiring
2035         larger messages must split at the application layer.
```

/src/lib.rs

```
395 fn pad_message(msg: &[u32], kb: &[u8], nonce: &[u8]) -> Vec<u32> {
```

```
    let n = msg.len();
    assert!(n <= 0xFFFF, "Message too long for 2-byte length prefix");
```

same file

```
    let padded = pad_message(message, &kb, &nonce);
```

same file

```
    // v7 low-level API: signature is `-> Vec<u32>`, so a malformed padded
    // buffer panics here exactly as it did before V3-CVF8. The v8 path
    // (`decrypt_core_v8`) returns the error instead.
    unpad_message(&padded).expect("v7 decrypt: malformed padded buffer")
```

same file

```
    let padded = pad_message(&codepoints, &kb, &nonce);
```

same file

```
    let padded = pad_message(&codepoints, &kb, &nonce);
```

same file

```
    let s: String = codepoints.into_iter().filter_map(char::from_u32).collect();
```

same file

```
    let padded = pad_message(&codepoints, &kb, &nonce);
```

same file

```
    if codepoints.len() > 0xFFFF {
        return Err("Message too long for 2-byte length prefix.".into());
    }
```

same file

```
    // `decrypt_core_v8` has already rejected any non-scalar value, so this
    // maps every recovered codepoint rather than silently dropping some.
    let s: String = codepoints
```

CVF-35: fixed Category: **behavior**

The FIPS 140-3 continuous-RNG health test is applied inconsistently across the crate's own v7 nonce paths, and the single process-global slot it compares against makes it a DRBG health test rather than the per-key uniqueness property a reader is likely to assume.

The crate implements a continuous random-number-generator test and cites the standard for it:

```
static PREV_NONCE: Mutex<Option<[u8; NONCE_SIZE]>> =
  Mutex::new(None);

/// Generate a cryptographically random 16-byte nonce, verifying it
  differs
/// from the previous one (FIPS 140-3 continuous RNG test, SP
  800-140B §4.9.2).
fn generate_nonce_with_crng_check() -> Result<[u8; NONCE_SIZE],
  String> {
    let mut nonce = [0u8; NONCE_SIZE];
    rand::thread_rng().fill_bytes(&mut nonce);
    let mut prev = PREV_NONCE.lock().map_err(|_| "CRNG mutex
  poisoned".to_string())?;
    if let Some(prev_nonce) = *prev {
        if ct_eq_bytes(&nonce, &prev_nonce) {
            return Err("CRNG failure: consecutive identical nonces —
  DRBG may be compromised".into());
        }
    }
    *prev = Some(nonce);
    Ok(nonce)
}
```

The specification says nothing about this: the v8 nonce is synthetic and there is no CSPRNG nonce draw anywhere in Π, so `generate_nonce_with_crng_check` is a v7-only, implementation-quality construct. Two observations about how it is applied.

It is bypassed by one of the crate's own public entry points. `encrypt_bytes` and `encrypt_raw` both call it. `encrypt` — also pub — does not; it draws its nonce inline:

```
pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
  Vec<u64>) {
    let mut nonce = [0u8; NONCE_SIZE];
    rand::thread_rng().fill_bytes(&mut nonce);
```

with no health check of any kind. So a caller using the low-level `encrypt` gets none of the protection the crate advertises for `encrypt_bytes`, and there is nothing in either signature or doc comment that would tell them so. Given what a repeated v7 nonce costs — the crate's own module comment describes it as permitting recovery of k from two ciphertexts sharing a nonce, "catastrophic key recovery, not merely a confidentiality loss" — an inconsistently applied health check on that exact quantity is worth making uniform. (`encrypt_bytes_with_nonce` takes its nonce from the caller and is `pub(crate)`, so it is outside this: nothing there can be checked.)

The single global slot is a DRBG health test, not a uniqueness guarantee. `PREV_NONCE` is one process-wide slot, so the value a call compares against is whichever call wrote last — across every thread, every key and every message in the process. As an SP 800-140B §4.9.2 continuous test that is exactly right and the implementation is correct for that purpose: the test is meant to catch a stuck generator, and one previous output suffices. But it must not be read as a per-key nonce-uniqueness check, because it is not one: it detects only *immediately consecutive* identical outputs, it can be defeated by interleaving two encryptions from different threads, and it says nothing about a nonce repeating at any distance greater than one. Nothing in the code or the documentation draws that line, and the error string — "DRBG may be compromised" — invites the stronger reading.

A third, smaller point: the comparison uses `ct_eq_bytes`, the crate's constant-time comparator, on two values that are both public — a freshly generated nonce and the previous nonce, both of which appear in the clear on the wire. There is nothing wrong with it, but it is a second call site for the comparator whose provenance CVF-12 records, and it is worth counting when that finding's recommendation to adopt `subtle` is applied.

No security claim in the paper depends on any of this, and no theorem covers the v7 path in any case. It is filed because the crate advertises a named FIPS control, applies it to two of its three self-nonce-generating public entry points, and leaves the third — the lowest-level one — uncovered.

Recommendation

Apply the check uniformly, or state where it applies.

Route `encrypt`'s nonce draw through `generate_nonce_with_crng_check` like the other two. The obstacle is the signature: `encrypt` returns `([u8; NONCE_SIZE], Vec<u64>)` with no error channel, so it cannot propagate a CRNG failure. Change it to `Result<([u8; NONCE_SIZE], Vec<u64>), String>`, which is a breaking change to a low-level v7 function and is worth doing only if the v7 encryptors are being retained — CVF-17 and CVF-28 both recommend withdrawing them, in which case this closes with them. If the signature must stay, say so in the doc comment: `encrypt` performs no continuous-RNG test and callers requiring one must use `encrypt_bytes`.

Document what the check is and is not. One sentence above `generate_nonce_with_crng_check` stating that `PREV_NONCE` is a single process-global slot, that the function implements the SP 800-140B §4.9.2 continuous test for a stuck generator, and that it is **not** a per-key nonce-uniqueness guarantee, would prevent the stronger reading. Consider softening the error string for the same reason: "consecutive identical DRBG outputs" is the observation; "DRBG may be compromised" is one inference from it.

If per-key nonce uniqueness is actually wanted on the v7 path, a single global slot cannot provide it and should not be extended to try. The right answer is the one v8 already took — a synthetic, message-bound nonce, which makes repetition impossible by construction rather than detectable after the fact. That is a further argument for migrating v7 consumers to the v8 entry points rather than hardening the v7 nonce path.

Finally, when CVF-12's recommendation is applied and `ct_eq_bytes` is replaced by `subtle`'s `ConstantTimeEq`, remember this call site as well as the three tag-

comparison ones. It compares two public values, so a plain == would also be correct here and would make the distinction between the security-relevant comparisons and this one visible in the code.

Client Comment

/src/lib.rs

```
47 static PREV_NONCE: Mutex<Option<[u8; NONCE_SIZE]>> =
Mutex::new(None);
```

same file

```
49 /// Generate a cryptographically random 16-byte nonce, verifying it
differs
50 /// from the previous one (FIPS 140-3 continuous RNG test, SP
800-140B §4.9.2).
51 fn generate_nonce_with_crng_check() -> Result<[u8; NONCE_SIZE],
String> {
52     let mut nonce = [0u8; NONCE_SIZE];
53     rand::thread_rng().fill_bytes(&mut nonce);
54     let mut prev = PREV_NONCE.lock().map_err(|_| "CRNG mutex
poisoned".to_string())?;
55     if let Some(prev_nonce) = *prev {
56         if ct_eq_bytes(&nonce, &prev_nonce) {
57             return Err("CRNG failure: consecutive identical nonces —
DRBG may be compromised".into());
58         }
59     }
60     *prev = Some(nonce);
61     Ok(nonce)
62 }
```

same file

```
56         if ct_eq_bytes(&nonce, &prev_nonce) {
```

same file

```
467 pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
Vec<u64>) {
468     let mut nonce = [0u8; NONCE_SIZE];
469     rand::thread_rng().fill_bytes(&mut nonce);
```

same file

```
734     let nonce = generate_nonce_with_crng_check()?;
```

same file

```
890     let nonce = generate_nonce_with_crng_check()?;
```

CVF-36: fixed Category: **documentation**

`Remark 3.3` states that "the reference implementations emit a warning below $K = 7$", but the delivered Rust port emits no warning of any kind, and eleven of the twelve positive vectors in the canonical corpus sit below that threshold with the paper itself advertising one of them.

`Remark 3.3` is careful, and deliberately so: it spends a paragraph establishing that K carries no security-theorem-driven minimum, that "$K = 10$ is an interoperability default, not a security parameter", and that "any $K \geq 1$ is permitted by the construction". It then makes a factual claim about the artifacts: "The reference implementations emit a warning below $K = 7$; that threshold is inherited from the earlier single-secret schedule, in which $H_\infty(\mathbf{k})$ did enter the bound, and no theorem in this paper requires it."

No such warning exists in the delivered Rust port. `validate_key` tests four conditions — non-empty, prime, at or above `MIN_KEY_PRIME`, pairwise distinct — and none of them concerns K. There is no `eprintln!`, no `log` call, no `#[deprecated]`, no feature-gated diagnostic and no `warn` of any kind anywhere in `src/lib.rs`; the library declares no logging dependency at all (`syslog` appears in the manifest, but only for the `ls_gateway` binary). `generate_v8_key(count, min_val, max_val)` accepts any `count` the caller passes, including 1, and returns without comment. The plural in "the reference implementations" cannot be checked further, because `napqes.py` and `C/napqes.c` were not delivered.

The delivered corpus then makes the gap conspicuous rather than academic. Of the twelve positive vectors in `tests/kat/v8_vectors.json`, **one uses $K = 1$, nine use $K = 4$, and only two use $K = 10$** — so eleven of the twelve canonical vectors are below the stated warning threshold, as are all eight negative vectors, which are $K = 4$. And Section 10 presents the $K = 1$ case as a deliberate feature of the corpus, listing "1-element and 10-element prime tuples **k**" among the properties the twelve positives cover. If the reference implementations warned below $K = 7$, generating and consuming the canonical corpus would produce a warning on eighteen of the twenty vectors.

There is **no security consequence**. `Remark 3.3` is explicit and correct that no theorem depends on $H_\infty(\mathbf{k})$ or on a minimum K, and Section 12 goes further, conceding that **k**'s secrecy "adds **no** independent confidentiality margin once sk is known". A missing warning about a parameter that carries no security requirement costs nothing directly.

It is filed because it is a false factual claim about the delivered artifacts, in a paper that elsewhere takes visible care to scope its evidence claims — Contribution 8 undertakes to "state in each case what the evidence does and does not establish" — and because the sentence is doing rhetorical work. Its function in the paragraph is to reassure a reader who has met the $K \geq 7$ rule elsewhere that the implementations still guard the old

threshold even though the theory no longer needs it. A reader who takes that at face value, sets $K = 2$ in a deployment and expects to be told about it will not be.

Recommendation

Either implement the warning or delete the sentence. Deleting it is the better choice, and the paragraph is stronger without it.

`Remark 3.3` has already done the work of establishing that K is an interoperability parameter rather than a security one. The warning sentence pulls against that conclusion: it tells a reader that the implementations treat $K < 7$ as worth flagging, immediately after arguing that nothing requires $K \geq 7$. Removing it leaves the argument intact and removes the claim about the artifacts. If the historical threshold is worth recording, the remaining half of the sentence — that it "is inherited from the earlier single-secret schedule, in which $H_\infty(\mathbf{k})$ did enter the bound" — can stand on its own as a note on provenance, phrased in the past tense.

If the warning is instead to be implemented, then implement it in all three ports and say where it surfaces. In Rust the natural place is `generate_v8_key`, not `validate_key`: a warning belongs at key *generation*, where the caller is choosing K, rather than on the per-message validation path, where it would fire on every encrypt and decrypt call for the lifetime of the key. The library declares no logging facade, so adding one for this is disproportionate; a `#[must_use]` result, a documented `Result` variant, or a `debug_assert!` would all be better than `eprintln!` from a library. Whatever mechanism is chosen, the canonical corpus must then be regenerated or the generator exempted, because eighteen of its twenty vectors would otherwise trip it.

In either case, correct Section 10's inventory if the $K = 1$ vector's status changes, and — independently of this finding — note that the sentence is one of several claims in the paper quantified over "the reference implementations" while only one of the three was delivered. That broader gap is recorded under CVF-25.

Client Comment

/docs/napseq-eprint-v4.tex

```
agree on $\mathbf{k}$ itself. The reference implementations emit a warning
below $K=7$; that threshold is inherited from the earlier single-secret
schedule, in which $H_\infty(\mathbf{k})$ did enter the bound, and no
theorem in this paper requires it. What the arithmetic layer is retained
```

same file

```
The corpus for the scheme specified in this paper is
```

```
\texttt{tests/kat/v8\_vectors.json}. It contains 20 vectors: 12 positive
(ids \texttt{W001}--\texttt{W012}) covering the empty message, a single
character, the block boundaries at 15, 16 and 32 codepoints, 1-element and
10-element prime tuples $\mathbf{k}$, empty, binary and 64-byte associated
```

/src/lib.rs

```
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of {}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        if key[..i].contains(&k) {
            return Err(format!(
                "Key element {} at index {} is a duplicate; all elements must be distinct.",
                k, i
            ));
        }
    }
    Ok(())
}
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) -> (Vec<u64>, [u8; SK_SIZE]) {
    let primes = generate_prime_numbers(count, min_val, max_val);
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    (primes, sk)
}
```

CVF-37: fixed Category: **flaw**

Neither zeroization helper is called anywhere in the library, and `sk_fmt` — the value that actually keys every derivation — is never wiped at all, so the crate protects the superseded secret and leaves the effective one on the stack of every call.

The crate takes key destruction seriously enough to cite the standards for it. `zeroize_key`'s doc comment reads: "Uses `ptr::write_volatile` to prevent the compiler from eliding the writes as dead-code optimisations, consistent with the `ct_eq_bytes` approach. Call this as soon as the key is no longer needed. Reference: FIPS 140-3 / SP 800-57 Part 1 Rev 5 §8.3 (key destruction)." Both helpers are correct as written — `write_volatile` per element is the right primitive.

Neither is ever called. The only call sites for `zeroize_key` and `zeroize_sk` in the entire crate are the two unit tests that exercise them, `zeroize_key_clears_memory` and `v8_zeroize_sk_clears_memory`. No library function invokes either; there is no `Drop` implementation on any type carrying key material; `Cargo.toml` declares neither the `zeroize` crate nor any `Zeroizing` wrapper.

The more consequential gap is what the helpers do not cover. `zeroize_sk` wipes sk — but as CVF-11 establishes, sk is not the key that keys anything except one two-byte HMAC. The value that keys every derivation is `sk_fmt = HMAC(sk, 0x0B||0x01)`, a `[u8; 32]` local computed at the head of `encrypt_bytes_v8_with_profile` and of `decrypt_bytes_v8` and dropped without a write at the end of each. **`sk_fmt` is sufficient on its own to encrypt, decrypt and forge**, with no knowledge of sk whatever, since every domain in Table 1 is keyed by it. So a deployment that follows the crate's own advice and calls `zeroize_sk` as soon as the key is no longer needed has wiped the value that grants nothing and left an unwiped copy of the value that grants everything on the stack of every call it ever made. The intended protection is inverted.

Three smaller residues in the same area:

- `generate_v8_key` returns `[u8; SK_SIZE]` **by value**, and `[u8; 32]` is `Copy`, so the function's own stack temporary and every subsequent assignment leave un-zeroized copies that `zeroize_sk` on the caller's binding does not reach.
- `key_bytes(key)` allocates a fresh `Vec<u8>` holding the serialised prime tuple at each of its seven legacy call sites and drops it unwiped, so the v7 HMAC key is left in the allocator on every v7 operation.
- `zeroize_key` takes `&mut [u64]`, so it cannot wipe a `Vec<u64>`'s spare capacity or any copy the caller made; nothing documents that limitation.

The specification offers no guidance to measure any of this against, which is itself worth recording. The 2,281-line source contains **zero** occurrences of "zeroiz", "erase", "destruct" or "key destruction". Section 12's key-material caveat says only that **k** and sk "must be protected with equal rigor, as a single unit" and stops there. So the code picks a key-lifetime policy unilaterally, cites FIPS 140-3 and SP 800-57 for it, and then does not apply it — in a paper that invokes FIPS 140-3 elsewhere.

No exploit follows directly. Recovering a dropped stack frame or a freed allocation requires either a memory-disclosure defect elsewhere, a core dump, a swap file or a VM snapshot, and none of the paper's theorems is stated in a model that covers any of those.

The finding is that the crate's stated intent and its behaviour do not match, and that the mismatch is in the direction that matters: what is wiped is the value whose compromise no longer suffices, and what is left is the value whose compromise does.

Recommendation

Wipe `sk_fmt`, and make wiping automatic rather than advisory.

The minimal change is to zeroize `sk_fmt` at the end of both v8 entry points, on every exit path including the error paths, since a rejected decryption has already computed it. Doing that by hand at each `return` is error-prone; the idiomatic fix is a guard type that wipes on drop:

```
struct Secret32([u8; 32]);
impl Drop for Secret32 {
    fn drop(&mut self) {
        for b in self.0.iter_mut() { unsafe {
  std::ptr::write_volatile(b, 0u8) }; }
    }
}
```

with `sk_fmt` held as a `Secret32` and passed as `&sk_fmt.0`. That covers every exit path automatically and costs one type. Applying the same wrapper to the `Vec<u8>` returned by `key_bytes` closes the v7 residue at all seven call sites at once.

Better still, take the dependency: the `zeroize` crate's `Zeroizing<T>` does exactly this, is a single well-reviewed dependency, handles the `Vec` spare-capacity case that a hand-rolled guard does not, and would let `zeroize_sk` and `zeroize_key` be retired rather than documented as functions nobody calls. The manifest already carries fifteen dependencies, so one more is not a meaningful addition to the audit surface.

Change `generate_v8_key` to hand the secret back in a wrapper rather than as a bare `[u8; 32]`, so that the `Copy` semantics stop silently multiplying it. If the signature must stay for compatibility, document that the returned array is `Copy` and that the caller is responsible for every copy it makes — at present the doc comment implies `zeroize_sk` is sufficient.

Update `zeroize_sk`'s doc comment either way. As written it describes itself as erasing "a v8 HMAC subkey", which is precisely what it does not do: it erases the master secret, and the subkey is the thing left behind. Once CVF-11 is resolved and $\mathrm{sk}_{\mathrm{fmt}}$ is a named quantity in the specification, this distinction becomes easy to state.

Finally, add one paragraph on key lifetime to Section 12, beside the existing "protected with equal rigor, as a single unit" sentence. It need not be long — that **k**, sk and any derived subkey should be erased when no longer needed, that implementations should use a volatile write or an equivalent the compiler cannot elide, and that the paper offers no side-channel or memory-disclosure guarantees. At present the specification is silent, each port is free to invent a policy, and the delivered one invented a policy it does not follow.

Client Comment

/Cargo.toml

```
[dependencies]
hmac = "0.12"
sha2 = "0.10"
base64 = "0.22"
rand = "0.8"
subtle = "2"
```

/docs/napseq-eprint-v4.tex

```
  \item[Scope of $\mathsf{sk}$ and $\mathbf{k}$ compromise.] $\mathsf{sk}$
        is the sole cryptographic secret in this construction: every HMAC
        domain derivation — the noise schedule, the padding, the
        per-position addends, the keystream, and the tag — is keyed by
        $\mathsf{sk}$ alone (Section~\ref{sec:domains}). Consequently, an
```

/src/lib.rs

```
fn key_bytes(key: &[u64]) -> Vec<u8> {
    let mut out = Vec::with_capacity(key.len() * 5);
    for &k in key {
        let b = k.to_be_bytes(); // 8 bytes
        out.extend_from_slice(&b[3..8]); // low 5 bytes, big-endian
    }
    out
}
```

same file

```
pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) -> (Vec<u64>, [u8; SK_SIZE]) {
    let primes = generate_prime_numbers(count, min_val, max_val);
    let mut sk = [0u8; SK_SIZE];
    rand::thread_rng().fill_bytes(&mut sk);
    (primes, sk)
}
```

same file

```
    let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```
1213     let sk_fmt = derive_format_subkey(sk, FORMAT_BLOCK_V8);
```

same file

```
1239 /// Securely erase a v8 HMAC subkey by overwriting it with zero.
1240 pub fn zeroize_sk(sk: &mut [u8; SK_SIZE]) {
1241     for x in sk.iter_mut() {
1242         unsafe { std::ptr::write_volatile(x, 0u8) };
1243     }
1244 }
```

same file

```
1246 // ── Key zeroization ──────────────────────────────────────────
1247
1248 /// Securely erase key material by overwriting each element with zero.
1249 ///
1250 /// Uses `ptr::write_volatile` to prevent the compiler from eliding the writes
1251 /// as dead-code optimisations, consistent with the `ct_eq_bytes` approach.
1252 /// Call this as soon as the key is no longer needed.
1253 ///
1254 /// Reference: FIPS 140-3 / SP 800-57 Part 1 Rev 5 §8.3 (key destruction).
1255 pub fn zeroize_key(key: &mut [u64]) {
1256     for x in key.iter_mut() {
1257         unsafe { std::ptr::write_volatile(x, 0u64) };
1258     }
1259 }
```

same file

```
1317     fn zeroize_key_clears_memory() {
```

same file

```
1410     fn v8_zeroize_sk_clears_memory() {
```

CVF-38: fixed Category: **documentation**

`Definition 3.1` makes key_bytes(**k**) part of KeyGen's output, but no v8 code path uses it — it survives only on the legacy surface — and meanwhile the one key encoding a second implementer genuinely must reproduce, the key-pair blob, is not specified at all.

`Definition 3.1` closes with a normative serialisation: "The serialised arithmetic key is

$$\text{key_bytes}(\mathbf{k}) = \text{be5}(k_1) \| \text{be5}(k_2) \| \cdots \| \text{be5}(k_K)$$

." It is stated as an output of KeyGen, on the same footing as **k** and sk.

The Rust port implements it faithfully — five bytes per element, big-endian, in tuple order — and then uses it nowhere in the specified scheme. `key_bytes` has exactly seven call sites, all on the legacy v7 path: `encrypt`, `decrypt`, `encrypt_bytes`, `encrypt_bytes_with_nonce`, `decrypt_bytes`, `encrypt_raw` and `decrypt_raw`, at each of which it supplies the HMAC key. **No v8 code path calls it**, because on the v8 path every derivation is keyed by `sk_fmt` and **k** is used only, directly and un-keyed by the arithmetic token map — which is precisely what `Remark 3.3` says it is for.

So key_bytes is normatively specified as part of the scheme and is functionally dead within it. Its only live role in the delivered crate is as the key schedule of the format the paper disclaims. That is a consistent picture once CVF-11 is resolved and the `0x0B` subkey layer is documented, but as the paper stands a reader implementing `Definition 3.1` produces a byte string that nothing in `Definition 3.10` consumes, and has no way to discover that from the text.

The encoding that is actually needed is the one that is missing. A second implementation must be able to load the key pair used by the known-answer corpus, and the paper never says how a key pair is serialised. There is no combined encoding, no parser, no `Key` type and no round-trip test in the crate either: `generate_v8_key` returns a bare `(Vec<u64>, [u8; SK_SIZE])` and every entry point takes the two halves as separate arguments. The corpus uses a third, ad-hoc format of its own — a JSON array of decimal integers for **k** and a hex string for sk. Since the key pair never appears on the wire, this is a completeness gap rather than an interoperability break in the normal sense; but Section 10 asserts cross-implementation byte parity, and the corpus's key encoding is the one thing a reader must reproduce exactly that the paper does not define. As written, `Definition 3.1` specifies an encoding the scheme does not use and omits the encoding the conformance test depends on.

There is also a latent hazard in `key_bytes` itself, recorded here for completeness because it belongs to the same three lines. Neither `key_bytes` nor the shared `be5` helper checks that its argument fits in five bytes:

```
let b = k.to_be_bytes();          // 8 bytes
out.extend_from_slice(&b[3..8]);  // low 5 bytes, big-endian
```

The top three bytes are discarded with no diagnostic. Since `validate_key` enforces no upper bound on key elements (CVF-16), any accepted prime at or above 2^{40} is silently reduced modulo 2^{40} when it is serialised, so on the legacy path two distinct keys congruent modulo 2^{40} yield the **identical** HMAC key while the token arithmetic uses the full 64-bit values — encryption and decryption then disagree, silently. The paper's Notation section states the precondition ("each field is fixed-width; x is assumed to fit") and `Remark 3.11` discharges it for the *token* field only, never for the `be5` key field. Enforcing `MAX_KEY_PRIME` in `validate_key` — CVF-16's recommendation — makes this unreachable, which is why it is recorded here rather than filed as its own finding.

Recommendation

Say what key_bytes is for, and specify the key-pair encoding that the conformance corpus requires.

In `Definition 3.1`, relabel key_bytes$(\mathbf{k})$ rather than deleting it. The natural framing, once CVF-11's subkey layer is documented, is that it is a storage and transport encoding of the arithmetic key — not an input to any algorithm in `Definition 3.10`. One clause is enough: note that **k** enters the construction only through the token map $c \mapsto c \cdot k_j + a$, so key_bytes plays no part in `Enc` or `Dec` and is given here for serialisation only. That removes the false impression that a conforming implementation must compute it, and it is consistent with what `Remark 3.3` already says about the two secrets' roles. If the intent is instead that key_bytes has no role at all in the specified scheme, delete it from `Definition 3.1` and note in Section 8.2 that the legacy v7 format uses it as its HMAC key.

Add the key-pair encoding to Section 10, where the corpus is described, rather than to Section 3. It is a test-vector concern, not a construction concern: state that a key pair is carried in the corpus as a JSON array of decimal prime values in tuple order together with a lowercase hex encoding of the 32-byte sk, matching the `key` and `sk_hex` fields the vectors actually use. Two sentences remove the only ambiguity a second implementer faces in loading the vectors, and Section 10 already asks them to reproduce those vectors byte for byte.

On the code side, if a `Key` newtype is introduced for the reasons given under CVF-16 and the key-validation timing finding, give it the parse and serialise pair at the same time, together with one round-trip test. That would replace three encodings — the `be5` concatenation, the loose `(Vec<u64>, [u8; 32])` tuple and the corpus JSON — with one, and would give the crate somewhere to enforce the five-byte bound that `key_bytes` currently assumes.

Finally, make `be5` and `key_bytes` reject an out-of-range element instead of truncating, or add a `debug_assert!(k < 1u64 << 40)` at minimum. Enforcing `MAX_KEY_PRIME` in `validate_key` per CVF-16 makes the case unreachable through the public API, but the helpers are shared with the `ct_pos` and `real_idx` counters and a bound stated at the encoder is cheaper to keep correct than one inferred from every caller.

Client Comment

/docs/napseq-eprint-v4.tex

```
251   \item $\mathrm{be4}(x)$, $\mathrm{be5}(x)$, $\mathrm{be8}(x)$
denote the
252         4-, 5-, and 8-byte big-endian encodings of the non-negative
integer
253         $x$ (each field is fixed-width; $x$ is assumed to fit).
```

same file

```
function relates $\mathsf{sk}$ to $\mathbf{k}$. The serialised
arithmetic
key is
\[
  \mathsf{key\_bytes}(\mathbf{k}) = \mathrm{be5}(k_1) \|
\mathrm{be5}(k_2) \| \cdots \| \mathrm{be5}(k_K).
\]
```

same file

```
claiming more codepoints than the padded buffer holds. Each positive
vector
fixes $(\mathbf{k},\mathsf{sk},A,M)$ and the expected
ciphertext; because $\mathsf{Enc}$ is a pure function of its inputs
(Definition~\ref{def:aead-triple}), no nonce field appears and none
is
needed. Conforming implementations must produce byte-identical
ciphertext
for every positive vector and reject every negative vector. The
generator
```

/src/lib.rs

```
fn key_bytes(key: &[u64]) -> Vec<u8> {
    let mut out = Vec::with_capacity(key.len() * 5);
    for &k in key {
        let b = k.to_be_bytes(); // 8 bytes
        out.extend_from_slice(&b[3..8]); // low 5 bytes, big-endian
    }
    out
}
```

same file

```
fn be5(n: u64) -> [u8; 5] {
    let b = n.to_be_bytes();
    let mut out = [0u8; 5];
    out.copy_from_slice(&b[3..8]);
    out
}
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
    let kb = key_bytes(key);
```

same file

```
794     let kb = key_bytes(key);
```

same file

```
842     let kb = key_bytes(key);
```

same file

```
892     let kb = key_bytes(key);
```

same file

```
944     let kb = key_bytes(key);
```

same file

```
1087 pub fn generate_v8_key(count: usize, min_val: u64, max_val: u64) ->
     (Vec<u64>, [u8; SK_SIZE]) {
1088     let primes = generate_prime_numbers(count, min_val, max_val);
1089     let mut sk = [0u8; SK_SIZE];
1090     rand::thread_rng().fill_bytes(&mut sk);
1091     (primes, sk)
1092 }
```

CVF-39: fixed Category: **documentation**

The `MAX_NOISE_RUN` doc comment justifies the ceiling fix against a "~13.4x average case" that is not the average under the paper's own $\theta(N)$ distribution; the correct capped mean is ≈8.42×, so the comment understates the ceiling's cost by more than half.

The doc comment on `MAX_NOISE_RUN` is the crate's only quantitative statement about pre-fix ciphertext expansion. It sets out the V2-CVF2 reasoning correctly and then closes: "After this fix, v8 ciphertext length is a deterministic function of the padding bucket alone, at the cost of always paying the worst-case ~20x expansion instead of the **~13.4x average case**."

Recomputed under the distribution the paper itself establishes, that baseline is wrong. Two figures are in play and the comment has taken the wrong one.

With the run cap in force, the token count per padded codepoint is $1 + \sum_{j=1}^{19} p^j$ with $p = \theta(N)/2^{64}$. `Remark 3.7` establishes that $\theta(N)$ is within total variation distance ≈ 0.033 of uniform on $[\theta_{\min}, \theta_{\max}]$ — we recomputed that distance and confirm it is exactly $1/30$ — so p is approximately uniform on $[0.75, 0.99]$. The mean of the capped count over that range is, in exact rational arithmetic, **8.4208**. Point values for orientation: 3.99 at $p = 0.75$, 18.21 at $p = 0.99$, and 7.218 at the midpoint $p = 0.87$.

The figure 13.4 is reachable two ways, and neither is the capped mean. It is the mean of the **uncapped** count $1/(1-p)$ over the same range, which is 13.412 — the expansion of a scheme with no run cap at all. It is also, coincidentally, the capped count evaluated

at the single point $p = 0.955$, which is 13.374, i.e. treating the noise probability as if it sat near the top of its range rather than at its mean. Both readings produce the printed number; neither is the average the sentence claims.

The consequence is that the comment understates what the ceiling actually costs. The true ratio is $20/8.42 = 2.38\times$, not $20/13.4 = 1.49\times$ — the fix is roughly sixty per-cent more expensive in bandwidth than the comment says. Since the comment exists precisely to record the price of the V2-CVF2 fix, a reader weighing whether to keep the ceiling, or comparing NAPQES's expansion against an alternative, is being handed the wrong number.

This is a code-documentation defect only. The paper states no average expansion — Section 7 and Section 12 both give the fixed 20-token, 160-byte-per-padded-codepoint figure, which is correct and is what `Lemma 3.9` requires — and no security claim, bound or theorem depends on the pre-fix average. The design decision the comment justifies is right on other grounds: a deterministic length is what the length-hiding argument needs, and no average expansion figure could change that.

Two of the four conformance regions in this pass arrived at the same correction independently, by different routes, and agree on 8.42. It is filed once. Note that a slightly different figure, 8.4221, appears in one region's working; the exact value computed in rationals is 8.42075, so 8.42 is the figure to use.

Recommendation

Replace "~13.4x average case" with the capped mean, and say which quantity is being averaged.

The clearest form states both, since both are informative and the ambiguity between them is what produced the error:

> at the cost of always paying the worst-case 20x expansion. Under the run cap alone, the mean token count per padded codepoint is about 8.4 for $\theta(N)$ approximately uniform on $[\theta_{\min}, \theta_{\max}]$ (`Remark 3.7`), ranging from 3.99 at $p = 0.75$ to 18.21 at $p = 0.99$; without the run cap it would be about 13.4. The ceiling therefore costs roughly 2.4x in bandwidth over the capped natural mean, and buys length determinism.

That is a small edit and it makes the trade-off legible: the reader sees the cost of the ceiling separately from the cost of the run cap, which the current sentence conflates.

If the intent was in fact to quote the uncapped mean — as a comparison against a hypothetical scheme with neither bound — then say so explicitly ("without the run cap, the mean would be 13.4"), because the sentence as printed attributes the figure to the code as it stood after the run cap was introduced and before the ceiling was.

Consider also adding the derivation, or a pointer to it, in one line: the capped count is $1 + \sum_{j=1}^{19} p^j$ and the uncapped one is $1/(1-p)$, both averaged over p approximately uniform on $[0.75, 0.99]$. The crate's comments are unusually thorough elsewhere and this is the only quantitative claim in them that a reader cannot check from the surrounding text.

No code change is required, and the ceiling should be kept. This finding is about the number in the comment, not about the design decision it justifies.

Client Comment

/docs/napseq-eprint-v4.tex

```
The noise threshold $\theta(N)$ (Section~\ref{sec:noiseprob}) deserves an
explicit correction here: it is \emph{not} bias-free. Although it uses exact
integer arithmetic and no modulo reduction, the fixed-point scaling
$\theta_{\min}+\lfloor\tau(\theta_{\max}-\theta_{\min})/2^{64}\rfloor$ maps
$2^{64}$ equally likely draws of $\tau$ onto approximately
$0.24\cdot2^{64}$ outputs, i.e.\ about $4.17$ preimages per output. Since
each output receives either $4$ or $5$ preimages, individual outputs
deviate from uniform on the range by up to $+20\%$ in relative terms, and
the total variation distance from the uniform distribution on
$[\theta_{\min},\theta_{\max}]$ is $\approx0.033$. This is harmless — the
only property required of $\theta(N)$ is that it land in the public
interval $[\theta_{\min},\theta_{\max}]$, which it does by construction,
and that $\mathsf{Dec}$ recompute it identically, which it does — but it
should not be described as exempt. The prime-index sampling of
Remark~\ref{rem:sampling} is genuinely bias-free, by rejection.
```

same file

```
To keep ciphertext length independent of the message-derived nonce's
particular noise realisation (Section~\ref{sec:wire-format}), the number
of \emph{consecutive} noise tokens emitted before each real token is capped
at a fixed constant $\mathrm{MAX\_NOISE\_RUN}=19$, and the total token
count is padded, with additional filler tokens structurally identical to
genuine noise tokens, up to a fixed ceiling of exactly
$R\cdot(\mathrm{MAX\_NOISE\_RUN}+1)$ tokens per message:
```

/src/lib.rs

```
/// Hard cap on consecutive noise tokens emitted before a real token in the
/// v8 token-emission loop (never applied to v7, matching `napqes.py`/
/// `C/napqes.c`). As of the V2-CVF2 fix, this cap also fixes the *total*
/// per-real-token budget: every v8 ciphertext is padded with additional
/// filler tokens up to a deterministic per-bucket ceiling of
/// `real_token_count * (MAX_NOISE_RUN + 1)` tokens (see
/// `encrypt_bytes_v8` / `decrypt_core_v8`). Without this, the *natural*
/// token count varies with the message-derived synthetic nonce even for a
/// fixed padding bucket, letting an observer who collects several
/// ciphertexts of one message under varying AAD average out the noise and
/// reliably recover the padding bucket (`docs/CAVEATS.md`, V2-CVF2). After
/// this fix, v8 ciphertext length is a deterministic function of the
/// padding bucket alone, at the cost of always paying the worst-case ~20x
/// expansion instead of the ~13.4x average case.
pub const MAX_NOISE_RUN: u64 = 19;
```

CVF-40: fixed Category: **readability**

Nine smaller inconsistencies in normative and near-normative text, none with a security consequence, collected as one finding: unbound indices, a 0-based/1-based clash between two statements of the same quantity, an over-general length-prefix claim its own lemma contradicts, an unreachable interval endpoint, two misleading analogies and descriptions, and one named parameter that exists only as a bare literal in the code.

Each item was checked against the source and, where a figure is involved, recomputed. They are grouped because each is a one-line or one-sentence repair and none justifies a separate finding; they are filed rather than dropped because they are all in text a conforming implementer reads as normative.

1. Section 3.3 over-claims length-prefixing, and `Lemma 3.5`'s own proof contradicts it. The derivation display is introduced with "every variable-width field inside ctx is length-prefixed (e.g. $\mathrm{be8}(|A|)\|A$ for AAD A)". That is not true of two fields: $\mathbb{B}$ in domain `0x03` and M in domain `0x0A` are both variable-width and carry no prefix. Injectivity for both comes from their being **last**, which is precisely how `Lemma 3.5`'s proof argues it — "because $\mathrm{be8}(|A|)$ fixes $|A|$, the split between A and $\mathbb{B}$ is unambiguous". So the

blanket sentence contradicts the lemma it introduces, and states a stronger property than the construction has or needs.

2. Table 1's output column uses an index that is never bound. Rows `0x01` and `0x05` give the output range as $[1, k_i - 1]$. Neither the table, its caption, nor the surrounding text ever says what i ranges over or which key element k_i denotes. `Definition 3.6` resolves it by taking k as a free parameter, but the table is the summary a reader consults first.

3. Section 3.7 indexes the key tuple 0-based where the rest of the paper is 1-based. Section 3.7 writes the key element as $k = \mathbf{k}[i \bmod K]$ — bracket notation, 0-based, and with i itself unbound, exactly as in item 2. `Dec` step (7) writes the same quantity as $k_{(\text{real_idx mod } K)+1}$: subscript notation, explicitly 1-based, over the tuple $(k_1, \ldots, k_K)$ that `Definition 3.1` declares. Both intend the same element, and the code agrees with the 1-based form and with Listing 1 (`primes[(real_idx % kk) as usize]`). But a reader implementing from Section 3.7's prose, using the paper's own 1-based tuple, lands one prime off for every token — a silent, total interoperability failure from a notation clash.

4. Table 1 does not record that domain `0x05`'s modulus comes from a different counter than its context. The noise-token addend's context is be5(ct_pos), but the modulus k it reduces against is drawn from `real_idx`, not from `ct_pos` — the noise tokens emitted inside a run use the key element of the real token they precede. Only Listing 1 records this, on one line. `Definition 3.6` takes k as a free parameter and so does not settle it, and Table 1 shows only the context. The code does the right thing.

5. θ_{max} is unreachable, so the stated interval is half-open in fact. `Remark 3.7` says the only property required of $\theta(N)$ "is that it land in the public interval $[\theta_{\text{min}}, \theta_{\text{max}}]$", and Table 1 gives the range as $[0.75, 0.99]$. Recomputed: with

$$\Delta = \theta_{\text{max}} - \theta_{\text{min}} = 4{,}427{,}218{,}577{,}690{,}292{,}387$$

, the maximum attainable value is

$$\theta_{\text{min}} + \lfloor (2^{64} - 1)\Delta / 2^{64} \rfloor = \theta_{\text{min}} + \Delta - 1 = \theta_{\text{max}} - 1$$

, so $p \in [0.75, 0.99)$ strictly. The closed interval is a superset, so nothing is wrong, but there is a related hair-splitting point in the same remark: the total variation distance is stated "from the uniform distribution on $[\theta_{\text{min}}, \theta_{\text{max}}]$", which contains $\Delta + 1$ integers, while the map's image contains only Δ. The difference is $O(1/\Delta)$ and does not change a printed digit — we recomputed the distance in exact rationals as $1/30 = 0.0333\ldots$, confirming the paper's ≈ 0.033.

6. "Agreed out of band, exactly like k and K" contradicts the sentence before it. Section 3.6 says the decryptor "needs *no* knowledge of the sender's profile", and then two sentences later that "The profile is a deployment parameter agreed out of band, exactly like **k** and K". The analogy is wrong in the direction that matters: **k** and K *must* be agreed or decryption fails outright, whereas the profile need not be agreed at all and a mismatch is undetectable and harmless to correctness. The crate's own comment gets this right — "The profile is a **sender-side** deployment parameter agreed out of band. It is never transmitted and `decrypt_bytes_v8` is profile-agnostic" — and `decrypt_bytes_v8` indeed takes no profile argument. The substantive consequence, that the profile is unauthenticated and untransmitted so `Corollary 4.14`'s guarantee is a property of the

sender's configuration alone, is CVF-5's territory; the misleading analogy is a separate one-line fix.

7. Table 2's two-class parenthetical understates the short class. The narrative says the two message classes "fall in different buckets (16 and 512) whatever the expansion factor". Recomputed: under `bucket` the short class (12–20 codepoints) straddles **two** buckets, $B = 16$ for $n \in [12, 15]$ and $B = 32$ for $n \in [16, 20]$; under `coarse(3)` it straddles $B = 16$ and $B = 128$. The long class (300–500) is $B = 512$ under `bucket` and $B = 1024$ under `coarse(3)`. The conclusion is unaffected — the class sets remain disjoint, and straddling a boundary makes the distinguisher no weaker — but the parenthetical describes the short class as a single bucket, which is wrong for both non-trivial profiles in the table. Relatedly, the `frame(1024)` row's 50% figure depends on $500 < 1024$, which the text does not spell out.

8. Section 12 calls the length prefix "2-byte" where it is two codepoints. `Definition 3.10` and `Dec` step (8) both correctly say "2-codepoint big-endian length prefix". Section 12's caveat says the scheme "stores the plaintext length as a 2-byte big-endian integer". On the wire the prefix is two *codepoints*, which become two real tokens, which occupy up to 40 token slots and 320 bytes. Both phrasings give the same 65,535 cap, so the caveat's conclusion is right, but its description of the mechanism understates the on-wire cost by a factor of 160.

9. `MAX_PLAINTEXT_CODEPOINTS` is a named parameter in the paper and a bare literal in the code. `Definition 3.10` names

$$\text{MAX_PLAINTEXT_CODEPOINTS} = 2^{16} - 1$$

, and the crate gives named `pub const` counterparts to every other public parameter — `MIN_KEY_PRIME`, `MAX_KEY_PRIME`, `MAX_NOISE_RUN`, `PAD_MIN_EXP`, `PAD_MAX_EXP`, `TOKEN_WIDTH`, `NONCE_SIZE`, `TAG_SIZE`, `SK_SIZE`. This one does not: it appears as `0xFFFF` at two sites, and those two sites have **diverged in failure mode** — one panics, the other returns an error. That divergence is the substance of a separate finding; the missing constant is what let it happen.

Two related observations, checked and found clean, recorded so they are not re-opened. `Enc` derives the synthetic nonce over the whole message before rejecting an over-long one, so a rejected 100 MB input pays a full-message HMAC first — wasted work on an error path, no security consequence, and a conforming implementation may reorder freely since both are input-domain guards. And the Notation section's blanket "each field is fixed-width; x is assumed to fit" never states the 2^{40} ceiling that be5 imposes on `ct_pos` and `real_idx`; we confirmed the precondition is discharged at current parameters, since the maximum reachable `ct_pos` is $20 \times 65{,}538 = 1{,}310{,}760 \ll 2^{40}$, but it is discharged by a bound the paper does not state.

Recommendation

Nine small edits, seven in the paper and two spanning both.

1. Narrow the Section 3.3 sentence to what `Lemma 3.5` actually proves: every variable-width field inside ctx is either length-prefixed or terminal. That covers A (prefixed) and B and M (terminal) and matches the proof exactly.
2. Bind the index in Table 1. Writing the output range as $[1, k-1]$ for the key element k selected at that position, and adding one clause to the caption saying which

counter selects it, removes the free variable without lengthening the table.

3. Make Section 3.7 use the paper's own 1-based tuple notation: $k = k_{(\text{real_idx mod } K)+1}$, matching `Dec` step (7) and Listing 1. This is the item to fix first — it is the only one in this finding whose misreading produces a wholly incorrect implementation, and the fix is two symbols.
4. Add a note to Table 1's `0x05` row, or a sentence to `Definition 3.6`, recording that the noise-token addend's modulus is the key element selected by `real_idx` while its context is `be5(ct_pos)`. At present a reader must reconstruct this from Listing 1.
5. In `Remark 3.7`, write the attainable range as $[\theta_{\min}, \theta_{\max})$ and note that $\theta_{\max}$ itself is not attained; adjust Table 1's range column to $[0.75, 0.99)$ for consistency. The total variation figure needs no change — ≈ 0.033 is correct, and the exact value is $1/30$ — but the sentence measuring it against uniform on the closed interval can say "on the image of the map" instead.
6. Replace "exactly like **k** and K" with the distinction the code already draws: the profile is a sender-side parameter that need not be agreed, unlike **k** and K, which must be. One clause.
7. Correct the parenthetical to name the buckets the classes actually occupy — $\{16, 32\}$ against $\{512\}$ under `bucket`, $\{16, 128\}$ against $\{1024\}$ under `coarse(3)` — and add the half-sentence explaining that `frame(1024)`'s 50% follows from 1024 exceeding both classes.
8. Change Section 12's "2-byte big-endian integer" to "2-codepoint big-endian prefix", matching `Definition 3.10`. If the caveat is meant to convey the storage cost, say that the two codepoints become two real tokens and occupy up to 320 bytes on the wire, since that is the number a deployer sizing a channel needs.
9. Add `pub const MAX_PLAINTEXT_CODEPOINTS: usize = 0xFFFF;` to the crate and use it at both existing sites. This is the item that pays for itself: naming the bound is what stops the two enforcement points drifting apart again, and they have already drifted once.

   Additionally, consider adding one sentence to the Notation section recording that be5 bounds any counter it encodes at 2^{40}, and that `ct_pos` and `real_idx` are within it because MAX_PLAINTEXT_CODEPOINTS and the token ceiling together cap `ct_pos` at $20(2^{16} + 1) = 1{,}310{,}760$. The blanket "assumed to fit" is doing real work here and the discharge is currently left to the reader.

Client Comment

/docs/napseq-eprint-v4.tex

```
251   \item $\mathrm{be4}(x)$, $\mathrm{be5}(x)$, $\mathrm{be8}(x)$
denote the
```

```
252         4-, 5-, and 8-byte big-endian encodings of the non-negative
integer
253         $x$ (each field is fixed-width; $x$ is assumed to fit).
```

same file

```
349 where $d$ is a 1-byte domain separator and every variable-width field
350 inside $\mathsf{ctx}$ is length-prefixed (e.g.\ $\mathrm{be8}(|A|) \|
A$
351 for AAD $A$). Domain $\texttt{0x0A}$ is the single exception: it does
not
```

same file

```
377 \texttt{0x01} & Real-token addend & $\mathrm{be5}(real\_idx)$ & $[1,
k_i - 1]$ \\
```

same file

```
381 \texttt{0x05} & Noise-token addend & $\mathrm{be5}(ct\_pos)$ & $[1,
k_i - 1]$ \\
```

same file

```
419 for domain $\texttt{0x03}$,
$\mathsf{ctx}=\mathrm{TagIn}(A,\widetilde{B})=\mathrm{be8}(|A|)\|A\|\widetilde{B
420 and because $\mathrm{be8}(|A|)$ fixes $|A|$, the split between $A$ and
421 $\widetilde{B}$ is unambiguous, so distinct $(A,\widetilde{B})$ pairs
never
422 collide. Within domain $\texttt{0x0A}$, whose input is
423 $\texttt{0x0A}\|\mathrm{be8}(|A|)\|A\|M$ with no nonce field at all,
the
424 same length-prefix argument shows $\mathrm{be8}(|A|)\|A\|M$ is
injective in
425 $(A,M)$. Every case is a statement about byte layout that holds for
```

same file

```
500 \emph{no} knowledge of the sender's profile: it recovers $R$ from the
token
501 count (Section~\ref{sec:wire-format}) and the true length $n$ from
the
```

same file

```
504 check against the same fixed set. The profile is a deployment
parameter
505 agreed out of band, exactly like $\mathbf{k}$ and $K$; it is never
506 transmitted and does not appear in the wire format.
```

same file

```
568 the total variation distance from the uniform distribution on
569 $[\theta_{\min},\theta_{\max}]$ is $\approx0.033$. This is harmless —
the
```

```
only property required of $\theta(N)$ is that it land in the public interval $[\theta_{\min},\theta_{\max}]$, which it does by construction,
```

same file

```
Real and noise tokens are integers $c\cdot k + a$ for codepoint $c$ (or a noise character), key element $k=\mathbf{k}[i \bmod K]$, and addend
$a$ from Definition~\ref{def:token-prims}. Because $k$ is prime and every
```

same file

```
        $\mathrm{MAX\_PLAINTEXT\_CODEPOINTS}=2^{16}-1$, the
        limit imposed by the 2-codepoint big-endian length prefix written
        during padding (Section~\ref{sec:padding}). The surrogate range is
```

same file

```
        excluded because the synthetic-nonce derivation
        (Definition~\ref{def:synthnonce}) hashes the UTF-8 encoding of $M$,
        and UTF-8 is undefined on unpaired surrogates; $\mathcal{M}$ is thus
```

same file

```
token $t$ and recover the padded codepoint as
```

same file

```
addend $\mathsf{Enc}$ used at that index; stop once $R$ real tokens have
been extracted, discarding any trailing filler; (8) read the 2-codepoint
big-endian length prefix $n$ and, if $n>R-2$, output $\bot$
```

same file

```
the two classes fall in different buckets ($16$ and $512$) whatever the
```

same file

```
  \item[Plaintext length cap.] The block padding scheme stores the
        plaintext length as a 2-byte big-endian integer, capping messages
```

/src/lib.rs

```rust
fn be5(n: u64) -> [u8; 5] {
    let b = n.to_be_bytes();
    let mut out = [0u8; 5];
```

```
    out.copy_from_slice(&b[3..8]);
    out
}
```

same file

```
/// The profile is a sender-side deployment parameter agreed out of band. It is
/// never transmitted and [`decrypt_bytes_v8`] is profile-agnostic.
```

same file

```
    assert!(n <= 0xFFFF, "Message too long for 2-byte length prefix");
```

same file

```
    let nonce = synthetic_nonce(&sk_fmt, aad, message.as_bytes());
    let codepoints: Vec<u32> = message.chars().map(|c| c as u32).collect();
```

same file

```
    if codepoints.len() > 0xFFFF {
```

CVF-42: fixed Category: **naming**

Three normative derivation parameters appear as bare literals in the v7 derivation functions while their v8 counterparts are named constants

The crate gives a named `pub const` to almost every public parameter of the scheme — `MIN_KEY_PRIME`, `MAX_KEY_PRIME`, `MAX_NOISE_RUN`, `PAD_MIN_EXP`, `PAD_MAX_EXP`, `TOKEN_WIDTH`, `NONCE_SIZE`, `TAG_SIZE`, `SK_SIZE`. Three values that `Definition 3.6` fixes normatively are exceptions, and in two of the three cases the *same* quantity is a named constant on the v8 path and a literal on the v7 path.

The noise-probability endpoints are the clearest instance. `derive_noise_p`, the v7 form, writes them inline:

```
let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
0.75 + t * (0.99 - 0.75)
```

while the v8 form five lines below declares them:

```
const THETA_MIN: u64 = ((75u128 << 64) / 100) as u64;
const THETA_MAX: u64 = ((99u128 << 64) / 100) as u64;
```

and `derive_noise_threshold_v8` uses those names. So $\theta_{\min}$ and $\theta_{\max}$ are named in one derivation and are four floating-point literals — `0.75`, `0.99`, and `0.75` again inside the difference — in the other. A change to either endpoint has to be made in two notations at two sites, one of which does not mention the parameter's name.

The noise-character alphabet is the second instance. `derive_noise_char` ends `(u32_from_be4(&d[..4]) as u64 % 96) + 32` with no constant and no comment. Its sibling in `pad_to_block` ends `(u32_from_be4(&d[..4]) % 95) + 32` with the comment `// [32, 126]`.

We checked whether the two moduli disagreeing is itself a defect, and it is not: `Definition 3.6` specifies mod 96 for `noise_char`, giving $[32, 127]$, and mod 95 for the padding codepoint, giving $[32, 126]$, and both sites conform. The two functions are five lines apart, differ by one in a magic modulus, and are correct — which is precisely the configuration in which a later editor "fixes" the discrepancy and silently changes the wire format. Only one of the two carries the comment that would stop them.

There is no defect here today and no security consequence; all three sites match the specification. It is filed because CVF-40's ninth item makes exactly this argument about `MAX_PLAINTEXT_CODEPOINTS`, where the parameter was left unnamed, appeared as `0xFFFF` at two sites, and **those two sites then diverged in failure mode**. The same precondition holds at all three sites here.

Recommendation

Name the three parameters and use the names at every site, following the pattern `THETA_MIN` and `THETA_MAX` already establish for the v8 path:

```
/// Noise-character alphabet, `Definition 3.6`: `(... mod
  NOISE_ALPHABET) + NOISE_ALPHABET_BASE`.
const NOISE_ALPHABET: u64 = 96;          // yields [32, 127]
/// Padding-codepoint alphabet, `Definition 3.6`: printable ASCII
  only.
const PAD_ALPHABET: u64 = 95;            // yields [32, 126]
const ASCII_PRINTABLE_BASE: u64 = 32;
```

and express the v7 endpoints in terms of the constants the v8 path already has, rather than as a second set of literals in a second numeric type:

```
const THETA_MIN_F64: f64 = 0.75;
const THETA_MAX_F64: f64 = 0.99;
...
THETA_MIN_F64 + t * (THETA_MAX_F64 - THETA_MIN_F64)
```

Add a one-line comment at `derive_noise_char` stating the resulting range, as `pad_to_block` already does, and stating that the difference from the padding modulus is intentional and normative. That comment is the part worth having even if the constants are not introduced: it is what prevents a future editor from reconciling 96 and 95 and changing the wire format in the process.

Apply the same naming to `MAX_PLAINTEXT_CODEPOINTS` at the same time — CVF-40 item 9 asks for it, the two sites there have already drifted, and doing all four in one change keeps the parameter table in the crate and the parameter table in `Definition 3.6` in one-to-one correspondence.

Client Comment

/src/lib.rs

```
196     (u32_from_be4(&d[..4]) as u64 % 96) + 32
```

same file

```
213     let t = u64_from_be8(&d[..8]) as f64 / TWO_POW_64;
214     0.75 + t * (0.99 - 0.75)
```

same file

```
219 const THETA_MIN: u64 = ((75u128 << 64) / 100) as u64;
220 const THETA_MAX: u64 = ((99u128 << 64) / 100) as u64;
```

same file

```
417         out.push((u32_from_be4(&d[..4]) % 95) + 32); // [32, 126]
```

CVF-44: fixed Category: **documentation**

`decrypt_core_v8`'s truncation error names one of the three conditions that reach it, and is inaccurate for the other two

The v8 decryptor's noise-run consumer is followed by a single exhaustion guard with a single message:

```
while noise_run < MAX_NOISE_RUN
    && ct_pos < n_tokens
    && is_noise_pos_v8(kb, nonce, ct_pos, noise_theta)
{
    ct_pos += 1;
    noise_run += 1;
}
if ct_pos >= n_tokens {
    return Err(
        "Truncated v8 ciphertext: token stream ended mid noise-run
  before the \
         expected real token.".into(),
    );
}
```

Three distinct states satisfy `ct_pos >= n_tokens` at that point, and the message describes only the first:

1. The inner loop exited on `ct_pos < n_tokens` with `noise_run > 0` — the stream genuinely ran out part-way through a noise run. The message is accurate.
2. The inner loop exited on `noise_run < MAX_NOISE_RUN`, having consumed a **complete** maximal run of `MAX_NOISE_RUN` tokens that happened to end exactly at `n_tokens`. Nothing was truncated mid-run; the run finished and the real token that should have followed it is absent. The message asserts the opposite of what happened.
3. The inner loop exited on `ct_pos < n_tokens` with `noise_run == 0` — the previous real token was the last in the stream and no noise run was ever entered. There is no noise run to be in the middle of.

Case 2 is the one Mikhail Vladimirov identified; case 3 falls out of the same reading. In all three the diagnosis a reader needs is the same — the token stream is shorter than the token count implies — and in two of the three the message misdirects them towards the noise schedule.

The condition is reachable only for a ciphertext that has already passed the tag check, the token-count divisibility check and the bucket check, so this is a diagnostic-quality issue with no security consequence. It is filed because `decrypt_core_v8`'s error strings are the only description of the post-authentication structural checks a client operator will ever see at run time, and because CVF-25 records that the negative KAT vectors currently assert only `is_err()` and never which error — so an inaccurate message is not caught by any test, and a vector aimed at one of these three states cannot presently be told apart from the others.

Recommendation

State the condition rather than one of its causes, and distinguish the cases the decryptor can actually tell apart:

```
if ct_pos >= n_tokens {
    return Err(format!(
        "Truncated v8 ciphertext: the token stream ended after {}
  of {} tokens \
         with {} of {} real tokens recovered; a real token was
  expected at \
         position {} (noise run length {}).",
        ct_pos, n_tokens, real_idx, real_count, ct_pos, noise_run
    ));
}
```

The counters are already in scope, and reporting them removes the need for the message to guess which of the three states produced the exhaustion — a reader can see it. If a fixed string is preferred over a formatted one, "Truncated v8 ciphertext: token stream ended before the expected real token" is accurate for all three cases and is the existing message with two words removed.

When CVF-25's recommendation to assert error text in the negative KAT tests is applied, add a vector for case 2 specifically: a ciphertext whose final bucket is a com-

plete `MAX_NOISE_RUN` run with the trailing real token removed. It is the only one of the three that the current message actively misreports, and it is cheap to construct from the existing generator.

Client Comment

/src/lib.rs

```
576     while real_idx < real_count {
577         let mut noise_run: u64 = 0;
```

same file

```
578         while noise_run < MAX_NOISE_RUN
579             && ct_pos < n_tokens
580             && is_noise_pos_v8(kb, nonce, ct_pos, noise_theta)
581         {
582             ct_pos += 1;
583             noise_run += 1;
584         }
```

same file

```
585         if ct_pos >= n_tokens {
586             return Err(
587                 "Truncated v8 ciphertext: token stream ended mid
noise-run before the \
588                  expected real token.".into(),
589             );
590         }
```

6. Round 5: Review of Fixes at Commit f6ced5d

This section reports the fifth review round: a review of the client's remediation of the round-4 findings listed above. The code under review is the client's commit

`f6ced5d94dfee098b310f8521a3dbdc2b3a4174b`

The hash is as stated by the client. The delivery contained no `.git` metadata, so it could not be verified locally.

What was reviewed. The review covered the client's seven in-scope files: the Rust sources `src/lib.rs`, `src/self_test.rs`, `src/kat_cross_check.rs`, `fuzz/fuzz_targets/decode_bytes.rs` and `examples/dudect_harness.rs`, together with `Cargo.toml` and `README.md`. The scope was extended to three files that are new in this release, `rust/src/key.rs` and the fuzz targets `fuzz/fuzz_targets/unauth_decode.rs` and `fuzz/fuzz_targets/validate_key.rs`, because the fixes for three round-4 findings were moved into them. The updated specification delivered with the commit served as the conformance reference. Despite its file name, `docs/napseq-eprint-v3.tex` is the newest revision (see CVF-V5-23). The review was static: none of the client's code was compiled or executed.

How. Every round-4 finding was re-checked against the new code. Separately, encryption was re-implemented from the updated specification alone, and the re-implementation reproduced all 22 positive v8 test vectors byte for byte. This settles the round-4 divergence between specification and implementation reported as CVF-11: the specification now describes the scheme the implementation computes.

Outcome for the round-4 findings. Of the 37 round-4 findings, 25 are fixed and 12 are partially fixed. None is unresolved and none has regressed. The statuses shown in the Findings section above reflect this review: fixed findings are marked fixed, and partially fixed findings remain open. Each partially fixed finding is carried into this section as a new finding that describes what remains.

New findings. This section reports 31 findings: 8 of moderate severity and 23 minor, with none major. Nineteen are new in this round. Seventeen of these were introduced by the remediation itself, and two are older defects reported for the first time (CVF-V5-15 and CVF-V5-21). The other twelve are the carried partial fixes.

Scale of the change. Compared with the round-4 code, the remediation adds 1,291 net-new lines of production code to the reviewed files, of which 934 are security-critical (cryptographic cores, key and nonce derivations, validation and gating). The count excludes blank lines, comments, test code and relocated code.

Numbering. Findings in this section carry identifiers of the form CVF-V5-N. A bare CVF-N anywhere in this report, including inside the findings below, denotes a round-4 finding from the Findings section above. The source excerpts quote the client's code and comments verbatim. Some of those comments cite the client's own labels for findings of earlier rounds (for example `CVF1`, `V2-CVF11` or "CVF-3-class"). These labels do not refer to findings in this report.

	Found	Fixed
Moderate	**8**	**0**
Minor	**23**	**0**

Fixed 0 out of 31 issues reported in this round.

6.1. Moderate

CVF-V5-1: open Category: **flaw**

The v8 streaming format transmits the exact plaintext length in clear in its sentinel, although the crate documentation lists the streaming entry points under the v8 length-hiding guarantee.

This defect was introduced by the round-5 remediation: the v4 crate contained no streaming implementation, only an unused `FORMAT_STREAM_AE_V8` constant.

Both streaming encryptors end the stream with a sentinel made of a zero chunk-length marker, the total number of real codepoints as an unencrypted 8-byte big-endian integer, and the sentinel tag. `encrypt_stream_ae_v8_inner` appends `total_real_cps.to_be_bytes()` directly to the output (`rust/src/lib.rs:2019-2023`), and `StreamV8Encryptor::finish` does the same (`rust/src/lib.rs:2333-2337`). The decryptor reads the value back from the clear bytes before verifying the sentinel tag (`rust/src/lib.rs:2129-2136`). The tag authenticates the count but does not hide it.

The crate-level documentation (`rust/src/lib.rs:17-25`) lists `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8` and `StreamV8Encryptor` among the v8 entry points and

then states "Length-hiding property holds per Corollary 4.14 of the paper". `SPEC.md` section 8.2 (`SPEC.md:513-594`) likewise claims "length hiding at chunk granularity", and the filler codepoints derived under domain `0x06` exist only to pad the last chunk to `F` codepoints. The filler hides nothing, because the exact count it is meant to hide is written next to it. For example, with `F = 16`, messages of 17 and 31 codepoints both produce 5,257-byte ciphertexts, but the 8 bytes at offset `len - 40` read 17 and 31 respectively. In every positive vector of `tests/kat/v8_stream_vectors.json` this field equals the length of the message.

The regression test `stream_v8_length_is_function_of_f_only` (`rust/src/lib.rs:3302-3311`) compares only the byte lengths of two ciphertexts, so it passes while the length is in the clear. Independently of the sentinel, the chunk count reveals the length to within one frame (`ceil(n/F)`), which with the default `F = 128` is far coarser than the at most 3.70 bits that the block format's bucketing leaks.

Evidence was established by reading the code and by an independent re-implementation of the streaming format, which reproduces all 14 stream vectors.

Recommendation

Do not transmit the real-codepoint count in clear. Carry it inside the encrypted token stream of the final chunk, as the block format does with its two-codepoint length prefix, or mask it with a keystream under a new domain while keeping it bound by the sentinel tag. State the residual leakage of the chunk count explicitly, remove the Corollary 4.14 claim for the streaming entry points, and replace the test with one that compares the full sentinel bytes for two messages with the same chunk count.

Client Comment

/rust/src/lib.rs

```
17 //! * **v8 (normative — use this).** Domain-`0x0B` format subkey
18 //!   ([`derive_format_subkey`], per CVF-11), synthetic nonce
19 //!   ([`synthetic_nonce`], domain `0x0A`), `MAX_NOISE_RUN`-capped
   emission
20 //!   loop, per-bucket token ceiling. Wire layout is
21 //!   `nonce(16) || masked_blob(fixed-width) || tag(32)`. Entry
   points:
22 //!   [`encrypt_bytes_v8`], [`decrypt_bytes_v8`],
   [`generate_v8_key`],
23 //!   [`encrypt_bytes_v8_with_profile`], [`encrypt_stream_ae_v8`],
24 //!   [`decrypt_stream_ae_v8`], [`StreamV8Encryptor`]. Length-hiding
   property
25 //!   holds per Corollary 4.14 of the paper.
```

same file

```
    let sentinel_tag = compute_stream_v8_sentinel_tag(
        sk_fmt, nonce, chunk_idx, aad, total_real_cps);
    stream.extend_from_slice(&0u32.to_be_bytes());
    stream.extend_from_slice(&total_real_cps.to_be_bytes());
    stream.extend_from_slice(&sentinel_tag);
```

same file

```
        if chunk_len == 0 {
            // SENTINEL: be8(total_real_cps) || tag(32)
            if stream.len() < pos + 8 + TAG_SIZE {
                return Err("Stream truncated: need sentinel body.".into());
            }
            let mut tr = [0u8; 8];
            tr.copy_from_slice(&stream[pos..pos + 8]);
            let total_real_cps = u64::from_be_bytes(tr);
```

same file

```
        let tag = compute_stream_v8_sentinel_tag(
            &self.sk_fmt, &self.nonce, self.chunk_idx, &self.aad, self.total_real_cps);
        out.extend_from_slice(&0u32.to_be_bytes());
        out.extend_from_slice(&self.total_real_cps.to_be_bytes());
        out.extend_from_slice(&tag);
```

same file

```
    #[test]
    fn stream_v8_length_is_function_of_f_only() {
        // Two messages that both land in the "2 chunks" bucket at F=16 must
        // produce ciphertexts of equal length -- the V2-CVF11-for-streaming
        // invariant.
        let (primes, sk) = generate_v8_key(DEFAULT_KEY_COUNT, MIN_KEY_PRIME, MAX_KEY_PRIME);
        let a = encrypt_stream_ae_v8(&"A".repeat(17), &primes, &sk, b"", 16).unwrap();
        let b = encrypt_stream_ae_v8(&"B".repeat(31), &primes, &sk, b"", 16).unwrap();
        assert_eq!(a.len(), b.len());
    }
```

CVF-V5-2: open Category: **documentation**

A complete new authenticated-encryption mode, the v8 streaming format, ships as a recommended normative API with no security definition, no theorem, an incomplete specification, and no restatement of the unauthenticated-streaming caveat dropped in round 3.

This was introduced by the round-5 remediation. Round 4 recorded only an unused `FORMAT_STREAM_AE_V8` constant and asked the client either to specify the format and restore or supersede caveat `CAV-001`, or to remove the constant. Version 5 instead ships about 630 lines of streaming code. The mode has its own CSPRNG nonce rather than the block format's synthetic nonce, a per-stream format subkey `HMAC(sk, 0x0B||0x02)`, three new domains (`0x0C` chunk tag, `0x0D` sentinel, `0x0E` sub-nonce), per-chunk tags and a terminating sentinel. The crate documentation lists it under "v8 (normative - use this)" and the README lists it under "Recommended (v8)".

No security claim covers it:

- The paper reserves format `0x02` for "a streaming-AE format that is outside the scope of this paper", pointing to its caveats section and to `SPEC.md` section 8.2. The caveats section never mentions streaming.
- The paper's domain table omits `0x0C` to `0x0E`.
- The abstract and section 3.3 assert separation between the block and streaming formats, but the security games query the format subkey only for `0x01`, so that separation is not proved.
- The crate's own comments disagree with each other: the streaming header calls the functions "misuse-resistant primitives", the entry point says "Not misuse-resistant: nonce reuse under the same sk is catastrophic", and the production core is labelled "Test-only".
- The README states that the ciphertext carries no format identifier, yet every stream begins with the byte `0x02`.

The specification is also not sufficient to implement the format. An independent reimplementation written from `SPEC.md` section 8.2 and the paper alone reproduced the 11 positive stream vectors only after trying 12 combinations of readings. The text does not determine:

- the base of the chunk index (0 or 1);
- the index fed to the filler derivation;
- whether the token position and real-codepoint counters restart for each chunk.

The text also states no lower bound on `F` (`F = 0` would collide with the zero sentinel marker), no valid range for the transmitted total, no per-token decoder checks for streaming, and no limit on the 32-bit chunk count. Its "verify-before-yield" rule releases each chunk before the sentinel is verified, which is the release-of-unverified-plaintext behaviour that `CAV-001` described. The Rust decryptor happens to buffer everything, but the specification does not require it to.

Further, `docs/CAVEATS.md` (`CAV-005`) gives the per-key data limit in streams, but the relevant birthday quantity is the total number of chunks, since every chunk uses a 128-bit truncated sub-nonce.

Recommendation

Either give the streaming format a stated security model (nonce-based online AE with a random nonce) and at least a proof sketch that reduces it to the PRF assumption on HMAC-SHA256, covering both format subkeys with a two-query hop, or remove it from the recommended and normative lists and place it behind a feature until that analysis exists. Complete `SPEC.md` section 8.2 so that it determines every byte: chunk-index base, filler index, counter reset per chunk, bounds on `F` and on the total, and decoder checks. Add `0x0C` to `0x0E` to the paper's domain table, restate the release-of-unverified-plaintext caveat, give the data limit in chunks, and correct the contradictory "misuse-resistant" and "Test-only" comments and the README's statement about the format identifier.

Client Comment

/SPEC.md

````
## 8.2 Streaming AE wire format v8 (`FORMAT_STREAM_AE_V8 = 0x02`)

*(napqes.py `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8`;
rust/src/lib.rs `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8`,
`StreamV8Encryptor`; C/napqes.c `napqes_encrypt_stream_ae_v8_bytes`,
`napqes_decrypt_stream_ae_v8_bytes`)*

The v8 streaming construction upgrades §8.1 to the full v8 primitive
family
(independent 256-bit subkey `sk` + per-format subkey
`sk_fmt = HMAC(sk, 0x0B || FORMAT_STREAM_AE_V8)`), uses fixed-width
8-byte
tokens (closing the per-token length leak retained by §8.1 for LEB128
tokens), and pads each chunk to a fixed `F * (MAX_NOISE_RUN + 1)`
token
ceiling so per-chunk ciphertext size is a pure function of the
sender-chosen
frame parameter `F`, which is public.

```
stream = 0x02 || uint32_be(F) || nonce (16 bytes)            # HEADER
(21 B)
       || [ uint32_be(F * 160)                                # CHUNK
FRAME × C
            || masked_chunk (F * 160 bytes)                     #
(each frame is
````
````

```
           || chunk_tag (32 bytes) ] × C                       #   4 + F*160 + 32 B,
                                                                 # fixed per stream)
       || uint32_be(0) || uint64_be(total_real_codepoints)      # SENTINEL (44 B)
            || sentinel_tag (32 bytes)
```

`F` = codepoints per chunk (sender-chosen; default 128, hard upper bound 16384). Every chunk carries exactly `F` real-codepoint slots; if the input stream ends mid-chunk, the remaining slots are filled with HMAC-derived filler codepoints (domain `0x06`, keyed by the per-chunk sub-nonce). The decoder strips filler using `total_real_codepoints` from the sentinel.

Sub-nonce (domain `0x0E`). Each chunk `i` is encrypted as a self-contained v8 primitive call keyed by `(sk_fmt, sub_nonce_i)` where

```
sub_nonce_i = HMAC(sk_fmt, 0x0E || nonce || uint32_be(i))[:16]
```

so domains `0x00`-`0x07` (position oracle, addend, noise char/addend, threshold, keystream) are reused verbatim from v8 block mode.

Per-chunk tag (domain `0x0C`).

```
chunk_tag_i = HMAC(sk_fmt,
    0x0C || nonce || uint32_be(i) || uint64_be(len(aad)) || aad || masked_chunk)
```

Sentinel tag (domain `0x0D`).

```
sentinel_tag = HMAC(sk_fmt,
    0x0D || nonce || uint32_be(C) || uint64_be(len(aad)) || aad
         || uint64_be(total_real_codepoints))
```

```
568
569 The sentinel binds *both* the chunk count `C` (anti-truncation) and
    the
570 real-codepoint count (needed to strip filler from the last chunk
    safely).
571
572 **Security properties:**
573 - **Verify-before-yield.** Per-chunk tag verifies before any of that
574   chunk's plaintext contributes to the output; the sentinel verifies
575   before the last chunk's real codepoints are emitted.
576 - **Anti-truncation.** Chunk count `C` is bound into `sentinel_tag`.
577 - **Anti-reorder.** `chunk_idx` is bound into every `chunk_tag_i`.
578 - **Length hiding at chunk granularity.** Per-chunk masked_blob
    length is
579   a pure function of `F`, which is public — the LEB128 residual
580   (V2-CVF11) that §8.1 retains is closed here.
581 - **Not misuse-resistant** — the 16-byte nonce is CSPRNG-drawn per
    stream,
582   not SIV-derived (SIV requires hashing the full message, which is
583   incompatible with streaming). Nonce reuse across two streams under
    the
584   same `sk` is catastrophic (CVF3-class hazard on aligned codepoint
585   positions). See `docs/CAVEATS.md` CAV-005.
586
587 **Compatibility.** v8 streams are **not** cross-compatible with:
588 - §5 (v7 block) — different keying and no format subkey.
589 - §8   (v7 streaming basic, deprecated) — different keying, no format
    subkey.
590 - §8.1 (v7 streaming AE) — different keying, different domain bytes.
591 - v8 *block* mode — different format subkey (`FORMAT_STREAM_AE_V8` vs
592   `FORMAT_BLOCK_V8`) and different domain bytes.
593
594 ---
```

/docs/napseq-eprint-v3.tex

```
421 $\mathrm{format\_id}=\texttt{0x01}$ (block mode). The identifier
422 $\mathrm{format\_id}=\texttt{0x02}$ is reserved for a streaming-AE
423 format that is outside the scope of this paper
424 (Section~\ref{sec:caveats}); its wire format is specified in
425 \texttt{SPEC.md} \S 8.2. Domain $\texttt{0x0B}$ takes no nonce and
```

same file

```
430 The purpose of this layer is \emph{cross-format tag separation}:
    because
431 every subsequent HMAC call is keyed by $\mathsf{sk}_{\mathrm{fmt}}$
432 rather than $\mathsf{sk}$, a tag or ciphertext produced under one
```

```
$\mathrm{format\_id}$ cannot verify under another sharing the same
$(\mathbf{k},\mathsf{sk})$. The ciphertext itself carries no
$\mathrm{format\_id}$ byte on the wire — format selection is an
out-of-band API contract — so a ciphertext produced under one format
subkey simply fails the tag comparison at the receiver and is
indistinguishable from a wrong-key ciphertext.

\paragraph{Per-message derivations.}
```

/rust/README.md

```
### Wire format

```text
v8 block:    nonce(16) || masked_blob(fixed-width tokens) || tag(32)
v8 streaming:                                          (see below)
```

The ciphertext carries no format identifier — format selection is an
out-of-band API contract. A v7 ciphertext handed to
`decrypt_bytes_v8`
```

same file

```
**Recommended (v8):**
`NapqesKey` with `encrypt_bytes_v8_key`, `decrypt_bytes_v8_key`,
`encrypt_bytes_v8_with_profile_key` (validates once, wipes on drop);
`encrypt_bytes_v8`, `encrypt_bytes_v8_with_profile`,
`decrypt_bytes_v8`,
`encrypt_stream_ae_v8`, `decrypt_stream_ae_v8`, `StreamV8Encryptor`,
`generate_v8_key`, `zeroize_key`, `zeroize_sk`.
```

/rust/src/lib.rs

```
//! * **v8 (normative — use this).** Domain-`0x0B` format subkey
//!   ([`derive_format_subkey`], per CVF-11), synthetic nonce
//!   ([`synthetic_nonce`], domain `0x0A`), `MAX_NOISE_RUN`-capped
emission
//!   loop, per-bucket token ceiling. Wire layout is
//!   `nonce(16) || masked_blob(fixed-width) || tag(32)`. Entry
points:
//!   [`encrypt_bytes_v8`], [`decrypt_bytes_v8`],
[`generate_v8_key`],
//!   [`encrypt_bytes_v8_with_profile`], [`encrypt_stream_ae_v8`],
//!   [`decrypt_stream_ae_v8`], [`StreamV8Encryptor`]. Length-hiding
property
//!   holds per Corollary 4.14 of the paper.
```

same file

```
// --- Streaming AE v8 (misuse-resistant primitives, streaming nonce) -----------
//
// Wire format (FORMAT_STREAM_AE_V8 = 0x02):
//   HEADER (21 B):     0x02 || be4(F) || nonce(16, CSPRNG)
//   CHUNK FRAME * C:   be4(F*160) || masked_chunk(F*160) || chunk_tag(32)
//   SENTINEL (44 B):   be4(0) || be8(total_real_codepoints) || sentinel_tag(32)
//
// Every chunk i is a self-contained v8 primitive call keyed by
// (sk_fmt, sub_nonce_i) where
//     sk_fmt      = HMAC(sk, 0x0B || FORMAT_STREAM_AE_V8)
//     sub_nonce_i = HMAC(sk_fmt, 0x0E || nonce || be4(i))[:16]
// so domains 0x00-0x07 (position oracle, addend, noise char/addend, threshold,
// keystream) are reused verbatim from v8 block mode.
//
// Security caveats (docs/CAVEATS.md, CAV-005):
//   * The stream nonce is CSPRNG-drawn -- not SIV-derived -- because SIV
//     requires hashing the full message. Nonce reuse across two streams under
//     the same sk is catastrophic (CVF3-class hazard).
//   * Fixed-width 8-byte tokens and F*20 per-chunk padding close the LEB128
//     length leak (V2-CVF11) that the v7 stream_ae format retains; per-chunk
//     size is a pure function of F, which is public.
//   * Truncation-safe: the sentinel binds the chunk count.
//   * Reorder-safe: chunk_idx is bound into every per-chunk tag.
```

same file

```
/// Deterministic v8 streaming encryption with caller-supplied nonce.
///
/// **Test-only.** Used by `kat_cross_check::stream_v8_*` to compare byte
/// output against the Python KAT vectors. Production callers must draw the
/// nonce from a CSPRNG per stream (CAV-005) -- reusing a nonce across two
/// streams under the same sk is catastrophic.
```

same file

```
/// **Not misuse-resistant**: nonce reuse under the same `sk` is catastrophic
```

```
2048 /// (CAV-005). Use the v8 *block* API (`encrypt_bytes_v8`) whenever
     the whole
2049 /// message fits in memory and MRAE is desired.
```

CVF-V5-3: open Category: **algorithm**

The normative prime interval was widened from [10^6, 9.9×10^6] to [10^6, 1.5×10^7) in this release although no finding asked for it, the remediation report does not disclose it, and the paper justifies it with a false claim and against its own statement that the key tuple is not a security parameter.

What changed. Round-4 CVF-16 found that `validate_key` omitted the normative upper bound `MAX_KEY_PRIME = 9_900_000` and asked for that bound to be enforced. v5 enforces an upper bound (lib.rs:425-430), but in the same release it moved the bound itself: `MAX_KEY_PRIME` is now `14_999_999` (lib.rs:127-141), the paper's set $\mathcal{P}$ is now the primes in $[10^6, 1.5 \times 10^7)$ with $|\mathcal{P}| = 892{,}206$ instead of $579{,}947$ (napseq-eprint-v3.tex:275-284), and the default key length went from $K = 10$ to $K = 13$ (lib.rs:156-160; tex 294-296). The Python and C ports were changed in step (`napqes.py:101,113`, `C/napqes.h:163`). The remediation report describes the CVF-16 change only as "upper bound enforced" (report §3 row 16, §4) and never mentions the new interval or the new K; the client's own `docs/audit_mitigation_responses.md` calls it "Not an audit finding" (the "PQ-128" change).

The doc comment at lib.rs:127-136 states the actual motive: $\mathcal{P}$ is "exactly the half-open interval `[10^6, 1.5e7)` already used by `kem.rs`". Round 4 had noted, as an out-of-scope observation, that `kem.rs` derives key elements from a second, wider interval than the AEAD's. Rather than bringing `kem.rs` into line with the specification, the specification and the AEAD were moved onto `kem.rs`'s interval. The KEM layer that is now cited as the reason (tex 282-284, "the key-establishment layer (Section~\ref{sec:keygen})") is not described anywhere in the paper; the reference resolves to the AEAD's own Key Generation section.

The stated justification is false and self-contradictory. Remark `rem:pq128` (tex 342-359) says the new default "is the smallest parameter set in this family reaching the 128-bit post-quantum target" and that "Raising K alone is not sufficient". Both statements are wrong:

- Over the old interval, $K = 14$ already gives $H_\infty(\mathbf{k}) = 268.04$ bits, i.e. 134.02 bits post-Grover. Raising K alone is sufficient.
- With $K = 13$, the 128-bit post-Grover target is already met with an inclusive upper bound of $14{,}257{,}301$ ($|\mathcal{P}| = 847{,}186$), so 1.5×10^7 is not the smallest such interval; it was chosen to match `kem.rs`.
- The same paper says $K = 13$ "is an interoperability default, not a security parameter" (tex 380-383) and that the **k** figure "is reported for completeness only. It is not

relevant to any theorem" (tex 1919-1923). On v8 every confidentiality and integrity claim rests on sk; a post-quantum target for **k** is meaningful only for the legacy v7 format, where the prime tuple is the whole key, and the paper disclaims v7.

Interoperability consequence. The paper makes the interval normative ("Implementations MUST use this interval: a different upper bound yields a different $\mathcal{P}$ … and hence different keys", tex 278-281), yet the change carries no version marker (the spec's `\date{July 2026}` is identical in rounds 3, 4 and 5 and its filename says v3). A port that conforms to the round-4 specification, and to CVF-16's requested bound, passes the entire v5 corpus, because no vector exercises the widened region: the largest key element in the 35-vector v8 corpus is $7{,}999{,}993$ and no vector uses $K = 13$. That port nevertheless rejects $1 - (579{,}947/892{,}206)^{13} = 99.63\%$ of keys generated under the new default. Conversely, a v5 port accepts keys a v4-conforming port must reject, and nothing in the KATs would reveal the disagreement.

Recomputed figures (own sieve; every one of them still holds under the new interval).

- $|\mathcal{P}| = 892{,}206$; largest prime $14{,}999{,}981$; both 10^6 and $14{,}999{,}999$ composite; $\log_2 |\mathcal{P}| = 19.7670$.
- $$H_\infty(\mathbf{k}) = \log_2(|\mathcal{P}|!/(|\mathcal{P}| - 13)!) = 256.9711$$

  bits at $K = 13$ (key space 2.270×10^{77}), 128.4855 post-Grover. Old interval: 248.8921 / 124.4461 at $K = 13$; 191.4555 / 95.7278 at the old default $K = 10$.
- Token-width bound of Remark `rem:token-fit` (tex 926-942): the maximum token $\texttt{0x10FFFF} \cdot k + (k - 1)$ is $16{,}711{,}658{,}831{,}871 \approx 2^{43.93}$ at the largest prime and $16{,}711{,}678{,}885{,}887 \approx 2^{43.93}$ at the inclusive bound $k = 14{,}999{,}999$, against $11{,}029{,}707{,}685{,}887 \approx 2^{43.33}$ in round 4. The 8-byte token field still never overflows; the margin $2^{64}/t$ shrinks from 1.672×10^6 to 1.104×10^6, and the paper's "over 1.1×10^6" is correct. `MAX_SAFE_KEY_PRIME` (lib.rs:149-154) is 1.10×10^6 times the new bound.
- $k < 2^{40}$ (5-byte serialisation) and $k - 1 < 2^{24}$ (the modulo-bias remark's $< 2^{-8}$ bound) still hold.
- Per-call `validate_key` cost at $K = 13$ rises to about 17,400 trial divisions on average (6,487 to 25,155), from about 11,000 at the old $K = 10$; see the per-call validation issues.

So the widening breaks no arithmetic invariant. The defect is that a normative parameter of the scheme changed silently, for a reason the paper misstates, without a version identifier and without test vectors that would let a second implementation notice.

Recommendation

Decide the interval deliberately and record the decision as a versioned change.

- If the wider interval is kept, state the real motive (alignment with the key-establishment layer) and either specify that layer or drop the reference to it; fix the `sec:keygen` cross-reference. Delete or correct "Raising K alone is not sufficient" and "the smallest parameter set in this family", and reconcile Remark `rem:pq128` with Remark `rem:key-roles` and §5, which say K and **k** carry no

security requirement. If a post-quantum figure for **k** matters at all, say that it matters only for v7.

- Give the specification a version identifier and name the interval change in its revision history and in the remediation report, so that ports written against round 4 know they must change.
- Add KAT vectors that pin the new interval: at least one positive vector with $K = 13$ and an element in $(9.9\times10^6, 1.5\times10^7)$, and a negative vector with an element just above the bound (for example $15{,}000{,}017$).
- Alternatively, keep the round-4 interval and change `kem.rs` (and the Python/C KEM) to sample from it; that brings the out-of-scope component into line with the specification rather than the reverse.

Client Comment

/docs/napseq-eprint-v3.tex

```
275   \item $\mathcal{P}$ denotes the set of prime integers in $[10^6,
      1.5 \times 10^7)$,
276         a fixed public system parameter with $|\mathcal{P}| =
      892{,}206$
277         (verified by sieve; the interval is half-open, equivalently
      the
278         inclusive interval $[10^6, 14{,}999{,}999]$, and both
      endpoints are
279         composite). Implementations MUST use this interval:
280         a different upper bound yields a different $\mathcal{P}$, a
281         different canonical enumeration, and hence different keys.
282         This is the same prime set from which the key-establishment
      layer
283         (Section~\ref{sec:keygen}) draws, so the AEAD and the KEM
      share one
284         parameter set.
```

same file

```
294 Draw $\mathbf{k} = (k_1,\ldots,k_K)$, an ordered tuple of $K$ distinct
295 primes sampled uniformly, without replacement, from $\mathcal{P}$ via
    a
296 cryptographically secure DRBG (default $K=13$); \emph{independently},
    draw
```

same file

```
342 \begin{remark}[Choice of the ``PQ-128'' parameter
    set]\label{rem:pq128}
```

```
The default $(\mathcal{P},K)$ is the smallest parameter set in this family
reaching the $128$-bit post-quantum target. Raising $K$ alone is not
sufficient: over the earlier interval $[10^6,9.9\times10^6]$
($|\mathcal{P}|=579{,}947$, $19.1456$ bits per element) one has
$H_\infty=248.89$ bits at $K=13$, i.e.\ only $124.45$ bits post-Grover.
Widening the interval to $[10^6,1.5\times10^7)$ raises the per-element
contribution to $19.7670$ bits and clears the target at the same $K=13$,
while simultaneously unifying $\mathcal{P}$ with the interval already used
by the key-establishment layer. Since $K$ affects neither the wire format
nor the ciphertext length (Proposition~\ref{prop:expansion-neutral}), the
only cost of moving the default from $K=10$ to $K=13$ is stored key
material, which grows from $5K+32=82$ to $97$ bytes; widening the interval
changes no serialised size. Validation enforces the same interval: every reference
implementation rejects a key element outside $\mathcal{P}$ and a tuple of
more than $128$ elements. Keys provisioned under the earlier interval lie
inside $\mathcal{P}$ and remain usable.
\end{remark}
```

same file

```
(Remark~\ref{rem:coarsening-not-inflation}). $K=13$ is an interoperability
default, not a security parameter: any $K\ge1$ is permitted by the
construction (the reference implementations cap $K\le128$ to bound the
per-call cost of key validation), and both parties must agree on it out of band exactly as they
```

/rust/src/lib.rs

```
/// Upper bound of the normative prime interval of the "PQ-128" profile.
/// Inclusive, and chosen so that `P` is exactly the half-open interval
/// `[10^6, 1.5e7)` already used by `kem.rs` — the AEAD and the KEM now draw
/// from the same prime set. `P` contains exactly 892_206 primes (verified by
```

```
/// sieve), giving `|P|!/(|P|-13)! = 2^256.9711` ordered 13-tuples
/// (`2^128.4855` post-Grover).
///
/// **CVF-16 fix (2026-09-22).** This bound is now enforced by
[`validate_key`]
/// on *every* key it sees (not just generation), matching the
normative
/// interval Remark 3.1 of the paper specifies. Prior revisions
accepted any
/// prime `>= MIN_KEY_PRIME` on decrypt paths, which allowed a key
element
/// large enough to make the token multiplication `c * k + addend`
wrap `u64`
/// silently in release builds. Matches `napqes.py::MAX_KEY_PRIME`
and
/// `C/napqes.h::NAPQES_MAX_KEY_PRIME`.
pub const MAX_KEY_PRIME: u64 = 14_999_999;
```

same file

```
/// Number of prime key elements generated by default ("PQ-128"
profile).
/// K=13 is the smallest count reaching the 128-bit post-quantum
target over
/// `P`; K=13 over the earlier `[10^6, 9.9e6]` interval would have
given only
/// 124.4 bits, which is why the interval was widened too.
pub const DEFAULT_KEY_COUNT: usize = 13;
```

CVF-V5-4: open Category: **flaw**

The v7 token-emission loop still has no noise-run cap and no token ceiling, and deprecation does not stop in-tree production traffic from using it, so v7 ciphertext length remains a nonce-dependent random variable and the loop remains unbounded.

This carries the residual of round-4 CVF-17, which is partially fixed.

Fixed in v5: the four public v7 encryptors are marked `#[deprecated]`: `encrypt` (`lib.rs:731-740`), `encrypt_bytes` (`lib.rs:1046-1052`), `encrypt_str` (`lib.rs:1182-1187`) and `encrypt_raw` (`lib.rs:1217-1225`). The token loop was extracted into one function, `emit_tokens_v7`.

Remaining:

- `emit_tokens_v7` (`lib.rs:994-1024`) still applies neither bound. Omitting them is correct for v7 wire compatibility, but every v7 ciphertext produced today has the length leak and the unbounded loop that round 4 described.

- `#[deprecated]` is a lint only. `ot_frame.rs` (out of scope, lines 33-40 and 303) suppresses it with `#[allow(deprecated)]`, and the client's report §8.4 confirms that the OT gateway still seals every frame with v7 `encrypt_raw`. The defect therefore applies to all in-tree production traffic.
- The robustness cap that round 4 asked for regardless of format ("cap the run anyway") was not added.
- The remediation report (row 17 and §5.2) says `encrypt_bytes_with_nonce` was deprecated; it was not, and it was already `pub(crate)` in v4.

Recommendation

Move the gateway (`ot_frame`) to the v8 format, then remove the v7 encryptors from the public API, keeping only the decryptors. Until then, add an iteration bound to `emit_tokens_v7` that returns an error rather than looping without limit, and correct the remediation report's statement about `encrypt_bytes_with_nonce`.

Client Comment

/rust/src/lib.rs

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
            ceiling (audit CVF-17): ciphertext length is a random function \
            of the CSPRNG-drawn nonce rather than of the padding bucket, so \
            the length-hiding property Corollary 4.14 proves for v8 does not \
            hold; v7 is keyed by the prime tuple, not sk, and no theorem of \
            the paper applies (CVF-28). Use encrypt_bytes_v8 / \
            encrypt_bytes_v8_with_profile."
)]
```

same file

```
fn emit_tokens_v7(
    padded: &[u32],
    kb: &[u8],
    nonce: &[u8],
    noise_p: f64,
    key: &[u64],
```

```
) -> Vec<u64> {
    let kk = key.len() as u64;
    let mut cypher: Vec<u64> = Vec::with_capacity(padded.len() * 4);
    let mut real_idx: u64 = 0;
    let mut ct_pos: u64 = 0;
    for &c in padded {
        loop {
            if is_noise_pos(kb, nonce, ct_pos, noise_p) {
                let k = key[(real_idx % kk) as usize];
                let nc = derive_noise_char(kb, nonce, ct_pos);
                let na = derive_noise_token_addend(kb, nonce, ct_pos,
k);
                cypher.push(nc * k + na);
                ct_pos += 1;
            } else {
                let k = key[(real_idx % kk) as usize];
                let addend = derive_addend(kb, nonce, real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
    cypher
}
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and
per-bucket \
            ceiling (audit CVF-17); v7 is keyed by the prime tuple,
not sk, \
            and no theorem of the paper applies (CVF-28). Use \
            encrypt_bytes_v8 / encrypt_bytes_v8_with_profile."
)]
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and
per-bucket \
            ceiling (audit CVF-17); v7 is keyed by the prime tuple,
not sk, \
```

```
            and no theorem of the paper applies (CVF-28). Use
encrypt_bytes_v8."
)]
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and
per-bucket \
            ceiling (audit CVF-17), and the byte-vs-codepoint encoding
\
             ambiguity of CVF-14 remains; v7 is keyed by the prime
tuple, not \
             sk, and no theorem of the paper applies (CVF-28). Use \
             encrypt_bytes_v8 for text or wrap the bytes in a
caller-chosen \
             text encoding."
)]
```

CVF-V5-5: open Category: **procedural**

The power-on self-test still has no software-integrity test, while the module's FIPS operator documents claim one, and the post-quantum KEM primitive is checked only for pairwise consistency.

This carries the residual of round-4 CVF-18, which is partially fixed.

Fixed in v5: `run_power_on_self_tests` (`self_test.rs:560-582`) now runs v8 known-answer tests KAT-4 to KAT-7 (encrypt, decrypt, tamper, structural reject) and a KEM derivation KAT-8. KAT-4 pins the SHA-256 of corpus vector W002 and was reproduced by an independent re-implementation. `kat_constants_match_corpus` stops the embedded constants drifting from the corpus. INT-1 was renamed to a build-provenance check and skips by default (`self_test.rs:715-727`) instead of comparing the version string with itself.

Remaining:

- There is no software-integrity test. The client defers the binary HMAC (remediation report §6 item 1), yet the operator documents `docs/fips/SECURITY_POLICY.md:180-181` and `MODULE_BOUNDARY.md:72,153-164` still describe a `build.rs` HMAC integrity check. INT-1 cannot fail in a default build, and its unit test `integrity_check_passes` therefore asserts nothing.
- KAT-8 checks only the key derivation as a known answer; FrodoKEM and the hybrid combiner get only a pairwise-consistency test.

The missing POST coverage of the new streaming format is a separate introduced-in-v5 issue.

Recommendation

Either implement the integrity test (an HMAC over the module binary computed at build time and verified at start-up) or correct the Security Policy and Module Boundary documents to say that no integrity test exists and that INT-1 is a provenance stub. Add a known-answer test for the KEM primitive (fixed-seed encapsulation and decapsulation against a pinned vector).

Client Comment

/rust/src/self_test.rs

```
560 pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
561     // CVF-47: a panicking KAT leaves POST_RUNNING, which refuses all
callers.
562     let _lock = lock_post();
563     POST_STATE.store(POST_RUNNING, Ordering::Release);
564     let _in_post = InPostFlag::set();
565     let result = (|| -> Result<(), SelfTestError> {
566         kat_encrypt()?;
567         kat_decrypt()?;
568         kat_tamper_rejection()?;
569         kat_v8_encrypt()?;
570         kat_v8_decrypt()?;
571         kat_v8_structural_reject()?;
572         kat_v8_tamper_rejection()?;
573         kat_kem()?;
574         build_provenance_check()?;
575         Ok(())
576     })();
577     match &result {
578         Ok(()) => POST_STATE.store(POST_PASSED, Ordering::Release),
579         Err(_) => POST_STATE.store(POST_FAILED, Ordering::Release),
580     }
581     result
582 }
```

same file

```
715 const BUILD_HASH: &str = env!("CARGO_PKG_VERSION");
716
717 fn build_provenance_check() -> Result<(), SelfTestError> {
```

```
718     // If the operator has pinned an attested version at build time,
    enforce
719     // strict equality. Otherwise (the default), succeed — this check
    is a
720     // build-provenance signal, not an integrity check, and no
    operator
721     // signal means no claim to verify against.
722     match option_env!("NAPQES_ATTESTED_VERSION") {
723         Some(attested) if attested == BUILD_HASH => Ok(()),
724         Some(_) => Err(SelfTestError::BuildProvenanceMismatch),
725         None => Ok(()),
726     }
727 }
```

CVF-V5-6: open Category: **architecture**

The crate still links the KEM, network and gateway modules unconditionally, and this release widens the build boundary further with two new unconditional native dependencies and a feature that cannot be built from the delivered tree.

This carries the residual of round-4 CVF-23, which is partially fixed, and the boundary has grown since round 4.

Fixed in v5: a `[profile.release]` with overflow checks, a committed `Cargo.lock`, a narrowed `tokio` dependency, and real use of `subtle`.

Remaining, deferred by the client:

- There is no feature gating: `pub mod kem`, `kem_exchange`, `ot_frame` and `protocols` are compiled unconditionally (`lib.rs:47-54`), so a consumer who wants only the AEAD still links the KEM, networking and gateway code.
- No `rust-version` (MSRV) is declared.
- `[profile.test]` sets nothing that differs from the default, and its comment (`Cargo.toml:52-61`) is stale.

Worsened in v5:

- `x25519-dalek` (`Cargo.toml:33-34`) and `pqcrypto-sphincsplus` (`Cargo.toml:47`) are new unconditional dependencies. No delivered `.rs`, `.py` or `.md` file uses SPHINCS+, yet `Cargo.lock` shows it pulling `pqcrypto-internals` and `cc`, so unreviewed native code is built into every consumer.
- The new `vale` feature (`Cargo.toml:14-15`) and the `[[bin]] vale_demo` target (`Cargo.toml:75-78`) refer to `src/vale` and `src/bin/vale_demo.rs`, which are gitignored or absent. The remediation report's `--features vale` test counts cannot be reproduced from the delivery.

Recommendation

Put `kem`, `kem_exchange`, `ot_frame` and `protocols` behind default-off features, together with their dependencies (`x25519-dalek`, `tokio`, the KEM crates). Make `pqcrypto-sphincsplus` optional under `vale` (`vale = ["dep:pqcrypto-sphincsplus"]`), and either deliver `vale` or remove the feature and the bin target. Declare `rust-version`, and delete or correct the `[profile.test]` block and its comment.

Client Comment

/rust/Cargo.toml

```
# Requires the unpublished rust/src/vale sources (gitignored).
vale = []
```

same file

```
# Classical half of the hybrid key establishment (ANSSI PQC doctrine).
x25519-dalek = { version = "2", features = ["static_secrets"] }
```

same file

```
pqcrypto-sphincsplus = { version = "0.7.2", default-features = false, features = ["std"] }
```

same file

```
# CVF-23: manifest hardening — release builds now trap on integer overflow
# (belt-and-braces backstop for CVF-16's checked_mul recommendation), and
# test builds keep debug_assert! live so CVF-12's ct_eq_bytes length check
# fires in `cargo test`.
[profile.release]
overflow-checks = true
debug-assertions = false

[profile.test]
overflow-checks = true
```

same file

```
[[bin]]
name = "vale_demo"
path = "src/bin/vale_demo.rs"
required-features = ["vale"]
```

/rust/src/lib.rs

```
47 pub mod self_test;
48 pub mod kem;
49 pub mod kem_exchange;
50 pub mod ot_frame;
51 pub mod protocols;
52 // rust/src/vale is not published (gitignored); build it with
   `--features vale`.
53 #[cfg(feature = "vale")]
54 pub mod vale;
```

CVF-V5-7: open Category: **architecture**

The public v7 surface still runs the single-secret key schedule that the v8 redesign was built to eliminate, and it still carries all of the gateway's production traffic.

This carries the residual of round-4 CVF-28, which is partially fixed.

Fixed in v5: `check_v7_aad_len` enforces the 4-byte AAD length prefix's range at every public v7 entry point, `be_len_prefix` asserts the width bound (and the assertion is unreachable from external input), and the v7 rustdoc now states the key schedule and that no theorem of the paper applies (`lib.rs:1131-1133`, `1255-1258`). The v7 encryptors are deprecated.

Remaining: the architectural core of the finding. The v7 encryptors `encrypt_bytes` (`lib.rs:1046-1052`) and `encrypt_raw` (`lib.rs:1217-1253`) and the decryptor `decrypt_raw` (`lib.rs:1255-1314`) are still public, deprecated rather than withdrawn. On v7 the prime tuple is the entire key: it derives both the HMAC key and the arithmetic, so there is no independent `sk`, and none of the paper's security theorems covers this format. The client's report §8.4 confirms that `ot_frame` still seals every gateway frame with v7 `encrypt_raw`, so the format the paper disclaims is the one in production use.

Recommendation

Migrate `ot_frame` to the v8 format with a separately provisioned `sk`, then remove the v7 encryptors from the public API and keep only the decryptors, behind a `legacy-v7` feature, for archived data. Until the migration, state in the README and the paper that gateway traffic uses a format with no security proof.

Client Comment

/rust/src/lib.rs

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
            ceiling (audit CVF-17); v7 is keyed by the prime tuple, not sk, \
            and no theorem of the paper applies (CVF-28). Use \
            encrypt_bytes_v8 / encrypt_bytes_v8_with_profile."
)]
```

same file

```
#[deprecated(
    since = "0.2.0",
    note = "v7 legacy format lacks the MAX_NOISE_RUN cap and per-bucket \
           ceiling (audit CVF-17), and the byte-vs-codepoint encoding \
            ambiguity of CVF-14 remains; v7 is keyed by the prime tuple, not \
            sk, and no theorem of the paper applies (CVF-28). Use \
            encrypt_bytes_v8 for text or wrap the bytes in a caller-chosen \
            text encoding."
)]
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    // CVF-34: cap at MAX_PLAINTEXT_CODEPOINTS bytes (each byte becomes a
    // codepoint under the raw-encode convention, so the same 2-codepoint
    // length prefix caps the byte count).
    if data.len() > MAX_PLAINTEXT_CODEPOINTS {
        return Err(format!(
            "data exceeds MAX_PLAINTEXT_CODEPOINTS ({}) bytes",
            MAX_PLAINTEXT_CODEPOINTS
        ));
```

```
    }
    let nonce = generate_nonce_with_crng_check()?;
    let codepoints: Vec<u32> = data.iter().map(|&b| b as
u32).collect();
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(&codepoints, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
// CVF-46
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
}
```

same file

```
/// Decrypt binary data previously encrypted with [`encrypt_raw`].
///
/// Legacy v7 decryptor, kept for archived ciphertexts; see
[`decrypt_bytes`]
/// for the v7 caveats.
///
/// Verifies the HMAC tag (constant-time) before decrypting.  Returns
/// `Err` on authentication failure — the caller must never use the
/// ciphertext for any purpose if this returns an error.
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
        return Err(format!(
            "Ciphertext too short: {} bytes; header+tag require at
least {}.",
            ciphertext.len(),
            NONCE_SIZE + TAG_SIZE
        ));
    }
    let kb = key_bytes(key);
    let split = ciphertext.len() - TAG_SIZE;
    let payload = &ciphertext[..split];
```

```
    let recv_tag = &ciphertext[split..];
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
    let nonce = &payload[..NONCE_SIZE];
    let masked = &payload[NONCE_SIZE..];
    let ks = varint_keystream(&kb, nonce, masked.len());
    let blob: Vec<u8> = masked.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let tokens = fixed_decode_tokens(&blob)
        .map_err(|e| format!("varint decode error: {}", e))?;
    let codepoints = decrypt(nonce, &tokens, key)?;
    // CVF-14: the previous body was `.map(|c| (c & 0xFF) as u8)` — a
silent
    // narrowing that discarded every recovered codepoint above 255
without
    // signalling. A ciphertext produced by `encrypt_bytes`
(codepoint-oriented,
    // e.g. `é` as one U+00E9 token) handed to `decrypt_raw` would
authenticate
    // and return a self-consistent-looking single byte, hiding the
fact that
    // the two ports use incompatible message-space encodings. We now
reject
    // any recovered value that does not fit in a byte.
    codepoints
        .into_iter()
        .enumerate()
        .map(|(i, c)| {
            if c > 0xFF {
                Err(format!(
                    "Malformed v7 raw ciphertext: recovered codepoint
{} at \
                     position {} is out of byte range [0, 255]. This
usually \
                     means the ciphertext was produced by
`encrypt_bytes` \
                     (codepoint-oriented) rather than `encrypt_raw` \
                    (byte-oriented); the two use incompatible message \
                     encodings (CVF-14).",
                    c, i
                ))
            } else {
                Ok(c as u8)
```

```
            }
        })
        .collect()
}
```

CVF-V5-8: open Category: **procedural**

The new FIPS self-test gate is off by default, leaves the v7 entry points and key generation ungated even when enabled, is misdescribed in the manifest and README, and has no test showing that it ever refuses an operation.

This carries the residual of round-4 CVF-47, which is partially fixed, together with two new defects in the remediation's own text and tests.

Fixed in v5: a four-state POST latch (`self_test.rs:92-130`) with no bypass through the RUNNING state and fail-closed panic handling, and a `require_post` check at every v8 block, streaming and `_key` entry point.

Remaining:

- The gate is behind the opt-in `fips_gate` feature and the default feature set is empty (`Cargo.toml:6-13`), the opposite of what round 4 recommended.
- With the gate on, all v7 entry points, including the four public encryptors `encrypt`, `encrypt_bytes`, `encrypt_str` and `encrypt_raw` (`lib.rs:741-767`, `1053-1081`, `1188-1194`, `1226-1253`) that `ot_frame` uses, and all key generation still run in the NOT_RUN and FAILED states. A FAILED state is cleared by any later re-run of POST.
- The manifest comment (`Cargo.toml:6-10`) says that otherwise "all crypto operations" return `NotOperational`, and the README's list of ungated functions (`README.md:100-114`) omits the v7 encryptors, so an operator would believe v7 encryption is inhibited in the error state.
- No test shows a refusal. Under `cfg(test)`, POST runs automatically on first gated use (`self_test.rs:119-130`), so NOT_RUN is unobservable in unit tests, and no fault-injection hook makes FAILED observable. The gate tests (`self_test.rs:816-890`) assert only success and would pass on an ungated crate. The two refusal tests the remediation report names do not exist.

The non-latching continuous RNG test is a separate introduced-in-v5 issue.

Recommendation

Enable `fips_gate` by default, gate the v7 entry points and key generation, and make FAILED sticky until the process restarts. Until the v7 functions are gated, list them as ungated in `Cargo.toml` and the README and remove "all crypto operations". Add an integration test under `rust/tests/` built with `--features fips_gate` that asserts every gated entry point returns `NotOperational` before POST, add a test-only fault-injection switch to cover the FAILED state, and correct the remediation report's test list.

Client Comment

/rust/Cargo.toml

```
# CVF-47: opt-in FIPS 140-3 self-test gating. When enabled, every public v8
# entry point calls `self_test::require_post()?` before doing crypto — the
# caller MUST first call `run_power_on_self_tests()` and it MUST succeed,
# else all crypto operations return `Err(NotOperational)`. Disabled
# by default so existing (non-FIPS) consumers see zero behaviour change.
[features]
default = []
fips_gate = []
```

/rust/README.md

```

## Power-on self-tests (FIPS 140-3)

`napqes::self_test::run_power_on_self_tests()` runs the SP 800-140B §4.9.1
KATs (v7 and v8 encrypt/decrypt/tamper, a v8 structural reject, and a KEM
derivation KAT plus FrodoKEM and hybrid pairwise-consistency checks) and
latches the outcome process-wide. Building with `--features fips_gate`
makes every public v8 encrypt/decrypt entry point (block, streaming,
`NapqesKey` variants) and every KEM encapsulate/decapsulate call refuse to
run (`module not operational`) until that call has returned `Ok(())`; a
failed POST keeps the module in the error state. Without the feature the
outcome is still latched but not enforced, and the feature is off by
default. Key generation, the v7 legacy decryptors, `ot_frame`, `protocols`
and `vale` are outside the gate. The bundled binaries and examples run POST
at start-up.
```

/rust/src/lib.rs

```
pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE], Vec<u64>) {
    // CVF-34: uphold the length cap even on this Vec<u64>-returning legacy
    // path. The .expect() surfaces the cap consistently with the deprecation
    // attribute and matches the pattern of the CVF-15 validate_key call.
    assert!(
        message.len() <= MAX_PLAINTEXT_CODEPOINTS,
        "message length {} exceeds MAX_PLAINTEXT_CODEPOINTS ({})",
        message.len(), MAX_PLAINTEXT_CODEPOINTS
    );
    // CVF-15: this v7 API returns `Vec<u64>` and cannot report a validation
    // error. Every other entry point returns `Result` and propagates via `?`.
    // The .expect() here surfaces the same validation failure as a panic,
    // consistent with `pub fn decrypt`'s prior panic contract and matched by
    // the #[deprecated] attribute (CVF-17) that steers callers to v8.
    validate_key(key).expect("CVF-15: encrypt called with invalid key material");
    // CVF-35: route through the checked CRNG helper like every other v7
    // encryptor. Signature is `-> (nonce, tokens)` so a CRNG failure is
    // surfaced as `.expect` — consistent with the deprecation attribute
    // that steers callers to v8.
    let nonce = generate_nonce_with_crng_check()
        .expect("CVF-35: CRNG stuck-generator check failed inside encrypt");
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(message, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
    (nonce, cypher)
}
```

same file

```
pub fn encrypt_bytes(message: &str, key: &[u64], aad: &[u8]) -> Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
```

```
    check_v7_aad_len(aad.len())?;
    // CVF-34: cap plaintext length at MAX_PLAINTEXT_CODEPOINTS at the entry
    // point (was an `assert!` in `pad_message`, which aborted release
    // builds — Section 12's "raises an error immediately; no silent
    // truncation" caveat is now honoured on v7 too).
    if message.chars().count() > MAX_PLAINTEXT_CODEPOINTS {
        return Err(format!(
            "message exceeds MAX_PLAINTEXT_CODEPOINTS ({})",
            MAX_PLAINTEXT_CODEPOINTS
        ));
    }
    let nonce = generate_nonce_with_crng_check()?;
    let codepoints: Vec<u32> = message.chars().map(|c| c as u32).collect();
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(&codepoints, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key); // CVF-46
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
}
```

same file

```
pub fn encrypt_str(message: &str, key: &[u64], aad: &[u8]) -> Result<String, String> {
    // CVF-15: transitively covered by encrypt_bytes, but early-rejecting a
    // bad key means we never allocate the base64 buffer for a doomed op.
    validate_key(key)?;
    #[allow(deprecated)]
    Ok(STANDARD.encode(encrypt_bytes(message, key, aad)?))
}
```

same file

```
pub fn encrypt_raw(data: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    // CVF-34: cap at MAX_PLAINTEXT_CODEPOINTS bytes (each byte
becomes a
    // codepoint under the raw-encode convention, so the same
2-codepoint
    // length prefix caps the byte count).
    if data.len() > MAX_PLAINTEXT_CODEPOINTS {
        return Err(format!(
            "data exceeds MAX_PLAINTEXT_CODEPOINTS ({}) bytes",
            MAX_PLAINTEXT_CODEPOINTS
        ));
    }
    let nonce = generate_nonce_with_crng_check()?;
    let codepoints: Vec<u32> = data.iter().map(|&b| b as
u32).collect();
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(&codepoints, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
// CVF-46
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
}
```

/rust/src/self_test.rs

```
fn admit(state: u8, in_post: bool) -> Result<(), SelfTestError> {
    match state {
        POST_PASSED => Ok(()),
        POST_RUNNING if in_post => Ok(()),
        _ => Err(SelfTestError::NotOperational),
    }
}

/// Crate-visible gate (CVF-47). Returns `Ok(())` iff the power-on
self-tests
```

```
/// have passed. A caller arriving while POST is running on another thread
/// blocks until it finishes, then sees the final outcome.
///
/// Called from every public v8 encrypt/decrypt entry point and every KEM
/// encapsulate/decapsulate function under `#[cfg(feature = "fips_gate")]`.
#[allow(dead_code)]
pub(crate) fn require_post() -> Result<(), SelfTestError> {
    if IN_POST.with(|f| f.get()) {
        return admit(POST_STATE.load(Ordering::Acquire), true);
    }
    ensure_post_for_unit_tests();
    if POST_STATE.load(Ordering::Acquire) == POST_RUNNING {
        let _guard = lock_post();
        return admit(POST_STATE.load(Ordering::Acquire), false);
    }
    admit(POST_STATE.load(Ordering::Acquire), false)
}

/// Unit tests share one process-global POST_STATE, so run POST on first use
/// instead of letting tests race to reset and re-run it.
#[cfg(test)]
fn ensure_post_for_unit_tests() {
    if POST_STATE.load(Ordering::Acquire) == POST_NOT_RUN {
        let _ = run_power_on_self_tests();
    }
}

#[cfg(not(test))]
#[allow(dead_code)]
fn ensure_post_for_unit_tests() {}
```

same file

```
    // CVF-47: latch transitions to PASSED on success and stays there on rerun.
    #[test]
    fn cvf47_post_state_transitions_to_passed() {
        run_power_on_self_tests().expect("POST failed unexpectedly");
        run_power_on_self_tests().expect("POST failed unexpectedly on rerun");
        let _guard = lock_post();
        assert_eq!(POST_STATE.load(Ordering::Acquire), POST_PASSED);
    }
```

```
824
825     // CVF-47: only PASSED admits other threads; RUNNING admits only
    the POST thread.
826     #[test]
827     fn cvf47_gate_decision_table() {
828         assert_eq!(admit(POST_NOT_RUN, false),
    Err(SelfTestError::NotOperational));
829         assert_eq!(admit(POST_NOT_RUN, true),
    Err(SelfTestError::NotOperational));
830         assert_eq!(admit(POST_RUNNING, false),
    Err(SelfTestError::NotOperational));
831         assert_eq!(admit(POST_RUNNING, true), Ok(()));
832         assert_eq!(admit(POST_PASSED, false), Ok(()));
833         assert_eq!(admit(POST_FAILED, false),
    Err(SelfTestError::NotOperational));
834         assert_eq!(admit(POST_FAILED, true),
    Err(SelfTestError::NotOperational));
835     }
836
837     // KAT-6 must fail on the structural check, not the tag.
838     #[test]
839     fn kat6_forgery_passes_the_tag() {
840         let ct = crate::encrypt_bytes_v8(KAT_V8_MESSAGE,
    KAT_V8_PRIMES, &KAT_V8_SK, KAT_V8_AAD).unwrap();
841         let body = &ct[..ct.len() - crate::TAG_SIZE];
842         assert_eq!(
843             crate::retag_block_v8(body, &KAT_V8_SK, KAT_V8_AAD)[..],
844             ct[ct.len() - crate::TAG_SIZE..]
845         );
846     }
847 }
848
849 // ─── FIPS gate integration tests (opt-in via --features fips_gate)
    ──────────
850
851 #[cfg(all(test, feature = "fips_gate"))]
852 mod fips_gate_tests {
853     //! With the fips_gate feature enabled, every public v8 entry
    point must
854     //! call `require_post()?` before doing any crypto. These tests
    exercise
855     //! the gate end-to-end. Run with:
856     //!   `cargo test --lib --features fips_gate fips_gate_tests`
857
858     use super::*;
859
```

```
    #[test]
    fn v8_entry_admits_after_post() {
        run_power_on_self_tests().unwrap();
        let key = crate::NapqesKey::generate().unwrap();
        let ct = crate::encrypt_bytes_v8_key("hi", &key,
b"").unwrap();
        assert_eq!(crate::decrypt_bytes_v8_key(&ct, &key,
b"").unwrap(), "hi");
    }

    // A second thread must wait for an in-flight POST instead of
being
    // admitted mid-run (the pre-fix behaviour) or spuriously refused.
    #[test]
    fn concurrent_callers_wait_for_post() {
        let key =
std::sync::Arc::new(crate::NapqesKey::generate().unwrap());
        let workers: Vec<_> = (0..4)
            .map(|_| {
                let key = key.clone();
                std::thread::spawn(move || {
                    for _ in 0..8 {
                        crate::encrypt_bytes_v8_key("hi", &key,
b"").unwrap();
                    }
                })
            })
            .collect();
        for _ in 0..4 {
            run_power_on_self_tests().unwrap();
        }
        for w in workers {
            w.join().unwrap();
        }
    }
}
```

6.2. Minor

CVF-V5-9: open Category: **flaw**

In an empty v8 stream the frame size in the header is authenticated by nothing, so any valid empty-stream ciphertext can be rewritten into further valid ciphertexts.

This was introduced by the round-5 remediation, since the streaming code is new in v5.

The stream header is `0x02 || be4(F) || nonce`. The sentinel tag covers the domain byte, nonce, chunk count, AAD and real-codepoint total, but neither the format byte nor `F`. The chunk tags do not include `F` either. For a non-empty stream `F` is pinned only indirectly: the decryptor requires every chunk to be exactly `F * 160` bytes, and the chunk bytes are the final HMAC input. A zero-chunk stream reaches the sentinel without any chunk-length check, and the header value of `F` is only range-checked to lie in `[1, 16384]`.

To trigger it, take the empty-stream vector `S001` from `tests/kat/v8_stream_vectors.json`, rewrite bytes 1 to 4 to any `F` in `[1, 16384]`, and the result still decrypts to the empty string under the same key and AAD. Both independent re-implementations made this round (one from the code, one from `SPEC.md`) confirm it. The forged ciphertext decrypts only to the same empty plaintext, so plaintext integrity is preserved, but integrity of ciphertexts fails and the header is malleable.

Recommendation

Bind the complete stream header (format byte, `be4(F)` and nonce) into the sentinel tag, and preferably into every chunk tag, and state this in `SPEC.md` section 8.2. Add a negative stream vector with a modified `F` on an empty stream.

Client Comment

/rust/src/lib.rs

```
1906 fn compute_stream_v8_sentinel_tag(
1907     sk_fmt: &[u8; 32],
1908     nonce: &[u8; NONCE_SIZE],
1909     chunk_count: u32,
```

```
    aad: &[u8],
    total_real_cps: u64,
) -> [u8; TAG_SIZE] {
    let mut buf = Vec::with_capacity(1 + NONCE_SIZE + 4 +
AAD_LEN_WIDTH_V8 + aad.len() + 8);
    buf.push(DOMAIN_STREAM_V8_SENTINEL);
    buf.extend_from_slice(nonce);
    buf.extend_from_slice(&chunk_count.to_be_bytes());
    buf.extend_from_slice(&be_len_prefix(aad.len(),
AAD_LEN_WIDTH_V8));
    buf.extend_from_slice(aad);
    buf.extend_from_slice(&total_real_cps.to_be_bytes());
    hmac_digest(sk_fmt, &buf)
}
```

same file

```
    let f = u32::from_be_bytes([stream[1], stream[2], stream[3],
stream[4]]);
    if f < 1 || f > STREAM_AE_V8_MAX_FRAME {
        return Err(format!(
            "Malformed v8 stream header: frame_codepoints F={} out of
[1, {}].",
            f, STREAM_AE_V8_MAX_FRAME
        ));
    }
```

CVF-V5-10: open Category: **architecture**

The Rust "streaming" decryptor is not incremental: it needs the whole stream ciphertext, about 160 times the plaintext size, in memory before it returns any plaintext.

This was introduced by the round-5 remediation, since the streaming code is new in v5.

`decrypt_stream_ae_v8` takes the complete ciphertext as one `&[u8]` and returns only after every chunk tag and the sentinel have been verified. No incremental decryptor exists in the crate, even though `docs/CAVEATS.md` names a `StreamV8Decryptor`. The README shows only an encrypt-side incremental API.

Buffering everything does mean the Rust port never releases unverified plaintext. But a receiver cannot process a stream of 10 million codepoints (about 1.6 GB of ciphertext at 20 tokens of 8 bytes per codepoint) without holding all of it. The all-at-once `encrypt_stream_ae_v8` likewise holds the whole plaintext in memory, yet uses a random nonce rather than the misuse-resistant synthetic nonce. For in-memory messages longer than 65,535 codepoints, the only option the crate offers is therefore the non-misuse-resistant

one. The Python port's decryptor, by contrast, is incremental (out of scope), so the two ports have different semantics under the same format name.

Recommendation

Either add an incremental decryptor that releases chunk `i` only after its tag verifies and documents that truncation is detected only at the sentinel, or document that Rust streaming is encrypt-side only. Consider deriving the nonce synthetically in the all-at-once `encrypt_stream_ae_v8`, since it already has the full message.

Client Comment

/rust/README.md

````
## Streaming AE (v8)

```rust
use napqes::{
    generate_v8_key, encrypt_stream_ae_v8, decrypt_stream_ae_v8,
    StreamV8Encryptor,
    DEFAULT_KEY_COUNT, MIN_KEY_PRIME, MAX_KEY_PRIME,
};

napqes::self_test::run_power_on_self_tests().expect("POST failed");

let (primes, sk) = generate_v8_key(DEFAULT_KEY_COUNT, MIN_KEY_PRIME,
MAX_KEY_PRIME).unwrap();

// All-at-once
let ct = encrypt_stream_ae_v8("hello, streaming!", &primes, &sk, b"",
128).unwrap();
let pt = decrypt_stream_ae_v8(&ct, &primes, &sk, b"").unwrap();

// Chunk-at-a-time
let (mut enc, header) = StreamV8Encryptor::new(&primes, &sk, b"",
128).unwrap();
let mut buf: Vec<u8> = header;
for c in "chunk-by-chunk".chars() {
    if let Some(frame) = enc.push(c).unwrap() {
buf.extend_from_slice(&frame); }
}
buf.extend_from_slice(&enc.finish().unwrap());
```

````

```
**Streaming caveat.** v8 streaming is NOT misuse-resistant — the 16-byte
per-stream nonce is CSPRNG-drawn, not SIV-derived. Nonce reuse under the
same `sk` is a CVF-3-class hazard. See `docs/CAVEATS.md` CAV-005. For
messages that fit in memory, prefer the SIV-protected block API
(`encrypt_bytes_v8` / `decrypt_bytes_v8`).
```

/rust/src/lib.rs

```
/// v8 streaming-AE decryption -- consumes the full ciphertext byte stream.
///
/// Convenience "all-at-once" API. Each per-chunk tag is verified before that
/// chunk's plaintext contributes to the output; the sentinel binds the
/// chunk count (anti-truncation) and the true real-codepoint count (used to
/// strip filler codepoints from the last chunk).
pub fn decrypt_stream_ae_v8(
    stream: &[u8],
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    aad: &[u8],
) -> Result<String, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    validate_key(primes)?;
    let sk_fmt = Secret32::new(derive_format_subkey(sk, FORMAT_STREAM_AE_V8));
    decrypt_stream_ae_v8_inner(stream, primes, sk_fmt.as_ref(), aad)
}
```

CVF-V5-11: open Category: **flaw**

The remediation adds new unwiped copies of key material while claiming to close the zeroization finding: `validate_key` leaves a sorted copy of the prime tuple on the heap on every call, `StreamV8Encryptor` holds `sk_fmt` outside the wipe guard and never wipes its plaintext buffer, and `NapqesKey::new` drops rejected key material unwiped.

All three were introduced by the round-5 remediation.

1. Sorted copy in `validate_key`. To fix the quadratic distinctness check (round-4 CVF-41), `validate_key` now makes `let mut sorted = key.to_vec()`, sorts it, and drops it without wiping. v4 used an in-place `contains` scan with no allocation. `validate_key` runs on every key the crate sees: inside `NapqesKey::new`, at the head of all nine v7 entry points (two or three times per v7 decrypt call), on every bare-slice v8 call, and on every stream open. For the v7 format the prime tuple is the entire key, and a sorted copy loses only the order. Thirteen elements have about 2^32.5 orderings, which one known plaintext can test. This sits alongside code that carefully wraps the v7 `key_bytes` buffer in the wipe-on-drop `SecretBytes` for exactly this reason.
2. `StreamV8Encryptor`. The client's report states that `sk_fmt` "is held in the `Secret32` guard on every v8 path". `StreamV8Encryptor`, however, stores `sk_fmt` as a raw `[u8; 32]` field. `new` copies a local `sk_fmt` into the struct and returns the struct by value inside a tuple, and each such move is a bitwise copy that leaves unwiped stack residue. Its `Drop` wipes `sk_fmt` and `primes` but not `cp_buf`, which holds up to `F - 1` plaintext codepoints between `push` calls. Nor is the memory released by `drain` in `push` wiped. The all-at-once streaming paths use `Secret32` correctly.
3. `NapqesKey::new`. When `validate_key` rejects the input, the moved `primes` vector and `sk` array are dropped without being wiped, although wipe-on-drop is the stated purpose of the type.

Round-4 CVF-37 itself is otherwise fixed, and these are the residual and new copies.

Recommendation

In `validate_key`, either wrap the scratch vector in a wipe-on-drop guard, or return to an in-place distinctness scan, which costs at most 8,128 comparisons for `K <= 128` and is negligible beside trial division. In `StreamV8Encryptor`, hold `sk_fmt` in a boxed `Secret32` so that moves do not copy it, and wipe `cp_buf` in `Drop` and after every drain. In `NapqesKey::new`, wipe the inputs on the error path. Correct the client report's statement about `Secret32` coverage.

Client Comment

/rust/src/key.rs

```
54      pub fn new(primes: Vec<u64>, sk: [u8; SK_SIZE]) -> Result<Self,
String> {
55          validate_key(&primes)?;
56          Ok(Self { primes, sk })
57      }
```

/rust/src/lib.rs

```
446     // Distinctness via sort-and-scan on a local clone (CVF-41).
447     let mut sorted = key.to_vec();
```

```
    sorted.sort_unstable();
    for w in sorted.windows(2) {
        if w[0] == w[1] {
            return Err(format!(
                "Key element {} is a duplicate; all elements must be distinct.",
                w[0]
            ));
        }
    }
```

same file

```
pub struct StreamV8Encryptor {
    primes: Vec<u64>,
    sk_fmt: [u8; 32],
    aad: Vec<u8>,
    nonce: [u8; NONCE_SIZE],
    f: u32,
    chunk_idx: u32,
    total_real_cps: u64,
    cp_buf: Vec<u32>,
    finished: bool,
}
```

same file

```
        let sk_fmt = derive_format_subkey(sk, FORMAT_STREAM_AE_V8);
        let mut header = Vec::with_capacity(1 + 4 + NONCE_SIZE);
        header.push(FORMAT_STREAM_AE_V8);
        header.extend_from_slice(&frame_codepoints.to_be_bytes());
        header.extend_from_slice(&nonce);
        Ok((
            Self {
                primes: primes.to_vec(),
                sk_fmt,
                aad: aad.to_vec(),
                nonce,
                f: frame_codepoints,
                chunk_idx: 0,
                total_real_cps: 0,
               cp_buf: Vec::with_capacity(frame_codepoints as usize),
                finished: false,
            },
            header,
        ))
```

same file

```
impl Drop for StreamV8Encryptor {
    fn drop(&mut self) {
        zeroize_sk(&mut self.sk_fmt);
        zeroize_key(&mut self.primes);
    }
}
```

CVF-V5-12: open Category: **efficiency**

The fix for round-4 CVF-15 makes every v7 decryptor run full trial-division key validation before the tag check, two or three times per call. A forged frame now costs thousands of key-dependent divisions where v4 spent one HMAC, and the rejection time depends on the secret primes.

This was introduced by the round-5 remediation. In v4 the v7 decryptors computed `key_bytes` and the tag first and did no validation.

Validation now runs at the head of `decrypt_raw` and again inside `decrypt`, which `decrypt_raw` calls after authentication. `decrypt_bytes` likewise validates first and again through `decrypt`. `decrypt_str` validates and then calls `decrypt_bytes`, which makes three validations.

`validate_key` trial-divides each element by every odd number up to its square root. For primes in the widened interval this is up to 1,935 divisions per element; for a `K = 13` key, about 17,400 divisions on average per validation, ranging from about 6,500 to 25,000 depending on the secret primes. An attacker who can reach the OT gateway's secure-side socket can send minimal 48-byte envelopes with random tags. `ot_frame::unwrap_pdu` (out of scope) passes every received envelope to `decrypt_raw`. The attacker gets roughly two orders of magnitude of CPU amplification over v4, plus a pre-authentication timing signal that is a function of the sum of the square roots of the key elements. On the v7 format the prime tuple is the whole key. Our estimate of the information exposed per observation is at most about 12.5 bits, which is why this is rated low.

This is the round-4 CVF-29 behaviour (per-call validation as a timing channel), which the `NapqesKey` type was introduced to remove, now extended to entry points that previously lacked it.

Recommendation

> In `decrypt_bytes` and `decrypt_raw`, verify the tag before validating the key. `key_bytes` and HMAC are defined for any key, and validation is needed only before the arithmetic in `decrypt_core`. Call `decrypt_core` directly rather than the validating `decrypt`, so that validation runs at most once per call. Preferably, add v7 decryptors that take a pre-validated key type, the counterpart of `NapqesKey`, so that a gateway validates once when it loads its key store. Extend the paper's section 8.4 to say that the v7 decryptors validate before authentication.

Client Comment

/rust/src/lib.rs

```rust
fn validate_key(key: &[u64]) -> Result<(), String> {
    if key.is_empty() {
        return Err("Key must be a non-empty list of primes.".into());
    }
    // Cheap checks first: trial division of an out-of-range u64 costs seconds.
    if key.len() > MAX_KEY_ELEMENTS {
        return Err(format!(
            "Key has {} elements, exceeding MAX_KEY_ELEMENTS ({}). Very large \
             keys are rejected because per-message primality/distinctness \
             validation is O(K²) with the classic scan and O(K log K) with \
             sort-and-scan; the cap makes the cost of legitimate use bounded.",
            key.len(),
            MAX_KEY_ELEMENTS
        ));
    }
    for (i, &k) in key.iter().enumerate() {
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of {}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        // CVF-16: enforce the normative upper bound on every key, not just on
        // generation.
        if k > MAX_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds MAX_KEY_PRIME ({}).",
                i, k, MAX_KEY_PRIME
            ));
        }
```

```
        // key_bytes keeps only the low 5 bytes; unreachable while
        // MAX_KEY_PRIME < 2^40, retained as a fail-safe.
        if k > MAX_SERIALISABLE_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds {}, the largest value \
                 representable in the 5-byte key serialisation.",
                i, k, MAX_SERIALISABLE_KEY_PRIME
            ));
        }
    }
    for (i, &k) in key.iter().enumerate() {
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
    }
    // Distinctness via sort-and-scan on a local clone (CVF-41).
    let mut sorted = key.to_vec();
    sorted.sort_unstable();
    for w in sorted.windows(2) {
        if w[0] == w[1] {
            return Err(format!(
                "Key element {} is a duplicate; all elements must be distinct.",
                w[0]
            ));
        }
    }
    Ok(())
}
```

same file

```
pub fn decrypt(nonce: &[u8], cypher: &[u64], key: &[u64]) -> Result<Vec<u32>, String> {
    validate_key(key)?; // CVF-15
```

same file

```
pub fn decrypt_bytes(ciphertext: &[u8], key: &[u64], aad: &[u8]) -> Result<String, String> {
    validate_key(key)?; // CVF-15
```

same file

```
    let codepoints = decrypt(nonce, &tokens, key)?;
```

same file

```
pub fn decrypt_str(cypher: &str, key: &[u64], aad: &[u8]) ->
Result<String, String> {
    validate_key(key)?; // CVF-15 — reject before base64 decode
```

same file

```
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
```

same file

```
    let codepoints = decrypt(nonce, &tokens, key)?;
```

CVF-V5-13: open Category: **architecture**

A new library feature, `noise_x10`, silently changes the v8 wire format and noise cap for every user of the crate in a build, and nothing in the key schedule or the tag records which variant produced a ciphertext.

This was introduced by the round-5 remediation. The v4 manifest had no features, and `MAX_NOISE_RUN = 19` was unconditional.

The manifest declares `noise_x10 = []` with the comment "Demo-only … Wire-incompatible with the default build and the Python/C KATs". Under the feature, `MAX_NOISE_RUN` becomes 9 instead of 19. That changes the emission cap, the per-bucket ceiling, the decoder's token-count checks and the streaming chunk size, so the ciphertext length becomes `48 + 80(B+2)` instead of the paper's `48 + 160(B+2)`. Cargo unifies features across a dependency graph. If any crate in a binary enables `napqes/noise_x10`, then every user of `napqes` in that binary gets the non-conforming format, and the public `MAX_NOISE_RUN` constant changes value under them.

Both variants use the same format identifier `0x01` for the format subkey, and the tag input does not include the cap. A ciphertext from one build therefore passes the HMAC check in the other. The default build's pre-tag length check also accepts it, because the real-token count is always even. It is rejected only after authentication, by the bucket-legality check (`2(B+2)` or `(B+2)/2` real tokens is never of the form `B+2`), and the operator sees a "Malformed" error rather than an authentication or format error. The separation is a coincidence of parity, not domain separation.

The feature is not tested anywhere, is not mentioned in the README or the remediation report, and the doc comment justifying the cap is attached only to the default variant. With `fips_gate` enabled, the power-on self-test catches it, because KAT-4 pins a 2,928-byte ciphertext. That makes the module permanently inoperable rather than misbehaving. Without `fips_gate`, which is the default, the build encrypts silently in the non-conforming format.

Recommendation

Remove `noise_x10` from the library; a demonstration that needs a different noise budget can define its own constant outside the crate. If the switch must stay, give that variant its own format identifier so that the subkey differs and cross-build ciphertexts fail at the tag, make it a compile error in combination with `fips_gate`, document it next to `MAX_NOISE_RUN`, and provide matching test vectors.

Client Comment

/rust/Cargo.toml

```
16 # Demo-only: MAX_NOISE_RUN 19 -> 9 (x10 tokens per codepoint instead
   of x20).
17 # Wire-incompatible with the default build and the Python/C KATs.
18 noise_x10 = []
```

/rust/src/lib.rs

```
1422 #[cfg(not(feature = "noise_x10"))]
1423 pub const MAX_NOISE_RUN: u64 = 19;
1424 /// Demo-only x10 token budget; ciphertexts are NOT interoperable
   with the default build.
1425 #[cfg(feature = "noise_x10")]
1426 pub const MAX_NOISE_RUN: u64 = 9;
```

same file

```
1428 /// Domain `0x0B` format-subkey identifier for v8 block mode.
1429 pub const FORMAT_BLOCK_V8: u8 = 0x01;
1430 /// Domain `0x0B` format-subkey identifier for the v8 streaming-AE
   format.
1431 pub const FORMAT_STREAM_AE_V8: u8 = 0x02;
```

same file

```
1440 fn derive_format_subkey(sk: &[u8], format_id: u8) -> [u8; 32] {
1441     hmac_digest(sk, &[0x0B, format_id])
1442 }
```

/rust/src/self_test.rs

```
462 /// Expected v8 ciphertext length for a one-codepoint message under
   the
463 /// default bucket profile: 16 (nonce) + 8 * R * (MAX_NOISE_RUN+1) +
   32 (tag)
464 /// where R = B + 2 = 18 for B = 16. The self-test asserts this to
   catch a
465 /// regression that changes ciphertext size independent of the tag
   pipeline.
```

```
const KAT_V8_EXPECTED_LEN: usize = 2928;
/// SHA-256 of W002's `ciphertext_hex`.
const KAT_V8_EXPECTED_SHA256: [u8; 32] = [
    0xba, 0x1c, 0x3f, 0x66, 0x4c, 0x62, 0xe9, 0x34,
    0x56, 0x83, 0x32, 0x74, 0xb5, 0x5f, 0x70, 0x3e,
    0xa8, 0x26, 0xf1, 0x28, 0x24, 0xa3, 0x90, 0x88,
    0x5d, 0x8e, 0x7f, 0x1e, 0xb3, 0x99, 0x4c, 0xed,
];
```

CVF-V5-14: open Category: **behavior**

`generate_v8_key` and `generate_prime_numbers` return `Ok` for a key count of zero or above `MAX_KEY_ELEMENTS`, producing keys that every entry point then rejects, contrary to the documented contract.

This was partly introduced by the round-5 remediation: the 128-element cap is new in v5 but was not carried over to key generation, while the zero-count case existed in v4.

`generate_prime_numbers` does not check `count`. With `count == 0` the loop never runs and it returns `Ok(vec![])`. With `count > 128` it returns `Ok` after a quadratic `contains` scan; `generate_prime_numbers(1_000_000, ...)` performs about 5 x 10^11 comparisons before returning a key that nothing accepts. The documentation of `generate_v8_key` says it returns `Err` when the key "would fail `validate_key`", but it checks only the interval. `validate_key` rejects both an empty key and one with more than 128 elements.

Recommendation

Reject `count == 0 || count > MAX_KEY_ELEMENTS` in `generate_v8_key`, and preferably in `generate_prime_numbers`. Alternatively, run `validate_key` on the output before returning it, so that the documented contract holds.

Client Comment

/rust/src/lib.rs

```
pub fn generate_prime_numbers(count: usize, min_val: u64, max_val:
u64) -> Result<Vec<u64>, String> {
    use rand::distributions::{Distribution, Uniform};
    if max_val <= min_val {
        return Err(format!(
```

```
            "generate_prime_numbers: empty range [{}, {}]",
            min_val, max_val
        ));
    }
    let mut rng = rand::thread_rng();
    let dist = Uniform::new_inclusive(min_val, max_val);
    let max_attempts = (count.max(1) as
u64).saturating_mul(KEYGEN_DRAWS_PER_PRIME);
    let mut primes: Vec<u64> = Vec::with_capacity(count);
    let mut attempts: u64 = 0;
    while primes.len() < count && attempts < max_attempts {
        // CVF-20: unbiased sample via rand::Uniform (Lemire
rejection),
        // replacing the modulo-bias line `min_val + (rng.next_u64()
% span)`.
        let num = dist.sample(&mut rng);
        if is_prime(num) && !primes.contains(&num) {
            primes.push(num);
        }
        attempts += 1;
    }
    if primes.len() < count {
        return Err(format!(
            "generate_prime_numbers: could not find {} distinct primes
in \
             [{}, {}] within {} draws; the range is too narrow or too
prime-poor \
             for the requested count.",
            count, min_val, max_val, max_attempts
        ));
    }
    Ok(primes)
}
```

same file

```
/// Generate a v8 key pair: an arithmetic-layer prime tuple and an
/// independently-sampled, uniformly random 256-bit HMAC subkey.
///
/// The two components MUST be generated independently (never one
derived
/// from the other) for the CVF8/CVF13 security argument above to
hold, and
/// MUST both be treated as secret key material.
///
/// Returns `Err` (CVF-20) if `[min_val, max_val]` is not a sub-range
of the
```

```
1452 /// normative interval `[MIN_KEY_PRIME, MAX_KEY_PRIME]` (such a key
     would fail
1453 /// `validate_key`), or if `count` distinct primes cannot be drawn
     from it.
1454 ///
1455 /// The returned `sk` is `Copy`: wipe every copy (see
     [`zeroize_sk`]), or use
1456 /// [`crate::NapqesKey::generate`], which owns both components and
     wipes them
1457 /// on drop.
1458 pub fn generate_v8_key(
1459     count: usize,
1460     min_val: u64,
1461     max_val: u64,
1462 ) -> Result<(Vec<u64>, [u8; SK_SIZE]), String> {
1463     if !(MIN_KEY_PRIME <= min_val && max_val <= MAX_KEY_PRIME) {
1464         return Err(format!(
1465             "generate_v8_key: range [{}, {}] lies outside
     [MIN_KEY_PRIME, MAX_KEY_PRIME]",
1466             min_val, max_val
1467         ));
1468     }
1469     let primes = generate_prime_numbers(count, min_val, max_val)?;
1470     let mut sk = [0u8; SK_SIZE];
1471     rand::thread_rng().fill_bytes(&mut sk);
1472     Ok((primes, sk))
1473 }
```

CVF-V5-15: open Category: **procedural**

The repository root commits `meta.json`, which contains a complete 13-element prime key tuple, and the fixed key validator accepts that tuple.

This file lies outside the Rust scope of this review. It is raised because committing key material to a repository is a procedural defect wherever it sits. It could not be compared with v4, because the round-4 delivery was a crate snapshot rather than the full repository.

`meta.json` is a single 1,364-byte line holding `key_primes`, `"key_count": 13` and `"sk_bits": 256`. It is not matched by `.gitignore`, so it is part of the client's stated commit. All 13 values are prime, distinct and inside `[10^6, 14,999,999]`. Five of them (10,008,853; 10,272,707; 11,046,967; 12,778,981; 11,042,657) exceed the round-4 bound of 9,900,000, which shows that the tuple was generated under the widened v5 interval. The v5 `validate_key` accepts the tuple: it has at most 128 elements, and every element

is within range, prime and distinct. Round-4 CVF-16's upper bound therefore does not stand between this file and use.

No producer or consumer of the file exists anywhere in the delivered tree. The producer is presumably under one of the paths `.gitignore` excludes (`tools/`, `verticals/`, `demos/`, `tmp/`).

What this exposes depends on how the tuple was used. The `profiles` and `sk_bits` fields point to a v8 benchmark or demonstration, where the tuple is only half the key and `sk` is not committed. But the `operation_samples` are Modbus and DNP3 frames, and the crate encrypts such frames through `ot_frame`, which uses the v7 `encrypt_raw` keyed by the prime tuple alone. If this tuple was ever provisioned to an OT gateway, it is a complete v7 key, and anyone with the repository can decrypt and forge that device's frames.

Recommendation

Remove `meta.json` from the repository and its history. Treat the tuple as compromised: never provision it, and rotate it wherever it was used. Add a secret-scanning step to CI that recognises prime-tuple key material as well as `sk`.

Client Comment

/meta.json

```
{"operations":["telemetry","breaker_status","setpoint",
"breaker_trip","config","firmware"],"operation_samples":
{"telemetry":"MODBUS FC03 RD HR 40001-40012 UNIT1 SUBST-GN3",
"breaker_status":"MODBUS FC02 RD DI 10001-10016 CB27 CB28 UNIT1
SUBST-GN3","setpoint":"MODBUS FC16 WR HR 40050=13800 40051=+2
40052=0.95 TAP-CHANGER UNIT1 FEEDER-GN3-A CONFIRM","breaker_trip":
"DNP3 g12v1 CB27 OPEN TRIP U1","config":"DNP3 CONFIG DL PROFILE
FEEDER-GN3-A THRESH OV=1.1 UV=0.9 OC=1200A ROCOF=0.5 DEADBAND=0.02
POLL=2000ms RETRY=3 TZ=KST DNP-ADDR=0x0003 MASTER=0x0001
SEQ-WINDOW=32","firmware":"DNP3 FW-BLOCK
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718
A1B2C3D4E5F60718A1B2C3D4E5F60718A1B2C3D4E5F60718"},"profiles":
["bucket","coarse","frame"],"key_primes":[REDACTED (13 primes)],
"key_count":13,"sk_bits":256}
```

CVF-V5-16: open Category: **documentation**

The v7 decryptors are documented as retained to decrypt existing v7 ciphertexts, but after the CVF-15 and CVF-14 fixes they reject some keys and plaintexts that v4 accepted, so archived v7 data is not guaranteed to decrypt.

In v4 the v7 decryptors (`decrypt` v4 lib.rs:523-526, `decrypt_bytes` 831-859, `decrypt_raw` 933-960) never called `validate_key`, and v4's own documentation of `MAX_KEY_PRIME` (v4 lib.rs:75-78) promised that decryption accepts "any prime ≥ MIN_KEY_PRIME, so keys generated before this bound was tightened remain usable". The v4 `pub fn generate_prime_numbers` accepted any range, and the v4 `pub fn encrypt` accepted any `u32`.

In v5 every v7 decryptor runs `validate_key` (lib.rs:401-458) first — `decrypt` at 821, `decrypt_bytes` at 1135, `decrypt_str` at 1199, `decrypt_raw` at 1264 — so a key is now rejected if any element is below 10^6 or above $14{,}999{,}999$, composite, duplicated, or if $K > 128$. In addition `decrypt_core` (lib.rs:793-807) now returns `Err` for a recovered value above `0x10FFFF` or in the surrogate range, which the v4 `pub fn encrypt` and the Python v7 `encrypt_bytes` (out of scope, `napqes.py:876`, no surrogate check)

could produce. Both changes are reasonable hardening, but the documentation still says the opposite: the crate header (lib.rs:27-31) says the v7 decryptors "remain to decode existing traffic", and `README.md:146-148` says they are retained to decrypt archived ciphertexts; the remediation report asserts that every v7 ciphertext decrypts byte-for-byte.

Keys produced by the in-tree generators are not affected: historical Rust keys are at most $9{,}999{,}999$, Python's at most $15{,}000{,}000$ and `kem.rs`'s below 1.5×10^7, all prime and distinct. The affected population is out-of-band keys (any prime $\geq 10^6$ above the new bound, or with an element that is composite or repeated) and archived plaintexts containing lone surrogates. That is why the severity is low; the defect is that a categorical compatibility claim is now false.

Recommendation

State the exact v7 compatibility envelope in the crate header, the README and the remediation report: which keys and which plaintexts written by earlier versions are still decryptable. If out-of-interval archives must remain readable, give the v7 decryptors a relaxed validator (non-empty, every element at least 2 and below 2^{40}, distinct) that is sufficient for the arithmetic, instead of the full v8 key policy.

Client Comment

/rust/README.md

```
146 applies (CVF-28). The corresponding decryptors (`decrypt`,
`decrypt_bytes`,
147 `decrypt_str`, `decrypt_raw`) remain to decrypt existing v7
ciphertexts and
148 are not deprecated.
```

/rust/src/lib.rs

```
27 //! * **v7 (legacy — decryptors retained for archived ciphertexts).**
28 //!   Fixed-width tokens (CVF1 fix). Wire layout same shape, but no
29 //!   `MAX_NOISE_RUN` cap, so ciphertext length varies with the nonce.
30 //!   Encryptors are `#[deprecated]` per CVF-17; decryptors
([`decrypt_bytes`],
31 //!   [`decrypt_str`], [`decrypt_raw`]) remain to decode existing
traffic.
```

same file

```
401 fn validate_key(key: &[u64]) -> Result<(), String> {
402     if key.is_empty() {
403         return Err("Key must be a non-empty list of primes.".into());
404     }
```

```
    // Cheap checks first: trial division of an out-of-range u64 costs seconds.
    if key.len() > MAX_KEY_ELEMENTS {
        return Err(format!(
            "Key has {} elements, exceeding MAX_KEY_ELEMENTS ({}). Very large \
             keys are rejected because per-message primality/distinctness \
             validation is O(K²) with the classic scan and O(K log K) with \
             sort-and-scan; the cap makes the cost of legitimate use bounded.",
            key.len(),
            MAX_KEY_ELEMENTS
        ));
    }
    for (i, &k) in key.iter().enumerate() {
        if k < MIN_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) is below the minimum of {}.",
                i, k, MIN_KEY_PRIME
            ));
        }
        // CVF-16: enforce the normative upper bound on every key, not just on
        // generation.
        if k > MAX_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds MAX_KEY_PRIME ({}).",
                i, k, MAX_KEY_PRIME
            ));
        }
        // key_bytes keeps only the low 5 bytes; unreachable while
        // MAX_KEY_PRIME < 2^40, retained as a fail-safe.
        if k > MAX_SERIALISABLE_KEY_PRIME {
            return Err(format!(
                "Key element at index {} ({}) exceeds {}, the largest value \
                 representable in the 5-byte key serialisation.",
                i, k, MAX_SERIALISABLE_KEY_PRIME
            ));
        }
    }
    for (i, &k) in key.iter().enumerate() {
```

```
        if !is_prime(k) {
            return Err(format!("Key element at index {} ({}) is not prime.", i, k));
        }
    }
    // Distinctness via sort-and-scan on a local clone (CVF-41).
    let mut sorted = key.to_vec();
    sorted.sort_unstable();
    for w in sorted.windows(2) {
        if w[0] == w[1] {
            return Err(format!(
                "Key element {} is a duplicate; all elements must be distinct.",
                w[0]
            ));
        }
    }
    Ok(())
}
```

same file

```
            if token < addend || (token - addend) % k != 0 {
                return Err(format!(
                    "Malformed v7 ciphertext: token at position {} is not of \
                     the form codepoint * k + addend for key element {}.",
                    ct_pos, k
                ));
            }
            let cp = (token - addend) / k;
            if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
                return Err(format!(
                    "Malformed v7 ciphertext: recovered value {} at position \
                     {} is not a Unicode scalar value.",
                    cp, ct_pos
                ));
            }
```

CVF-V5-17: open Category: **behavior**

The deprecated `pub fn encrypt` still accepts any `u32` codepoint, while its inverse `pub fn decrypt` now rejects surrogates and values above `0x10FFFF`, so the pair no longer round-trips.

`encrypt` (lib.rs:741-767) takes `&[u32]` and checks only the length and the key; it encrypts surrogates and out-of-range values without complaint. `decrypt` (lib.rs:820-824) calls `decrypt_core`, whose new CVF-14 guard (lib.rs:793-807) returns `Err` for exactly those values. In v4 (`encrypt` v4 lib.rs:467-498, `decrypt` 504-526) `decrypt(encrypt([0xD800]))` returned `[0xD800]`.

Trigger: `let (n, t) = encrypt(&[0xD800], &k); decrypt(&n, &t, &k)` returns `Err`. The function is deprecated and has no in-tree caller, hence low severity, but an encryptor whose own decryptor rejects its output is a contract defect the remediation introduced.

Recommendation

Reject non-scalar input in `encrypt`, consistently with its other preconditions (a documented panic, or better a `Result` signature), and document the accepted input domain in its rustdoc.

Client Comment

/rust/src/lib.rs

```
741 pub fn encrypt(message: &[u32], key: &[u64]) -> ([u8; NONCE_SIZE],
Vec<u64>) {
742     // CVF-34: uphold the length cap even on this Vec<u64>-returning
legacy
743     // path. The .expect() surfaces the cap consistently with the
deprecation
744     // attribute and matches the pattern of the CVF-15 validate_key
call.
745     assert!(
746         message.len() <= MAX_PLAINTEXT_CODEPOINTS,
747         "message length {} exceeds MAX_PLAINTEXT_CODEPOINTS ({})",
748         message.len(), MAX_PLAINTEXT_CODEPOINTS
749     );
750     // CVF-15: this v7 API returns `Vec<u64>` and cannot report a
validation
751     // error. Every other entry point returns `Result` and propagates
via `?`.
752     // The .expect() here surfaces the same validation failure as a
panic,
753     // consistent with `pub fn decrypt`'s prior panic contract and
matched by
```

```
    // the #[deprecated] attribute (CVF-17) that steers callers to v8.
    validate_key(key).expect("CVF-15: encrypt called with invalid key material");
    // CVF-35: route through the checked CRNG helper like every other v7
    // encryptor. Signature is `-> (nonce, tokens)` so a CRNG failure is
    // surfaced as `.expect` — consistent with the deprecation attribute
    // that steers callers to v8.
    let nonce = generate_nonce_with_crng_check()
        .expect("CVF-35: CRNG stuck-generator check failed inside encrypt");
    let kb = key_bytes(key);
    let noise_p = derive_noise_p(&kb, &nonce);
    let padded = pad_message(message, &kb, &nonce);
    let cypher = emit_tokens_v7(&padded, &kb, &nonce, noise_p, key);
    (nonce, cypher)
}
```

same file

```
            if token < addend || (token - addend) % k != 0 {
                return Err(format!(
                    "Malformed v7 ciphertext: token at position {} is not of \
                     the form codepoint * k + addend for key element {}.",
                    ct_pos, k
                ));
            }
            let cp = (token - addend) / k;
            if cp > 0x10FFFF || char::from_u32(cp as u32).is_none() {
                return Err(format!(
                    "Malformed v7 ciphertext: recovered value {} at position \
                     {} is not a Unicode scalar value.",
                    cp, ct_pos
                ));
            }
```

same file

```
pub fn decrypt(nonce: &[u8], cypher: &[u64], key: &[u64]) -> Result<Vec<u32>, String> {
    validate_key(key)?; // CVF-15
    let kb = key_bytes(key);
    decrypt_core(nonce, cypher, key, &kb)
```

```
824 }
```

CVF-V5-18: open Category: **procedural**

The power-on self-test has no known-answer test for the v8 streaming format added in this release, which recreates for a new algorithm path the defect round-4 CVF-18 reported for v8.

`run_power_on_self_tests` (self_test.rs:560-582) runs KAT-1..8 and INT-1: v7 KATs, v8 block KATs and the KEM derivation. None exercises `encrypt_stream_ae_v8`, `decrypt_stream_ae_v8` or `StreamV8Encryptor`, which are public, listed among the normative v8 entry points (lib.rs:17-25) and gated by `fips_gate` (lib.rs:2057-2059, 2076-2078, 2246-2248). The stream-only code — the `0x02` format subkey, `derive_stream_v8_sub_nonce`, the filler codepoints and the sentinel tag — is therefore never checked at start-up. `README.md:103-106` says POST runs "v7 and v8 encrypt/decrypt/tamper" KATs without qualification.

A deterministic helper already exists: `encrypt_stream_ae_v8_with_nonce` (`pub(crate)`, lib.rs:1960-1973), and the corpus `tests/kat/v8_stream_vectors.json` carries `nonce_hex` for S001–S014, so a known-answer test costs a few lines. Without it, a regression in the sub-nonce derivation, the sentinel binding or the filler leaves POST returning `Ok(())`, and under `fips_gate` the module keeps operating.

Recommendation

Add a streaming encrypt known-answer test (hash of one S-vector produced through `encrypt_stream_ae_v8_with_nonce`), a streaming decrypt test, and a sentinel or truncation rejection test to POST, and add their constants to `kat_constants_match_corpus`. Update the README's POST description accordingly.

Client Comment

/rust/README.md

```
103 `napqes::self_test::run_power_on_self_tests()` runs the SP 800-140B
    §4.9.1
104 KATs (v7 and v8 encrypt/decrypt/tamper, a v8 structural reject, and a
    KEM
105 derivation KAT plus FrodoKEM and hybrid pairwise-consistency checks)
    and
106 latches the outcome process-wide. Building with `--features
    fips_gate`
```

/rust/src/self_test.rs

```
pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
    // CVF-47: a panicking KAT leaves POST_RUNNING, which refuses all callers.
    let _lock = lock_post();
    POST_STATE.store(POST_RUNNING, Ordering::Release);
    let _in_post = InPostFlag::set();
    let result = (|| -> Result<(), SelfTestError> {
        kat_encrypt()?;
        kat_decrypt()?;
        kat_tamper_rejection()?;
        kat_v8_encrypt()?;
        kat_v8_decrypt()?;
        kat_v8_structural_reject()?;
        kat_v8_tamper_rejection()?;
        kat_kem()?;
        build_provenance_check()?;
        Ok(())
    })();
    match &result {
        Ok(()) => POST_STATE.store(POST_PASSED, Ordering::Release),
        Err(_) => POST_STATE.store(POST_FAILED, Ordering::Release),
    }
    result
}
```

CVF-V5-19: open Category: **behavior**

A failure of the continuous random-number-generator test fails only the current call and does not put the module into the error state, although that generator now also supplies the nonces of the v8 streaming format.

`generate_nonce_with_crng_check` (lib.rs:103-119) returns `Err` when two consecutive draws are equal and leaves `POST_STATE` untouched; only `run_power_on_self_tests` writes the state (self_test.rs:57-62, 560-582). The next call simply draws again and proceeds with the module still PASSED. Under the standard the crate cites, a conditional self-test failure must inhibit operation until the module is re-initialised.

Two things introduced in v5 make this matter. The POST latch itself is new (the round-4 CVF-47 fix), so there is now an error state the CRNG test could feed and does not. And the v8 streaming entry points `encrypt_stream_ae_v8` (lib.rs:2050-2062) and `StreamV8Encryptor::new` (lib.rs:2240-2271) now draw their 16-byte nonce through this function. Streaming is not misuse-resistant: nonce reuse under one `sk`

reveals token differences and allows chunk splicing. A stuck DRBG therefore fails one `StreamV8Encryptor::new` and the very next call goes ahead.

The test itself is weak (round-4 CVF-35: it detects only identical consecutive draws), which keeps the severity low.

Recommendation

On a continuous-test failure, set the POST state to FAILED under `POST_LOCK`, so that every gated entry point refuses until POST is re-run, and document this in the README's POST section.

Client Comment

/rust/src/lib.rs

```
103 fn generate_nonce_with_crng_check() -> Result<[u8; NONCE_SIZE],
String> {
104     let mut nonce = [0u8; NONCE_SIZE];
105     rand::thread_rng().fill_bytes(&mut nonce);
106     let mut prev = PREV_NONCE.lock().map_err(|_| "CRNG mutex
poisoned".to_string())?;
107     if let Some(prev_nonce) = *prev {
108         if ct_eq_bytes(&nonce, &prev_nonce) {
109             // CVF-35: SP 800-140B §4.9.2 "stuck-generator" check. The
110             // single process-global PREV_NONCE slot is not a per-key
111             // uniqueness guarantee — it catches only an
immediately-stuck
112            // DRBG, not restart-and-replay across processes or per-key
113             // reuse across time. The previous message overstated the
scope.
114             return Err("CRNG stuck-generator check failed:
consecutive draws matched (SP 800-140B §4.9.2)".into());
115         }
116     }
117     *prev = Some(nonce);
118     Ok(nonce)
119 }
```

same file

```
2050 pub fn encrypt_stream_ae_v8(
2051     plaintext: &str,
2052     primes: &[u64],
2053     sk: &[u8; SK_SIZE],
2054     aad: &[u8],
```

```
    frame_codepoints: u32,
) -> Result<Vec<u8>, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    let nonce = generate_nonce_with_crng_check()?;
    encrypt_stream_ae_v8_with_nonce(plaintext, primes, sk, &nonce, aad, frame_codepoints)
}
```

same file

```
    pub fn new(
        primes: &[u64],
        sk: &[u8; SK_SIZE],
        aad: &[u8],
        frame_codepoints: u32,
    ) -> Result<(Self, Vec<u8>), String> {
        // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
        #[cfg(feature = "fips_gate")]
        crate::self_test::require_post().map_err(|e| e.to_string())?;
        validate_key(primes)?;
        validate_stream_v8_frame(frame_codepoints)?;
        let nonce = generate_nonce_with_crng_check()?;
        let sk_fmt = derive_format_subkey(sk, FORMAT_STREAM_AE_V8);
        let mut header = Vec::with_capacity(1 + 4 + NONCE_SIZE);
        header.push(FORMAT_STREAM_AE_V8);
        header.extend_from_slice(&frame_codepoints.to_be_bytes());
        header.extend_from_slice(&nonce);
        Ok((
            Self {
                primes: primes.to_vec(),
                sk_fmt,
                aad: aad.to_vec(),
                nonce,
                f: frame_codepoints,
                chunk_idx: 0,
                total_real_cps: 0,
               cp_buf: Vec::with_capacity(frame_codepoints as usize),
                finished: false,
            },
            header,
        ))
    }
```

/rust/src/self_test.rs

```
const POST_NOT_RUN: u8 = 0;
const POST_RUNNING: u8 = 1;
const POST_PASSED: u8 = 2;
const POST_FAILED: u8 = 3;

static POST_STATE: AtomicU8 = AtomicU8::new(POST_NOT_RUN);
```

same file

```
pub fn run_power_on_self_tests() -> Result<(), SelfTestError> {
    // CVF-47: a panicking KAT leaves POST_RUNNING, which refuses all callers.
    let _lock = lock_post();
    POST_STATE.store(POST_RUNNING, Ordering::Release);
    let _in_post = InPostFlag::set();
    let result = (|| -> Result<(), SelfTestError> {
        kat_encrypt()?;
        kat_decrypt()?;
        kat_tamper_rejection()?;
        kat_v8_encrypt()?;
        kat_v8_decrypt()?;
        kat_v8_structural_reject()?;
        kat_v8_tamper_rejection()?;
        kat_kem()?;
        build_provenance_check()?;
        Ok(())
    })();
    match &result {
        Ok(()) => POST_STATE.store(POST_PASSED, Ordering::Release),
        Err(_) => POST_STATE.store(POST_FAILED, Ordering::Release),
    }
    result
}
```

CVF-V5-20: open Category: **procedural**

Several regression tests added in this release and named after round-4 findings cannot fail for the defect they are named for, which recreates the round-4 CVF-19 defect class against the remediation report's claim that every fix is covered by an automated check named after it.

- `cvf43_unpad_rejects_noncanonical_length_prefix` (lib.rs:2722-2735) only round-trips an honest message, so it passes on v4. Its comment says the rejection path "is covered by an internal test below via a helper that mimics decrypt_core_v8's

final step"; no such test or helper exists in `lib.rs`, `self_test.rs` or `kat_cross_check.rs`, and the test module never calls `unpad_message`. The real evidence for CVF-43 is corpus vector W-N11, which does fail on v4.

- `cvf42_named_alphabet_constants_are_correct` (lib.rs:2802-2810) compares the new constants with the literals they replaced; it fails on v4 only by not compiling. The real evidence is KAT byte parity.
- `cvf20_generate_prime_numbers_produces_valid_key` (lib.rs:2703-2719) checks primality, range, distinctness and `validate_key`, properties v4's modulo-biased sampler also had; it cannot detect the bias CVF-20 was about.
- `napqeskey_matches_bare_slice_api_byte_for_byte` (lib.rs:2875-2886) compares two paths that both call `encrypt_bytes_v8_core`, and carries a stray `#[allow(deprecated)]` on a call that is not deprecated.

The remediation report (§7) states that "Every finding's fix is covered by at least one automated check named after it". For these findings that is true only of the name. No defect escapes today, because the corpus vectors or the code itself carry the evidence, hence low severity.

Recommendation

Test `unpad_message` directly on non-canonical prefixes such as `[0, 300, …]`, `[1, 256, …]` and `[0x100, 0, …]` (with a buffer of length B+2) and assert an `Err` containing "byte-valued"; delete the false comment. Drop or rename the CVF-42 constant test. For CVF-20, either test the distribution over a deliberately small span or state that the fix is verified by inspection. Correct the §7 claim in the remediation report.

Client Comment

/rust/src/lib.rs

```
2703     #[test]
2704     fn cvf20_generate_prime_numbers_produces_valid_key() {
2705         for _ in 0..8 {
2706             let ps = generate_prime_numbers(DEFAULT_KEY_COUNT,
MIN_KEY_PRIME, MAX_KEY_PRIME);
2707             assert_eq!(ps.len(), DEFAULT_KEY_COUNT);
2708             for &p in &ps {
2709                 assert!(is_prime(p), "sampler returned non-prime: {}",
p);
2710                  assert!(p >= MIN_KEY_PRIME && p <= MAX_KEY_PRIME);
2711             }
2712             let mut sorted = ps.clone();
2713             sorted.sort();
2714             sorted.dedup();
```

```
            assert_eq!(sorted.len(), ps.len(), "sampler returned duplicates: {:?}", ps);
            // The sampler's output must pass validate_key unconditionally.
            validate_key(&ps).expect("sampler output failed validate_key");
        }
    }
```

same file

```

    #[test]
    fn cvf43_unpad_rejects_noncanonical_length_prefix() {
        // Directly test unpad_message via decrypt_bytes_v8 with a hand-
        // crafted padded buffer would require constructing a valid ciphertext;
        // easier to exercise via a small helper. Instead, verify by proxy:
        // any v8 round-trip must still succeed (byte-identical to before).
        // The direct rejection path is covered by an internal test below via
        // a helper that mimics decrypt_core_v8's final step.
        let (primes, sk) = generate_v8_key(DEFAULT_KEY_COUNT, MIN_KEY_PRIME, MAX_KEY_PRIME);
        let msg = "CVF-43 sanity";
        let ct = encrypt_bytes_v8(msg, &primes, &sk, b"").unwrap();
        assert_eq!(decrypt_bytes_v8(&ct, &primes, &sk, b"").unwrap(), msg);
    }
```

same file

```
    #[test]
    fn cvf42_named_alphabet_constants_are_correct() {
        // Byte-parity check: existing KATs would fail if PAD_ALPHABET/
        // NOISE_ALPHABET/ASCII_PRINTABLE_BASE were changed. This asserts
        // the constants match the paper's numeric values.
        assert_eq!(NOISE_ALPHABET, 96);
        assert_eq!(PAD_ALPHABET, 95);
        assert_eq!(ASCII_PRINTABLE_BASE, 32);
    }
```

same file

```
    #[test]
```

```
    fn napqeskey_matches_bare_slice_api_byte_for_byte() {
        // The NapqesKey path must produce byte-identical ciphertexts to the
        // bare-slice path (CVF-29 fix skips validate_key but must not change
        // the wire format).
        let k = NapqesKey::generate().unwrap();
        let msg = "byte parity check";
        let via_key = encrypt_bytes_v8_key(msg, &k, b"aad").unwrap();
        #[allow(deprecated)]
        let via_slice = encrypt_bytes_v8(msg, k.primes(), k.sk(), b"aad").unwrap();
        assert_eq!(via_key, via_slice);
    }
```

CVF-V5-21: open Category: **procedural**

The v7 negative known-answer test silently skips corpus vector N009, the cross-port empty-ciphertext rejection vector for round-4 CVF-13, and would fail its new reason check if it ran.

`legacy_v7_negative_returns_err` (kat_cross_check.rs:228-300) skips every vector whose `tampered_hex` is empty (241-244). N009 in `tests/kat/v6_vectors.json` is exactly such a vector — the empty ciphertext, `"tampered_hex": ""` — so it is never exercised. The guard at 295-299 requires only `tested > 0`, and `corpus_sizes_are_pinned` (64-80) counts corpus entries rather than exercised ones, so the skip of one vector is invisible and the harness reports a clean run.

If the skip were removed, N009 would fail: its `expected_exception` is "not a valid authenticated v7 payload", while Rust now returns "Ciphertext too short..." (lib.rs:1137-1143). N002 was given `expected_error_contains: "too short"` for exactly this reason; N009 was not.

The skip already existed in v4 (v4 kat_cross_check.rs:197-200), so this is not introduced in v5; it matters now because N009 is the cross-port evidence for the CVF-13 fix. The behaviour itself is covered by the unit test `empty_ciphertext_is_rejected` (lib.rs:2432-2443), hence low severity.

Recommendation

Remove the skip (an empty `tampered_hex` is the vector, not a missing field), add `expected_error_contains: "too short"` to N009, and assert the exact number of exercised negatives instead of `tested > 0`. Apply the same exact-count assertion to every harness test that has a skip path.

Client Comment

/rust/src/kat_cross_check.rs

```rust
#[test]
fn legacy_v7_negative_returns_err() {
    let vectors = load_vectors();
    let mut tested = 0;

    for vec in vectors
        .iter()
        .filter(|v| v["kind"] == "negative" && v["api"] != "stream_ae")
    {
        let id = vec["id"].as_str().unwrap();
       let tampered_hex = vec["tampered_hex"].as_str().unwrap_or("");
        let aad_hex = vec["aad_hex"].as_str().unwrap_or("");

        if tampered_hex.is_empty() {
            eprintln!("[SKIP] {} — no tampered_hex", id);
            continue;
        }

        let key: Vec<u64> = vec["key"]
            .as_array()
            .unwrap()
            .iter()
            .map(|v| v.as_u64().unwrap())
            .collect();

        let ct = hex_decode(tampered_hex);
        let aad = hex_decode(aad_hex);

        let result = decrypt_bytes(&ct, &key, &aad);
        match result {
            Ok(pt) => panic!("[{}] expected Err but got Ok({:?})", id, pt),
            Err(err_text) => {
                // CVF-25: if the corpus supplies an `expected_error_contains`
                // string, check that the error message text contains it —
```

```
                // this distinguishes rejection at the tag check from
               // rejection at a post-authentication structural check
                // (Dec step (3), (6), (8)). Vectors that need to reach the
                // structural checks (W-N06/W-N07/W-N08) rely on this
                // discriminator to prove they actually do.
                // Corpus emits `expected_exception` (a substring of the
                // Python ValueError message); the same words also appear in
                // the Rust error text, so we can enforce them cross-port.
                // `expected_error_contains` remains reserved for cases where
                // the two languages diverge.
                if let Some(want) = vec
                    .get("expected_error_contains")
                    .and_then(|v| v.as_str())
                    .or_else(||
vec.get("expected_exception").and_then(|v| v.as_str()))
                {
                    assert!(
                        err_text.contains(want),
                       "[{}] error message did not contain expected \
                         substring {:?}; got: {}",
                        id, want, err_text
                    );
                }
                eprintln!("[PASS] {} — correctly returned Err", id);
                tested += 1;
            }
        }
    }

    eprintln!("\nRust KAT negative: {} passed", tested);
    // CVF-19: a negative-KAT test that never submitted a tampered ciphertext
    // is a hard failure. The previous eprintln! was the auditor's headline
    // example of a test that could not report failure: "Not a hard failure"
    // is exactly the wrong disposition for a rejection-oracle test.
    assert!(
        tested > 0,
        "negative_returns_err: no tampered vectors submitted — the \
         rejection oracle was never exercised (CVF-19)"
```

```
299     );
300 }
```

CVF-V5-22: open Category: **documentation**

Comments written for the round-4 fixes no longer match the code, including a crate header that claims no panic is reachable from a `&[u64]` key.

- The crate header (`lib.rs:39-45`) says every entry point taking a `&[u64]` key runs `validate_key` "as its first statement" and that "no panic path is reachable from `&[u64]` input". The deprecated `pub fn encrypt` panics by design on an over-length message (`assert!` at `lib.rs:745-749`) and on an invalid key (`validate_key(key).expect(...)` at `lib.rs:755`), and the crate's own test `cvf15_pub_encrypt_panics_on_invalid_key` (`#[should_panic]`) asserts that panic. `encrypt_stream_ae_v8` also draws its nonce before validating, so "first statement" is not literal either. The v4 header made no such claim.
- The inline comment in `encrypt` (`lib.rs:742-744`) says ".expect() surfaces the cap", but the cap is enforced with `assert!`. `encrypt` has no rustdoc, and none of its three panic conditions (length, key, CRNG failure) is documented.
- The `ct_eq_bytes` doc (`lib.rs:940-945`) says it is used at "the three tag verification sites" plus the CRNG check. It is now reached from seven sites, including the streaming tag checks and the self-test.
- `Cargo.toml:52-55` says test builds keep `debug_assert!` live "so CVF-12's ct_eq_bytes length check fires". That check was removed by the CVF-12 fix, and `[profile.test]` does not set `debug-assertions` at all.
- The `varint_keystream` doc (`lib.rs:673-677`) still justifies the mask by the LEB128 continuation-bit bias under NIST SP 800-22. The LEB128 format is gone; round-4 CVF-33 asked for this justification to be replaced.

None of these changes behaviour, but each misleads a reader about the security-relevant contract of the code it sits beside.

Recommendation

Restate the header's guarantee with its exception (the deprecated `encrypt` panics), or give `encrypt` a `Result` signature and a rustdoc listing its failure conditions. Correct the `encrypt` inline comment, the `ct_eq_bytes` call-site list and the `Cargo.toml` profile comment. Rewrite the `varint_keystream` doc to state what the mask is for in the current format, or remove the LEB128 reference.

Client Comment

/rust/Cargo.toml

```
# CVF-23: manifest hardening — release builds now trap on integer overflow
# (belt-and-braces backstop for CVF-16's checked_mul recommendation), and
# test builds keep debug_assert! live so CVF-12's ct_eq_bytes length check
# fires in `cargo test`.
```

/rust/src/lib.rs

```
//! # Validation
//!
//! Every entry point that accepts a `&[u64]` key runs [`validate_key`]
//! (private) as its first statement, per CVF-15. A key that is empty,
//! composite, below [`MIN_KEY_PRIME`], above [`MAX_KEY_PRIME`], or contains
//! duplicates is rejected with a descriptive `String` error before any
//! HMAC or arithmetic runs — no panic path is reachable from `&[u64]` input.
```

same file

```
/// HMAC-CTR keystream for masking the varint blob — domain byte 0x07.
///
/// Each 32-byte block is `HMAC(key_bytes, 0x07 || nonce || uint32_be(block))`.
/// XOR-masking the raw LEB128 blob eliminates the 3:1 MSB continuation-bit
/// bias that otherwise causes systematic NIST SP 800-22 failures.
```

same file

```
    // CVF-34: uphold the length cap even on this Vec<u64>-returning legacy
    // path. The .expect() surfaces the cap consistently with the deprecation
    // attribute and matches the pattern of the CVF-15 validate_key call.
```

same file

```
940 /// Used at the three tag verification sites (v7 `decrypt_bytes`,
941 /// `decrypt_raw`, v8 `decrypt_bytes_v8`) on `[u8; TAG_SIZE]`
    operands, and
942 /// once inside `generate_nonce_with_crng_check` on `[u8;
    NONCE_SIZE]`
943 /// operands (CVF-35 flags that the latter compares public values — no
944 /// constant-time requirement — but the comparator's uniform length
    handling
945 /// makes it safe there too).
```

CVF-V5-23: open Category: **procedural**

The remediated specification carries no version marker and is filed as `docs/napseq-eprint-v3.tex`, although it is newer than the round-4 specification, so a reader cannot tell which revision a citation or a port refers to.

The round-5 paper (2,647 lines) is not the round-3 file of the same name (1,258 lines); against the round-4 source it differs by 57 hunks. Yet the rounds 3, 4 and 5 sources all carry the same title and `\date{July 2026}` (`napseq-eprint-v3.tex:48`), and no version string appears anywhere in the file. Only text outside the paper calls it "NAPQES v4": `lib.rs:11-13` and `README.md:161-162` say "the filename lags the version tag one revision", but there is no version tag to lag. The remediation report lists the rename as still pending.

This matters more in this round because a normative parameter changed (the prime interval and default `K`), so ports written against the round-4 text must be able to tell that the text they hold is out of date. Two in-repo pointers are also stale: `self_test.rs:143` cites `docs/napseq-eprint-preprint.tex` "rem:domsep", which is the May 2026 preprint describing the v7 layout, while the rest of `rust/` cites the current text under the v3 name. The shipped `docs/napseq-eprint-v3.pdf` matches the tex but has 32 pages, not the 31 the client states.

Recommendation

Give the paper an explicit version identifier (in the title block or `\date`) and a revision history that names the normative changes of each revision, including the interval change. Rename the file to match that version, update every pointer in `rust/` together, and repoint `self_test.rs:143` at the current paper's remark.

Client Comment

/docs/napseq-eprint-v3.tex

```
\date{July 2026}
```

/rust/README.md

```
* Paper: EPINeon NAPQES v4 (`docs/napseq-eprint-v3.tex`; will be renamed
  to `napseq-eprint-v4.tex` in the next revision).
```

/rust/src/lib.rs

```
//! Specification: EPINeon NAPQES v4 (see `docs/napseq-eprint-v3.tex` — the
//! filename lags the version tag one revision and will be renamed to
//! `napseq-eprint-v4.tex` in a follow-up).
```

/rust/src/self_test.rs

```
// introduced by the CVF2 fix (see SPEC.md §3 and docs/napseq-eprint-preprint.tex
```

CVF-V5-24: open Category: **documentation**

The specification's remediation edits left residue: the `sk` to `sk_fmt` restatement missed the security preamble and the correctness proof, a new sentence calls a roughly 2^{-20} event negligible, a remark cites a non-existent finding "CV-11", and §9 claims 15 statistical tests where 14 were run.

- Stale `sk` wording (round-4 CVF-11). The format subkey `sk_fmt` now keys every per-message HMAC, but the §4 preamble (`napseq-eprint-v3.tex:1115-1125`) still describes a single external oracle "keyed by the challenger's sk" and says NAPQES has no MAC key "separate from sk". The proof of Proposition 3.14 (1044-1077) argues from "the same sk" and "a function of sk and N", and does not discharge the new rejection branches of `Dec` steps (7) and (8a). Lemma nonce-collision (1269-1283) is stated "under the same sk" but hops against `HMAC(sk_fmt, ·)` without a uniformity premise; it is used only inside game G_1, so no bound breaks. `Enc` has a step (0) deriving `sk_fmt` (964-978) while `Dec` has none, and `Dec` step (7) defines the addend `a` as the raw `Derive_0x01` output (1002-1004) rather than the reduced addend.

- Remark key-roles (384-390, round-4 CVF-36). The new sentence says the tag is "keyed by sk alone" (it is keyed by `sk_fmt`) and that a mismatched tuple yields a wrong plaintext only "with that negligible probability". A real token under a differing element `k'` passes the step (7) checks with probability about $1/k' \in [2^{-23.8}, 2^{-19.9}]$, which is small but not negligible.
- The remark at 548 is titled "Audit finding CV-11"; there is no such identifier. Its text (562-563 onward) asserts that the theorems and tables "have all been restated", which the residue above contradicts.
- The noise thresholds (614-615) are written as $\lfloor 0.99 \times 2^{64} \rfloor$ without saying whether the product is exact; the exact and IEEE-754 readings differ by 163, and the KAT corpus does not distinguish them. The adjacent CVF-30 footnote points to a `docs/CAVEATS.md` entry that does not exist.
- §9 (2272-2273) says "All 15 tests, 40 scored sub-results, passed". `sts_report.json` contains 14 distinct tests; Overlapping Template Matching is not implemented (round-4 CVF-33 lineage).

Recommendation

Make one pass replacing `sk` by `sk_fmt` wherever HMAC keying is meant; state Lemma nonce-collision for a uniform `sk_fmt` (or under G_0'), add a `Dec` step (0), and extend the Proposition 3.14 proof to the new rejection branches. Replace "negligible" in Remark key-roles with the per-token probability, correct "CV-11" and the "all been restated" claim, state that $\lfloor 0.99 \times 2^{64} \rfloor$ is computed exactly (and give the integer), fix or remove the CAVEATS.md pointer, and say 14 tests in §9 or implement the missing one.

Client Comment

/docs/napseq-eprint-v3.tex

```
384 agree on $\mathbf{k}$ itself. A receiver configured with a different
    tuple
385 (including a different $K$) still verifies the tag, which is keyed by
386 $\mathsf{sk}$ alone, and then fails the structural checks of
387 $\mathsf{Dec}$ step (7) with overwhelming probability as soon as a
    real
388 token uses a differing element; a mismatch therefore surfaces as a
389 decryption error, never (except with that negligible probability) as a
390 wrong plaintext.\footnote{CVF-36: an earlier revision of this
```

same file

```
548 \begin{remark}[Audit finding CV-11: undocumented $\texttt{0x0B}$
```

same file

```
implementation written from the earlier text produces \emph{none} of
the
twelve positive vectors. The defect was in the specification, not the
```

same file

```
  \theta_{\min} = \lfloor 0.75 \times 2^{64} \rfloor, \qquad
  \theta_{\max} = \lfloor 0.99 \times 2^{64} \rfloor,
```

same file

```
\paragraph{$\mathsf{Enc}(\mathbf{k}, \mathsf{sk}, A, M) \to C$.}
(0) derive $\mathsf{sk}_{\mathrm{fmt}}$
(Definition~\ref{def:fmtkey}; it may be cached per key install); (1)
compute $N=N(\mathsf{sk}_{\mathrm{fmt}},A,M)$
(Definition~\ref{def:synthnonce}); (2) pad $M$ to $P_d$
(Section~\ref{sec:padding}); (3) run the token-emission loop of
Section~\ref{sec:token-construction} over $P_d$ using
$(\mathsf{sk}_{\mathrm{fmt}},N)$ to obtain the token vector
$(\mathrm{token}[0],\ldots,\mathrm{token}[N_{\mathrm{tok}}-1])$; (4)
serialise $B$; (5) mask $\widetilde{B}=B\oplus\mathrm{ks}(N)$; (6)
compute
$T=\mathrm{Derive}_{0x03}(\mathsf{sk}_{\mathrm{fmt}},N,\mathrm{TagIn}(A,\widetil
(Definition~\ref{def:taginput});
(7) output $C=N\|\widetilde{B}\|T$. The public encryption oracle is
$\mathsf{Encrypt}(\mathbf{k},\mathsf{sk},A,M)\triangleq\mathsf{Enc}(\mathbf{k},\
— a \emph{pure function} of $(A,M)$, since $N$ is computed
synthetically
rather than sampled.
```

same file

```
implementations perform), where $a$ is the
$\mathrm{real\_idx}$-th addend
$\mathrm{Derive}_{0x01}(\mathsf{sk}_{\mathrm{fmt}},N,\mathrm{real\_idx})$
and
```

same file

```
\begin{proof}
Let $C=N\|\widetilde{B}\|T$ be the output of
$\mathsf{Enc}(\mathbf{k},\mathsf{sk},A,M)$. We first check that the
two tag
inputs coincide, rather than assuming tag agreement.

By Lemma~\ref{lem:length-det},
$|C|=16+8R(\mathrm{MAX\_NOISE\_RUN}+1)+32$
with $R=B(|M|)+2\ge1$, so $|C|\ge2928>48$ and
$N_{\mathrm{tok}}=(|C|-48)/8=R(\mathrm{MAX\_NOISE\_RUN}+1)$ is a
non-negative integer divisible by $\mathrm{MAX\_NOISE\_RUN}+1$: steps
(1)
```

```
and (3) of $\mathsf{Dec}$ do not reject, and step (3) recovers exactly the
$R$ used by $\mathsf{Enc}$. Step (2) parses the leading 16 bytes as $N$ and
the trailing 32 as $T'=T$, leaving the middle bytes as exactly the
$\widetilde{B}$ that $\mathsf{Enc}$ emitted; in particular $N$ is
\emph{read from the ciphertext}, not re-derived, so no circularity with the
synthetic-nonce derivation arises. Hence $\mathsf{Dec}$ step (4) evaluates
$\mathrm{Derive}_{0x03}$ on $\mathrm{TagIn}(A,\widetilde{B})$ with the same
$\mathsf{sk}$, the same $N$, the same caller-supplied $A$, and the same
$\widetilde{B}$ as $\mathsf{Enc}$ step (6) did
(Definition~\ref{def:taginput}). The two tag inputs are byte-identical, so
$T=T'$ and step (5) does not reject. Since $R=B(|M|)+2$ with
$B(|M|)\in\{2^4,\ldots,2^{16}\}$, step (6) does not reject either, and since
the length prefix written by $\mathsf{Enc}$ satisfies $n=|M|\le B(|M|)=R-2$,
neither does step (8).

Step (7) then unmasks with $\mathrm{ks}(N)$, the same keystream
$\mathsf{Enc}$ masked with (a function of $\mathsf{sk}$ and the recovered
$N$ alone), recovering $B$ and hence the token vector. $\mathsf{Enc}$'s
token-emission loop and $\mathsf{Dec}$'s loop-inversion step both compute
the noise-position oracle, addends, and padding codepoints as pure,
deterministic functions of $(\mathsf{sk},N,\mathrm{position})$, so with the
same $\mathsf{sk}$ and $N$ they compute identical intermediate values at
every position; emission and inversion are therefore exact inverses, and
stripping the length prefix and padding yields $M$.
\end{proof}
```

same file

```
Every theorem in this section is unconditional under the standard,
uniform-key PRF assumption on HMAC-SHA256 stated below: because
$\mathsf{sk}$ is sampled uniformly and independently of $\mathbf{k}$, every
```

```
reduction constructed in this section may sample $\mathbf{k}$ on its
own
(to run the arithmetic layer) while forwarding every HMAC call to an
external oracle keyed by the challenger's $\mathsf{sk}$ — the
reduction
never needs to extract $\mathsf{sk}$ from $\mathbf{k}$, or vice
versa, so
no simulation gap and no non-standard key-distribution assumption
arises.
NAPQES does not define a standalone MAC verification key separate from
$\mathsf{sk}$, so EUF-CMA unforgeability of the tag and INT-CTXT
unforgeability of the ciphertext coincide here.
```

same file

```
\begin{lemma}[Nonce collision]\label{lem:nonce-collision}
For any two distinct queried pairs $(A_1,M_1)\neq(A_2,M_2)$ under the
same
$\mathsf{sk}$, there is a PRF adversary $\mathcal{B}$ with similar
running
time such that
$\Pr[N(\mathsf{sk}_{\mathrm{fmt}},A_1,M_1)=N(\mathsf{sk}_{\mathrm{fmt}},A_2,M_2)
\le \mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-
}SHA256}}(\mathcal{B}) +
2^{-128}$.
\end{lemma}

\begin{proof}
By Lemma~\ref{lem:domsep}, $(A_1,M_1)\neq(A_2,M_2)$ implies the two
domain
$\texttt{0x0A}$ inputs are distinct, so replacing
$\mathrm{HMAC}(\mathsf{sk}_{\mathrm{fmt}},\cdot)$
by a truly random function $\rho$ (a PRF hop of advantage
$\mathrm{Adv}^{\mathrm{PRF}}_{\mathrm{HMAC\text{-
}SHA256}}(\mathcal{B})$)
makes the two 128-bit truncated outputs independent and uniform,
colliding
with probability exactly $2^{-128}$.
\end{proof}
```

same file

```
suite correctly rejects the repetition). All 15 tests, 40 scored
sub-results, passed at the $\alpha=0.01$ significance level; the
smallest
```

CVF-V5-25: open Category: **behavior**

The v7 byte and codepoint conventions are still not separated, so a v7 ciphertext produced under one convention decrypts without error to a different plaintext under the other for values up to U+00FF.

This carries the residual of round-4 CVF-14, which is partially fixed.

Fixed in v5: `decrypt_raw` no longer masks recovered values to their low byte and rejects anything above `0xFF` (`lib.rs:1295-1313`); `decrypt_bytes` maps codepoints fallibly instead of dropping invalid ones (`lib.rs:1167-1179`); `decrypt_core` checks the token shape, the `0x10FFFF` bound and scalar validity. The tests `cvf14_...rejected_by_decrypt_raw_for_non_ascii` and `cvf14_decrypt_returns_result_not_panic` fail on v4.

Remaining: the primary recommendation, making one encoding normative or domain-separating the two conventions in the tag, was deferred by the client (remediation report §8.4). Both conventions still share one key and one tag construction, so the confusion survives for every value in `[0, 0xFF]`:

- `encrypt_bytes("é")` followed by `decrypt_raw` (`lib.rs:1263-1314`) returns `[0xE9]` with no error;
- `encrypt_raw([0xC3, 0xA9])` followed by `decrypt_bytes` (`lib.rs:1134-1180`) returns ″Ã©″.

The severity is reduced to low because the v7 surface is deprecated and out-of-range values are now rejected rather than silently corrupted.

Recommendation

Bind the plaintext convention into the v7 tag input (for example a convention byte in the authenticated data), or withdraw `encrypt_raw`/`decrypt_raw` and document `decrypt_raw` as decoding only ciphertexts produced by `encrypt_raw`. At minimum, state in both functions' rustdoc that cross-convention decryption succeeds silently for values up to U+00FF.

Client Comment

/rust/src/lib.rs

```
1134 pub fn decrypt_bytes(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
     Result<String, String> {
1135     validate_key(key)?; // CVF-15
1136     check_v7_aad_len(aad.len())?;
1137     if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
1138         return Err(format!(
1139             "Ciphertext too short: {} bytes; header+tag require at
     least {}.",
```

```
            ciphertext.len(),
            NONCE_SIZE + TAG_SIZE
        ));
    }
    let kb = key_bytes(key);
    let split = ciphertext.len() - TAG_SIZE;
    let payload = &ciphertext[..split];
    let recv_tag = &ciphertext[split..];
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
    let nonce = &payload[..NONCE_SIZE];
    let masked = &payload[NONCE_SIZE..];
    let ks = varint_keystream(&kb, nonce, masked.len());
    let blob: Vec<u8> = masked.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let tokens = fixed_decode_tokens(&blob)
        .map_err(|e| format!("varint decode error: {}", e))?;
    let codepoints = decrypt(nonce, &tokens, key)?;
    // CVF-14: the previous body was `filter_map(char::from_u32)`,
which
    // silently dropped every recovered value that was not a Unicode
scalar
    // (surrogates, > 0x10FFFF). That turned a malformed ciphertext
into a
    // shorter-but-syntactically-valid String rather than into an
error, so a
    // ciphertext produced under the byte-oriented convention
(`encrypt_raw`)
    // could decrypt to a truncated codepoint-oriented string with no
signal.
    // Match `decrypt_bytes_v8`'s fallible mapping: a non-scalar
value is an
    // error.
    codepoints
        .into_iter()
        .enumerate()
        .map(|(i, c)| {
            char::from_u32(c).ok_or_else(|| {
                format!(
                    "Malformed v7 plaintext: recovered value {} at
position {} \
                     is not a Unicode scalar value.",
                    c, i
```

```
                )
            })
        })
        .collect::<Result<String, String>>()
}
```

same file

```
pub fn decrypt_raw(ciphertext: &[u8], key: &[u64], aad: &[u8]) ->
Result<Vec<u8>, String> {
    validate_key(key)?; // CVF-15
    check_v7_aad_len(aad.len())?;
    if ciphertext.len() < NONCE_SIZE + TAG_SIZE {
        return Err(format!(
            "Ciphertext too short: {} bytes; header+tag require at
least {}.",
            ciphertext.len(),
            NONCE_SIZE + TAG_SIZE
        ));
    }
    let kb = key_bytes(key);
    let split = ciphertext.len() - TAG_SIZE;
    let payload = &ciphertext[..split];
    let recv_tag = &ciphertext[split..];
    let calc_tag = compute_auth_tag(&kb, aad, payload,
AAD_LEN_WIDTH_V7);
    if !ct_eq_bytes(recv_tag, calc_tag.as_ref()) {
        return Err("Authentication failed: invalid HMAC tag.".into());
    }
    let nonce = &payload[..NONCE_SIZE];
    let masked = &payload[NONCE_SIZE..];
    let ks = varint_keystream(&kb, nonce, masked.len());
    let blob: Vec<u8> = masked.iter().zip(ks.iter()).map(|(a, b)| a ^
b).collect();
    let tokens = fixed_decode_tokens(&blob)
        .map_err(|e| format!("varint decode error: {}", e))?;
    let codepoints = decrypt(nonce, &tokens, key)?;
    // CVF-14: the previous body was `.map(|c| (c & 0xFF) as u8)` — a
silent
    // narrowing that discarded every recovered codepoint above 255
without
    // signalling. A ciphertext produced by `encrypt_bytes`
(codepoint-oriented,
    // e.g. `é` as one U+00E9 token) handed to `decrypt_raw` would
authenticate
    // and return a self-consistent-looking single byte, hiding the
fact that
```

```
    // the two ports use incompatible message-space encodings. We now reject
    // any recovered value that does not fit in a byte.
    codepoints
        .into_iter()
        .enumerate()
        .map(|(i, c)| {
            if c > 0xFF {
                Err(format!(
                    "Malformed v7 raw ciphertext: recovered codepoint {} at \
                     position {} is out of byte range [0, 255]. This usually \
                     means the ciphertext was produced by `encrypt_bytes` \
                     (codepoint-oriented) rather than `encrypt_raw` \
                    (byte-oriented); the two use incompatible message \
                     encodings (CVF-14).",
                    c, i
                ))
            } else {
                Ok(c as u8)
            }
        })
        .collect()
}
```

CVF-V5-26: open Category: **procedural**

The fuzzing set-up still cannot exercise the unauthenticated decoder in depth, has no target for the new streaming decryptor, and has no committed corpus or recorded run.

This carries the residual of round-4 CVF-22, which is partially fixed.

Fixed in v5: `fuzz/fuzz_targets/decode_bytes.rs` (lines 68-124) now targets v8 and re-tags each mutated input through `fuzz_retag_block_v8`, which is compiled only under the `fuzzing` feature (`fuzz/Cargo.toml:13-15`), so it reaches the post-authentication decode paths. A `validate_key` target with malformed key fixtures was added and is adequate.

Remaining:

- The new `unauth_decode.rs` target (lines 54-67) feeds random token vectors to the decoder, but each real token must have the form `c*k + a`, which a random input

meets with probability about $1/k$. The target therefore almost never passes the first real-token check, the same structural limit round 4 described.

- The targets use the Bucket profile only, with `B <= 256`, and never mutate the nonce.
- There is no fuzz target for `decrypt_stream_ae_v8`, the largest new parser in this release.
- No corpus is committed, no run is recorded (`docs/FUZZ_ATTESTATION.md` is unchanged and still describes the v6 target; the client's §8.4 concedes this), and CI does not build the fuzz crate.
- The paper's sentence on Python fuzzing (`napseq-eprint-v3.tex` 2358-2361) is still unsupported.

Recommendation

Make `unauth_decode` construct structurally valid tokens (choose `c` and compute `c*k + a` from the fuzzer's bytes, then perturb) so that it reaches the later checks. Add a streaming-decryptor target that re-tags chunks and the sentinel, cover all padding profiles and nonce mutation, commit a seed corpus, build the fuzz crate in CI, and record a run in `docs/FUZZ_ATTESTATION.md` before the paper cites it.

Client Comment

/rust/fuzz/Cargo.toml

```
[dependencies.napqes]
path = ".."
features = ["fuzzing"]
```

/rust/fuzz/fuzz_targets/decode_bytes.rs

```
fuzz_target!(|data: &[u8]| {
    // Header: [key_sel, aad_sel, sk_sel, flags, mutate_index_lo,
    //          mutate_index_hi, mutate_xor_byte, msg_len, ..msg..]
    // flags bit0: flip a byte; bits1..3: drop that many token groups.
    if data.len() < 8 {
        return;
    }
    let key = key_for(data[0]);
    // Sanity: keys start above MIN_KEY_PRIME by construction.
    if key.iter().any(|&k| k < MIN_KEY_PRIME) {
        return;
    }
    let aad = aad_for(data[1]);
    let sk = sk_for(data[2]);
```

```
    let flip = data[3] & 1 == 1;
    let drop_groups = ((data[3] >> 1) & 0x07) as usize;
    let mut_lo = data[4] as usize;
    let mut_hi = data[5] as usize;
    let mut_xor = data[6];
    let msg_len = (data[7] as usize).min(data.len() - 8);
    let msg_bytes = &data[8..8 + msg_len];

    // Restrict to valid UTF-8 slices — v8 encrypts &str. Fall back to a
    // trivial ASCII message on invalid UTF-8 to keep coverage on the
    // encrypt/decrypt path.
    let message: &str = match std::str::from_utf8(msg_bytes) {
        Ok(s) => s,
        Err(_) => "A",
    };

    // Encrypt once — the fuzzer proves this cannot panic on any
    // validate_key-accepting key with any UTF-8 message.
    let ct = match encrypt_bytes_v8(message, &key, &sk, aad) {
        Ok(ct) => ct,
        Err(_) => return, // e.g. message too long — accepted refusal
    };

    // Unmutated round-trip: must succeed and reach every branch of the
    // decrypt pipeline. Any panic here is a bug.
    let rt = decrypt_bytes_v8(&ct, &key, &sk, aad).expect("v8 round trip failed");
    assert_eq!(rt, message);

    if !flip && drop_groups == 0 {
        return;
    }
    let mut payload = ct[..ct.len() - TAG_SIZE].to_vec();
    let masked_len = payload.len() - NONCE_SIZE;
    let drop = (drop_groups * TOKEN_GROUP).min(masked_len);
    payload.truncate(payload.len() - drop);
    if flip && payload.len() > NONCE_SIZE {
        let idx = NONCE_SIZE + ((mut_hi << 8 | mut_lo) % (payload.len() - NONCE_SIZE));
        payload[idx] ^= mut_xor;
    }
    let tag = fuzz_retag_block_v8(&payload, &sk, aad);
    payload.extend_from_slice(&tag);
```

```
    let _ = decrypt_bytes_v8(&payload, &key, &sk, aad);
});
```

/rust/fuzz/fuzz_targets/unauth_decode.rs

```
    // Target 1: pub fn decrypt — unauthenticated token slice + nonce.
    // Result<Vec<u32>, String>; must never panic on any tokens, any
key.
    let _ = decrypt(nonce, &tokens, &key);

    // Target 2: decrypt_raw — tag-first (safer surface), fed
arbitrary
    // bytes. Should always return Err from the parse floor or the tag
    // check; must never panic.
    let _ = decrypt_raw(data, &key, b"");

    // Target 3: decrypt_str — base64-decode then decrypt_bytes.
Should
    // report base64 or auth failure, never panic.
    if let Ok(s) = std::str::from_utf8(data) {
        let _ = decrypt_str(s, &key, b"");
    }
```

CVF-V5-27: open Category: **procedural**

The constant-time evidence for the Rust port is still a single whole-call dudect run on an undelivered commit, with no comparator-level measurement and no CI gate.

This carries the residual of round-4 CVF-26, which is partially fixed.

Fixed in v5: the paper's §8.4 (`napseq-eprint-v3.tex` 2252-2259) now cites the delivered Rust harness and its attestation instead of an undelivered C study.

Remaining:

- `examples/dudect_harness.rs` (lines 29-69) times the entire `decrypt_bytes_v8_key` call with the first versus the last tag byte flipped. The comparator is measured only inside an HMAC over the whole payload; the attestation itself puts the signal at about 0.1% of the window, so the pass (one run, $|t| = 1.113$ over 29.7 M samples) is low-power evidence.
- The key is random and unpinned per run (`NapqesKey::generate()`), and the `_key` path skips `validate_key`, the one operation the paper lists as not constant-time.
- The attestation names commit `6db914e` plus working-tree changes, not the delivered `f6ced5d`.

- The comparator-only harness is deferred (remediation report §6 item 3), and there is no CI gate.

Recommendation

Add a harness that times `ct_eq_bytes` alone on equal-length inputs differing in the first versus the last byte, run it on the delivered commit, record the result in the attestation, and add the run to CI with a fixed |t| threshold. State in the paper that the whole-call measurement is low-power evidence.

Client Comment

/rust/examples/dudect_harness.rs

```
fn fixed_key() -> &'static NapqesKey {
    static KEY: OnceLock<NapqesKey> = OnceLock::new();
    KEY.get_or_init(|| {
        napqes::self_test::run_power_on_self_tests().expect("dudect: POST failed");
        NapqesKey::generate().expect("dudect: NapqesKey::generate failed")
    })
}

/// One valid v8 ciphertext produced once at startup. Both classes derive
/// from this by cloning + corrupting one tag byte in-place; allocator
/// typically reuses the same slot, giving both classes similar cache state.
fn base_ct() -> &'static Vec<u8> {
    static CT: OnceLock<Vec<u8>> = OnceLock::new();
    CT.get_or_init(|| {
        encrypt_bytes_v8_key(
            "napqes v8 dudect constant-time timing harness (CVF-26 retarget)",
            fixed_key(),
            b"",
        )
        .expect("dudect: v8 encrypt failed")
    })
}

fn bench_tag_comparison(runner: &mut CtRunner, rng: &mut BenchRng) {
    let key = fixed_key();
```

```
    let base = base_ct();
    let n = base.len();

    let mut ct = base.clone();

    // Left: tag byte 0 wrong  (a hypothetical early-exit comparator would
    // fail immediately). Right: tag byte 31 wrong (would scan all 31 bytes).
    // With subtle::ConstantTimeEq the two must be timing-indistinguishable.
    if rng.gen::<bool>() {
        ct[n - TAG_SIZE] ^= 0xFF;
        runner.run_one(Class::Left, || decrypt_bytes_v8_key(&ct, key, b""));
    } else {
        ct[n - 1] ^= 0xFF;
        runner.run_one(Class::Right, || decrypt_bytes_v8_key(&ct, key, b""));
    }
}
```

CVF-V5-28: open Category: **algorithm**

`Dec` step (7) is now total in the specification, but Remark 3.13 still justifies totality with the argument round 4 showed to be false, and the new step (7) text uses symbols before binding them.

This carries the residual of round-4 CVF-27, which is partially fixed. The code side was already correct in v4 and is unchanged: the checks sit in `decrypt_core_v8` after tag verification, and the negative vectors W-N09, W-N10 and W-N12 were added.

Fixed in v5: step (7) now carries the `t < a` and divisibility guards and the scalar-value check (`napseq-eprint-v3.tex` 992-1012), so `Dec` is total as specified.

Remaining, all on the paper side:

- Remark 3.13 (1015-1034) still says that steps (6) and (8) make step (7) total, which is the claim round 4 refuted; its vector map omits W-N12.
- The sentence that the scheme is ”indifferent to which subset of [0, 2^21)” is retained (913-915), although the scalar check now makes the subset matter.
- Step (7) uses `a` and `k` before binding them, and defines `a` as the raw `Derive_0x01` output rather than the reduced addend, so `t < a` literally compares a 64-bit token with a 256-bit digest.

- §8.4's "What is not" list (2242-2246) still names only steps (6) and (8).

Recommendation

Rewrite Remark 3.13 to say that totality comes from the explicit guards in step (7), and add W-N12 to its vector map. Delete the "indifferent to which subset" sentence. In step (7), bind k and the reduced addend before the guards use them, and update §8.4 to list step (7)'s checks.

Client Comment

/docs/napseq-eprint-v3.tex

```
913         weaker is needed and nothing stronger is assumed: the
arithmetic
914         layer itself operates on the integer codepoint values and is
915         indifferent to which subset of $[0,2^{21})$ they range over.
```

same file

```
992 (7) else unmask $B=\widetilde{B}\oplus\mathrm{ks}(N)$, deserialise
    the
993 $N_{\mathrm{tok}}$ fixed-width tokens, and invert the token-emission
    loop
994 using the same $(\mathsf{sk}_{\mathrm{fmt}},N)$-derived
    noise-position oracle: skip at most
995 $\mathrm{MAX\_NOISE\_RUN}$ consecutive noise positions, then read one
    real
996 token $t$; if $t<a$ or $(t-a)\not\equiv 0\pmod{k}$ output $\bot$
997 (CVF-27: totality preconditions on the recovery); otherwise recover
    the
998 padded codepoint as
999 $(t-a)/k_{(\mathrm{real\_idx}\bmod K)+1}$; if the recovered value
    exceeds
1000 $\mathtt{0x10FFFF}$ or is a Unicode surrogate output $\bot$
1001 (also CVF-27, matching the codepoint-range check the reference
1002 implementations perform), where $a$ is the
1003 $\mathrm{real\_idx}$-th addend
1004 $\mathrm{Derive}_{0x01}(\mathsf{sk}_{\mathrm{fmt}},N,\mathrm{real\_idx})$
    and
1005 $\mathrm{real\_idx}$ counts real tokens from $0$ — exactly the prime
    and
1006 addend $\mathsf{Enc}$ used at that index; stop once $R$ real tokens
    have
```

```
been extracted, discarding any trailing filler; (8) read the
2-codepoint
big-endian length prefix $(p_0,p_1)$; if $p_0>255$ or $p_1>255$
output
$\bot$ (CVF-43: canonical, byte-valued prefix), else set
$n=256p_0+p_1$
and, if $n>R-2$, output $\bot$
(Remark~\ref{rem:dec-structural}); (9) output the $n$ codepoints
following
the prefix.
```

same file

```
\begin{remark}[The post-authentication structural checks]
\label{rem:dec-structural}
Step (3) depends only on $|C|$, a public quantity, and is performed
\emph{before} the tag is computed, so a ciphertext of illegal length
is
rejected without an HMAC evaluation and the rejection reveals nothing
the
adversary did not already know.
Steps (6) and (8) are performed \emph{after} the tag comparison of
step (5),
so they are never reachable on an input an adversary without
$\mathsf{sk}$
could construct, and they play no part in the INT-CTXT argument. They
are
specified nonetheless because step (7) is not total without them: $R$
is
recovered from $|C|$ and $n$ from the decrypted buffer, so a
malformed but
correctly tagged ciphertext — which a key holder can trivially
produce —
would otherwise drive an out-of-range read or an allocation sized by
a field
the caller did not choose. Divisibility by
$\mathrm{MAX\_NOISE\_RUN}+1$
alone does not imply step (6): $R=20$ passes step (3) and fails step
(6).
All three reference implementations enforce these checks in this
order;
vector \texttt{W-N06} of Section~\ref{sec:kats} pins step (3),
\texttt{W-N07} step (6), \texttt{W-N09}--\texttt{W-N10} the per-token
checks of step (7), and \texttt{W-N08} and \texttt{W-N11} step (8).
\end{remark}
```

CVF-V5-29: open Category: **flaw**

Per-call key validation by trial division, with timing that depends on the secret primes, is still performed on the bare-slice v8 entry points and on every streaming call.

This carries the residual of round-4 CVF-29, which is partially fixed.

Fixed in v5: the new `NapqesKey` type validates once at construction (`key.rs:54-57`), and its `_key` entry points skip validation; the paper's §8.4 now lists key validation as not constant-time.

Remaining:

- The bare-slice v8 functions still validate on every message (`lib.rs:1517-1530`, `1611-1625`) and are not deprecated in favour of the `_key` variants.
- The streaming format has no `_key` variant at all: `encrypt_stream_ae_v8_with_nonce` (`lib.rs:1960-1975`), `decrypt_stream_ae_v8` (`lib.rs:2070-2082`) and `StreamV8Encryptor::new` (`lib.rs:2240-2271`) each validate the key on every stream.
- Under the widened prime interval and the new default `K = 13`, one validation costs about 17,400 trial divisions on average (6,487 to 25,155 depending on the secret primes), up from about 11,000 in round 4.

The information exposed is at most about 12.5 bits per observation. On v8 it concerns only the prime tuple, which is not a security parameter, hence low severity. The spread of the same behaviour onto the v7 decryptors is a separate introduced-in-v5 issue.

Recommendation

Add `NapqesKey` variants of the streaming entry points, deprecate the bare-slice v8 functions in favour of the `_key` ones, and document in each bare-slice function that it validates per call with secret-dependent timing.

Client Comment

/rust/src/lib.rs

```
1517 pub fn encrypt_bytes_v8_with_profile(
1518     message: &str,
1519     primes: &[u64],
1520     sk: &[u8; SK_SIZE],
1521     aad: &[u8],
1522     pad_profile: PadProfile,
1523 ) -> Result<Vec<u8>, String> {
1524     // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
1525     #[cfg(feature = "fips_gate")]
1526     self_test::require_post().map_err(|e| e.to_string())?;
```

```
    validate_key(primes)?;
    encrypt_bytes_v8_core(message, primes, sk, aad, pad_profile)
}

```

same file

```
pub fn decrypt_bytes_v8(
    ciphertext: &[u8],
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    aad: &[u8],
) -> Result<String, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    validate_key(primes)?;
    decrypt_bytes_v8_core(ciphertext, primes, sk, aad)
}

/// CVF-29 fix: shared v8-decrypt core that skips per-message
`validate_key`.
/// See [`encrypt_bytes_v8_core`] for the caller contract.
```

same file

```
pub(crate) fn encrypt_stream_ae_v8_with_nonce(
    plaintext: &str,
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    nonce: &[u8; NONCE_SIZE],
    aad: &[u8],
    frame_codepoints: u32,
) -> Result<Vec<u8>, String> {
    validate_key(primes)?;
    validate_stream_v8_frame(frame_codepoints)?;

    let sk_fmt = Secret32::new(derive_format_subkey(sk,
FORMAT_STREAM_AE_V8));
    encrypt_stream_ae_v8_inner(plaintext, primes, sk_fmt.as_ref(),
nonce, aad, frame_codepoints)
}

fn encrypt_stream_ae_v8_inner(
```

same file

```
pub fn decrypt_stream_ae_v8(
    stream: &[u8],
```

```
    primes: &[u64],
    sk: &[u8; SK_SIZE],
    aad: &[u8],
) -> Result<String, String> {
    // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
    #[cfg(feature = "fips_gate")]
    self_test::require_post().map_err(|e| e.to_string())?;
    validate_key(primes)?;
    let sk_fmt = Secret32::new(derive_format_subkey(sk, FORMAT_STREAM_AE_V8));
    decrypt_stream_ae_v8_inner(stream, primes, sk_fmt.as_ref(), aad)
}
```

same file

```
    pub fn new(
        primes: &[u64],
        sk: &[u8; SK_SIZE],
        aad: &[u8],
        frame_codepoints: u32,
    ) -> Result<(Self, Vec<u8>), String> {
        // CVF-47: FIPS 140-3 gate (opt-in via the `fips_gate` feature).
        #[cfg(feature = "fips_gate")]
        crate::self_test::require_post().map_err(|e| e.to_string())?;
        validate_key(primes)?;
        validate_stream_v8_frame(frame_codepoints)?;
        let nonce = generate_nonce_with_crng_check()?;
        let sk_fmt = derive_format_subkey(sk, FORMAT_STREAM_AE_V8);
        let mut header = Vec::with_capacity(1 + 4 + NONCE_SIZE);
        header.push(FORMAT_STREAM_AE_V8);
        header.extend_from_slice(&frame_codepoints.to_be_bytes());
        header.extend_from_slice(&nonce);
        Ok((
            Self {
                primes: primes.to_vec(),
                sk_fmt,
                aad: aad.to_vec(),
                nonce,
                f: frame_codepoints,
                chunk_idx: 0,
                total_real_cps: 0,
               cp_buf: Vec::with_capacity(frame_codepoints as usize),
                finished: false,
            },
            header,
        ))
```

```
2271     }
```

CVF-V5-30: open Category: **efficiency**

Prime generation and primality testing keep three of the four inefficiencies round 4 reported, and key generation still accepts an uncapped element count.

This carries the residual of round-4 CVF-41, which is partially fixed.

Fixed in v5: `MAX_KEY_ELEMENTS = 128` is enforced, `validate_key` checks length and range before primality, and the quadratic distinctness check was replaced by a sort-and-scan. The test `cvf41_validate_key_rejects_key_above_max_elements` fails on v4.

Remaining:

- `is_prime` (`lib.rs:172-190`) is unchanged: it does not use a small-prime table and recomputes the square-root bound rather than hoisting it.
- `generate_prime_numbers` (`lib.rs:208-239`) still checks distinctness with a linear `contains` per candidate, which is quadratic in `count`, and `count` is not capped. The uncapped count is reported in the separate introduced-in-v5 key-generation issue.
- Per-message validation remains on every non-`_key` path (see the carried per-call validation issue).

The sort-and-scan introduced a new unwiped sorted copy of the key, which is filed as a separate introduced-in-v5 issue.

Recommendation

In `is_prime`, compute the square-root bound once and trial-divide first by a small table of primes. In `generate_prime_numbers`, cap `count` at `MAX_KEY_ELEMENTS` and track membership with a sorted vector or a hash set rather than `contains`.

Client Comment

/rust/src/lib.rs

```
172 pub fn is_prime(n: u64) -> bool {
173     if n < 2 {
174         return false;
175     }
176     if n == 2 {
```

```
        return true;
    }
    if n % 2 == 0 {
        return false;
    }
    let mut i: u64 = 3;
    while i.saturating_mul(i) <= n {
        if n % i == 0 {
            return false;
        }
        i += 2;
    }
    true
}
```

same file

```
pub fn generate_prime_numbers(count: usize, min_val: u64, max_val: u64) -> Result<Vec<u64>, String> {
    use rand::distributions::{Distribution, Uniform};
    if max_val <= min_val {
        return Err(format!(
            "generate_prime_numbers: empty range [{}, {}]",
            min_val, max_val
        ));
    }
    let mut rng = rand::thread_rng();
    let dist = Uniform::new_inclusive(min_val, max_val);
    let max_attempts = (count.max(1) as u64).saturating_mul(KEYGEN_DRAWS_PER_PRIME);
    let mut primes: Vec<u64> = Vec::with_capacity(count);
    let mut attempts: u64 = 0;
    while primes.len() < count && attempts < max_attempts {
        // CVF-20: unbiased sample via rand::Uniform (Lemire rejection),
        // replacing the modulo-bias line `min_val + (rng.next_u64() % span)`.
        let num = dist.sample(&mut rng);
        if is_prime(num) && !primes.contains(&num) {
            primes.push(num);
        }
        attempts += 1;
    }
    if primes.len() < count {
        return Err(format!(
           "generate_prime_numbers: could not find {} distinct primes in \
```

```
            [{}, {}] within {} draws; the range is too narrow or too prime-poor \
             for the requested count.",
            count, min_val, max_val, max_attempts
        ));
    }
    Ok(primes)
}
```

CVF-V5-31: open Category: **architecture**

The token-emission loop is still hand-copied: the v8 block loop and the new v8 streaming-chunk loop are two statement-identical copies, and the v7 sealing tail is still triplicated.

This carries the residual of round-4 CVF-46, which is partially fixed.

Fixed in v5: the four v7 copies were replaced by one function, `emit_tokens_v7`. A statement-by-statement comparison of the v4 copies shows that the extraction does not change behaviour, and the loop is pinned by re-encryption of vectors V001 to V022 and by KAT-1.

Remaining:

- The v8 block loop (`lib.rs:1569-1598`) and the streaming-chunk loop (`lib.rs:1805-1835`) are separate hand copies of the same statements. The streaming copy is new in v5, which is exactly the failure mode the finding warned about: the next change to the emission rule must be made twice.
- The v7 sealing tail (masking, tag computation and output assembly) is still copied three times (`lib.rs:1072-1080`, `1120-1128`, `1244-1252`).
- The restructuring that hoists the two statements duplicated in both branches of the loop was not done.

The number of copies fell from five to three, so the severity is reduced to low.

Recommendation

Extract one `emit_tokens_v8` function used by both the block and the streaming encryptors, parameterised by the chunk's counters, and one v7 sealing helper used by the three v7 encryptors. Hoist the statements common to both loop branches.

Client Comment

/rust/src/lib.rs

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    payload
```

same file

```
    let blob = fixed_encode_tokens(&cypher);
    let ks = varint_keystream(&kb, &nonce, blob.len());
    let masked: Vec<u8> = blob.iter().zip(ks.iter()).map(|(a, b)| a ^ b).collect();
    let mut payload = Vec::with_capacity(NONCE_SIZE + masked.len());
    payload.extend_from_slice(&nonce);
    payload.extend_from_slice(&masked);
    let tag = compute_auth_tag(&kb, aad, &payload, AAD_LEN_WIDTH_V7);
    payload.extend_from_slice(&tag);
    Ok(payload)
```

same file

```
    for &c in &padded {
        let mut noise_run: u64 = 0;
```

```
        loop {
            if noise_run < MAX_NOISE_RUN && is_noise_pos_v8(sk_fmt,
&nonce, ct_pos, noise_theta) {
                let k = primes[(real_idx % kk) as usize];
                let nc = derive_noise_char(sk_fmt, &nonce, ct_pos);
                let na = derive_noise_token_addend(sk_fmt, &nonce,
ct_pos, k);
                cypher.push(nc * k + na);
                ct_pos += 1;
                noise_run += 1;
            } else {
                let k = primes[(real_idx % kk) as usize];
                let addend = derive_addend(sk_fmt, &nonce, real_idx,
k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }
    // V2-CVF2 fix: pad up to the fixed, bucket-only ceiling so
ciphertext
    // length never depends on the message-derived nonce's noise
realisation.
    let ceiling = (padded.len() as u64) * (MAX_NOISE_RUN + 1);
    while (cypher.len() as u64) < ceiling {
        let k = primes[(real_idx % kk) as usize];
        let nc = derive_noise_char(sk_fmt, &nonce, ct_pos);
       let na = derive_noise_token_addend(sk_fmt, &nonce, ct_pos, k);
        cypher.push(nc * k + na);
        ct_pos += 1;
    }
```

same file

```
    for &c in chunk_cps {
        let mut noise_run: u64 = 0;
        loop {
            if noise_run < MAX_NOISE_RUN
                && is_noise_pos_v8(sk_fmt, sub_nonce, ct_pos,
noise_theta)
            {
                let k = primes[(real_idx % kk) as usize];
               let nc = derive_noise_char(sk_fmt, sub_nonce, ct_pos);
                let na = derive_noise_token_addend(sk_fmt, sub_nonce,
ct_pos, k);
```

```
                cypher.push(nc * k + na);
                ct_pos += 1;
                noise_run += 1;
            } else {
                let k = primes[(real_idx % kk) as usize];
                let addend = derive_addend(sk_fmt, sub_nonce,
real_idx, k);
                cypher.push(c as u64 * k + addend);
                ct_pos += 1;
                real_idx += 1;
                break;
            }
        }
    }

    let ceiling = f * (MAX_NOISE_RUN + 1);
    while (cypher.len() as u64) < ceiling {
        let k = primes[(real_idx % kk) as usize];
        let nc = derive_noise_char(sk_fmt, sub_nonce, ct_pos);
        let na = derive_noise_token_addend(sk_fmt, sub_nonce, ct_pos,
k);
        cypher.push(nc * k + na);
        ct_pos += 1;
    }
```

7. Conclusion

This report covers the Rust reference implementation of NAPQES v4, with the version 4 paper serving as the conformance oracle rather than as the subject; the specification was audited separately and its findings are reported elsewhere. On its own terms the implementation is competent and careful, and, byte for byte, almost exactly the construction the paper describes. What the round establishes is that "almost" is doing real work in that sentence. We report **thirty-seven findings: three high, seventeen moderate and seventeen low**.

The first of the three high findings is the reason, and it is the central result of the round. Every derivation in the delivered v8 implementation is keyed by an undocumented format subkey $\mathsf{sk}_{\mathsf{fmt}} = \mathsf{HMAC}\big(\mathsf{sk}, \texttt{0x0B||0x01}\big)$ rather than by sk, and we did not infer this from the source alone: an independent re-implementation of Enc written from the paper's text reproduces **0 of 12** of the delivered positive known-answer vectors when keyed by sk as Table 1 specifies, and **12 of 12**, byte for byte, under that one substitution and no other change. Two consequences are structural, and both survive the narrowing of this report. Section 10 requires conforming implementations to produce byte-identical ciphertext for every positive vector, and we have shown experimentally that no Table-1-faithful implementation can do so — the specification's own conformance test rejects the specification, and because the failure is a whole-ciphertext disagreement the corpus cannot distinguish it from an ordinary bug. And since $\mathsf{sk}_{\mathsf{fmt}}$ is a pseudorandom value rather than a uniform draw, the uniform-key assumption does not reach it directly; `Theorem 4.8`, `Theorem 4.12`, `Theorem 4.18` and `Theorem 4.20` each need one further $\mathsf{Adv}^{\mathsf{PRF}}_{\mathsf{HMAC}}(\mathcal{B}_0)$ term for the extra hop. That composition is sound and the added term is negligible, so this is not a loss of security — but it means the object proved about is not the object computed, and it means an implementer working from the paper cannot interoperate with the reference port. We stress that the extra layer is a good idea: it separates wire formats cryptographically, all three ports apply it, and the right repair is to specify it, not to remove it.

The other two high findings are in the code and are independent of each other. Six public entry points on the legacy v7 surface short-circuit an empty ciphertext to a successful empty plaintext before any tag is computed, so a zero-length ciphertext verifies under every key and every associated-data value — a universal forgery of the empty message, reachable from outside the crate and, through the crate's own frame layer, from the wire. And the FIPS 140-3 power-on self-test, which a module claiming that conformance must pass before it produces any output, exercises only the superseded v7 format: none of its three known-answer tests touches a line of the v8 scheme the paper specifies, and its software-integrity test compares the crate's version string to the literal `0.1.0`, so it passes today and fails on the next version bump for a reason unrelated to integrity. A

companion moderate finding completes that picture: whatever the self-tests conclude is never latched and never consulted, so every encryption and decryption path behaves identically whether the tests passed, failed, or never ran at all.

Of the seventeen moderate findings, the largest group concerns the legacy v7 surface, which the crate ships alongside v8 and which the paper neither specifies nor disclaims clearly enough. It still runs the single-secret key schedule that the whole v8 redesign exists to replace, still uses a four-byte associated-data length prefix that truncates silently where v8 uses eight, computes its noise threshold in floating point, applies neither the noise-run cap nor the token ceiling — so on v7 the ciphertext length is a random function of a fresh nonce rather than a deterministic function of the padding bucket, which is the property the length-hiding result depends on — and validates no key at any entry point. The cause of the last of these is itself a finding: the token-emission loop is hand-copied into five encryption entry points, the copies have already drifted apart, and the missing noise-run cap is simply a fix that landed in one copy of five. The same legacy surface dominates the delivered evidence. The fuzz harness targets the v7 decryptor rather than the v8 one the paper names; three of the six known-answer parity tests consume the corpus the paper itself places out of scope; and the statistical study of Section 9 was run on a bitstream generated by the v7 encryptor. The crate ships two schemes, the paper describes one, and the evidence mostly exercises the other. A further moderate finding records that the Rust port realises both of the competing message-space encodings the paper attributes to different ports, so which one a caller gets depends on which entry point they reach.

The second group of moderate findings is a set of claims about the implementation that the implementation does not support. The paper credits the constant-time tag comparison to the `subtle` crate; `subtle` is a declared dependency and is never called, and the hand-written comparator that replaces it uses a zip over two slices, so on a length mismatch it compares the common prefix and returns true — including for an empty candidate tag. Its only guard is a debug assertion, and the manifest declares no release profile, so that assertion, and the integer-overflow checks that another finding depends on, are both absent from the build the client ships. Key validation deliberately omits the normative upper bound on the prime elements, leaving the paper's unconditional no-overflow claim unenforced. The length prefix is reconstructed from two codepoints without a range check, so distinct codepoint pairs collapse onto the same length and an authenticated ciphertext can have more than one valid reading — which, for a scheme whose headline claim is byte-parity across three ports, is a real interoperability hazard. Key generation reintroduces, by an unrejected modulo reduction, precisely the bias that a remark of the paper exists to eliminate. The paper's no-panic attestation covers one entry point where five more are reachable. And the README documents about five of some thirty-three public items, omits the v8 API entirely, describes a retired wire format as current, and offers one usage example that does not compile.

Two moderate findings stand outside both groups. The remark that justifies the two post-authentication decoder checks does so with a totality argument that is false: those checks bound the two length fields and constrain no token value, so a key holder can still drive the decryption arithmetic below zero — a gap the Rust port closes with two rejection rules that appear nowhere in the specification, which is to say the code is safe and the conformance statement is not. The last is a cluster of four avoidable inefficiencies in the prime-generation and key-validation paths.

The seventeen low findings are precision and hygiene defects rather than design problems. They record documentation that has fallen out of date with the crate it describes, a timing channel in key validation that the paper's own inventory of non-constant-time operations omits, zeroization helpers that are never called on the value that actually keys the scheme, padding-profile bounds that are checked for one profile of three, tests that cannot fail, error messages that misreport which of several states occurred, normative parameters that appear in the code as bare literals, and a bundle of nine smaller inconsistencies between the specification's near-normative text and the code that realises it, filed as one. None of them is urgent. Collectively they indicate a codebase and a document that have been revised repeatedly and never re-read end to end against each other.

Two things this report cannot settle should be said plainly. The Python and C ports have never been delivered in any round, so every statement here about the three ports agreeing or disagreeing rests on the Rust source, on the stored corpora, and on what the paper and the crate's comments assert — not on their code. And the delivered tree is considerably larger than the reviewed surface: the KEM, the frame layer, the protocol adapters and the three binaries were not audited, although an in-scope file exports them as public modules of the same crate. During the review we came into incidental contact with some of that code and with one stored evidence artifact, and rather than discard what we saw we have recorded it in the two clearly-labelled sections that follow this one. Neither section contains audited findings, and both say so; they are there because the client should have the information, not because we are asserting it.

Our overall assessment is that the implementation is well built where it matters most and misdescribed where it matters most. The v8 path is sound and we found no way to break it from outside; its defect is that it computes a scheme the specification does not describe, and the repair is a small edit to the specification with a large reach — it removes the leading finding, makes Section 10's conformance requirement satisfiable, and costs the security argument one negligible term that should be written down anyway. The risk in this crate is concentrated elsewhere, in the legacy v7 surface that sits beside v8 sharing a public API and a key type, carrying an authentication bypass, and standing in for v8 in most of the evidence the paper offers. We recommend that the client decide explicitly whether v7 is still supported — and if it is not, delete it, because every high finding and most moderate findings in the implementation either live there or are obscured by it.

8. Out-of-Scope Observations (Code Not Reviewed)

The code discussed in this section was not part of the reviewed surface. None of the items below is an audited finding, none has been assessed to the standard the findings above were held to, and none should be treated as one. They are recorded because we encountered them incidentally while establishing the reach of findings that are in scope, and because a client is better served by seeing them than by not.

The two we would most want the client to look at first both sit in the transport and key-establishment code that the audited crate re-exports as public modules of itself.

The first concerns the prime interval. Section 3.1 of the paper carries the document's only **MUST**: implementations must draw key elements from $\mathcal{P}$, the primes in $[10^6,\ 9.9 \times 10^6]$. The audited file honours it. `src/kem.rs`, which does not, declares its own interval $[10^6,\ 1.5 \times 10^7)$ and samples thirteen key elements from it. By an independent sieve there are $892{,}206$ primes in that interval, of which $312{,}259$ — exactly 35.00% — lie above max $\mathcal{P}$. A tuple produced there therefore carries 4.55 out-of-interval elements on average, and the probability that all thirteen are in range is about 0.37%. The audited key-validation routine accepts every one of them, because it enforces no upper bound. This matters for a finding that is in scope: it converts that finding's overflow precondition from a hypothetical caller error into a routine product of the crate's own code. Read with `kem.rs` in view, the in-scope finding is no longer "a caller who supplies an oversized prime overflows the token multiplication" but "the crate's own key-derivation path produces out-of-interval keys on essentially every invocation, and only the size of the excess keeps it below the overflow threshold". The same function also samples by an unrejected modulo reduction, disagrees with the audited sampler on whether the interval is closed at the top, and runs an unbounded search loop with no failure path.

The second concerns replay protection. `ot_frame::unwrap_pdu` validates and commits the per-peer sequence counter before it authenticates the payload: the envelope header is parsed, the counter is checked and written, and only then is a tag verified. An attacker who can inject a single frame on the wire, holding no key material, therefore advances that counter to any value they choose, after which every genuine frame from the real peer is rejected as a replay. Choosing the maximum value locks the affected peer out permanently; the state is process-global, so recovery requires restarting the gateway, and the attack can be repeated immediately. This holds for any frame whose tag fails, so it is independent of the empty-envelope authentication bypass reported above — though the two compose badly, because an empty envelope is accepted rather than merely counted. The standard remedy is to move the counter write after tag verification; since the sequence number is already covered by the associated data, a frame that authenticates has proved the sender knew the key when it chose that number, which is exactly

the property the counter needs.

Separately, and because it bears directly on the reach of a major finding: `ot_frame::unwrap_pdu` reaches the legacy decryption entry point, so the empty-ciphertext authentication bypass reported above is reachable from the frame layer, and through it from the wire. That is part of why we assess that finding as major rather than moderate.

Four further observations are recorded in our working notes and can be supplied in full on request: the transport path derives no independent 256-bit secret at all and runs the pre-v8 single-secret key schedule, so no theorem in the paper applies to any traffic the gateway carries; the file-backed key store loads device key material from an unvalidated plaintext JSON file with no validation, no independent secret and no zeroization; a key-derivation salt still carries a v6 domain label in a v8 crate; and the generator that produced the Section 9 bitstream calls the legacy encryptor, which is where the code-side half of an in-scope finding would be fixed.

We would put one question to the client ahead of any of this. If the position is that these modules are demonstration or integration code rather than part of the product, then most of the above becomes a scoping and documentation matter rather than a security one. Nothing in the crate, the README or the paper currently says that, and the audited file exports all of them as public modules, so the artifacts do not presently support that reading.

9. Evidence Provenance: The Section 9 Statistical Report

This section records a discrepancy in a delivered evidence artifact. It is not an audited finding and we are not asserting a conclusion about it; we are asking the client to explain it.

Section 9 of the paper reports a NIST SP 800-22 statistical study. The delivered tree contains both the pipeline that runs such a study, `tests/sts_pipeline.py`, and a stored result, `tests/sts_report_v6.json`. The stored result cannot have been produced by that pipeline, and it reports an outcome the pipeline is incapable of producing.

The schemas do not match. Each per-test record the pipeline emits carries the fields `name`, `passed`, `skipped_nistrng_bug`, `p_value` and `p_values`, and its summary block counts `total`, `passed`, `failed` and `skipped_nistrng_bug`. Every record in the stored file instead carries `name`, `passed`, `skipped`, `p_value` and `note`; there is no `p_values` array anywhere in it, and its summary counts `skipped` rather than `skipped_nistrng_bug`. The granularity differs too: the stored file lists forty entries, breaking the random-excursion families out into one row per state in the manner of the original NIST suite, where the pipeline emits one row per test as its library returns them.

The substantive point is the second one. The pipeline declares a set `_NISTRNG_BROKEN_TESTS` naming eight tests — the discrete Fourier transform, linear complexity, non-overlapping template matching, serial, approximate entropy, random excursion, random excursion variant and Maurer's universal test — which it treats as unusable because of a defect in the library it drives, and which it counts as skipped rather than as passed or failed. Any genuine run of that pipeline must therefore report at least eight skips. The stored file reports every one of those eight as a clean pass, with `skipped` false on each record and a summary skip count of zero, and reports forty passes out of forty with no failures and no skips.

We draw no conclusion from this beyond the obvious one, that the stored artifact and the delivered pipeline are not the same evidence. The file is also labelled as version v6, two format revisions behind the scheme the paper specifies, which is consistent with an in-scope finding reporting that the Section 9 result is asserted of a scheme other than the one it was measured on. We ask the client to say which tool produced `sts_report_v6.json`, against which version of the scheme, and whether the eight tests the delivered pipeline considers unusable were in fact run.

Client Comment



ABDK
Consulting

About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

EMail

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

@ABDKConsulting

LinkedIn

company/abdk-consulting